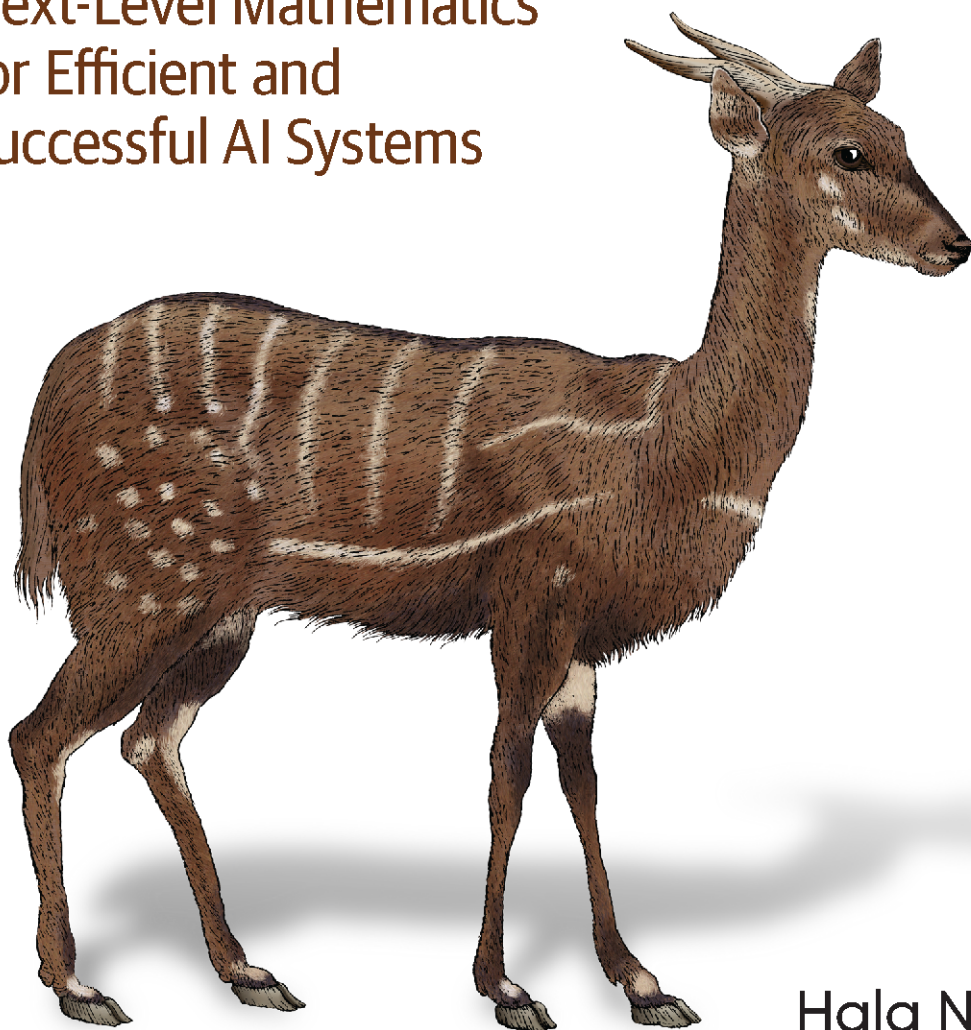


O'REILLY®

Essential Math for AI

Next-Level Mathematics
for Efficient and
Successful AI Systems



Hala Nelson

Essential Math for AI

Many sectors and industries are eager to integrate AI and data-driven technologies into their systems and operations. But to build truly successful AI systems, you need a firm grasp of the underlying mathematics. This comprehensive guide bridges the current gap in presentation between the unlimited potential and applications of AI and its relevant mathematical foundations.

Rather than discussing dense academic theory, author Hala Nelson surveys the mathematics necessary to thrive in the AI field, focusing on real-world applications and state-of-the-art models. You'll explore topics such as regression, neural networks, convolution, optimization, probability, Markov processes, differential equations, and more within an exclusive AI context. Engineers, data scientists, mathematicians, and scientists will gain a solid foundation for success in the AI and math fields.

You'll be able to:

- Comfortably speak the languages of AI, machine learning, data science, and mathematics
- Unify machine learning models and natural language models under one mathematical structure
- Handle graph and network data with ease
- Explore real data, visualize space transformations, reduce dimensions, and process images
- Decide on which models to use for different data-driven projects
- Explore the various implications and limitations of AI

"Technology and AI markets are like a river, where some parts are moving faster than others. Successfully applying AI requires the skill of assessing the direction of the flow and complementing it with a strong foundation, which this book enables in an engaging, delightful, and inclusive way. Hala has made math fun for a spectrum of participants in the AI-enabled future!"

—Adri Purkayastha

Group Head, AI Operational Risk and
Digital Risk Analytics, BNP Paribas

Hala Nelson is an associate professor of mathematics at James Madison University who specializes in mathematical modeling and consults for the public sector on emergency and infrastructure services. She has a PhD in mathematics from the Courant Institute of Mathematical Sciences at New York University.

AI

US \$79.99

CAN \$99.99

ISBN: 978-1-098-10763-5



5 7 9 9 9

Twitter: @oreillymedia
[linkedin.com/company/oreilly-media](https://www.linkedin.com/company/oreilly-media)
[youtube.com/oreillymedia](https://www.youtube.com/oreillymedia)

Praise for *Essential Math for AI*

Technology and AI markets are like a river, where some parts are moving faster than others. Successfully applying AI requires the skill of assessing the direction of the flow and complementing it with a strong foundation, which this book enables, in an engaging, delightful, and inclusive way. Hala has made math fun for a spectrum of participants in the AI-enabled future!

—Adri Purkayastha, Group Head, AI Operational Risk and Digital Risk Analytics, BNP Paribas

Texts on artificial intelligence are usually either technical manuscripts written by experts for other experts, or cursory, math-free introductions catered to general audiences. This book takes a refreshing third path by introducing the mathematical foundations for readers in business, data, and similar fields without advanced mathematics degrees. The author weaves elegant equations and pithy observations throughout, all the while asking the reader to consider the very serious implications artificial intelligence has on society.

I recommend *Essential Math for AI* to anyone looking for a rigorous treatment of AI fundamentals viewed through a practical lens.

—George Mount, Data Analyst and Educator

Hala has done a great job in explaining crucial mathematical concepts. This is a must-read for every serious machine learning practitioner. You'd love the field more once you go through the book.

—Umang Sharma, Senior Data Scientist and Author

To understand artificial intelligence, one needs to understand the relationship between math and AI. Dr. Nelson made this easy by giving us the foundation on which the symbiotic relationship between the two disciplines is built.

—Huan Nguyen, Rear Admiral (Ret.),
Cyber Engineering, NAVSEA

Essential Math for AI

*Next-Level Mathematics for Efficient and
Successful AI Systems*

Hala Nelson

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Essential Math for AI

by Hala Nelson

Copyright © 2023 Hala Nelson. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Aaron Black

Development Editor: Angela Rufino

Production Editor: Kristen Brown

Copyeditor: Sonia Saruba

Proofreader: JM Olejarz

Indexer: nSight, Inc.

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

January 2023: First Edition

Revision History for the First Edition

2023-01-04: First Release

2023-02-03: Second Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098107635> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Essential Math for AI*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-10763-5

[LSI]

Table of Contents

Preface.....	xvii
1. Why Learn the Mathematics of AI?.....	1
What Is AI?	2
Why Is AI So Popular Now?	3
What Is AI Able to Do?	3
An AI Agent's Specific Tasks	4
What Are AI's Limitations?	6
What Happens When AI Systems Fail?	8
Where Is AI Headed?	8
Who Are the Current Main Contributors to the AI Field?	10
What Math Is Typically Involved in AI?	10
Summary and Looking Ahead	11
2. Data, Data, Data.....	13
Data for AI	14
Real Data Versus Simulated Data	16
Mathematical Models: Linear Versus Nonlinear	16
An Example of Real Data	18
An Example of Simulated Data	21
Mathematical Models: Simulations and AI	25
Where Do We Get Our Data From?	27
The Vocabulary of Data Distributions, Probability, and Statistics	29
Random Variables	30
Probability Distributions	30
Marginal Probabilities	31

The Uniform and the Normal Distributions	31
Conditional Probabilities and Bayes' Theorem	31
Conditional Probabilities and Joint Distributions	31
Prior Distribution, Posterior Distribution, and Likelihood Function	32
Mixtures of Distributions	32
Sums and Products of Random Variables	32
Using Graphs to Represent Joint Probability Distributions	33
Expectation, Mean, Variance, and Uncertainty	33
Covariance and Correlation	33
Markov Process	34
Normalizing, Scaling, and/or Standardizing a Random Variable or Data Set	34
Common Examples	34
Continuous Distributions Versus Discrete Distributions (Density Versus Mass)	35
The Power of the Joint Probability Density Function	37
Distribution of Data: The Uniform Distribution	38
Distribution of Data: The Bell-Shaped Normal (Gaussian) Distribution	40
Distribution of Data: Other Important and Commonly Used Distributions	43
The Various Uses of the Word "Distribution"	47
A/B Testing	48
Summary and Looking Ahead	48
3. Fitting Functions to Data.....	51
Traditional and Very Useful Machine Learning Models	53
Numerical Solutions Versus Analytical Solutions	55
Regression: Predict a Numerical Value	56
Training Function	58
Loss Function	60
Optimization	71
Logistic Regression: Classify into Two Classes	85
Training Function	85
Loss Function	86
Optimization	88
Softmax Regression: Classify into Multiple Classes	88
Training Function	90
Loss Function	92
Optimization	92
Incorporating These Models into the Last Layer of a Neural Network	93
Other Popular Machine Learning Techniques and Ensembles of Techniques	94
Support Vector Machines	94
Decision Trees	98

Random Forests	107
k-means Clustering	108
Performance Measures for Classification Models	109
Summary and Looking Ahead	110
4. Optimization for Neural Networks.	113
The Brain Cortex and Artificial Neural Networks	113
Training Function: Fully Connected, or Dense,	
Feed Forward Neural Networks	115
A Neural Network Is a Computational Graph Representation of the	
Training Function	117
Linearly Combine, Add Bias, Then Activate	117
Common Activation Functions	122
Universal Function Approximation	125
Approximation Theory for Deep Learning	131
Loss Functions	131
Optimization	133
Mathematics and the Mysterious Success of Neural Networks	134
Gradient Descent $\vec{\omega}^{i+1} = \vec{\omega}^i - \eta \nabla L(\vec{\omega}^i)$	135
Explaining the Role of the Learning Rate Hyperparameter η	137
Convex Versus Nonconvex Landscapes	140
Stochastic Gradient Descent	143
Initializing the Weights $\vec{\omega}^0$ for the Optimization Process	144
Regularization Techniques	145
Dropout	145
Early Stopping	146
Batch Normalization of Each Layer	146
Control the Size of the Weights by Penalizing Their Norm	147
Penalizing the l^2 Norm Versus Penalizing the l^1 Norm	150
Explaining the Role of the Regularization Hyperparameter α	151
Hyperparameter Examples That Appear in Machine Learning	152
Chain Rule and Backpropagation: Calculating $\nabla L(\vec{\omega}^i)$	153
Backpropagation Is Not Too Different from How Our Brain Learns	154
Why Is It Better to Backpropagate?	155
Backpropagation in Detail	155
Assessing the Significance of the Input Data Features	157
Summary and Looking Ahead	158

5. Convolutional Neural Networks and Computer Vision.	161
Convolution and Cross-Correlation	163
Translation Invariance and Translation Equivariance	167
Convolution in Usual Space Is a Product in Frequency Space	167
Convolution from a Systems Design Perspective	168
Convolution and Impulse Response for Linear and Translation Invariant Systems	169
Convolution and One-Dimensional Discrete Signals	171
Convolution and Two-Dimensional Discrete Signals	172
Filtering Images	174
Feature Maps	178
Linear Algebra Notation	179
The One-Dimensional Case: Multiplication by a Toeplitz Matrix	182
The Two-Dimensional Case: Multiplication by a Doubly Block Circulant Matrix	182
Pooling	183
A Convolutional Neural Network for Image Classification	184
Summary and Looking Ahead	186
 6. Singular Value Decomposition: Image Processing, Natural Language Processing, and Social Media.	 187
Matrix Factorization	188
Diagonal Matrices	191
Matrices as Linear Transformations Acting on Space	193
Action of A on the Right Singular Vectors	194
Action of A on the Standard Unit Vectors and the Unit Square Determined by Them	195
Action of A on the Unit Circle	196
Breaking Down the Circle-to-Ellipse Transformation According to the Singular Value Decomposition	197
Rotation and Reflection Matrices	198
Action of A on a General Vector $\vec{x}$	199
Three Ways to Multiply Matrices	200
The Big Picture	201
The Condition Number and Computational Stability	203
The Ingredients of the Singular Value Decomposition	204
Singular Value Decomposition Versus the Eigenvalue Decomposition	204
Computation of the Singular Value Decomposition	206
Computing an Eigenvector Numerically	207
The Pseudoinverse	208

Applying the Singular Value Decomposition to Images	209
Principal Component Analysis and Dimension Reduction	212
Principal Component Analysis and Clustering	214
A Social Media Application	214
Latent Semantic Analysis	215
Randomized Singular Value Decomposition	216
Summary and Looking Ahead	216
7. Natural Language and Finance AI: Vectorization and Time Series.....	219
Natural Language AI	222
Preparing Natural Language Data for Machine Processing	223
Statistical Models and the log Function	226
Zipf's Law for Term Counts	226
Various Vector Representations for Natural Language Documents	227
Term Frequency Vector Representation of a Document or Bag of Words	227
Term Frequency-Inverse Document Frequency Vector Representation of a Document	228
Topic Vector Representation of a Document Determined by Latent Semantic Analysis	228
Topic Vector Representation of a Document Determined by Latent Dirichlet Allocation	232
Topic Vector Representation of a Document Determined by Latent Discriminant Analysis	233
Meaning Vector Representations of Words and of Documents Determined by Neural Network Embeddings	234
Cosine Similarity	241
Natural Language Processing Applications	243
Sentiment Analysis	243
Spam Filter	244
Search and Information Retrieval	244
Machine Translation	246
Image Captioning	247
Chatbots	247
Other Applications	247
Transformers and Attention Models	247
The Transformer Architecture	248
The Attention Mechanism	251
Transformers Are Far from Perfect	255
Convolutional Neural Networks for Time Series Data	255
Recurrent Neural Networks for Time Series Data	257
How Do Recurrent Neural Networks Work?	258

Gated Recurrent Units and Long Short-Term Memory Units	260
An Example of Natural Language Data	261
Finance AI	261
Summary and Looking Ahead	262
8. Probabilistic Generative Models.....	263
What Are Generative Models Useful For?	264
The Typical Mathematics of Generative Models	265
Shifting Our Brain from Deterministic Thinking to Probabilistic Thinking	268
Maximum Likelihood Estimation	270
Explicit and Implicit Density Models	272
Explicit Density-Tractable: Fully Visible Belief Networks	273
Example: Generating Images via PixelCNN and Machine Audio via WaveNet	273
Explicit Density-Tractable: Change of Variables Nonlinear Independent Component Analysis	276
Explicit Density-Intractable: Variational Autoencoders Approximation via Variational Methods	277
Explicit Density-Intractable: Boltzman Machine Approximation via Markov Chain	279
Implicit Density-Markov Chain: Generative Stochastic Network	279
Implicit Density-Direct: Generative Adversarial Networks	280
How Do Generative Adversarial Networks Work?	281
Example: Machine Learning and Generative Networks for High Energy Physics	283
Other Generative Models	285
Naive Bayes Classification Model	286
Gaussian Mixture Model	287
The Evolution of Generative Models	288
Hopfield Nets	290
Boltzmann Machine	290
Restricted Boltzmann Machine (Explicit Density and Intractable)	291
The Original Autoencoder	292
Probabilistic Language Modeling	293
Summary and Looking Ahead	295
9. Graph Models.....	297
Graphs: Nodes, Edges, and Features for Each	299
Example: PageRank Algorithm	302
Inverting Matrices Using Graphs	307
Cayley Graphs of Groups: Pure Algebra and Parallel Computing	308

Message Passing Within a Graph	309
The Limitless Applications of Graphs	310
Brain Networks	311
Spread of Disease	312
Spread of Information	312
Detecting and Tracking Fake News Propagation	312
Web-Scale Recommendation Systems	314
Fighting Cancer	314
Biochemical Graphs	315
Molecular Graph Generation for Drug and Protein Structure Discovery	316
Citation Networks	316
Social Media Networks and Social Influence Prediction	316
Sociological Structures	317
Bayesian Networks	317
Traffic Forecasting	317
Logistics and Operations Research	318
Language Models	318
Graph Structure of the Web	320
Automatically Analyzing Computer Programs	321
Data Structures in Computer Science	321
Load Balancing in Distributed Networks	322
Artificial Neural Networks	323
Random Walks on Graphs	324
Node Representation Learning	326
Tasks for Graph Neural Networks	327
Node Classification	327
Graph Classification	328
Clustering and Community Detection	329
Graph Generation	329
Influence Maximization	329
Link Prediction	330
Dynamic Graph Models	330
Bayesian Networks	331
A Bayesian Network Represents a Compactified Conditional Probability Table	333
Making Predictions Using a Bayesian Network	334
Bayesian Networks Are Belief Networks, Not Causal Networks	334
Keep This in Mind About Bayesian Networks	335
Chains, Forks, and Colliders	336
Given a Data Set, How Do We Set Up a Bayesian Network for the Involved Variables?	337

Graph Diagrams for Probabilistic Causal Modeling	338
A Brief History of Graph Theory	340
Main Considerations in Graph Theory	341
Spanning Trees and Shortest Spanning Trees	341
Cut Sets and Cut Vertices	342
Planarity	342
Graphs as Vector Spaces	343
Realizability	343
Coloring and Matching	344
Enumeration	344
Algorithms and Computational Aspects of Graphs	344
Summary and Looking Ahead	345
10. Operations Research.....	347
No Free Lunch	349
Complexity Analysis and $O()$ Notation	350
Optimization: The Heart of Operations Research	353
Thinking About Optimization	356
Optimization: Finite Dimensions, Unconstrained	357
Optimization: Finite Dimensions, Constrained Lagrange Multipliers	357
Optimization: Infinite Dimensions, Calculus of Variations	360
Optimization on Networks	365
Traveling Salesman Problem	365
Minimum Spanning Tree	366
Shortest Path	367
Max-Flow Min-Cut	368
Max-Flow Min-Cost	369
The Critical Path Method for Project Design	369
The n-Queens Problem	370
Linear Optimization	371
The General Form and the Standard Form	372
Visualizing a Linear Optimization Problem in Two Dimensions	373
Convex to Linear	374
The Geometry of Linear Optimization	377
The Simplex Method	379
Transportation and Assignment Problems	386
Duality, Lagrange Relaxation, Shadow Prices, Max-Min, Min-Max, and All That	386
Sensitivity	401
Game Theory and Multiagents	402
Queuing	404

Inventory	405
Machine Learning for Operations Research	405
Hamilton-Jacobi-Bellman Equation	406
Operations Research for AI	407
Summary and Looking Ahead	407
11. Probability.....	411
Where Did Probability Appear in This Book?	412
What More Do We Need to Know That Is Essential for AI?	415
Causal Modeling and the Do Calculus	415
An Alternative: The Do Calculus	417
Paradoxes and Diagram Interpretations	420
Monty Hall Problem	420
Berkson's Paradox	422
Simpson's Paradox	422
Large Random Matrices	424
Examples of Random Vectors and Random Matrices	424
Main Considerations in Random Matrix Theory	427
Random Matrix Ensembles	429
Eigenvalue Density of the Sum of Two Large Random Matrices	430
Essential Math for Large Random Matrices	430
Stochastic Processes	432
Bernoulli Process	433
Poisson Process	433
Random Walk	434
Wiener Process or Brownian Motion	435
Martingale	435
Levy Process	436
Branching Process	436
Markov Chain	436
Itô's Lemma	437
Markov Decision Processes and Reinforcement Learning	438
Examples of Reinforcement Learning	438
Reinforcement Learning as a Markov Decision Process	439
Reinforcement Learning in the Context of Optimal Control and Nonlinear Dynamics	441
Python Library for Reinforcement Learning	441
Theoretical and Rigorous Grounds	441
Which Events Have a Probability?	442
Can We Talk About a Wider Range of Random Variables?	443
A Probability Triple (Sample Space, Sigma Algebra, Probability Measure)	443

Where Is the Difficulty?	444
Random Variable, Expectation, and Integration	445
Distribution of a Random Variable and the Change of Variable Theorem	446
Next Steps in Rigorous Probability Theory	447
The Universality Theorem for Neural Networks	448
Summary and Looking Ahead	448
12. Mathematical Logic.....	451
Various Logic Frameworks	452
Propositional Logic	452
From Few Axioms to a Whole Theory	455
Codifying Logic Within an Agent	456
How Do Deterministic and Probabilistic Machine Learning Fit In?	456
First-Order Logic	457
Relationships Between For All and There Exist	458
Probabilistic Logic	460
Fuzzy Logic	460
Temporal Logic	461
Comparison with Human Natural Language	462
Machines and Complex Mathematical Reasoning	462
Summary and Looking Ahead	463
13. Artificial Intelligence and Partial Differential Equations.....	465
What Is a Partial Differential Equation?	466
Modeling with Differential Equations	467
Models at Different Scales	468
The Parameters of a PDE	468
Changing One Thing in a PDE Can Be a Big Deal	469
Can AI Step In?	471
Numerical Solutions Are Very Valuable	472
Continuous Functions Versus Discrete Functions	472
PDE Themes from My Ph.D. Thesis	474
Discretization and the Curse of Dimensionality	477
Finite Differences	478
Finite Elements	484
Variational or Energy Methods	489
Monte Carlo Methods	490
Some Statistical Mechanics: The Wonderful Master Equation	493
Solutions as Expectations of Underlying Random Processes	495
Transforming the PDE	495
Fourier Transform	495

Laplace Transform	498
Solution Operators	499
Example Using the Heat Equation	499
Example Using the Poisson Equation	501
Fixed Point Iteration	503
AI for PDEs	509
Deep Learning to Learn Physical Parameter Values	509
Deep Learning to Learn Meshes	510
Deep Learning to Approximate Solution Operators of PDEs	512
Numerical Solutions of High-Dimensional Differential Equations	519
Simulating Natural Phenomena Directly from Data	520
Hamilton-Jacobi-Bellman PDE for Dynamic Programming	522
PDEs for AI?	528
Other Considerations in Partial Differential Equations	528
Summary and Looking Ahead	530
14. Artificial Intelligence, Ethics, Mathematics, Law, and Policy.....	531
Good AI	533
Policy Matters	534
What Could Go Wrong?	536
From Math to Weapons	536
Chemical Warfare Agents	537
AI and Politics	538
Unintended Outcomes of Generative Models	539
How to Fix It?	539
Addressing Underrepresentation in Training Data	539
Addressing Bias in Word Vectors	540
Addressing Privacy	540
Addressing Fairness	541
Injecting Morality into AI	542
Democratization and Accessibility of AI to Nonexperts	543
Prioritizing High Quality Data	543
Distinguishing Bias from Discrimination	544
The Hype	545
Final Thoughts	546
Index.....	549

Preface

Why I Wrote This Book

AI is built on mathematical models. We need to know how.

I wrote this book in purely colloquial language, leaving most of the technical details out. It is a math book about AI with very few mathematical formulas and equations, no theorems, no proofs, and no coding. My goal is *to not keep this important knowledge in the hands of the very few elite, and to attract more people to technical fields*. I believe that many people get turned off by math before they ever get a chance to know that they might love it and be naturally good at it. This also happens in college or in graduate school, where many students switch their majors from math, or start a Ph.D. and never finish it. The reason is not that they do not have the ability, but that they saw no motivation or an end goal for learning torturous methods and techniques that did not seem to transfer to anything useful in their lives. It is like going to a strenuous mental gym every day only for the sake of going there. No one even wants to go to a real gym every day (this is a biased statement, but you get the point). In math, formalizing objects into functions, spaces, measure spaces, and entire mathematical fields comes *after* motivation, not *before*. Unfortunately, it gets taught in reverse, with formality first and then, if we are lucky, some motivation.

The most beautiful thing about math is that it has the expressive ability to connect seemingly disparate things together. A field as big and as consequential as AI not only builds on math, as that is a given; it also needs the binding ability that only math can provide in order to tell its big story concisely. In this book I will extract the math required for AI in a way that does not deviate at all from the real-life AI application in mind. It is infeasible to go through existing tools in detail and not fall into an encyclopedic and overwhelming treatment. What I do instead is try to teach you *how to think* about these tools and *view them from above*, as a means to an end that we can tweak and adjust when we need to. I hope that you will get out of this book a way of seeing how things relate to each other and why we develop or use certain methods

among others. In a way, this book provides a platform that launches you to whatever area you find interesting or want to specialize in.

Another goal of this book is to democratize mathematics, and to build more confidence to ask about how things work. Common answers such as “It’s complicated mathematics,” “It’s complicated technology,” or “It’s complex models,” are no longer satisfying, especially since the technologies that build on mathematical models currently affect every aspect of our lives. We do not need to be experts in every field in mathematics (no one is) in order to understand how things are built and why they operate the way they do. There is one thing about mathematical models that everyone needs to know: they *always* give an answer. They always spit out a number. A model that is vetted, validated, and backed with sound theory gives an answer. Also, a model that is *complete trash* gives an answer. Both compute mathematical functions. Saying that our decisions are based on mathematical models and algorithms does not make them sacred. What are the models built on? What are their assumptions? Limitations? Data they were trained on? Tested on? What variables did they take into account? And what did they leave out? Do they have a feedback loop for improvement, ground truths to compare to and improve on? Is there any theory backing them up? We need to be *transparent* with this information when the models are ours, and ask for it when the models are deciding our livelihoods for us.

The unorthodox organization of the topics in this book is intentional. I wanted to avoid getting stuck in math details before getting to the applicable stuff. My stand on this is that we do not ever need to dive into background material unless we happen to be personally practicing something, and that background material becomes an unfulfilled gap in our knowledge that is stopping us from making progress. Only then it is worth investing serious time to learn the intricate details of things. It is much more important to see how it all ties together and where everything fits. In other words, this book provides a map for how everything between math and AI interacts nicely together.

I also want to make a note to newcomers about the era of large data sets. Before working with large data, real or simulated, structured or unstructured, we might have taken computers and the internet for granted. If we came up with a model or needed to run analytics on small and curated data sets, we might have assumed that our machine’s hardware would handle the computations, or that the internet would just give more curated data when we needed it, or more information about similar models. The reality and limitations to access data, errors in the data, errors in the outputs of queries, hardware limitations, storage, data flow between devices, and vectorizing unstructured data such as natural language or images and movies hits us really hard. That is when we start getting into parallel computing, cloud computing, data management, databases, data structures, data architectures, and data engineering in order to understand the compute infrastructure that allows us to run our models. What kind of infrastructure do we have? How is it structured? How did it

evolve? Where is it headed? What is the architecture like, including the involved solid materials? How do these materials work? And what is all the fuss about quantum computing? We should not view the software as separate from the hardware, or our models separate from the infrastructure that allows us to simulate them. This book focuses only on the math, the AI models, and some data. There are neither exercises nor coding. In other words, we focus on the *soft*, the *intellectual*, and the *I do not need to touch anything* side of things. But we need to keep learning until we are able to comprehend the technology that powers many aspects of our lives as the one interconnected body that it actually is: hardware, software, sensors and measuring devices, data warehouses, connecting cables, wireless hubs, satellites, communication centers, physical and software security measures, and mathematical models.

Who Is This Book For?

I wrote this book for:

- A person who knows math but wants to get into AI, machine learning, and data science.
- A person who practices AI, data science, and machine learning but wants to brush up on their mathematical thinking and get up-to-date with the mathematical ideas behind the state-of-the-art models.
- Undergraduate or early graduate students in math, data science, computer science, operations research, science, engineering, or other domains who have an interest in AI.
- People in management positions who want to integrate AI and data analytics into their operations but want a deeper understanding of how the models that they might end up basing their decisions on actually work.
- Data analysts who are primarily doing business intelligence, and are now, like the rest of the world, driven into *AI-powered business intelligence*. They want to know what that actually means before adopting it into business decisions.
- People who care about the ethical challenges that AI might pose to the world and want to understand the inner workings of the models so that they can argue for or against certain issues such as autonomous weapons, targeted advertising, data management, etc.
- Educators who want to put together courses on math and AI.
- Any person who is curious about AI.

Who Is This Book Not For?

This book is not for a person who likes to sit down and do many exercises to master a particular mathematical technique or method, a person who likes to write and prove theorems, or a person who wants to learn coding and development. This is not a math textbook. There are many excellent textbooks that teach calculus, linear algebra, and probability (but few books relate this math to AI). That said, this book has many in-text pointers to the relevant books and scientific publications for readers who want to dive into technicalities, rigorous statements, and proofs. This is also not a coding book. The emphasis is on concepts, intuition, and general understanding, rather than on implementing and developing the technology.

How Will the Math Be Presented in This Book?

Writing a book is ultimately a decision-making process: how to organize the material of the subject matter in a way that is most insightful into the field, and how to choose what and what not to elaborate on. I will detail some math in a few places, and I will omit details in others. This is on purpose, as my goal is to not get distracted from telling the story of:

Which math do we use, why do we need it, and where exactly do we use it in AI?

I always define the AI context, with many applications. Then I talk about the related mathematics, sometimes with details and other times only with the general way of thinking. Whenever I skip details, I point out the relevant questions that we should be asking and how to go about finding answers. I showcase the math, the AI, and the models as one connected entity. I dive deeper into math only if it must be part of the foundation. Even then, I favor intuition over formality. The price I pay here is that on very few occasions, I might use some technical terms before defining them, secretly hoping that you might have encountered these terms before. In this sense, I adopt AI's *transformer* philosophy (see [Google Brain's 2017 article: "Attention Is All You Need"](#)) for natural language understanding: the model learns word meanings from their context. So when you encounter a technical term that I have not defined before, focus on the term's surrounding environment. Over the course of the section within which it appears, you will have a very good intuition about its meaning. The other option, of course, is to google it. Overall, I avoid jargon and I use zero acronyms.

Since this book lies at the intersection of math, data science, AI, machine learning, and philosophy, I wrote it expecting a diverse audience with drastically different skill sets and backgrounds. For this reason, depending on the topic, the same material might feel trivial to some but complicated to others. I hope I do not insult anyone's mind in the process. That said, this is a risk that I am willing to take, so that all readers will find useful things to learn from this book. For example, mathematicians

will learn the AI application, and data scientists and AI practitioners will learn deeper math.

The sections go in and out of technical difficulty, so if a section gets too confusing, make a mental note of its existence and skip to the next section. You can come back to what you skipped later.

Most of the chapters are independent, so readers can jump straight to their topics of interest. When chapters are related to other chapters, I point that out. Since I try to make each chapter as self-contained as possible, I may repeat a few explanations across different chapters. I push the probability chapter to all the way near the end ([Chapter 11](#)), but I use and talk about probability distributions all the time (especially the joint probability distribution of the features of a data set). The idea is to get used to the language of probability and how it relates to AI models before learning its grammar, so when we get to learning the grammar, we have a good idea of the context that it fits in.

I believe that there are two types of learners: those who learn the specifics and the details, then slowly start formulating a bigger picture and a map for how things fit together; and those who first need to understand the big picture and how things relate to each other, then dive into the details only when needed. Both are equally important, and the difference is only in someone's type of brain and natural inclination. I tend to fit more into the second category, and this book is a reflection of that: how does it all look from above, and how do math and AI interact with each other? The result might feel like a whirlwind of topics, but you'll come out on the other side with a great knowledge base for both math and AI, plus a healthy dose of confidence.

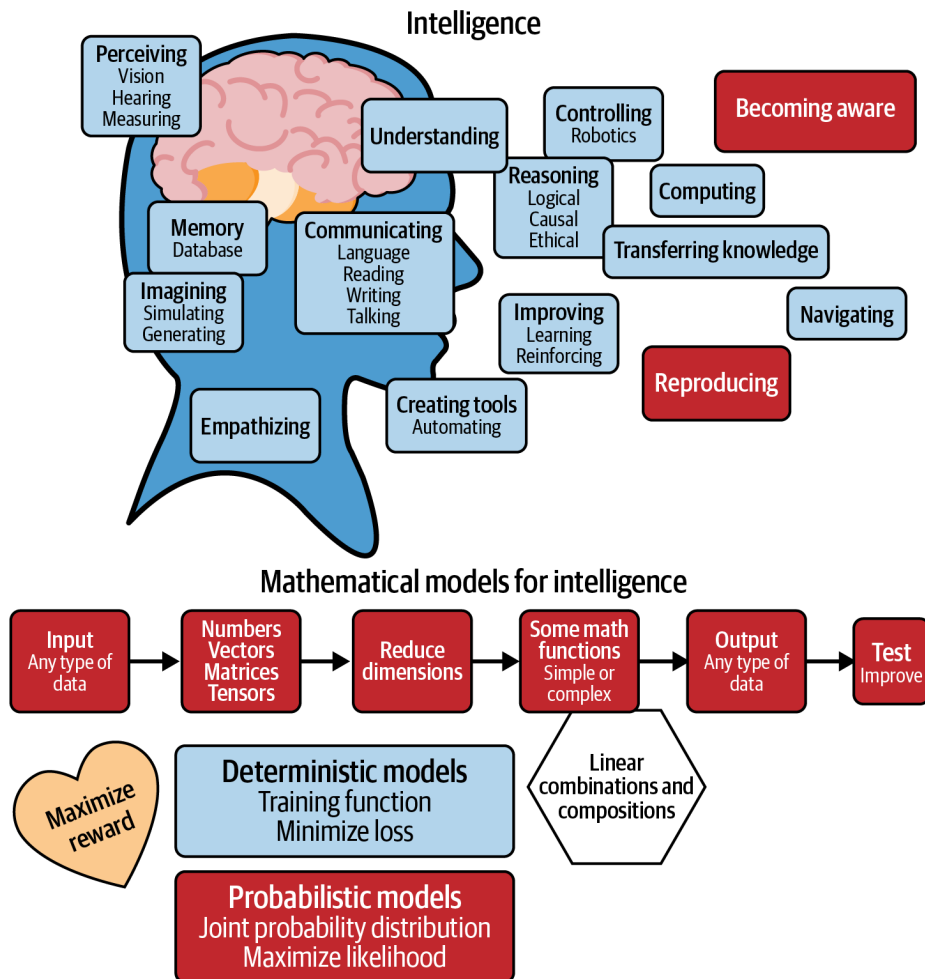
When my dad taught me to drive, he sat in the passenger's seat and asked me to drive. Ten minutes in, the road became a cliffside road. He asked me to stop, got out of the car, then said: "Now drive, just try not to fall off the cliff, don't be afraid, I am watching" (like that was going to help). I did not fall off the cliff, and in fact I love cliffside roads the most. Now tie this to training self-driving cars by reinforcement learning, with the distinction that the cost of falling off the cliff would've been minus infinity for me. I could not afford that; I am a real person in a real car, not a simulation.

This is how you'll do math and AI in this book. There are no introductions, conclusions, definitions, theorems, exercises, or anything of the like. There is immersion.

You are already in it. You know it. Now drive.

Infographic

I accompany this book with an infographic, visually tying all the topics together. You can also find this on the [book's GitHub page](#).



What Math Background Is Expected from You to Be Able to Read This Book?

This book is self-contained in the sense that we motivate everything that we need to use. I do hope that you have been exposed to calculus and some linear algebra, including vector and matrix operations, such as addition, multiplication, and some matrix decompositions. I also hope that you know what a function is and how it

maps an input to an output. Most of what we do mathematically in AI involves constructing a function, evaluating a function, optimizing a function, or composing a bunch of functions. You need to know about derivatives (these measure how fast things change) and the chain rule for derivatives. You do not necessarily need to know how to compute them for each function, as computers, Python, Desmos, and/or Wolfram|Alpha mathematics do a lot for us nowadays, but you need to know their meaning. Some exposure to probabilistic and statistical thinking are helpful as well. If you do not know any of the above, that is totally fine. You might have to sit down and do some examples (from some other books) on your own to familiarize yourself with certain concepts. The trick here is to know *when* to look up the things that you do not know...*only when you need them*, meaning only when you encounter a term that you do not understand, and you have a good idea of the context within which it appeared. If you are truly starting from scratch, you are not too far behind. This book tries to avoid technicalities at all costs.

Overview of the Chapters

We have a total of 14 chapters.

If you are a person who cares for math and the AI technology as they relate to ethics, policy, societal impact, and the various implications, opportunities, and challenges, then read Chapters 1 and 14 first. If you do not care for those, then we make the case that you should. In this book, we treat math as the binding agent of seemingly disparate topics, rather than the usual presentation of math as an oasis of complicated formulas, theorems, and Greek letters.

Chapter 13 might feel separate from the book if you've never encountered differential equations (ODEs and PDEs), but you will appreciate it if you are into mathematical modeling, the physical and natural sciences, simulation, or mathematical analysis, and you would like to know how AI can benefit your field, and in turn how differential equations can benefit AI. Countless scientific feats build on differential equations, so we cannot leave them out when we are at a dawn of a computational technology that has the potential to address many of the field's long-standing problems. This chapter is not essential for AI per se, but it is essential for our general understanding of mathematics as a whole, and for building theoretical foundations for AI and neural operators.

The rest of the chapters are essential for AI, machine learning, and data science. There is no optimal location for Chapter 6 on the singular value decomposition (the essential math for principal component analysis and latent semantic analysis, and a great method for dimension reduction). Let your natural curiosity dictate when you read this chapter: before or after whichever chapter you feel would be the most fitting. It all depends on your background and which industry or academic discipline you happen to come from.

Let's briefly overview Chapters 1 through 14:

Chapter 1, "Why Learn the Mathematics of AI?"

Artificial intelligence is here. It has already penetrated many areas of our lives, is involved in making important decisions, and soon will be employed in every sector of our society and operations. The technology is advancing very fast and its investments are skyrocketing. What is artificial intelligence? What is it able to do? What are its limitations? Where is it headed? And most importantly, how does it work, and why should we really care about knowing how it works? In this introductory chapter we briefly survey important AI applications, the problems usually encountered by companies trying to integrate AI into their systems, incidents that happen when systems are not well implemented, and the math typically used in AI solutions.

Chapter 2, "Data, Data, Data"

This chapter highlights the fact that data is central to AI. It explains the differences between concepts that are usually a source of confusion: structured and unstructured data, linear and nonlinear models, real and simulated data, deterministic functions and random variables, discrete and continuous distributions, prior probabilities, posterior probabilities, and likelihood functions. It also provides a map for the probability and statistics needed for AI without diving into any details, and introduces the most popular probability distributions.

Chapter 3, "Fitting Functions to Data"

At the core of many popular machine learning models, including the highly successful neural networks that brought artificial intelligence back into the popular spotlight since 2012, lies a very simple mathematical problem: fit a given set of data points into an appropriate function, then make sure this function performs well on new data. This chapter highlights this widely useful fact with a real data set and other simple examples. We discuss regression, logistic regression, support vector machines, and other popular machine learning techniques, with one unifying theme: training function, loss function, and optimization.

Chapter 4, "Optimization for Neural Networks"

Neural networks are modeled after the brain cortex, which involves millions of neurons arranged in a layered structure. The brain learns by reinforcing neuron connections when faced with a concept it has seen before, and weakening connections if it learns new information that undoes or contradicts previously learned concepts. Machines only understand numbers. Mathematically, stronger connections correspond to larger numbers (weights), and weaker connections correspond to smaller numbers. This chapter explains the optimization and backpropagation steps used when training neural networks, similar to how learning happens in our brain (not that humans fully understand this). It also walks through various regularization techniques, explaining their advantages,

disadvantages, and use cases. Furthermore, we explain the intuition behind approximation theory and the universal approximation theorem for neural networks.

Chapter 5, “Convolutional Neural Networks and Computer Vision”

Convolutional neural networks are widely popular for computer vision and natural language processing. In this chapter we start with the convolution and cross-correlation operations, then survey their uses in systems design and filtering signals and images. Then we integrate convolution with neural networks to extract higher-order features from images.

Chapter 6, “Singular Value Decomposition: Image Processing, Natural Language Processing, and Social Media”

Diagonal matrices behave like scalar numbers and hence are highly desirable. Singular value decomposition is a crucially important method from linear algebra that transforms a dense matrix into a diagonal matrix. In the process, it reveals the action of a matrix on space itself: rotating and/or reflecting, stretching and/or squeezing. We can apply this simple process to *any* matrix of numbers. This wide applicability, along with the ability to dramatically reduce the dimensions while retaining essential information, make singular value decomposition popular in the fields of data science, AI, and machine learning. It is the essential mathematics behind principal component analysis and latent semantic analysis. This chapter walks through singular value decomposition along with its most relevant and up-to-date applications.

Chapter 7, “Natural Language and Finance AI: Vectorization and Time Series”

We present the mathematics in this chapter in the context of natural language models, such as identifying topics, machine translation, and attention models. The main barrier to overcome is moving from words and sentences that carry meaning to low-dimensional vectors of numbers that a machine can process. We discuss state-of-the-art models such as Google Brain’s transformer (starting in 2017), among others, while we keep our attention only the relevant math. Time series data and models (such as recurrent neural networks) appear naturally here. We briefly introduce finance AI, as it overlaps with natural language both in terms of modeling and how the two fields feed into each other.

Chapter 8, “Probabilistic Generative Models”

Machine-generated images, including those of humans, are becoming increasingly realistic. It is very hard nowadays to tell whether an image of a model in the fashion industry is that of a real person or a computer-generated image. We have generative adversarial networks (GANs) and other generative models to thank for this progress, where it is harder to draw a line between the virtual and the real. Generative adversarial networks are designed to repeat a simple mathematical process using two neural networks until the machine itself cannot tell the

difference between a real image and a computer-generated one, hence the “very close to reality” success. Game theory and zero-sum games occur naturally here, as the two neural networks “compete” against each other. This chapter surveys generative models, which mimic imagination in the human mind. These models have a wide range of applications, from augmenting data sets to completing masked human faces to high energy physics, such as simulating data sets similar to those produced at the CERN Large Hadron Collider.

Chapter 9, “Graph Models”

Graphs and networks are everywhere: cities and roadmaps, airports and connecting flights, the World Wide Web, the cloud (in computing), molecular networks, our nervous system, social networks, terrorist organization networks, even various machine learning models and artificial neural networks. Data that has a natural graph structure can be better understood by a mechanism that exploits and preserves that structure, building functions that operate directly on graphs, as opposed to embedding graph data into existing machine learning models that attempt to artificially reshape the data before analyzing it. This is the same reason convolutional neural networks are successful with image data, recurrent neural networks are successful with sequential data, and so on. The mathematics behind graph neural networks is a marriage among graph theory, computing, and neural networks. This chapter surveys this mathematics in the context of many applications.

Chapter 10, “Operations Research”

Another suitable name for operations research would be *optimization for logistics*. This chapter introduces the reader to problems at the intersection of AI and operations research, such as supply chain, traveling salesman, scheduling and staffing, queuing, and other problems whose defining features are high dimensionality, complexity, and the need to balance competing interests and limited resources. The math required to address these problems draws from optimization, game theory, duality, graph theory, dynamic programming, and algorithms.

Chapter 11, “Probability”

Probability theory provides a systematic way to quantify randomness and uncertainty. It generalizes logic to situations that are of paramount importance in artificial intelligence: when information and knowledge are uncertain. This chapter highlights the essential probability used in AI applications: Bayesian networks and causal modeling, paradoxes, large random matrices, stochastic processes, Markov chains, and reinforcement learning. It ends with rigorous probability theory, which demystifies measure theory and introduces interested readers to the universal approximation theorem for neural networks.

Chapter 12, “Mathematical Logic”

This important topic is positioned near the end to not interrupt the book’s natural flow. Designing agents that are able to gather knowledge, reason logically about the environment within which they exist, and make inferences and good decisions based on this logical reasoning is at the heart of artificial intelligence. This chapter briefly surveys propositional logic, first-order logic, probabilistic logic, fuzzy logic, and temporal logic, within an intelligent knowledge-based agent.

Chapter 13, “Artificial Intelligence and Partial Differential Equations”

Differential equations model countless phenomena in the real world, from air turbulence to galaxies to the stock market to the behavior of materials and population growth. Realistic models are usually very hard to solve and require a tremendous amount of computational power when relying on traditional numerical techniques. AI has recently stepped in to accelerate solving differential equations. The first part of this chapter acts as a crash course on differential equations, highlighting the most important topics and arming the reader with a bird’s-eye view of the subject. The second part explores new AI-based methods that simplify the whole process of solving differential equations. These have the potential to unlock long-standing problems in the natural sciences, finance, and other fields.

Chapter 14, “Artificial Intelligence, Ethics, Mathematics, Law, and Policy”

I believe this chapter should be the first chapter in any book on artificial intelligence; however, this topic is so wide and deep that it needs multiple books to cover it completely. This chapter only scratches the surface and summarizes various ethical issues associated with artificial intelligence, including: equity, fairness, bias, inclusivity, transparency, policy, regulation, privacy, weaponization, and security. It presents each problem along with possible solutions (mathematical or with policy and regulation).

My Favorite Books on AI

There are many excellent and incredibly insightful books on AI and on topics intimately related to the field. The following is not even close to being an exhaustive list. Some are technical books heavy on mathematics, and others are either introductory or completely nontechnical. Some are code-oriented (Python 3) and others are not. I have learned a lot from all of them:

- Brunton, Steven L. and J. Nathan Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems and Control* (Cambridge University Press, 2022)
- Crawford, Kate, *Atlas of AI* (Yale University Press, 2021)

- Ford, Martin, *Architects of Intelligence* (Packt Publishing, 2018)
- Géron, Aurélien, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow* (O'Reilly, 2022)
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville, *Deep Learning* (MIT Press, 2016)
- Grus, Joel, *Data Science from Scratch* (O'Reilly, 2019)
- Hawkins, Jeff, *A Thousand Brains* (Basic Books, 2021)
- Izenman, Alan J., *Modern Multivariate Statistical Techniques* (Springer, 2013)
- Jones, Herbert, *Data Science: The Ultimate Guide to Data Analytics, Data Mining, Data Warehousing, Data Visualization, Regression Analysis, Database Querying, Big Data for Business and Machine Learning for Beginners* (Bravex Publications, 2020)
- Kleppmann, Martin, *Designing Data-Intensive Applications* (O'Reilly, 2017)
- Lakshmanan, Valliappa, Sara Robinson, and Michael Munn, *Machine Learning Design Patterns* (O'Reilly, 2020)
- Lane, Hobson, Hannes Hapke, and Cole Howard, *Natural Language Processing in Action* (Manning, 2019)
- Lee, Kai-Fu, *AI Superpowers* (Houghton Mifflin Harcourt, 2018)
- Macey, Tobias, ed., *97 Things Every Data Engineer Should Know* (O'Reilly, 2021)
- Marr, Bernard and Matt Ward, *Artificial Intelligence in Practice* (Wiley, 2019)
- Moroney, Laurence, *AI and Machine Learning for Coders* (O'Reilly, 2021)
- Mount, George, *Advancing into Analytics: From Excel to Python and R* (O'Reilly, 2021)
- Norvig, Peter and Stuart Russell, *Artificial Intelligence: A Modern Approach* (Pearson, 2021)
- Pearl, Judea, *The Book of Why* (Basic Books, 2020)
- Planche, Benjamin and Eliot Andres, *Hands-On Computer Vision with TensorFlow2* (Packt Publishing, 2019)
- Potters, Marc, and Jean-Philippe Bouchaud, *A First Course in Random Matrix Theory for Physicists, Engineers, and Data Scientists* (Cambridge University Press, 2020)
- Rosenthal, Jeffrey S., *A First Look at Rigorous Probability Theory* (World Scientific Publishing, 2016)
- Roshak, Michael, *Artificial Intelligence for IoT Cookbook* (Packt Publishing, 2021)
- Strang, Gilbert, *Linear Algebra and Learning from Data* (Wellesley Cambridge Press, 2019)

- Stone, James V., *Artificial Intelligence Engines* (Sebtel Press, 2020)
- Stone, James V., *Bayes' Rule, A Tutorial Introduction to Bayesian Analysis* (Sebtel Press, 2013)
- Stone, James V., *Information Theory: A Tutorial Introduction* (Sebtel Press, 2015)
- Vajjala, Sowmya et al., *Practical Natural Language Processing* (O'Reilly, 2020)
- Van der Hofstad, Remco, *Random Graphs and Complex Networks* (Cambridge, 2017)
- Vershynin, Roman, *High-Dimensional Probability: An Introduction with Applications in Data Science* (Cambridge University Press, 2018)

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Using Code Examples

The very few code examples that we have in this book are available for download at <https://github.com/halanelson/Essential-Math-For-AI>.

If you have a technical question or a problem using the code examples, please send an email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Essential Math for AI* by Hala Nelson (O'Reilly). Copyright 2023 Hala Nelson, 978-1-098-10763-5.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

O'REILLY® For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/essentialMathAI>.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>

Follow us on Twitter: <https://twitter.com/oreillymedia>

Watch us on YouTube: <https://www.youtube.com/oreillymedia>

Acknowledgments

My dad, Yousef Zein, who taught me math, and made sure to always remind me: *Don't think that the best thing we gave you in this life is land, or money. These come and go. Humans create money, buy assets, and create more money. What we did give you is a brain, a really good brain. That's your real asset, so go out and use it.* I love your brain, this book is for you, dad.

My mom, Samira Hamdan, who taught me both English and philosophy, and who gave up everything to make sure we were happy and successful. I wrote this book in English, not my native language, thanks to you, mom.

My daughter, Sary, who kept me alive during the most vulnerable times, and who is the joy of my life.

My husband, Keith, who gives me the love, passion, and stability that allow me to be myself, and to do so many things, some of them unwise, like writing a five-hundred-or-so-page book on math and AI. I love you.

My sister, Rasha, who is my soulmate. This says it all.

My brother, Haitham, who went against all our cultural norms and traditions to support me.

The memory of my uncle Omar Zein, who also taught me philosophy, and who made me fall in love with the mysteries of the human mind.

My friends Sharon and Jamie, who let me write massive portions of this book at their house, and were great editors any time I asked.

My lifetime friend Oren, who on top of being one of the best friends anyone can wish for, agreed to read and review this book.

My friend **Huan Nguyen**, whose story should be its own book, and who also took the time to read and review this book. Thank you, admiral.

My friend and colleague John Webb, who read every chapter word by word, and provided his invaluable pure math perspective.

My wonderful friends Deb, Pankaj, Jamie, Tamar, Sajida, Jamila, Jen, Mattias, and Karen, who are part of my family. I love life with you.

My mentors Robert Kohn (New York University) and John Schotland (Yale University), to whom I owe reaching many milestones in my career. I learned a great deal from you.

The memory of Peter, whose impact was monumental, and who will forever inspire me.

The reviewers of this book, who took time and care despite their busy schedules to make it much better. Thank you for your great expertise and for generously giving me your unique perspectives from all your different domains.

All the waiters and waitresses in many cities in the world, who tolerated me sitting at my laptop at their restaurants for hours and hours and hours, writing this book. I got so much energy and happiness from you.

My incredible, *patient*, cheerful, and always supportive editors, Angela Rufino and Kristen Brown.

Why Learn the Mathematics of AI?

It is not until someone said, “It is intelligent,” that I stopped searching, and paid attention.

—H.

Artificial intelligence, known as AI, is here. It has penetrated multiple aspects of our lives and is increasingly involved in making very important decisions. Soon it will be employed in every sector of our society, powering most of our daily operations. The technology is advancing very fast and its investments are skyrocketing. At the same time, it feels like we are in the middle of an AI frenzy. Every day we hear about a new AI accomplishment. AI beats the best human player at a Go game. AI outperforms human vision in classification tasks. AI makes deep fakes. AI generates high energy physics data. AI solves difficult partial differential equations that model the natural phenomena of the world. Self-driving cars are on the roads. Delivery drones are hovering in some parts of the world.

We also hear about AI’s seemingly unlimited potential. AI will revolutionize health-care and education. AI will eliminate global hunger. AI will fight climate change. AI will save endangered species. AI will battle disease. AI will optimize the supply chain. AI will unravel the origins of life. AI will map the observable universe. Our cities and homes will be smart. Eventually, we cross into science fiction territory. Humans will upload their brains into computers. Humans will be *enhanced* by AI. Finally, the voices of fear and skepticism emerge: AI will take over and destroy humanity.

Amid this frenzy, where the lines between reality, speculation, exaggeration, aspiration, and pure fiction are blurred, we must first define AI, at least within the context of this book. We will then discuss some of its limitations, where it is headed, and set the stage for the mathematics that is used in today’s AI. My hope is that when you understand the mathematics, you will be able to look at the subject from a relatively deep perspective, and the blurring lines between fiction, reality, and everything in between will become more clear. You will also learn the main ideas behind state-of-

the-art math in AI, arming you with the confidence needed to use, improve, or even create entirely new AI systems.

What Is AI?

I have yet to come across a unified definition of AI. If we ask two AI experts, we hear two different answers. Even if we ask the same expert on two different days, they might come up with two different definitions. The reason for this inconsistency and seeming inability to define AI is that until now it has not been clear what the definition of the *I* is. What is intelligence? What makes us human and unique? What makes us conscious of our own existence? How do neurons in our brain aggregate tiny electric impulses and translate them into images, sounds, feelings, and thoughts? These are vast topics that have fascinated philosophers, anthropologists, and neuroscientists for centuries. I will not attempt to go there in this book. I will, however, address artificial intelligence in terms of an AI agent and list the following defining principles for the purposes of this book. In 2022, an AI agent can be one or more of the following:

- An AI agent can be pure software or have a physical robotic body.
- An AI agent can be geared toward a specific task, or be a flexible agent exploring and manipulating its environment, building knowledge with or without a specific aim.
- An AI agent *learns* with experience, that is, it gets better at performing a task with more practice at that task.
- An AI agent perceives its environment, then builds, updates, and/or evolves a model for this environment.
- An AI agent perceives, models, analyzes, and makes decisions that lead to accomplishing its goal. This goal can be predefined and fixed, or variable and changing with more input.
- An AI agent understands cause and effect, and can tell the difference between patterns and causes.

Whenever a mathematical model for AI is inspired by the way our brain works, I will point out the analogy, hence keeping AI and human intelligence in comparison, without having to define either. Even though today's AI is nowhere close to human intelligence, except for specific tasks such as image classification, AlphaGo, etc., so many human brains have recently converged to develop AI that the field is bound to grow and have breakthroughs in the coming years.

It is also important to note that some people use the terms artificial intelligence, machine learning, and data science interchangeably. These three domains overlap but they are not the same. The fourth very important but slightly less hyped area is that

of robotics, where physical parts and motor skills must be integrated into the learning and reasoning processes, merging mechanical engineering, electrical engineering, and bioengineering with information and computer engineering. One fast way to think about the interconnectivity of these fields is: *data fuels machine learning algorithms that in turn power many popular AI and/or robotics systems*. The mathematics in this book is useful, in different proportions, for all four domains.

Why Is AI So Popular Now?

In the past decade, AI has sprung into worldwide attention due to the successful combination of following factors:

Generation and digitization of massive amounts of data

This may include text data, images, videos, health records, e-commerce, network, and sensor data. Social media and the Internet of Things have played a very significant role here with their continuous streaming of great volumes of data.

Advances in computational power

This occurs through parallel and distributed computing as well as innovations in hardware, allowing for efficient and relatively cheap processing of large volumes of complex structured and unstructured data.

Recent success of neural networks in making sense of big data

AI has surpassed human performance in certain tasks such as image recognition and the Go game. When **AlexNet** won the **ImageNet Large Scale Visual Recognition Challenge** in 2012, it spurred a myriad of activity in convolutional neural networks (supported by graphical processing units), and in 2015, **PRelu-Net (ResNet)** was the first to outperform humans in image classification.

When we examine these factors, we realize that today's AI is not the same as science fiction AI. Today's AI is centered around big data (all kinds of data), machine learning algorithms, and is heavily geared toward performing one task extremely well, as opposed to developing and adapting varied intelligence types and goals as a response to the surrounding environment.

What Is AI Able to Do?

There are many more areas and industries where AI can be successfully applied than there are AI experts who are well suited to respond to this ever-growing need. Humans have always strived for automating processes, and AI carries a great promise to do exactly that, at a massive scale. Large and small companies have volumes of raw data that they would like to analyze and turn into insights for profits, optimal strategies, and allocation of resources. The health industry suffers a severe shortage of doctors, and AI has innumerable applications and unlimited potential there.

Worldwide financial systems, stock markets, and banking industries have always depended heavily on our ability to make good predictions, and have suffered tremendously when those predictions failed. Scientific research has progressed significantly with our increasing ability to compute, and today we are at a new dawn where advances in AI enable computations at scales thought impossible a few decades ago.

Efficient systems and operations are needed everywhere, from the power grid, transportation, and the supply chain to forest and wildlife preservation, battling world hunger, disease, and climate change. Automation is even sought after in AI itself, where an AI system spontaneously decides on the optimal pipelines, algorithms, and parameters, readily producing the desired outcomes for given tasks, thus eliminating the need for human supervision altogether.

An AI Agent's Specific Tasks

In this book, as I work through the math, I will focus on popular application areas of AI, in the context of an AI agent's specified tasks. Nevertheless, the beneficial mathematical ideas and techniques are readily transferable across different application domains. The reason for this seeming easiness and wide applicability is that we happen to be at the age of AI *implementation*, in the sense that the main ideas for addressing certain tasks have already been developed, and with only a little tweaking, they can be *implemented* across various industries and domains. Our AI topics and/or tasks include:

Simulated and real data

Our AI agent processes data, provides insights, and makes decisions based on that data (using mathematics and algorithms).

The brain neocortex

Neural networks in AI are modeled after the neocortex, or the *new brain*. This is the part of our brain responsible for high functions such as perception, memory, abstract thought, language, voluntary physical action, decision making, imagination, and consciousness. The neocortex has many layers, six of which are mostly distinguishable. It is flexible and has a tremendous learning ability. The *old brain* and the *reptilian brain* lie below the neocortex, and are responsible for emotions and more basic and primitive survival functions such as breathing, regulating the heartbeat, fear, aggression, sexual urges, and others. The old brain keeps records of actions and experiences that lead to favorable or unfavorable feelings, creating our emotional memory that influences our behavior and future actions. Our AI agent, in a very basic way, emulates the neocortex and sometimes the old brain.

Computer vision

Our AI agent senses and recognizes its environment through cameras, sensors, etc. It peeks into everything, from our daily pictures and videos, to our MRI scans, and all the way into images of distant galaxies.

Natural language processing

Our AI agent communicates with its environment and automates tedious and time-consuming tasks such as text summarization, language translation, sentiment analysis, document classification and ranking, captioning images, and chatting with users.

Financial systems

Our AI agent detects fraud in our daily transactions, assesses loan risks, and provides 24-hour feedback and insights about our financial habits.

Networks and graphs

Our AI agent processes network and graph data, such as animal social networks, infrastructure networks, professional collaboration networks, economic networks, transportation networks, biological networks, and many others.

Social media

Our AI agent has social media to thank for providing the large amount of data necessary for its learning. In return, our AI agent attempts to characterize social media users, identifying their patterns, behaviors, and active networks.

The supply chain

Our AI agent is an optimizing expert. It helps us predict optimal resource needs and allocation strategies at each level of the production chain. It also finds ways to end world hunger.

Scheduling and staffing

Our AI agent facilitates our daily operations.

Weather forecasting

Our AI agent solves partial differential equations used in weather forecasting and prediction.

Climate change

Our AI agent attempts to fight climate change.

Education

Our AI agent delivers personalized learning experiences.

Ethics

Our AI agent strives to be fair, equitable, inclusive, transparent, unbiased, and protective of data security and privacy.

What Are AI's Limitations?

Along with the impressive accomplishments of AI and its great promise to enhance or revolutionize entire industries, there are some real limitations that the field needs to overcome. Some of the most pressing limitations are:

Intelligence

Current AI is not even remotely close to being *intelligent* in the sense that we humans consider ourselves uniquely intelligent. Even though AI has outperformed humans in innumerable tasks, it cannot naturally switch and adapt to new tasks. For example, an AI system trained to recognize humans in images cannot recognize cats without retraining, or generate text without changing its architecture and algorithms. In the context of the three types of AI, we have thus far only partially accomplished *artificial narrow intelligence*, which has a narrow range of abilities. We have accomplished neither *artificial general intelligence*, on par with human abilities, nor *artificial super intelligence*, which is more capable than humans'. Moreover, machines today are incapable of experiencing any of the beautiful human emotions, such as love, closeness, happiness, pride, dignity, caring, sadness, loss, and many others. Mimicking emotions is different than experiencing and genuinely providing them. In this sense, machines are nowhere close to replacing humans.

Large volumes of labeled data

Most popular AI applications need large volumes of labeled data, for example, MRI images can be labeled cancer or not-cancer, YouTube videos can be labeled safe for children or unsafe, or house prices can be available with the house district, number of bedrooms, median family income, and other features—in this case the house price is the label. The limitation is that the data required to train a system is usually not readily available, and not cheap to obtain, label, maintain, or warehouse. A substantial amount of data is confidential, unorganized, unstructured, biased, incomplete, and unlabeled. Obtaining the data, curating it, preprocessing it, and labeling it become major obstacles requiring large time and resource investments.

Multiple methods and hyperparameters

For a certain AI task, there are sometimes many methods, or algorithms, to accomplish it. Each task, data set, and/or algorithm has parameters, called hyperparameters, that can be tuned during implementation, and it is not always clear what the best values for these hyperparameters are. The variety of methods and hyperparameters available to tackle a specific AI task mean that different methods can produce extremely different results, and it is up to humans to assess which methods' decisions to rely on. In some applications, such as which dress styles to recommend for a certain customer, these discrepancies may be inconsequential. In other areas, AI-based decisions can be life-changing: a patient is told

they do not have a certain disease, while in fact they do; an inmate is mislabeled as highly likely to reoffend and gets their parole denied as a consequence; or a loan gets rejected for a qualified person. Research is ongoing on how to address these issues, and I will expand on them as we progress through the book.

Resource limitations

Human abilities and potential are limited to our brainpower, the capacity of our biological bodies, and the resources available on Earth and in the universe that we are able to manipulate. These are again limited by the power and capacity of our brains. AI systems are similarly limited by the computing power and hardware capability of the systems supporting the AI software. Recent studies have suggested that computation-intensive deep learning is approaching its computational limits, and new ideas are needed to improve algorithm and hardware efficiency, or discover entirely new methods. Progress in AI has heavily depended on large increases in computing power. This power, however, is not unlimited, is extremely costly for large systems processing massive data sets, and has a substantial carbon footprint that cannot be ignored, for example, the power required to run and cool down data warehouses, individual devices, keep the cloud connected, etc. Moreover, data and algorithmic software do not exist in the vacuum. Devices such as computers, phones, tablets, batteries, and the warehouses and systems needed to store, transfer, and process data and algorithms are made of real physical materials harvested from Earth. It took Earth millions of years to make some of these materials, and the type of infinite supply required to forever sustain these technologies is just not there.

Security costs

Security, privacy, and adversarial attacks remain a primary concern for AI, especially with the advent of interconnected systems. A lot of research and resources are being allocated to address these important issues. Since most of the current AI is software and most of the data is digital, the arms race in this area is never-ending. This means that AI systems need to be constantly monitored and updated, requiring more expensive-to-hire AI and cybersecurity specialists, probably at a cost that defeats the initial purpose of automation at scale.

Broader impacts

The AI research and implementation industries have thus far viewed themselves as slightly separate from the economical, social, and security consequences of their advancing technologies. Usually these ethical, social, and security implications of the AI work are acknowledged as important and needing to be attended to, but beyond the scope of the work itself. As AI becomes widely deployed and its impacts on the fabric and nature of society, markets, and potential threats are felt more strongly, the field as a whole has to become more intentional in the way it attends to these issues of paramount importance. In this sense, the AI development community has been limited in the resources it allocates to

addressing the broader impacts of the implementation and deployment of its new technologies.

What Happens When AI Systems Fail?

A very important part of learning about AI is learning about its incidents and failures. This helps us foresee and avoid similar outcomes when designing our own AI, *before* deploying out into the real world. If the AI fails after being deployed, the consequences can be extremely undesirable, dangerous, or even lethal.

One online repository for AI failures, called the [AI Incident Database](#), contains more than a thousand such incidents. Examples from this website include:

- A self-driving car kills a pedestrian.
- Self-driving cars lose contact with their company's server for a full 20 minutes and all stall at once in the streets of San Francisco (June 28 and May 18 of 2022).
- A trading algorithm causes a market *flash crash* where billions of dollars automatically transfer between parties.
- A facial recognition system causes an innocent person to be arrested.
- Microsoft's infamous chatbot Tay is shut down only 16 hours after its release, since it quickly learned and tweeted offensive, racist, and highly inflammatory remarks.

Such bad outcomes can be mitigated but require a deep understanding of how these systems work, at all levels of production, as well as of the environment and users they are deployed for. Understanding the mathematics behind AI is one crucial step in this discerning process.

Where Is AI Headed?

To be able to answer, or speculate on, where AI is headed, it is best to recall the field's original goal since its inception: mimic human intelligence. This field was conceived in the fifties. Examining its journey over the past seventy years *might* tell us something about its future direction. Moreover, studying the history of the field and its trends enables us to have a bird's-eye view of AI, putting everything in context and providing a better perspective. This also makes learning the mathematics involved in AI a less overwhelming experience. The following is a very brief and nontechnical overview of AI's evolution and its eventual thrust into the limelight thanks to the recent impressive progress of deep learning.

In the beginning, AI research attempted to mimic intelligence using rules and logic. The idea was that all we needed to do is feed machines facts and logical rules of reasoning about these facts (we will see examples of this logical structure in

Chapter 12). There was no emphasis on the learning process. The challenge here was that, in order to capture human knowledge, there are too many rules and constraints to be tractable for a coder, and the approach seemed unfeasible.

In the late 1990s and the early 2000s, various machine learning methods became popular. Instead of programming the rules, and making conclusions and decisions based on these preprogrammed rules, machine learning infers the rules from the data. The more data a machine learning system is able to handle and process, the better its performance. Data and the ability to process and *learn from* large amounts of data economically and efficiently became the main goals. Popular machine learning algorithms in that time period were support vector machines, Bayesian networks, evolutionary algorithms, decision trees, random forests, regression, logistic regression, and others. These algorithms are still popular now.

After 2010, and particularly in 2012, a tidal wave of neural networks and deep learning took over after the success of AlexNet's convolutional neural network in image recognition.

Most recently, in the last five years, reinforcement learning gained popularity after DeepMind's AlphaGo beat the world champion in the very complicated ancient Chinese game of Go.

Note that this glimpse of history is very rough: regression has been around since Legendre and Gauss in the very early 1800s, and the first artificial neurons and neural networks were formulated in the late 1940s and early 1950s with the works of neurophysiologist Warren McCulloch, mathematician Walter Pitts, and psychologists Donald Hebb and Frank Rosenblatt. The Turing Test, originally called the Imitation Game, was introduced in 1950 by Alan Turing, a computer scientist, cryptanalyst, mathematician, and theoretical biologist, in his paper “**Computing Machinery and Intelligence**”. Turing proposed that a machine possesses artificial intelligence if its responses are indistinguishable from those of a human. Thus, a machine is considered intelligent if it is able to imitate human responses. The Turing Test, however, for a person outside the field of computer science, sounds limiting in its definition of intelligence, and I wonder if the Turing Test might have inadvertently limited the goals or the direction of AI research.

Even though machines are able to mimic human intelligence in some specific tasks, the original goal of replicating human intelligence has not been accomplished yet, so it might be safe to assume that is where the field is headed, even though it could involve rediscovering old ideas or inventing entirely new ones. The current level of investment in the area, combined with the explosion in research and public interest, are bound to produce new breakthroughs. Nonetheless, breakthroughs brought about by recent AI advancements are already revolutionizing entire industries eager to implement these technologies. These contemporary AI advancements involve plenty of important mathematics that we will be exploring throughout this book.

Who Are the Current Main Contributors to the AI Field?

The main AI race has been between the United States, Europe, and China. Some of the world leaders in the technology industry have been Google and its parent company Alphabet, Amazon, Facebook, Microsoft, Nvidia, and IBM in the United States, DeepMind in the UK and the United States (owned by Alphabet), and Baidu and Tencent in China. There are major contributors from the academic world as well, but these are too many to enumerate. If you are new to the field, it is good to know the names of the big players, their histories and contributions, and the kinds of goals they are currently pursuing. It is also valuable to learn about the controversies, if any, surrounding their work. This general knowledge comes in handy as you navigate through and gain more experience in AI.

What Math Is Typically Involved in AI?

When I say the word “math,” what topics and subjects come to your mind?

Whether you are a math expert or a beginner, whatever math topic that you thought of to answer the question is most likely involved in AI. Here is a commonly used list of the most useful math subjects for AI implementation: calculus, linear algebra, optimization, probability, and statistics; however, you do not need to be an expert in all of these fields to succeed in AI. What you do need is a deep understanding of certain useful topics drawn from these math subjects. Depending on your specific application area, you might need special topics from: random matrix theory, graph theory, game theory, differential equations, and operations research.

In this book we will walk through these topics without presenting a textbook on each one. AI application and implementation are the unifying themes for these varied and intimately interacting mathematical subjects. Using this approach, I might offend some math experts by simplifying a lot of technical definitions or omitting whole theorems and delicate details, and I might as well offend AI or specialized industry experts, again omitting details involved in certain applications and implementations. The goal, however, is to keep the book simple and readable, while at the same time covering most of the math topics that are important for AI applications. Interested readers who want to dive deeper into the math or the AI field can then read more involved books on the particular area they want to focus on. My hope is that this book is a concise summary and a thorough overview, hence a reader can afterward branch out confidently to whatever AI math field or AI application area interests them.

Summary and Looking Ahead

Human intelligence reveals itself in perception, vision, communication through natural language, reasoning, decision making, collaboration, empathy, modeling and manipulating the surrounding environment, transfer of skills and knowledge across populations and generations, and generalization of innate and learned skills into new and uncharted domains. Artificial intelligence aspires to replicate all aspects of human intelligence. In its current state, AI addresses only one or few aspects of intelligence at a time. Even with this limitation, AI has been able to accomplish impressive feats, such as modeling protein folding and predicting protein structures, which are the building blocks of life. The implications of this one AI application (among many) for understanding the nature of life and battling all kinds of diseases are boundless.

When you enter the AI field, it is important to remain mindful of which aspect of intelligence you are developing or using. Is it perception? Vision? Natural language? Navigation? Control? Reasoning? Which mathematics to focus on and why then follow naturally, since you already know where in the AI field you are situated. It will then be easy to attend to the mathematical methods and tools used by the community developing that particular aspect of AI. The recipe in this book is similar: first the AI type and application, then the math.

In this chapter, we addressed general questions. What is AI? What is AI able to do? What are AI's limitations? Where is AI headed? How does AI work? We also briefly surveyed important AI applications, the problems usually encountered by companies trying to integrate AI into their systems, incidents that happen when systems are not well implemented, and the math subjects typically needed for AI implementations.

In the next chapter, we dive into data and affirm its intimate relationship to AI. When we talk data, we also talk data distributions, and that plunges us straight into probability theory and statistics.

Data, Data, Data

Maybe if I know where it all came from, and why, I would know where it's all headed, and why.

—H.

Data is the fuel that powers most AI systems. In this chapter, we will understand how data, and devising methods for extracting useful and actionable information from data, is at the heart of *perception AI*.

Perception AI is based on statistical learning from data, where an AI agent, or a machine, perceives data from its environment, then detects patterns within this data, allowing it to draw conclusions and/or make decisions.

Perception AI is different from the three other types of AI:

Understanding AI

Where an AI system understands that the image it classified as a chair serves the function of sitting, the image it classified as cancer means that the person is sick and needs further medical attention, or the textbook it read about linear algebra can be used to extract useful information from data.

Control AI

This has to do with controlling the physical parts of the AI agent in order to navigate spaces, open doors, serve coffee, etc. Robotics have made significant progress in this area. We need to augment robots with “brains” that include perception AI and understanding AI, and connect those to the control AI. Ideally, like humans, control AI then learns from its physical interactions with its environment by passing that information to its perception and understanding systems, which in turn pass control commands to the agent’s control systems.

Awareness AI

Where an AI agent has an inner experience similar to the human experience. Since we do not know yet how to mathematically define awareness, we do not visit this concept at all in this book.

Ideally, true human-like intelligence combines all four aspects: perception, understanding, control, and awareness. The main focus of this chapter and the next few chapters is perception AI. Recall that AI and data have become intertwined to the extent that it is now common, though erroneous, to use the terms data science and AI synonymously.

Data for AI

At the core of many popular machine learning models, including the highly successful neural networks that brought artificial intelligence back into the popular spotlight with AlexNet in 2012, lies a very simple mathematical problem:

Fit a given set of data points into an appropriate function (mapping an input to an output) that picks up on the important signals in the data and ignores the noise, then make sure this function performs well on new data.

Complexity and challenges, however, arise from various sources:

Hypothesis and features

Neither the true function that generated the data nor all the features it actually depends on are known. We simply observe the data, then try to estimate a hypothetical function that generated it. Our function tries to learn which features of the data are important for our predictions, classifications, decisions, or general purposes. It also learns how these features interact in order to produce the observed results. One of the great potentials of AI in this context is its ability to pick up on subtle interactions between features of data that humans do not usually pick up on, since we are very good at observing strong features but may miss more subtle ones. For example, we as humans can tell that a person's monthly income affects their ability to pay back a loan, but we might not observe that their daily commute, or morning routine, may have a nontrivial effect on that as well. Some feature interactions are much simpler than others, such as linear interactions. Others are more complex and are nonlinear. From a mathematical point of view, whether our feature interactions are simple (linear) or complex (nonlinear), we still have the same goal: find the hypothetical function that fits your data and is able to make good predictions on new data. One extra complication arises here: there are many hypothetical functions that can *fit* the same data set—how do we know which ones to choose?

Performance

Even after computing a hypothetical function that fits our data, how do we know whether it will perform well on new and unseen data? How do we know which performance measure to choose, and how to monitor this performance after deploying into the real world? Real-world data and scenarios do not come to us all labeled with ground truths, so we cannot easily measure whether our AI system is doing well and making correct or appropriate predictions and decisions. We do not know what to measure the AI system's results against. If real-world data and scenarios were labeled with ground truths, then we would all be out of business since we would know what to do in every situation, there would be peace on Earth, and we would live happily ever after (not really, I wish it was that simple).

Volume

Almost everything in the AI field is very high-dimensional! The number of data instances, observed features, and unknown weights to be computed could be in the millions, and the required computation steps in the billions. Efficient storage, transport, exploration, preprocessing, structuring, and computation on such volumes of data become center goals. In addition, exploring the landscapes of the involved high-dimensional mathematical functions is a nontrivial endeavor.

Structure

Most of the data created by the modern world is unstructured. It is not organized in easy-to-query tables that contain labeled fields such as names, phone numbers, genders, ages, zip codes, house prices, income levels, etc. Unstructured data is everywhere: posts on social media, user activity, word documents, PDF files, images, audio and video files, collaboration software data, traffic or seismic or weather data, GPS, military movement, emails, instant messenger, mobile chat data, and many others. Some of these examples, such as email data, can be considered semistructured, since emails come with headings that include the email's metadata: From, To, Date, Time, Subject, Content-Type, Spam Status, etc. Moreover, large volumes of important data are not available in digital format and are fragmented over multiple and noncommunicating databases. Examples here include historical military data, museum archives, and hospital records. Presently, there is great momentum toward digitalizing our world and our cities to leverage more AI applications. Overall, it is easier to draw insights from structured and labeled data than from unstructured data. Mining unstructured data requires innovative techniques that are currently driving forces in the fields of data science, machine learning, and artificial intelligence.

Real Data Versus Simulated Data

When we work with data, it is very important to know the difference between real data and simulated data. Both types of data are extremely valuable for human discovery and progress.

Real data

This data is collected through real-world observations, using measuring devices, sensors, surveys, structured forms like medical questionnaires, telescopes, imaging devices, websites, stock markets, controlled experiments, etc. This data is often imperfect and noisy due to inaccuracies and failures in measuring methods and instruments. Mathematically, *we do not know the exact function or probability distribution that generated the real data*, but we can hypothesize about it using models, theories, and simulations. We can then test our models, and finally use them to make predictions.

Simulated data

This is data generated using a *known* function or randomly sampled from a *known* probability distribution. Here, we have our known mathematical function(s), or *model*, and we plug numerical values into the model to generate our data points. Examples are plentiful: numerical solutions of partial differential equations modeling all kinds of natural phenomena on all kinds of scales, such as turbulent flows, protein folding, heat diffusion, chemical reactions, planetary motion, fractured materials, traffic, and even and even enhancing Disney movie animations, such as simulating natural water movement in *Moana* or Elsa's hair movement in *Frozen*.

In this chapter, we present two examples about human height and weight data to demonstrate the difference between real and simulated data. In the first example, we visit an online public database, then download and explore two real data sets containing measurements of the heights and weights of real individuals. In the second example, we simulate our own data set of heights and weights *based on a function that we hypothesize*: we assume that the weight of an individual depends linearly on their height. This means that when we plot the weight data against the height data, we expect to see a straight, or flat, visual pattern.

Mathematical Models: Linear Versus Nonlinear

Linear dependencies model flatness in the world, like one-dimensional straight lines, two-dimensional flat surfaces (called *planes*), and higher-dimensional hyperplanes. The graph of a linear function, which models a linear dependency, is forever flat and does not bend. Every time you see a flat object, like a table, a rod, a ceiling, or a bunch of data points huddled together around a straight line or a flat surface, know that their representative function is linear. Anything that isn't flat is nonlinear, so

functions whose graphs bend are nonlinear, and data points that congregate around bending curves or surfaces are generated by nonlinear functions.

The formula for a linear function, representing a linear dependency of the function output on the *features*, or *variables*, is very easy to write down. The features appear in the formula as just themselves, with no powers or roots, and are not embedded in any other functions, such as denominators of fractions, sine, cosine, exponential, logarithmic, or other calculus functions. They can only be multiplied by *scalars* (real or complex numbers, not vectors or matrices), and added to or subtracted from each other. For example, a function that depends linearly on three features x_1, x_2 , and x_3 can be written as:

$$f(x_1, x_2, x_3) = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3$$

where the parameters $\omega_0, \omega_1, \omega_2$, and ω_3 are scalar numbers. The parameters or *weights* ω_1, ω_2 , and ω_3 linearly combine the features, and produce the outcome of $f(x_1, x_2, x_3)$ after adding the bias term ω_0 . In other words, the outcome is produced as a result of linear interactions between the features x_1, x_2 , and x_3 , plus bias.

The formula for a nonlinear function, representing a nonlinear dependency of the function output on the features, is very easy to spot as well. One or more features appear in the function formula with a power other than one, or multiplied or divided by other features, or embedded in some other calculus functions, such as sines, cosines, exponentials, logarithms, etc. The following are three examples of functions depending nonlinearly on three features x_1, x_2 , and x_3 :

$$f(x_1, x_2, x_3) = \omega_0 + \omega_1 \sqrt{x_1} + \omega_2 \frac{x_2}{x_3}$$

$$f(x_1, x_2, x_3) = \omega_0 + \omega_1 x_1^2 + \omega_2 x_2^2 + \omega_3 x_3^2$$

$$f(x_1, x_2, x_3) = \omega_1 e^{x_1} + \omega_2 e^{x_2} + \omega_3 \cos(x_3)$$

As you can tell, we can come up with all kinds of nonlinear functions, and the possibilities related to what we can do and how much of the world we can model using nonlinear interactions are limitless. In fact, neural networks are successful because of their ability to pick up on the relevant *nonlinear* interactions between the data features.

We will use the previous notation and terminology throughout the book, so you will become very familiar with terms like linear combination, weights, features, and linear and nonlinear interactions between features.

An Example of Real Data

You can find the Python code to investigate the data and produce the figures in the following two examples at the [book's GitHub page](#).



Structured Data

The two data sets for height, weight, and gender that we will work with here are examples of *structured* data sets. They come organized in rows and columns. Columns contain the features, such as weight, height, gender, health index, etc. Rows contain the feature scores for each data instance, in this case, each person. On the other hand, data sets that are a bunch of audio files, Facebook posts, images, or videos are all examples of unstructured data sets.

I downloaded two data sets from the [Kaggle website](#) for data scientists. Both data sets contain height, weight, and gender information for a certain number of individuals. My goal is to learn how the weight of a person depends on their height. Mathematically, I want to write a formula for the weight as a function of one feature, the height:

$$weight = f(height)$$

so that if I am given the height of a *new* person, I would be able to *predict* their weight. Of course, there are other features than height that a person's weight depends on, such as their gender, eating habits, workout habits, genetic predisposition, etc. However, for the data sets that I downloaded, we only have height, weight, and gender data available. Unless we want to look for more detailed data sets, or go out and collect new data, we have to work with what we have. Moreover, the goal of this example is only to illustrate the difference between real data and simulated data. We can work with more involved data sets with larger numbers of features when we have more involved goals.

For the [first data set](#), I plot the weight column against the height column in [Figure 2-1](#), and obtain something that seems to have no pattern at all!

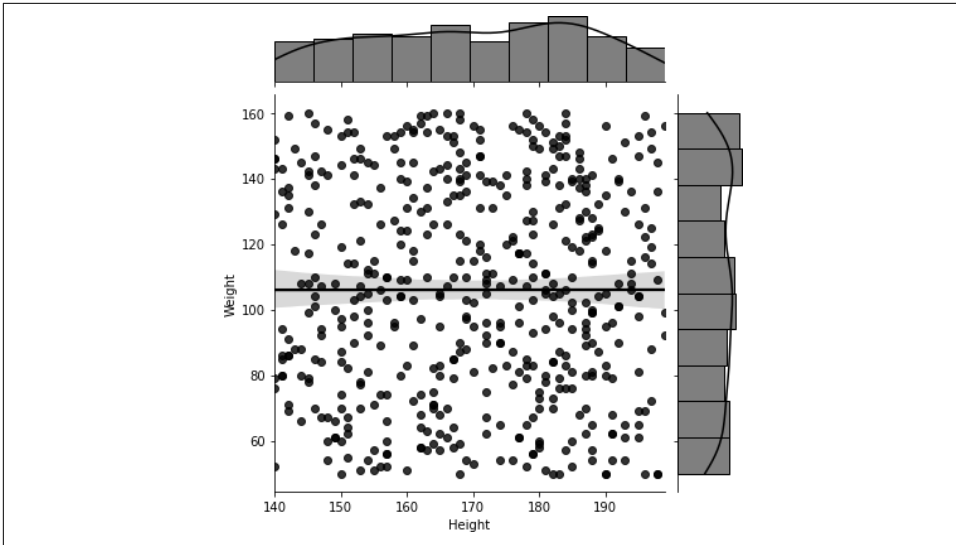


Figure 2-1. When plotting the weight against the height for *the first data set*, we cannot detect a pattern. The graphs on the top and on the right of the scatterplot show the respective histograms and empirical distributions of the height and weight data.

For the *second data set*, I do the same, and I can visually observe an obvious linear dependency in Figure 2-2. The data points seem to congregate around a straight line!

So what is going on? Why does my first real data set reflect no dependency between the height and weight of a person whatsoever, but my second one reflects a linear dependency? We need to look deeper into the data.

This is one of the many challenges of working with real data. We do not know what function generated the data, and why it looks the way it looks. We investigate, gain insights, detect patterns, if any, and we propose a hypothesis function. Then we test our hypothesis, and if it performs well based on our measures of performance, which have to be thoughtfully crafted, we deploy it into the real world. We make predictions using our deployed model, until new data tells us that our hypothesis is no longer valid, in which case we investigate the updated data and formulate a new hypothesis. This process and feedback loop keeps going for as long as our models are in business.

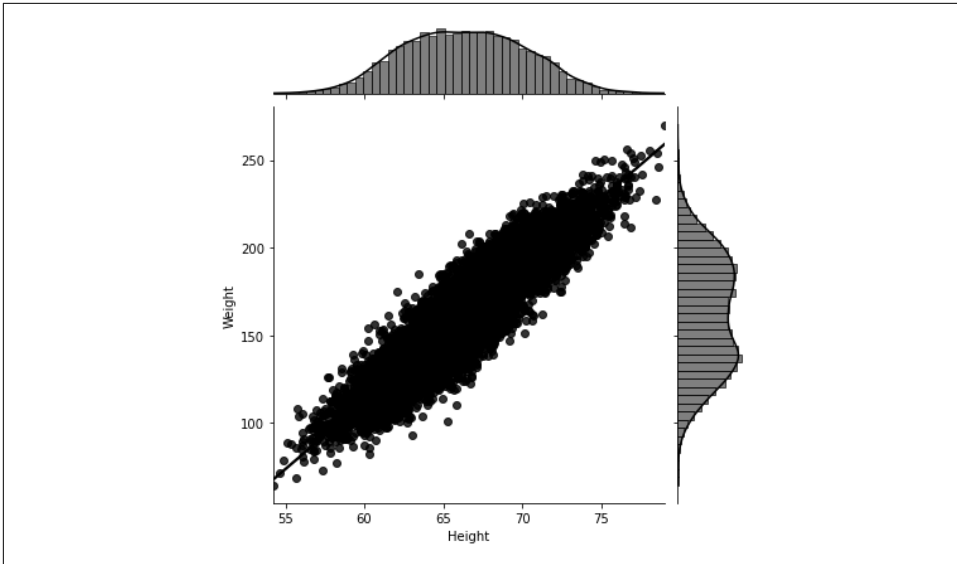


Figure 2-2. When plotting the weight against the height for *the second data set*, we observe a linear pattern. Note that the empirical distribution of the weight data is plotted on the righthand side of the figure, and the empirical distribution of the height data is plotted at the top of the figure. Both seem to have two peaks (bimodal), suggesting the existence of a mixture distribution. In fact, both the height and the weight data sets can be modeled using a mixture of two normal distributions, called Gaussian mixtures, representing mixing the underlying distributions for the male and female data. So if we plot the data for either the female or male subpopulations alone, as in *Figure 2-6*, we observe that the height and weight data are normally distributed (bell-shaped).

Before moving on to simulated data, let's explain why the first data set seemed to have no insight at all about the relationship between the height and the weight of an individual. Upon further inspection, we notice that the data set has an overrepresentation of individuals with Index scores 4 and 5, referring to obesity and extreme obesity. So, I decided to split the data by Index score, and plot the weight against the height for all individuals with similar Index scores. This time around, a linear dependency between the height and the weight is evident in *Figure 2-3*, and the mystery is resolved. This might feel like we are cheating our way to linearity, by conditioning on individuals' Index scores. All is fair game in the name of data exploration.

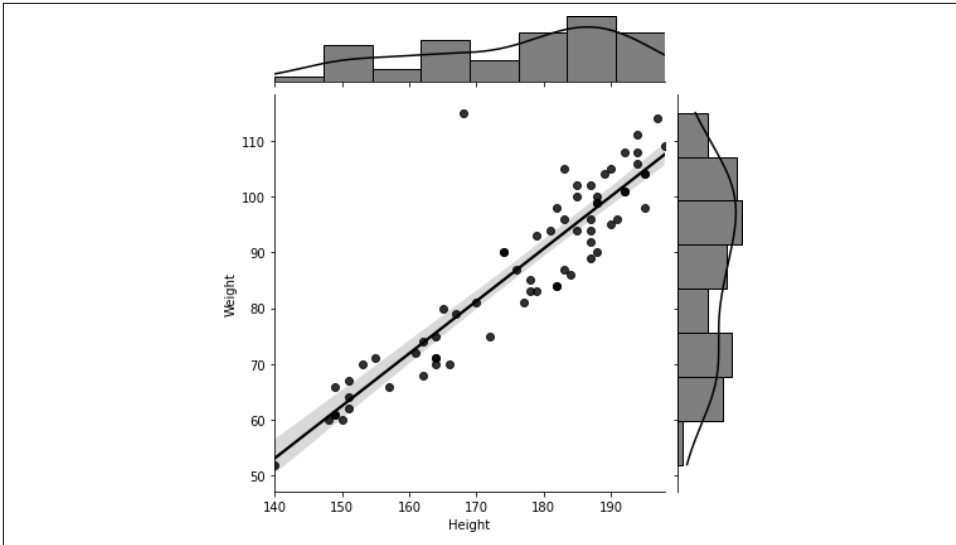


Figure 2-3. When plotting the weight against the height for individuals with similar Index scores in the *first data set*, we observe a linear pattern. This figure shows the weight against the height for individuals with Index score 3.

Now we can safely go ahead and hypothesize that the weight depends linearly on the height:

$$weight = \omega_0 + \omega_1 \times height$$

Of course, we are left with the task of finding appropriate values for the parameters ω_0 and ω_1 . [Chapter 3](#) teaches us how to do exactly that. In fact, the bulk of activity in machine learning, including deep learning, is about *learning* these ω 's *from the data*. In our very simple example, we only have two ω 's to learn, since we only had one feature, the height, and we *assumed* linear dependency after observing a linear pattern in the real data. In the next few chapters, we will encounter some deep learning networks with millions of ω 's to learn, yet we will see that the mathematical structure of the problem is in fact the same exact structure that we will learn in [Chapter 3](#).

An Example of Simulated Data

In this example, I simulate my own height-weight data set. Simulating our own data circumvents the trouble of searching for data from the web, the real world, or even building a lab to obtain controlled measurements. This is incredibly valuable when the required data is not available or very expensive to obtain. It also helps test different scenarios by only changing numbers in a function, as opposed to, say,

creating new materials or building labs and running new experiments. Simulating data is so convenient because all we need is a mathematical function, a probability distribution if we want to involve randomness and/or noise, and a computer.

Let's again assume linear dependency between the height and the weight, so the function that we will use is:

$$weight = \omega_0 + \omega_1 \times height$$

For us to be able to simulate numerical $(height, weight)$ pairs, or data points, we must assume numerical values for the parameters ω_0 and ω_1 . Without having insights from real data about the correct choices for these ω 's, we are left with making educated guesses from the context of the problem and experimenting with different values. Note that for the height-weight case in this example, we happen to have real data that we can use to learn appropriate values for the ω 's, and one of the goals of [Chapter 3](#) is to learn how to do that. However, in many other scenarios, we do not have real data, so the only way to go is experimenting with various numerical values for these ω 's.

In the following simulations, we set $\omega_0 = -314.5$ and $\omega_1 = 7.07$, so the function becomes:

$$weight = -314.5 + 7.07 \times height$$

Now we can generate as many numerical $(height, weight)$ pairs as we want. For example, plugging $height = 60$ in the formula for the weight function, we get $weight = -314.5 + 7.07 \times 60 = 109.7$. So our linear model *predicts* that a person whose height is 60 inches weighs 109.7 pounds, and the data point that we can plot on the height-weight graph has coordinates $(60, 109.7)$. In [Figure 2-4](#), we generate 5,000 of these data points: we choose 5,000 values for the height between 54 and 79 inches and plug them into the weight function. We notice that the graph in [Figure 2-4](#) is a perfect straight line, with no noise or variation in the simulated data, since we did not incorporate those into our linear model.

That's a hallmark of simulated data: it does what the function(s) it was generated from does. If we understand the function (called *model*) that we used to build our simulation, and if our computation does not accumulate too many numerical errors and/or very large numbers that go rogue, then we understand the data that our model generates, and we can use this data in any way we see fit. There isn't much space for surprises. In our example, our proposed function is linear, so its equation is that of a straight line, and as you see in [Figure 2-4](#), the generated data lies perfectly on this straight line.

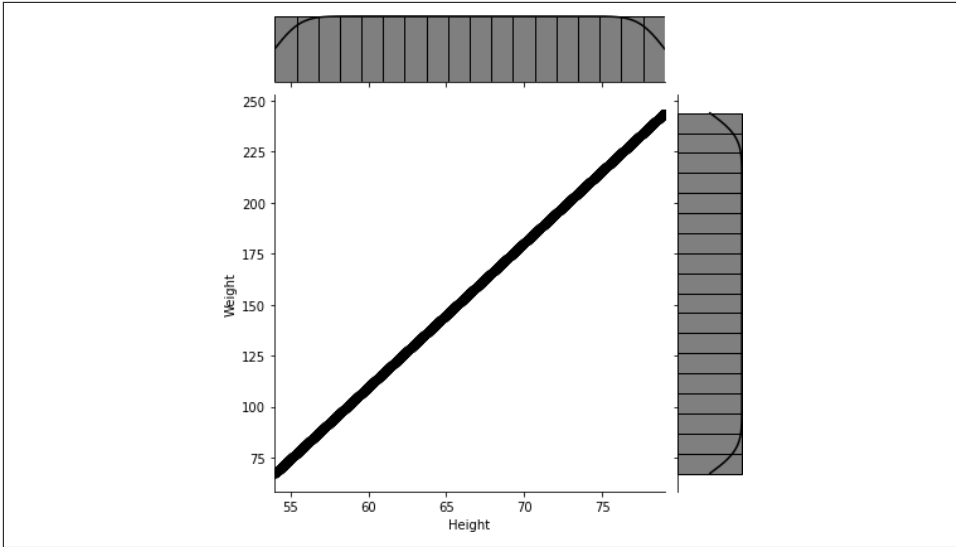


Figure 2-4. Simulated data: we generated 5,000 (height, weight) points using the linear function $weight = -314.5 + 7.07 \times height$

What if we want to simulate more realistic data for height and weight? Then we can sample the height values from a more realistic distribution for the heights of a human population: the bell-shaped normal distribution! Again, we *know* the probability distribution that we are sampling from, which is different from the case for real data. After we sample the height values, we plug those into the linear model for the weight, then we add some noise, since we want our simulated data to be realistic. Since noise has a random nature, we must also pick the probability distribution it will be sampled from. We again choose the bell-shaped normal distribution, but we could have chosen the uniform distribution to model uniform random fluctuations. Our more realistic height-weight model becomes:

$$weight = -314.5 + 7.07 \times height + noise$$

We obtain the results seen in [Figure 2-5](#).

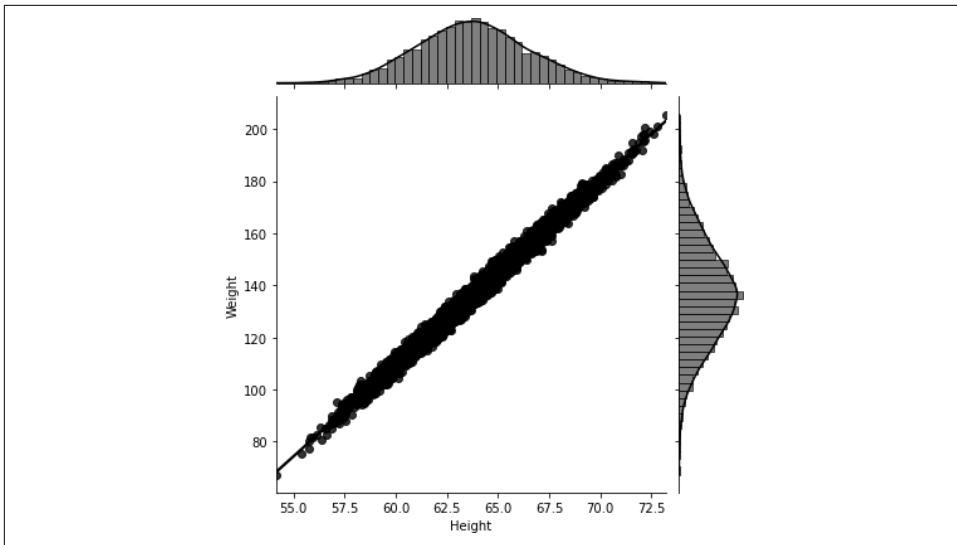


Figure 2-5. Simulated data: we generated 5,000 (height, weight) points using the linear function $weight = -314.5 + 7.07 \times height$. The height points are normally distributed and we added normally distributed noise as well. Note that the distributions of the weight and height data at the righthand side and at the top of the figure, respectively, are normally distributed. This is not surprising since we designed them that way in our simulation.

Now compare [Figure 2-5](#) containing our simulated height-weight data to [Figure 2-6](#) containing real height-weight data of 5,000 females from [the second data set](#) that we used. Not too bad, given that it only took five minutes of code writing to generate this data, as opposed to collecting real data! Had we spent more time tweaking the values of our ω 's, and the parameters for the normally distributed noise that we added (mean and standard deviation), we would've obtained an even better-looking simulated data set. However, we will leave this simulation here since very soon our whole focus will be on *learning* the appropriate parameter values for our hypothesized models.

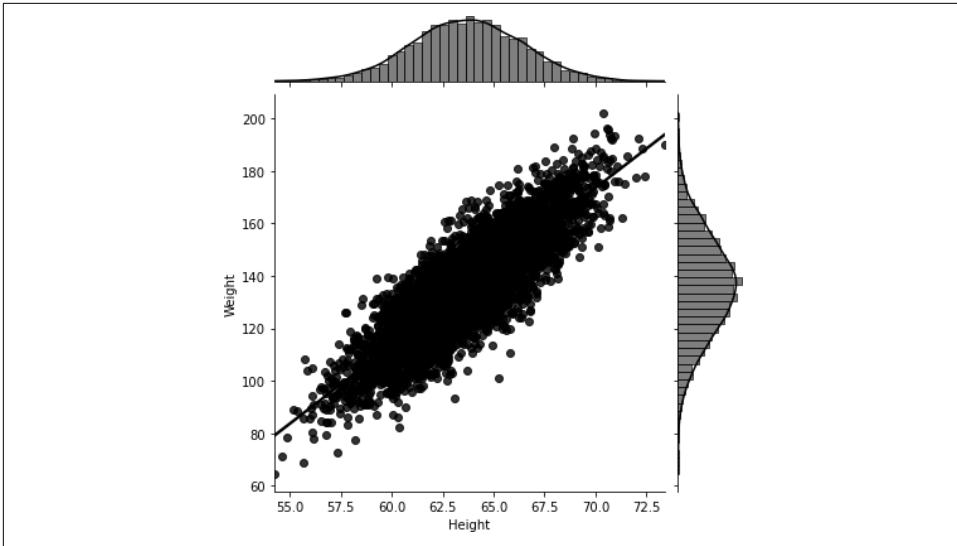


Figure 2-6. Real data: weight data plotted against the height data of the 5,000 females in *the second data set*. Note that the distributions of the female weight and height data at the righthand side and at the top of the figure, respectively, are normally distributed. For more details, see the *book's GitHub page* for more details.

Mathematical Models: Simulations and AI

We can always adjust our mathematical models to make them more realistic. We are the designers, so we get to decide what goes into these models. It is often the case that the more a model mimics nature, the more mathematical objects get incorporated within it. Therefore, while building a mathematical model, the usual trade-off is between getting closer to reality, and the model's simplicity and accessibility for mathematical analysis and computation. Different designers come up with different mathematical models, and some capture certain phenomena better than others. These models keep improving and evolving as the quest to capture natural behaviors continues. Thankfully, our computational capabilities have dramatically improved in the past decades, enabling us to create and test more involved and realistic mathematical models.

Nature is at the same time very finely detailed and enormously vast. Interactions in nature range from the subatomic quantum realm all the way to the intergalactic scale. We, as humans, are forever trying to understand nature and capture its intricate components with their numerous interconnections and interplays. Our reasons for this are varied. They include pure curiosity about the origins of life and the universe, creating new technologies, enhancing communication systems, designing drugs and discovering cures for diseases, building weapons and defense

systems, and traveling to distant planets and perhaps inhabiting them in the future. Mathematical models provide an excellent and almost miraculous way to describe nature with all its details using only numbers, functions, equations, and invoking quantified randomness through probability when faced with uncertainty. Computer simulations of these mathematical models enable us to investigate and visualize various simple and complex behaviors of the modeled systems or phenomena. In turn, insights from computer simulations aid in model enhancement and design, in addition to supplying deeper mathematical insights. This incredibly positive feedback cycle makes mathematical modeling and simulations an indispensable tool that is enhanced greatly with our increased computational power.

It is a mystery of the universe that its various phenomena can be accurately modeled using the abstract language of mathematics, and it is a marvel of the human mind that it can discover and comprehend mathematics, and build powerful technological devices that are useful for all kinds of applications. Equally impressive is that these devices, at their core, are doing nothing but computing or transmitting mathematics, more specifically, a bunch of zeros and ones.

The fact that humans are able to generalize their understanding of simple numbers all the way to building and applying mathematical models for natural phenomena at all kinds of scales is a spectacular example of *generalization* of learned knowledge, and is a hallmark of human intelligence. In the AI field, a common goal for both general AI (human-like and super-AI) and narrow AI (specific task oriented) is *generalization*: the ability of an AI agent to *generalize* learned abilities to new and unknown situations. In [Chapter 3](#), we will understand this principle for narrow and task-oriented AI: *an AI agent learns from data, then produces good predictions for new and unseen data*.

AI interacts in three ways with mathematical models and simulations:

Mathematical models and simulations create data for AI systems to train on.

Self-driving cars are considered by some to be a benchmark for AI. It will be inconvenient to let intelligent car prototypes drive off cliffs, hit pedestrians, or crash into new work zones before the car's AI system learns that these are unfavorable events that must be avoided. Training on simulated data is especially valuable here, as simulations can create all kinds of hazardous virtual situations for a car to train on before releasing it out on the roads. Similarly, simulated data is tremendously helpful for training AI systems for rovers on Mars, drug discovery, materials design, weather forecasting, aviation, military training, and so on.

AI enhances existing mathematical models and simulations.

AI has great potential to assist in areas that have traditionally been difficult and limiting for mathematical models and simulations, such as learning appropriate values for the parameters involved in the models, appropriate probability

distributions, mesh shapes and sizes when discretizing equations (fine meshes capture fine details and delicate behaviors at various spatial and time scales), and scaling computational methods to longer times or to larger domains with complicated shapes. Fields like navigation, aviation, finance, materials science, fluid dynamics, operations research, molecular and nuclear sciences, atmospheric and ocean sciences, astrophysics, physical and cyber security, and many others rely heavily on mathematical modeling and simulations. Integrating AI capabilities into these domains is starting to take place with very positive outcomes. We will come across examples of AI enhancing simulations in later chapters of this book.

AI itself is a mathematical model and simulation.

One of the big aspirations of AI is to *computationally* replicate human intelligence. Successful machine learning systems, including neural networks with all their architectures and variations, are mathematical models aimed at simulating tasks that humans associate with intelligence, such as vision, pattern recognition and generalization, communication through natural language, and logical reasoning. Understanding, emotional experience, empathy, and collaboration are also associated with intelligence and have contributed tremendously to the success and domination of humankind, so we must also find ways to replicate them if we want to achieve general AI while gaining a deeper understanding of the nature of intelligence and the workings of the human brain. Efforts in these areas are already on the way. What we want to keep in mind is that in all these areas, what machines are doing is *computing*. Machines *compute* meanings of documents for natural language processing, combine and *compute* digital image pixels for computer vision, convert audio signals to vectors of numbers and *compute* new audio for human-machine interaction, and so on. It is then easy to see how software AI is one big mathematical model and simulation. This will become more evident as we progress in this book.

Where Do We Get Our Data From?

When I first decided to enter the AI field, I wanted to apply my mathematical knowledge to help solve real-world problems that I felt passionate about. I grew up in a war-torn country, and saw many problems erupt, disrupt, then eventually dissipate or get resolved, either by direct fixes or by the human network adjusting around them and settling into completely new (unstable) equilibria. Common problems in war were sudden and massive disruptions to different supply chains, sudden destruction of large parts of the power grid, sudden paralysis of entire road networks by targeted bombing of certain bridges, sudden emergence of terrorist networks, black markets, trafficking, inflation, and poverty. The number of problems that math can help solve in these scenarios, including war tactics and strategy, is limitless. From the safety of the United States, my Ph.D. in mathematics, and my tenure at a university, I started approaching companies, government agencies, and the military, looking for real

projects with real data to work on. I offered to help find solutions to their problems for free. What I did not know, and learned the hard way, was that getting real data was the biggest hurdle. There are many regulations, privacy issues, institutional review boards, and other obstacles standing in the way. Even after jumping through all these hoops, the companies, institutions, and organizations tend to hold on to their data, even when they know they are not making the best use of it, and one almost has to beg in order to get real data. It turned out that the experience I had was not unique. The same had happened to many others in the field.

This story is not meant to discourage you from getting the real data that you need to train your AI systems. The point is to not get surprised and disheartened if you encounter hesitation and resistance from the owners of the data that you need. Keep asking, and someone will be willing to take that one leap of faith.

Sometimes the data you need is available publicly on the web. For the simple models in this chapter, I am using data sets from the [Kaggle website](#). There are other great public data repositories, which I will not list here, but a simple Google search with keywords like “best data repositories” will return excellent results. Some repositories are geared toward computer vision, others toward natural language processing, audio generation and transcription, scientific research, and so on.

Crawling the web to acquire data is common, but you have to abide by the rules of the websites you are crawling. You also have to learn how to crawl (some people say that the difference between data scientists and statisticians is that data scientists know how to hack!). Some websites require you to obtain written permission before you crawl. For example, if you are interested in social media user behavior, or in collaboration networks, you can crawl social media and professional networks—Facebook, Instagram, YouTube, Flickr, LinkedIn, etc.—for statistics on user accounts, such as the number of friends or connections, likes, comments, and activity on these sites. You will end up with very large data sets with hundreds of thousands of records, which you can then do your computations on.

To gain an intuitive understanding of how data gets integrated into AI, and the type of data that goes into various systems, while at the same time avoiding feeling overwhelmed by all the information and the data that is out there, it is beneficial to develop a habit of exploring the data sets that successful AI systems were trained on, if they are available. You do not have to download them and work on them. Browsing the data set, its metadata, what features and labels (if any) it comes with, etc., is enough to get you comfortable with data. For example, DeepMind’s [WaveNet](#) (2016), which we will learn about in [Chapter 7](#), is a neural network that generates raw machine audio, with realistic-sounding human voices or enjoyable pieces of music. It accomplishes tasks like text-to-audio conversion with natural-sounding human voice connotations, even with a specific person’s voice if the network gets conditioned on this person’s voice. We will understand the mathematical meaning of conditioning

when we study WaveNet in [Chapter 7](#). For now, think of it as a restriction imposed artificially on a problem so it restricts its results to a certain set of outcomes. So what data was WaveNet trained on? For multispeaker audio generation that is not conditioned on text, WaveNet was trained on a data set of audio files consisting of 44 hours of audio from 109 different speakers: the [English Multispeaker Corpus from CSTR Voice Cloning Toolkit](#) (2012). For converting text to speech, WaveNet was trained on the [North American English data set](#), which contains 24 hours of speech data, and on the [Mandarin Chinese data set](#), which has 34.8 hours of speech data. For generating music, WaveNet was trained on the [YouTube Piano data set](#), which has 60 hours of solo piano music obtained from YouTube videos, and the [MagnaTagATune data set](#) (2009), which consists of about 200 hours of music audio, where each 29-second clip is labeled with 188 tags describing the genre, instrumentation, tempo, volume, mood, and various other labels for the music. Labeled data is extremely valuable for AI systems, because it provides a ground truth to measure the output of your hypothesis function against. We will learn this in the next few sections.

How about the famous image classification (for computer vision) AlexNet (2012)? What data was its convolutional neural network trained and tested on? AlexNet was trained on [ImageNet](#), a data set containing millions of images (scraped from the internet) and labeled (crowdsourced human labelers) with thousands of classes.

Note that all of these examples were all examples of unstructured data.

If the data that a certain system was trained on is not publicly available, it is good to look up the published paper on the system or its documentation and read about how the required data was obtained. That alone will teach you a lot.

Before moving on to doing mathematics, keep in mind the following takeaways:

- AI systems need digital data.
- Sometimes, the data you need is not easy to acquire.
- There is a movement to digitalize our whole world.

The Vocabulary of Data Distributions, Probability, and Statistics

When you enter a new field, the first thing you want to learn is the vocabulary of that field. It is similar to learning a new language. You can learn it in a classroom, and suffer, or you can travel to a country that speaks the language, and listen to frequently used terms. You don't have to know what "bonjour" means in French. But while you are in France, you notice that people say it to each other all the time, so you start saying it as well. Sometimes you will not use it in the right context, like when you have to say "bonsoir" instead of "bonjour." But slowly, as you find yourself staying longer in France, you will be using the right vocabulary in the right context.

One more advantage of learning the vocabulary as fast as you can, without necessarily mastering any of the details, is that different fields refer to the same concepts with different terms, since there is a massive vocabulary collision out there. This ends up being a big source of confusion, therefore the embodiment of *language barriers*. When you learn the common vocabulary of the field, you will realize that you might already know the concepts, except that now you have new names for them.

The vocabulary terms from probability and statistics that you want to know for the purposes of AI applications are not too many. I will define each term once we get to use it, but note that the goal of probability theory is to make deterministic statements about random or stochastic quantities and events, since humans hate uncertainty and like their world to be controllable and predictable. Watch for the following language from the fields of probability and statistics whenever you are reading about AI, machine learning, or data science. Again, you do not have to know any of the definitions yet; you just need to hear the terms discussed in the following sections and be familiar with the way they progress after each other.

Random Variables

It all starts with *random variables*. Math people talk about functions nonstop. Functions have certain or deterministic outcomes. When you evaluate a function, you know exactly what value it will return. Evaluate the function x^2 at 3, and you are certain you will get $3^2 = 9$. Random variables, on the other hand, do not have deterministic outcomes. Their outcomes are uncertain, unpredictable, or stochastic. When you call a random variable, you do not know, before you actually see the outcome, what value it will return. Since you cannot aim for certainty anymore, what you can instead aim for is quantifying how likely it is to get an outcome. For example, when you roll a die, you can confidently say that your chance to get the outcome 4 is 1/6, assuming that the die you rolled is not tampered with. You never know ahead of time what outcome you will get before you actually roll the die. If you did, then casinos would run out of business, and the finance sector would eliminate its entire predictive analytics and risk management departments. Just like deterministic functions, a random variable can return outcomes from a discrete set (discrete random variable) or from the continuum (continuous random variable). The key distinction between a random variable and a function is in the randomness versus the certainty of the outcomes.

Probability Distributions

After random variables we define *probability density functions* for continuous random variables and *probability mass functions* for discrete random variables. We call both *distributions* in order to add to our confusion. Usually, whether a distribution represents a discrete or a continuous random variable is understood from the context.

Using this terminology, we sometimes say that one random variable, whether continuous or discrete, is *sampled* from a probability distribution, and multiple random variables are sampled from a *joint probability distribution*. In practice, it is rare that we know the full joint probability distribution of all the random variables involved in our data. When we do, or if we are able to *learn it from the data*, it is a powerful thing.

Marginal Probabilities

Marginal probability distributions sit literally on the margins of a joint probability distribution (if we represent the joint probability distribution with a table containing the probabilities of all the combined states of the involved variables; see, for example, the first table on [this Wikipedia page](#)). In this setting, you are lucky enough to have access to the full joint probability distribution of multiple random variables, and you are interested in finding out the probability distribution of only one or few of them. You can find these *marginal probability distributions* easily using the *sum rule* for probabilities, for example:

$$p(x) = \sum_{y \in \text{all states of } y} p(x, y).$$

The Uniform and the Normal Distributions

The *uniform distribution* and the *normal distribution* are the most popular continuous distributions, so we start with them. The normal distribution and the fundamental *central limit theorem* from probability theory are intimately related. There are many other useful distributions representing the different random variables involved in our data, but we do not need them right away, so we postpone them until we need to use them.

Conditional Probabilities and Bayes' Theorem

The moment we start dealing with multiple random variables (such as our gender, height, weight, and health index data), which is almost always the case, we introduce *conditional probabilities*, *Bayes' Rule or Theorem*, and the *product or chain rule* for conditional probabilities, along with the concepts of *independent and conditionally independent random variables* (knowing the value of one does not change the probability of the other).

Conditional Probabilities and Joint Distributions

Both conditional probabilities and joint distributions involve multiple random variables, so it makes sense that they have something to do with each other. Slice the graph of a joint probability distribution (when we fix the value of one of the variables) and we get a conditional probability distribution (see [Figure 2-7](#) later on).



Bayes' Rule Versus Joint Probability Distribution

It is very important to keep the following in mind: if we happen to have access to the full joint probability distribution of all the multiple random variables that we care for in our setting, then we would not need Bayes' Rule. In other words, Bayes' Rule helps us calculate the desired conditional probabilities *when we do not have access* to the full joint probability distribution of the involved random variables.

Prior Distribution, Posterior Distribution, and Likelihood Function

From logical and mathematical standpoints, we can define conditional probabilities, then move on smoothly with our calculations and our lives. Practitioners, however, give different names to different conditional probabilities, depending on whether they are conditioning on data that has been observed or on weights (also called parameters) that they still need to estimate. The vocabulary words here are: *prior distribution* (general probability distribution for the weights of our model prior to observing any data), *posterior distribution* (probability distribution for the weights given the observed data), and the *likelihood function* (function encoding the probability of observing a data point given a particular weight distribution). All of these can be related through the Bayes' Rule, as well as through the joint distribution.



We Say Likelihood Function not Likelihood Distribution

We refer to the likelihood as a function and not as a distribution because probability distributions *must* add up to one (or integrate to one if we are dealing with continuous random variables), but the likelihood function does not necessarily add up to one (or integrate to one in the case of continuous random variables).

Mixtures of Distributions

We can mix probability distributions and produce *mixtures of distributions*. *Gaussian mixtures* are pretty famous. The earlier height data that contains measurements for both males and females is a good example of a Gaussian mixture.

Sums and Products of Random Variables

We can add or multiply random variables sampled from simple distributions to produce new random variables with more complex distributions, representing more complex random events. The natural question that's usually investigated here is: *what is the distribution of the sum random variable or the product random variable?*

Using Graphs to Represent Joint Probability Distributions

Finally, we use directed and undirected graph representations (diagrams) to efficiently decompose joint probability distributions. This makes our computational life much cheaper and tractable.

Expectation, Mean, Variance, and Uncertainty

Four quantities are central to probability, statistics, and data science: *expectation* and *mean*, quantifying an average value, and the *variance* and *standard deviation*, quantifying the spread around that average value, hence encoding uncertainty. Our goal is to have control over the variance in order to reduce the uncertainty. The larger the variance, the more error you can commit when using your average value to make predictions. Therefore, when you explore the field, you often notice that mathematical statements, inequalities, and theorems mostly involve some control over the expectation and variance of any quantities that involve randomness.

When we have one random variable with a corresponding probability distribution, we calculate the *expectation* (expected average outcome), *variance* (expected squared distance from the expected average), and *standard deviation* (expected distance from the average). For data that has been already sampled or observed, for example, our height and weight data above, we calculate the *sample mean* (average value), variance (average squared distance from the mean), and standard deviation (average distance from the mean, so this measures the spread around the mean). So if the data we care for has not been sampled or observed yet, we speculate on it using the language of *expectations*, but if we already have an observed or measured sample, we calculate its *statistics*. Naturally we are interested in how far off our speculations are from our computed statistics for the observed data, and what happens in the limiting (but idealistic) case where we can in fact measure data for the entire population. The *law of large numbers* answers that for us and tells us that in this limiting case (when the sample size goes to infinity), our expectation matches the sample mean.

Covariance and Correlation

When we have two or more random variables, we calculate the *covariance*, *correlation*, and *covariance matrix*. This is when the field of linear algebra with its language of vectors, matrices, and matrix decompositions (such as eigenvalues and singular value decompositions) gets married to the field of probability and statistics. The variance of each random variable sits on the diagonal of the covariance matrix, and the covariances of each possible pair sit off the diagonal. The covariance matrix is symmetric. When you diagonalize it, using standard linear algebra techniques, you *uncorrelate* the involved random variables.

Meanwhile, we pause and make sure we know the difference between independence and zero covariance. Covariance and correlation are all about capturing a linear relationship between two random variables. Correlation works on *normalized* random variables, so that we can still detect linear relationships even if random variables or data measurements have vastly different scales. When you normalize a quantity, its scale doesn't matter anymore. It wouldn't matter whether it is measured on a scale of millions or on a 0.001 scale. Covariance works on unnormalized random variables. Life is not all linear. Independence is stronger than zero covariance.

Markov Process

Markov processes are very important for AI's reinforcement learning paradigm. They are characterized by all possible states of a system, a set of all possible actions that can be performed by an agent (move left, move right, etc.), a matrix containing the transition probabilities between all states, the probability distribution for what states an agent will transition to after taking a certain action, and a reward function, which we want to maximize. Two popular examples from AI include board games and a smart thermostat such as Nest. We will go over these in [Chapter 11](#).

Normalizing, Scaling, and/or Standardizing a Random Variable or Data Set

This is one of the many cases where there is a vocabulary collision. Normalizing, scaling, and standardizing are used synonymously in various contexts. The goal is always the same. Subtract a number (shift) from the data or from all possible outcomes of a random variable, then divide by a constant number (scale). If you subtract the mean of your data sample (or the expectation of your random variable) and divide by their standard deviation, then you get new *standardized* or *normalized* data values (or a new standardized or normalized random variable) that have a mean equal to zero (or expectation zero) and standard deviation equal to one. If instead you subtract the minimum and divide by the range (max value minus min value), then you get new data values or a new random variable with outcomes all between zero and one. Sometimes we talk about normalizing vectors of numbers. In this case, what we mean is we divide every number in our vector by the length of the vector itself, so that we obtain a new vector of length one. So whether we say we are normalizing, scaling, or standardizing a collection of numbers, the goal is to try to control the values of these numbers, center them around zero, and/or restrict their spread to be less than or equal to one while at the same time preserving their inherent variability.

Common Examples

Mathematicians like to express probability concepts in terms of flipping coins, rolling dice, drawing balls from urns, drawing cards from decks, trains arriving at stations,

customers calling a hotline, customers clicking on an ad or a website link, diseases and their symptoms, criminal trials and evidence, and the time until something happens, such as a machine failing. Do not be surprised that these examples are everywhere, as they generalize nicely to many other real-life situations.

In addition to this map of probability theory, we will borrow very few terms and functions from statistical mechanics (for example, the *partition function*) and information theory (for example, signal versus noise, entropy, and the *cross-entropy function*). We will explain these when we encounter them in later chapters.

Continuous Distributions Versus Discrete Distributions (Density Versus Mass)

When we deal with continuous distributions, it is important to use terms like observing or sampling a data point *near* or *around* a certain value instead of observing or sampling an *exact* value. In fact, the probability of observing an exact value in this case is zero.

When our numbers are in the continuum, there is no discrete separation between one value and the next value. Real numbers have an infinite precision. For example, if I measure the height of a male and I get 6 feet, I wouldn't know whether my measurement is exactly 6 feet or 6.00000000785 feet or 5.999911134255 feet. It's better to set my observation in an interval around 6 feet, for example $5.95 < \text{height} < 6.05$, then quantify the probability of observing a height between 5.95 and 6.05 feet.

We do not have such a worry for discrete random variables, as we can easily separate the possible values from each other. For example, when we roll a die, our possible values are 1, 2, 3, 4, 5, or 6. So we can confidently assert that the probability of rolling an exact 5 is $1/6$. Moreover, a discrete random variable can have nonnumerical outcomes; for example, when we flip a coin, our possible values are heads or tails. A continuous random variable can only have numerical outcomes.

Because of that reasoning, when we have a continuous random variable, we define its probability *density* function, not its probability *mass* function, as in the case of discrete random variables. A density specifies how much of a substance is present within a certain length or area or volume of space (depending on the dimension we're in). To find the mass of a substance in a specified region, we multiply the density by the length, area, or volume of the considered region. If we are given the density per an infinitesimally small region, then we must integrate over the whole region to find the mass within that region, because an integral is akin to a sum over infinitely many infinitesimally small regions.

We will elaborate on these ideas and mathematically formalize them in [Chapter 11](#). For now, we stress the following:

- If we only have one continuous random variable, such as the height of males in a certain population, then we use a one-dimensional probability density function to represent its probability distribution: $f(x_1)$. To find the probability of the height being between $5.95 < \text{height} < 6.05$, we integrate the probability density function $f(x_1)$ over the interval $(5.95, 6.05)$, and we write:

$$P(5.95 < \text{height} < 6.05) = \int_{5.95}^{6.05} f(x_1) dx_1$$

- If we have two continuous random variables, such as the height and weight of males in a certain population, or the true height and the measured height of a person (which usually includes random noise), then we use a two-dimensional probability density function to represent their *joint probability distribution*: $f(x_1, x_2)$. So in order to find the joint probability that the height is between $5.95 < \text{height} < 6.05$ *and* the weight is between $160 < \text{weight} < 175$, we *double integrate* the joint probability density function $f(x_1, x_2)$, assuming that we know the formula for $f(x_1, x_2)$, over the intervals $(5.95, 6.05)$ and $(160, 175)$, and we write:

$$P(5.95 < \text{height} < 6.05, 160 < \text{weight} < 175) = \int_{160}^{175} \int_{5.95}^{6.05} f(x_1, x_2) dx_1 dx_2$$

- If we have more than two continuous random variables, then we use a higher-dimensional probability density function to represent their joint distribution. For example, if we have the height, weight, and blood pressure of males in a certain population, then we use a three-dimensional joint probability distribution function: $f(x_1, x_2, x_3)$. Similar to the reasoning explained previously, to find the joint probability that the first random variable is between $a < x_1 < b$, the second random variable is between $c < x_2 < d$, and the third random variable is between $e < x_3 < f$, we *triple integrate* the joint probability density function over the intervals (a, b) , (c, d) , and (e, f) , and we write:

$$P(a < x_1 < b, c < x_2 < d, e < x_3 < f) = \int_e^f \int_c^d \int_a^b f(x_1, x_2, x_3) dx_1 dx_2 dx_3$$

Not all our worries (that things will add up mathematically) are eliminated even after defining the probability density function for a continuous random variable. Again, the culprit is the infinite precision of real numbers. If we allow *all sets* to have probability, we encounter paradoxes in the sense that we can construct disjoint sets (such as fractal-shaped sets or sets formulated by transforming the set of rational numbers) whose probabilities add up to more than one! One must admit that these sets are pathological and must be carefully constructed by a person who has plenty of time on their hands; however, they exist and they produce paradoxes. *Measure theory*

in mathematics steps in and provides a mathematical framework where we can work with probability density functions without encountering paradoxes. It defines sets of measure zero (these occupy no volume in the space that we are working in), then gives us plenty of theorems that allow us to do our computations *almost everywhere*, that is, except on sets of measure zero. This turns out to be more than enough for our applications.

The Power of the Joint Probability Density Function

Having access to the joint probability distribution of many random variables is a powerful but rare thing. The reason is that the joint probability distribution encodes within it the probability distribution of each separate random variable (marginal distributions), as well as all the possible co-occurrences (and the conditional probabilities) that we ever encounter between these random variables. This is akin to seeing a whole town from above, rather than being inside the town and observing only one intersection between two or more alleys.

If the random variables are independent, then the joint distribution is simply the product of each of their individual distributions. However, when the random variables are not independent, such as the height and the weight of a person, or the observed height of a person (which includes measurement noise) and the true height of a person (which doesn't include noise), accessing the joint distribution is much more difficult and expensive storage-wise. The joint distribution in the case of dependent random variables is not separable, so we cannot only store each of its parts alone. We need to store every value for every co-occurrence between the two or more variables. This exponential increase in storage requirements (and computations or search spaces) as you increase the number of dependent random variables is one embodiment of the infamous *curse of dimensionality*.

When we *slice* a joint probability density function, say $f(x_1, x_2)$, meaning when we fix one of the random variables (or more in higher dimensions) to be an exact value, we retrieve a distribution proportional to the *posterior probability distribution* (probability distribution of the model parameters given the observations), which we are usually interested in. For example, slice through $f(x_1, x_2)$ at $x_1 = a$, and we get $f(a, x_2)$, which happens to be proportional to the probability distribution $f(x_2 | x_1 = a)$ (see [Figure 2-7](#)).

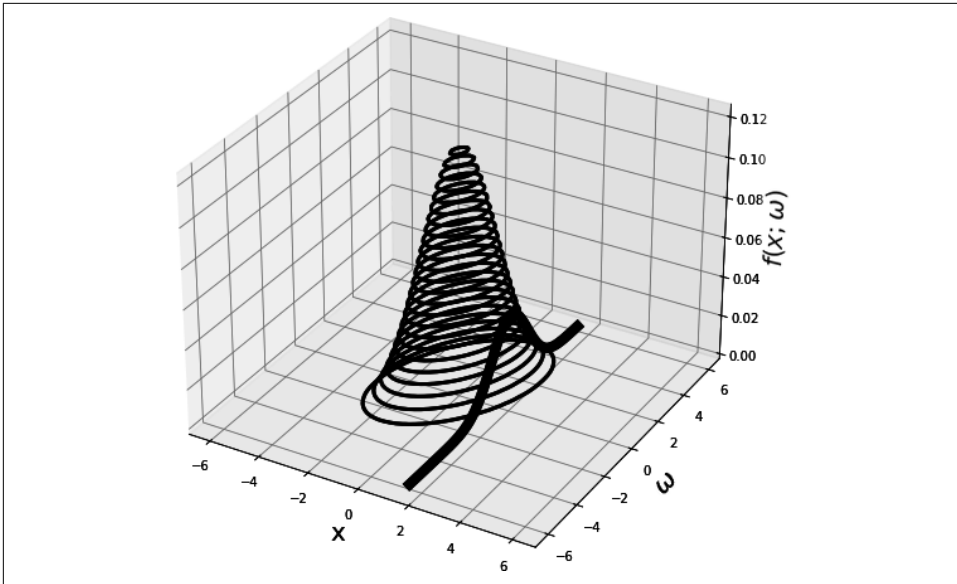


Figure 2-7. Slicing through the joint probability distribution

Again, this is in the luxurious case that we know the joint probability distribution, otherwise, we use Bayes' Rule to obtain the same posterior probability distribution (using the prior distribution and the likelihood function).

In some AI applications, the AI system learns the joint probability distribution by separating it into a product of conditional probabilities using the product rule for probabilities. Once it learns the joint distribution, it then samples from it to generate new and interesting data. DeepMind's WaveNet does that in its process of generating raw audio.

The next sections introduce the most useful probability distributions for AI applications. Two ubiquitous continuous distributions are the *uniform distribution* and the *normal distribution* (also known as the *Gaussian distribution*), so we start there. Refer to the [Jupyter Notebook](#) for reproducing figures and more details.

Distribution of Data: The Uniform Distribution

To intuitively understand the uniform distribution, let's give an example of a *nonuniform* distribution, which we have already seen earlier in this chapter. In our real height-weight data sets, we cannot use the *uniform distribution* to model the height data. We also cannot use it to model the weight data. The reason is that human heights and weights are not evenly distributed. In the general population, it is not *equally likely* to encounter a person with height around 7 feet as it is to encounter a person with height around 5 feet 6 inches.

The *uniform distribution* only models data that is evenly distributed. If we have an interval (x_{min}, x_{max}) containing all the values in the continuum between x_{min} and x_{max} of our data, and our data is uniformly distributed over our interval, then the probability of observing a data point near any particular value in our interval is the same for all values in this interval. That is, if our interval is $(0,1)$, it is equally likely to pick a point near 0.2 as it is to pick a point near 0.75.

The probability density function for the uniform distribution is therefore constant. For one random variable x over an interval (x_{min}, x_{max}) , the formula for the probability density function for the continuous uniform distribution is given by:

$$f(x; x_{min}, x_{max}) = \frac{1}{x_{max} - x_{min}} \text{ for } x_{min} < x < x_{max}$$

and zero otherwise.

Let's plot the probability density function for the uniform distribution over an interval (x_{min}, x_{max}) . The graph in [Figure 2-8](#) is a straight segment because uniformly distributed data, whether real or simulated, appears evenly distributed across the entire interval under consideration. No data values within the interval are more favored to appear than others.

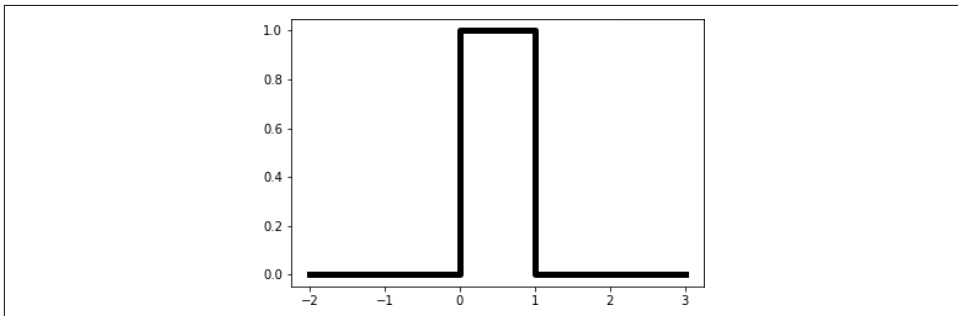


Figure 2-8. Graph of the probability density function of the uniform distribution over the interval $[0,1]$

The uniform distribution is extremely useful in computer simulations for generating random numbers *from any other probability distribution*. If you peek into the random number generators that Python uses, you would see the uniform distribution used somewhere in the underlying algorithms.

Distribution of Data: The Bell-Shaped Normal (Gaussian) Distribution

A continuous probability distribution better suited to model human height data (when restricted to one gender) is the *bell-shaped normal distribution*, also called the *Gaussian distribution*. Samples from the normal distribution tend to congregate around an average value where the distribution peaks, called the *mean* μ , then dwindle symmetrically as we get farther away from the mean. How far from the mean the distribution spreads out as it dwindles down is controlled by the second parameter of the normal distribution, called the *standard deviation* σ . About 68% of the data falls within one standard deviation of the mean, 95% of the data falls within two standard deviations of the mean, and about 99.7% of the data falls within three standard deviations of the mean (Figure 2-9).

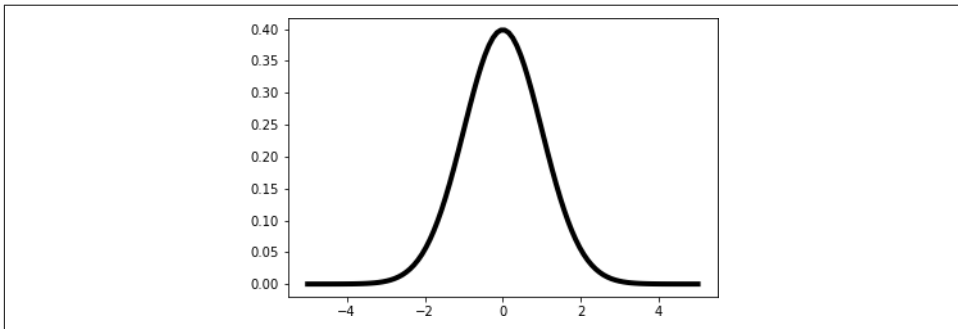


Figure 2-9. Graph of the probability density function of the bell-shaped normal distribution with parameters $\mu = 0$ and $\sigma = 1$

Values near the mean are more likely to be picked (or to occur, or to be observed) when we are sampling data from the normal distribution, and values that are very small ($\rightarrow -\infty$) or very large ($\rightarrow \infty$) are less likely to be picked. This peaking near the mean value and decaying on the outer skirts of the distribution give this distribution its famous bell shape. Note that there are other bell-shaped continuous distributions out there, but the normal distribution is the most prevalent. It has a neat mathematical justification for this well-deserved fame, based on an important theorem in probability theory called the *central limit theorem* (CLT).

The central limit theorem states that the average of many independent random variables that all have the same distribution (not necessarily the normal distribution) is normally distributed. This explains why the normal distribution appears everywhere in society and nature. It models baby birth weights, student grade distributions, countries' income distributions, distribution of blood pressure measurements, etc. There are special statistical tests that help us determine whether a real data set can

be modeled using the normal distribution. We will expand on these ideas later in [Chapter 11](#).

If you happen to find yourself in a situation where you are uncertain and have no prior knowledge about which distribution to use for your application, the normal distribution is usually a reasonable choice. In fact, among all choices of distributions with the same variance, the normal distribution is the choice with maximum *uncertainty*, so it does in fact encode the least amount of prior knowledge into your model.

The formula for the probability density function of the normal distribution for one random variable x (univariate) with mean μ and standard deviation σ is:

$$g(x; \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

and its graph for $\mu = 0$ and $\sigma = 1$ is plotted in [Figure 2-9](#).

The formula for the probability density function for the normal distribution of two random variables x and y (bivariate) is:

$$g(x, y; \mu_1, \sigma_1, \mu_2, \sigma_2, \rho) = \frac{1}{\sqrt{(2\pi)^2 \det \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}}} e^{-\frac{1}{2} \begin{pmatrix} x - \mu_1 & y - \mu_2 \end{pmatrix} \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}^{-1} \begin{pmatrix} x - \mu_1 \\ y - \mu_2 \end{pmatrix}}$$

and its graph is plotted in [Figure 2-10](#).

We can write the above bivariate formula in more compact notation using the language of linear algebra:

$$g(x, y; \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^2 \det(\Sigma)}} e^{-\frac{1}{2} (u - \mu)^T \Sigma^{-1} (u - \mu)}$$

In [Figure 2-11](#), we sample 6,000 points from the bivariate normal distribution. Points near the center are more likely to be picked, and points away from the center are less likely to be picked. The lines roughly trace the contour lines of the normal distribution, had we only observed the sample points without knowing which distribution they were sampled from.

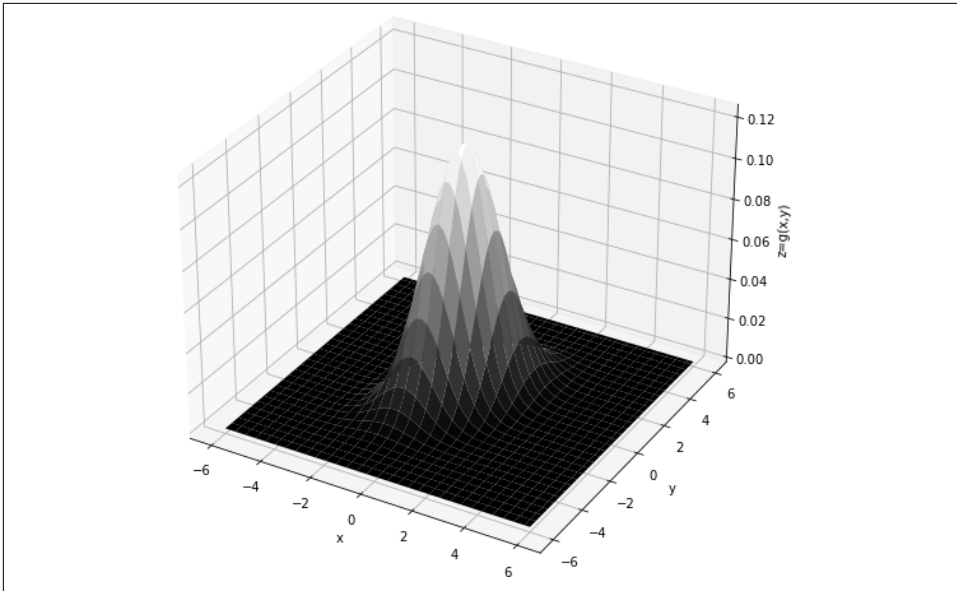


Figure 2-10. Graph of the probability density function of the bell-shaped bivariate normal distribution

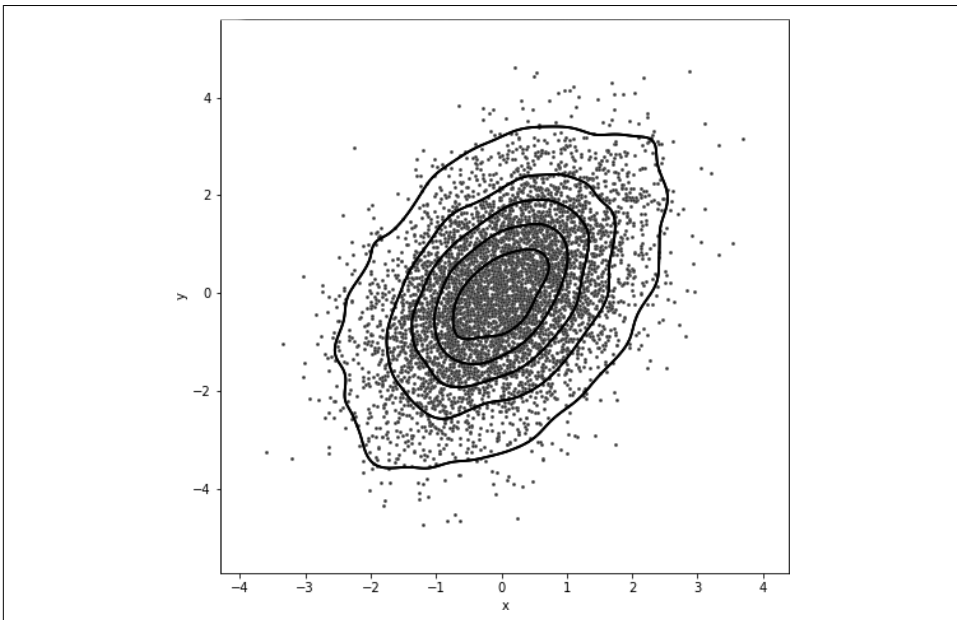


Figure 2-11. Sampling 6,000 points from the bivariate normal distribution

Let's pause and compare the formula of the probability density function for the bivariate normal distribution to the formula of the probability density function for the univariate normal distribution:

- When there is only one random variable, we only have one mean μ and one standard deviation σ .
- When there are two random variables, we have two means $\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}$ and two standard deviations $\begin{pmatrix} \sigma_1 \\ \sigma_2 \end{pmatrix}$. The product σ^2 will be replaced by the covariance matrix $\Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$ and its determinant. ρ is the correlation between the two random variables, which is the covariance of the two normalized versions of the random variables.

The same exact formula for the probability density function of the bivariate normal distribution generalizes to any dimension, where we have many random variables instead of only two random variables. For example, if we have 100 random variables, representing 100 features in a data set, the mean vector in the formula will have 100 entries in it, and the covariance matrix will have the size 100×100 , with the variance of each random variable on the diagonal and the covariance of each of the 4,950 pairs of random variables off the diagonal.

Distribution of Data: Other Important and Commonly Used Distributions

Almost everything you did not understand in this chapter will be revisited many times throughout the book, and [Chapter 11](#) focuses exclusively on probability. The concepts will get reinforced as they appear again and again in various interesting contexts. Our goal for this chapter is to get exposed to the vocabulary of probability and statistics, and have a guiding map for the important ideas that frequently appear in AI applications. We also want to acquire a good probabilistic intuition for the following chapters without having to take a deep dive and delay our progress for no necessary reason.

There are many probability distributions out there. Each models a different type of real-world scenario. The uniform and normal distributions are very common, but we have other important distributions that frequently appear in the AI field. Recall that our goal is to model the world around us in order to make good designs, predictions, and/or decisions. Probability distributions help us make predictions when our models involve randomness or when we are uncertain about our outcomes.

When we study distributions, one frustrating part is that most of them have weird names that provide zero intuition about what kind of phenomena a given distribution would be useful for. This makes us either expend extra mental energy to memorize these names, or keep a distribution cheat sheet in our pocket. I prefer to keep a cheat sheet. Another frustrating part is that most textbook examples involve flipping a coin, rolling a die, or drawing colored balls from urns. This leaves us with no real-life examples or motivation to understand the subject, as I never met anyone walking around flipping coins and counting heads or tails, except for Two-Face (aka Harvey Dent) in *The Dark Knight* (a really good 2008 movie, where the Joker [played by Heath Ledger] says some of my favorite and profound statements about randomness and chance, like this one: “The world is cruel. And the only morality in a cruel world...is chance. Unbiased. Unprejudiced. Fair.”). I will try to amend this as much as I can in this book, pointing to as many real-world examples as my page limit allows.

Some of the following distributions are mathematically related to each other, or follow naturally from others. We will explore these relationships in [Chapter 10](#). For now, let’s name a popular distribution, state whether it is discrete (predicts a count of something that we care for) or continuous (predicts a quantity that exists in the continuum, such as the time needed to elapse before something happens; careful, this is not the number of hours, since the number of hours is discrete, but it is the length of the time period), state the parameters that control it, and state its defining properties that are useful for our AI applications:

Binomial distribution

This is discrete. It represents the probability of obtaining a certain number of successes when repeating one experiment, independently, multiple times. Its controlling parameters are n , the number of experiments we perform, and p , the *predefined* probability of success. Real-world examples include predicting the number of patients that will develop side effects for a vaccine or a new medication in a clinical trial, the number of ad clicks that will result in a purchase, and the number of customers that will default on their monthly credit card payments. When we model examples from the real world using a probability distribution that requires independent trials, it means that we *are assuming* independence even if the real-world trials are not really independent. It is good etiquette to point our models’ assumptions.

Poisson distribution

This is discrete. It predicts the number of rare events that will occur in a given period of time. These events are independent or weakly dependent, meaning that the occurrence of the event once does not affect the probability of its next occurrence in the same time period. They also occur at a *known and constant* average rate λ . Thus, we know the average rate, and we want to predict *how many* of these events will happen during a certain time period. The Poisson distribution’s controlling parameter is the predefined rare event rate λ . Real-world examples

include predicting the number of babies born in a given hour, the number of people in a population who age past 98, the number of alpha particles discharged from a radioactive system during a certain time period, the number of duplicate bills sent out by the IRS, the number of a not-too-popular product sold on a particular day, the number of typos that one page of this book contains, the number of defective items produced by a certain machine on a certain day, the number of people entering a store at a certain hour, the number of car crashes an insurance company needs to cover within a certain time period, and the number of earthquakes happening within a particular time period.

Geometric distribution

This is discrete. It predicts the number of trials needed before we obtain a success when performing independent trials, each with a *known* probability p for success. The controlling parameter here is obviously the probability p for success. Real-world examples include estimating the number of weeks that a company can function without experiencing a network failure, the number of hours a machine can function before producing a defective item, or the number of people we need to interview before meeting someone who opposes a certain political bill that we want to pass. Again, for these real-world examples, we might be assuming independence if modeling using the geometric distribution, while in reality the trials might not be independent.

Exponential distribution

This is continuous. If we happen to know that a certain event occurs at a constant rate λ , then exponential distribution predicts the waiting time until this event occurs. It is *memoryless*, in the sense that the remaining lifetime of an item that belongs to this exponential distribution is also exponential. The controlling parameter is the constant rate λ . Real-world examples include the amount of time we have to wait until an earthquake occurs, the time until someone defaults on a loan, the time until a machine part fails, or the time before a terrorist attack strikes. This is very useful for the reliability field, where the reliability of a certain machine part is calculated, hence statements such as a 10-year guarantee, etc.

Weibull distribution

This is continuous. It is widely used in engineering in the field of predicting product lifetimes (10-year warranty statements are appropriate here as well). Here, a product consists of many parts, and if any of its parts fail, then the product stops working. For example, a car will not work if the battery fails, or if a fuse in the gearbox burns out. A Weibull distribution provides a good approximation for the lifetime of a car before it stops working, after accounting for its many parts and their *weakest link* (assuming we are not maintaining the car and resetting the clock). It is controlled by three parameters: shape, scale, and location. The exponential distribution is a special case of this distribution, because the exponential distribution has a constant rate of event occurrence, but

the Weibull distribution can model rates of occurrence that increase or decrease with time.

Log-normal distribution

This is continuous. If we take the logarithms of each value provided in this distribution, we get normally distributed data. Meaning that in the beginning, your data might not appear normally distributed, but if you try transforming it using the log function, you will see normally distributed data. This is a good distribution to use when encountering skewed data with low mean value, large variance, and *assuming only positive values*. Just like the normal distribution appears when you average many independent samples of a random variable (using the central limit theorem), the log-normal distribution appears when you take the *product* of many positive sample values. Mathematically, this is due to an awesome property of log functions: the log of a product is a sum of the logs. This distribution is controlled by three parameters: shape, scale, and location. Real-world examples include the volume of gas in a petroleum reserve, and the ratio of the price of a security at the end of one day to its price at the end of the day before.

Chi-squared distribution

This is continuous. It is a distribution for the sum of squares of normally distributed independent random variables. You might wonder why would we care about squaring normally distributed random variables, then adding them up. The answer is that this is how we usually compute the variance of a random variable or of a data sample, and one of our main goals is controlling the variance in order to lower our uncertainties. There are two types of significance tests associated with this distribution: the goodness of fit test, which measures how far off our expectation is from our observation, and the independence and homogeneity of data features test.

Pareto distribution

This is continuous. It is useful for many real-world applications, such as the time to complete a job assigned to a supercomputer (think machine learning computations), the household income level in a certain population, the number of friends in a social network, and the file size of internet traffic. This distribution is controlled by only one parameter α , and it is *heavy tailed* (its tail is heavier than the exponential distribution).

Let's throw in a few other distributions before moving on, without fussing about any of the details. These are all more or less related to the aforementioned distributions:

Student's t-distribution

Continuous, similar to the normal distribution, but used when the sample size is small and the population variance is unknown.

Beta distribution

Continuous, produces random values in a given interval.

Cauchy distribution

Continuous, pathological because neither its mean nor its variance is defined, can be obtained using the tangents of randomly chosen angles.

Gamma distribution

Continuous, has to do with the waiting time until n independent events occur, instead of only one event, as in the exponential distribution.

Negative binomial distribution

Discrete, has to do with the number of independent trials needed to obtain a certain number of successes.

Hypergeometric distribution

Discrete, similar to the binomial but the trials are not independent.

Negative hypergeometric distribution

Discrete, captures the number of dependent trials needed before we obtain a certain number of successes.

The Various Uses of the Word “Distribution”

You might have already noticed that the word *distribution* refers to many different (but related) concepts, depending on the context. This inconsistent use of the same word could be a source of confusion and an immediate turnoff for some people who are trying to enter the field.

Let's list the different concepts that the word distribution refers to, so that we easily recognize its intended meaning in a given context:

- If you have real data, such as the height-weight data in this chapter, and plot the histogram of one feature of your data set, such as the height, then you get the *empirical distribution* of the height data. You usually do not know the underlying probability density function of the height of the entire population, also called distribution, since the real data you have is only a sample of that population. So you try to estimate it, or model it, using the probability distributions given by probability theory. For the height and weight features, when separated by gender, a Gaussian distribution is appropriate.
- If you have a discrete random variable, the word distribution could refer to either its probability mass function or its cumulative distribution function (which specifies the probability that the random variable is less than or equal to a certain value, $f(x) = \text{prob}(X \leq x)$).

- If you have a continuous random variable, the word distribution could refer to either its probability density function or its cumulative distribution function, whose integral gives the probability that the random variable is less than or equal to a certain value.
- If you have multiple random variables (discrete, continuous, or a mix of both), then the word distribution refers to their joint probability distribution.

A common goal is to establish an appropriate correspondence between an idealized mathematical function, such as a random variable with an appropriate distribution, and real observed data or phenomena, with an observed empirical distribution. When working with real data, each feature of the data set can be modeled using a random variable. So in a way, a mathematical random variable with its corresponding distribution is an idealized version of our measured or observed feature.

Finally, distributions appear everywhere in AI applications. We will encounter them plenty of times in the next chapters, for example, the distribution of the weights at each layer of a neural network, and the distribution of the noise and errors committed by various machine learning models.

A/B Testing

Before leaving this chapter, we make a tiny detour into the world of A/B testing, also called *split testing*, or *randomized single-blind* or *double-blind trials*. We make this detour because this is a topic important for data scientists: countless companies rely on data from A/B tests to increase engagement, revenue, and customer satisfaction. Microsoft, Amazon, LinkedIn, Google, and others each conduct thousands of A/B tests annually.

The idea of an A/B test is simple: split the population into two groups. Roll out a version of something you want to test (a new web page design, a different font size, a new medicine, a new political ad) to one group, the test group, and keep the other group as a control group. Compare the data between the two groups.

The test is *single blind* if the subjects do not know which group they belong to (some do not even know that they are in a test at all), but the experimenters know. The test is *double blind* if neither the experimenters nor the subjects know which group they are interacting with.

Summary and Looking Ahead

In this chapter, we emphasized the fact that data is central to AI. We also clarified the differences between concepts that are usually a source of confusion: structured and unstructured data, linear and nonlinear models, real and simulated data, deterministic functions and random variables, discrete and continuous distributions, and

posterior probabilities and likelihood functions. We also provided a map for the probability and statistics needed for AI without diving into any of the details, and we introduced the most popular probability distributions.

If you find yourself lost in some new probability concept, you might want to consult the map provided in this chapter and see how that concept fits within the big picture of probability theory, and most importantly, how it relates to AI. Without knowing how a particular mathematical concept relates to AI, you are left with having some tool that you know how to turn on, but you have no idea what it is used for.

We have not yet mentioned *random matrices* and *high-dimensional probability*. In these fields, probability theory, with its constant tracking of distributions, expectations, and variances of any relevant random quantities, merges with linear algebra, with its hyperfocus on eigenvalues and various matrix decompositions. These fields are very important for the extremely high-dimensional data that is involved in AI applications. We discuss them in [Chapter 11](#) on probability.

In the next chapter, we learn how to fit our data into a function, then use this function to make predictions and/or decisions. Mathematically, we find the weights (the ω 's) that characterize the strengths of various interactions between the features of our data. When we characterize the involved types of interactions (the formula of the fitting function, called the *learning* or *training* function) and the strengths of these interactions (the values of the ω 's), we can make our predictions. In AI, this one concept of *characterizing the fitting function with its suitable weight values* can be used successfully for computer vision, natural language processing, predictive analytics (like house prices, time until maintenance, etc.), and many other applications.

Fitting Functions to Data

Today it fits. Tomorrow?

—H.

In this chapter, we introduce the core mathematical ideas lying at the heart of many AI applications, including the mathematical engines of neural networks. Our goal is to internalize the following structure of the machine learning part of an AI problem:

Identify the problem

The problem depends on the specific use case: classify images, classify documents, predict house prices, detect fraud or anomalies, recommend the next product, predict the likelihood of a criminal reoffending, predict the internal structure of a building given external images, convert speech to text, generate audio, generate images, generate video, etc.

Acquire the appropriate data

This is about training our models to do the right thing. We say that our models *learn* from the data. Make sure this data is clean, complete, and if necessary, depending on the specific model we are implementing, transformed (normalized, standardized, some features aggregated, etc.). This step is usually way more time-consuming than implementing and training the machine learning models.

Create a hypothesis function

We use the terms hypothesis function, *learning function*, *prediction function*, *training function*, and *model* interchangeably. Our main assumption is that this input/output mathematical function explains the observed data, and it can be used later to make predictions on new data. We give our model features, like a person's daily habits, and it returns a prediction, like this person's likelihood to pay back a loan. In this chapter, we will give our model the length measurements of a fish, and it will return its weight.

Find the numerical values of weights

We will encounter many models (including neural networks) where our training function has unknown parameters called *weights*. The goal is to find the numerical values of these weights using the data. After we find these weight values, we can use the trained function to make predictions by plugging the features of a new data point into the formula of the trained function.

Create an error function

To find the values of the unknown weights, we create another function called the *error function*, *cost function*, *objective function*, or *loss function* (everything in the AI field has three or more names). This function has to measure some sort of distance between the ground truth and our predictions. Naturally, we want our predictions to be as close to the ground truth as possible, so we search for weight values that minimize our loss function. Mathematically, we solve a minimization problem. The field of *mathematical optimization* is essential to AI.

Decide on mathematical formulas

Throughout this process, we are the engineers, so we get to decide on the mathematical formulas for training functions, loss functions, optimization methods, and computer implementations. Different engineers decide on different processes, with different performance results, and that is OK. The judge, in the end, is the performance of the deployed model, and contrary to popular belief, mathematical models are flexible and can be tweaked and altered when needed. It is crucial to monitor performance after deployment.

Find a way to search for minimizers

Since our goal is to find the weight values that minimize the error between our predictions and ground truths, we need to find an efficient mathematical way to search for these *minimizers*: those special weight values that produce the least error. The *gradient descent* method plays a key role here. This powerful yet simple method involves calculating *one derivative* of our error function. This is one reason we spent half of our calculus classes calculating derivatives (and the gradient: this is one derivative in higher dimensions). There are other methods that require computing two derivatives. We will encounter them and comment on the benefits and the downsides of using higher-order methods.

Use the backpropagation algorithm

When data sets are enormous and our model happens to be a layered neural network, we need an efficient way to calculate this one derivative. The *backpropagation algorithm* steps in at this point. We will walk through gradient descent and backpropagation in [Chapter 4](#).

Regularize a function

If our learning function fits the given data too well, then it will not perform well on new data. A function with too good of a fit with the data picks up on the noise in the data as well as the signal (for example, the function on the left in [Figure 3-1](#)). We do not want to pick up on noise. This is where *regularization* helps. There are multiple mathematical ways to regularize a function, which means to make it smoother and less oscillatory and erratic. In general, a function that follows the noise in the data oscillates too much. We want more regular functions. We visit regularization techniques in [Chapter 4](#).

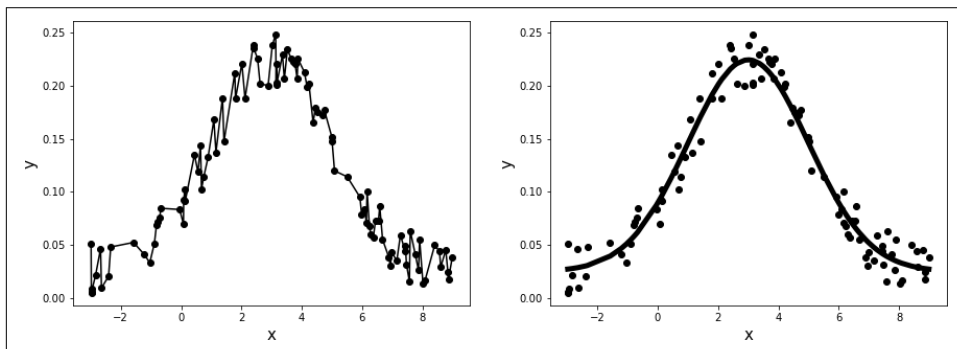


Figure 3-1. Left: the fitting function fits the data perfectly; however, it is not a good prediction function since it fits the noise in the data instead of the main signal. Right: a more regular function fitting the same data set. Using this function will give better predictions than the function in the left subplot, even though the function in the left subplot matches the data points better.

In the following sections, we explore this structure of an AI problem with real, but simple, data sets. We will see in subsequent chapters how the same concepts generalize to much more involved tasks.

Traditional and Very Useful Machine Learning Models

All the data used in this chapter is *labeled* with ground truths, and the goal of our models is to *predict* the labels of new (unseen) and unlabeled data. This is *supervised* learning.

In the next few sections, we fit training functions into our labeled data using the following popular machine learning models. While you may hear so much about the latest and greatest developments in AI, in a typical business setting you are probably better off starting with these more traditional models:

Linear regression

Predict a numerical value.

Logistic regression

Classify into two classes (binary classification).

Softmax regression

Classify into multiple classes.

Support vector machines

Classify into two classes, or regression (predict a numerical value).

Decision trees

Classify into any number of classes, or regression (predict a numerical value).

Random forests

Classify into any number of classes, or regression (predict a numerical value).

Ensembles of models

Bundle up the results of many models by averaging the prediction values, voting for the most popular class, or some other bundling mechanism.

k-means clustering

Classify into any number of classes, or regression

We try multiple models on the same data sets to compare performance. In the real world, it is rare that any model ever gets deployed without having been compared with many other models. This is the nature of the computation-heavy AI industry, and why we need parallel computing, which enables us to train multiple models at once (except for models that build and improve on the results of other models, like in the case of *stacking*; for those we cannot use parallel computing).

Before we dive into any machine learning models, it is extremely important to note that it has been reported again and again that only about 5% of a data scientist's time, and/or an AI researcher's time, is spent on training machine learning models. The majority of the time is consumed by acquiring data, cleaning data, organizing data, creating appropriate pipelines for data, etc., *before* feeding the data into machine learning models. So machine learning is only one step in the production process, and it is an easy step once the data is ready to train the model. We will discover how these machine learning models work: most of the mathematics we need resides in these models. AI researchers are always trying to enhance machine learning models and automatically fit them into production pipelines. It is therefore important for us to eventually learn about the whole pipeline, from raw data (including its storage, hardware, query protocols, etc.) to deployment to monitoring. Learning machine learning is only one piece of a bigger and more interesting story.

We must start with *regression* since the ideas of regression are so fundamental for most of the AI models and applications that will follow. Only for *linear regression* do we find our minimizing weights using an *analytical* method, giving an explicit formula for the desired weights directly in terms of the training data set and its target labels. The simplicity of the linear regression model allows for this explicit analytical solution. Most other models do not have such explicit solutions, and we have to find their minimizers using numerical methods, among which the gradient descent is extremely popular.

In regression and many other upcoming models, including the neural networks of the next few chapters, watch for the following progression in the modeling process:

1. The training function
2. The loss function
3. Optimization

Numerical Solutions Versus Analytical Solutions

It is important to be aware of the differences between numerical solutions and analytical solutions of mathematical problems. A mathematical problem can be anything, such as:

- Find the minimizer of some function.
- Find the best way to go from destination *A* to destination *B*, with a constrained budget.
- Find the best way to design and query a data warehouse.
- Find the solution of a mathematical equation (where a lefthand side with math stuff equals a righthand side with math stuff). These equations could be algebraic equations, ordinary differential equations, partial differential equations, integro-differential equations, systems of equations, or any sort of mathematical equations. Their solutions could be static or evolving in time. They could model anything from the physical, biological, socioeconomic, or natural worlds.

Here is the vocabulary:

Numerical

Has to do with numbers

Analytical

Has to do with analysis

As a rule of thumb, numerical solutions are much easier to obtain and much more accessible than analytical solutions, provided that we have enough computational

power to simulate and compute these solutions. All we usually need to do is discretize some continuous spaces and/or functions (change the continuum into a bunch of points), albeit sometimes in very clever ways, and evaluate functions on these discrete quantities. The only problem with numerical solutions is that they are only approximate solutions. Unless they are backed by estimates on how far off they are from the true analytical solutions and how fast they converge to these true solutions, which in turn requires mathematical background and analysis, numerical solutions are not exact. They do, however, provide incredibly useful insights about the true solutions. In many cases, numerical solutions are the only ones available, and many scientific and engineering fields would not have advanced at all had they not relied on numerical solutions of complex problems. If those fields waited for analytical solutions and proofs to happen, or, in other words, for mathematical theory to catch up, they would have had very slow progress.

Analytical solutions, on the other hand, are exact, robust, and have a whole mathematical theory backing them up. They come accompanied with theorems and proofs. When analytical solutions are available, they are very powerful. They are, however, not easily accessible, sometimes impossible to obtain, and they do require deep knowledge and domain expertise in fields such as calculus, mathematical analysis, algebra, theory of differential equations, etc. Analytical methods, however, are extremely valuable for describing important properties of solutions (even when explicit solutions are not available), guiding numerical techniques, and providing ground truths to compare approximate numerical methods against (in the lucky cases when these analytical solutions are available).

Some researchers are purely analytical and theoretical, others are purely numerical and computational, and the best place to exist is somewhere near the intersection, where we have a decent understanding of the analytical and the numerical aspects of our mathematical problems.

Regression: Predict a Numerical Value

A quick search on the [Kaggle website](#) for data sets for regression returns many excellent data sets and related notebooks. I randomly chose a simple [Fish Market](#) data set that we will use to explain our upcoming mathematics. Our goal is to build a model that predicts the weight of a fish given its five different length measurements, or features, labeled in the data set as Length1, Length2, Length3, Height, and Width (see [Figure 3-2](#)). For the sake of simplicity, we choose not to incorporate the categorical feature, Species, into this model, even though we could (and that would give us better predictions, since a fish's type is a good predictor of its weight). If we choose to include the Species feature, then we would have to convert its values into numerical values, using *one-hot encoding*, which means exactly what it sounds like: assign a code for each fish made up of ones and zeros based on its category (type). Our Species

feature has seven categories: Perch, Bream, Roach, Pike, Smelt, Parkki, and Whitefish. So if our fish is a Pike, we would code its species as (0,0,0,1,0,0,0), and if it is a Bream we would code its species as (0,1,0,0,0,0,0). Of course this adds seven more dimensions to our feature space and seven more weights to train.

	Species	Weight	Length1	Length2	Length3	Height	Width
0	Bream	242.0	23.2	25.4	30.0	11.5200	4.0200
1	Bream	290.0	24.0	26.3	31.2	12.4800	4.3056
2	Bream	340.0	23.9	26.5	31.1	12.3778	4.6961
3	Bream	363.0	26.3	29.0	33.5	12.7300	4.4555
4	Bream	430.0	26.5	29.0	34.0	12.4440	5.1340

Figure 3-2. The first five rows of the Fish Market data set downloaded from [Kaggle](#). The Weight column is the target feature, and our goal is to build a model that predicts the weight of a new fish given its length measurements.

Let's save ink space and relabel our five features as x_1 , x_2 , x_3 , x_4 , and x_5 , then write the fish weight as a function of these five features, $y = f(x_1, x_2, x_3, x_4, x_5)$. This way, once we settle on an acceptable formula for this function, all we have to do is input the feature values for a certain fish and our function will output the predicted weight of that fish.

This section builds a foundation for everything to come, so it is important to first see how it is organized:

Training function

- Parametric models versus nonparametric models.

Loss function

- The predicted value versus the true value.
- The absolute value distance versus the squared distance.
- Functions with singularities (pointy points).
- For linear regression, the loss function is the mean squared error.
- Vectors in this book are always column vectors.
- The training, validation, and test subsets.
- When the training data has highly correlated features.

Optimization

- Convex landscapes versus nonconvex landscapes.
- How do we locate minimizers of functions?
- Calculus in a nutshell.
- A one-dimensional optimization example.
- Derivatives of linear algebra expressions that we use all the time.
- Minimizing the mean squared error loss function.
- Caution: multiplying large matrices by each other is very expensive—multiply matrices by vectors instead.
- Caution: we never want to fit the training data too well.

Training Function

A quick exploration of the data, as in plotting the weight against the various length features, allows us to assume a linear model (even though a nonlinear one could be better in this case). That is, we assume that the weight depends linearly on the length features (see [Figure 3-3](#)).

This means that the weight of a fish, y , can be computed using a *linear combination* of its five different length measurements, plus a bias term ω_0 , giving the following *training function*:

$$y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \omega_4 x_4 + \omega_5 x_5$$

After our major decision in the modeling process to use a linear training function $f(x_1, x_2, x_3, x_4, x_5)$, all we have to do is to find the appropriate values of the parameters $\omega_0, \omega_1, \omega_2, \omega_3, \omega_4$, and ω_5 . We will *learn* the best values for our ω 's from the data. The process of using the data to find the appropriate ω 's is called *training* the model. A *trained* model is then a model where the ω values have been decided on.

In general, training functions, whether linear or nonlinear, including those representing neural networks, have unknown parameters, ω 's, that we need to learn from the given data. For linear models, each parameter gives each feature a certain weight in the prediction process. So if the value of ω_2 is larger than the value of ω_5 , then the second feature plays a more important role than the fifth feature in our prediction, assuming that the second and fifth features have comparable scales. This is one of the reasons it is good to scale or normalize the data before training the model. If, on the other hand, the value ω_3 associated with the third feature dies, meaning becomes zero or negligible, then the third feature can be omitted from the data set, as it plays no role in our predictions. Therefore, learning our ω 's from the data allows us to mathematically compute the contribution of each feature to our predictions (or the

importance of feature combinations if some features were combined during the data preparation stage, before training). In other words, the models learn how the data features interact and how strong these interactions are. The moral is that through a trained learning function, we can quantify how features come together to produce both observed and yet-to-be observed results.

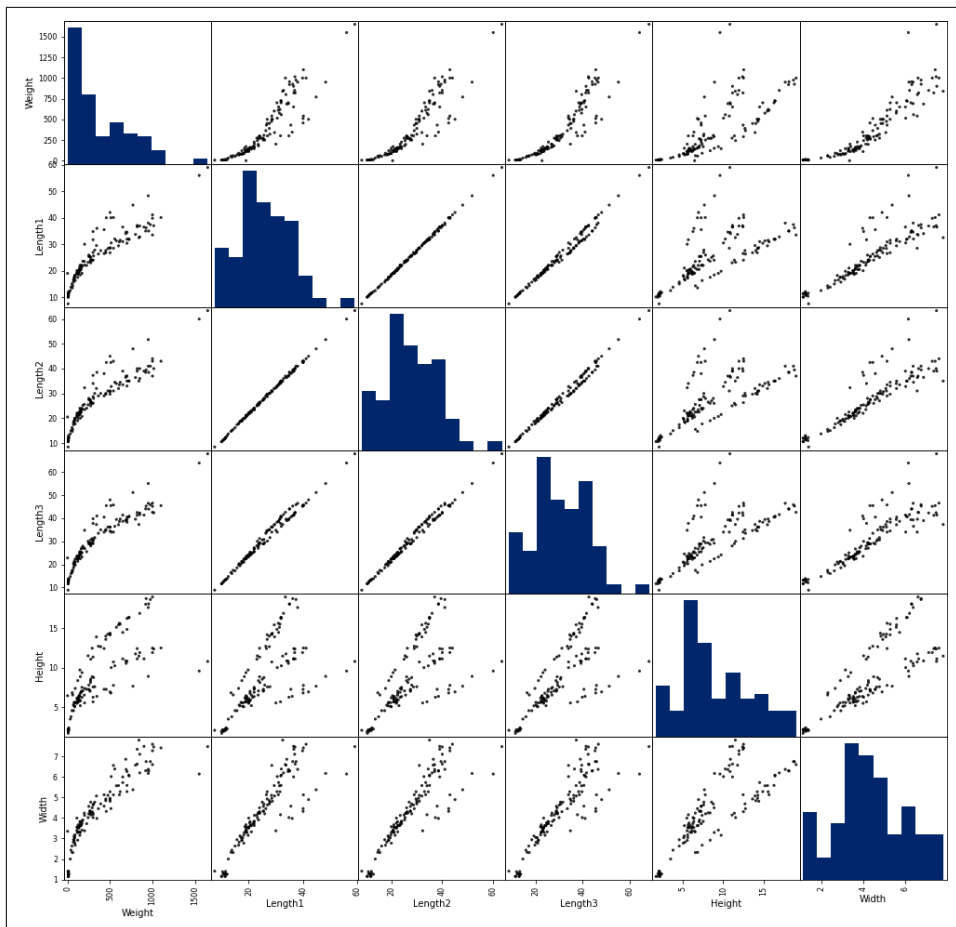


Figure 3-3. Scatterplots of the Fish Market numerical features. For more details, see the [book's GitHub page](#), or some of the public [notebooks on Kaggle](#) associated with this data set.

Parametric Models Versus Nonparametric Models

A model that has parameters (we are calling them weights) pre-built into its formula, such as the ω 's in our current linear regression model:

$$y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \omega_4 x_4 + \omega_5 x_5$$

(and later the ω 's of neural networks) is called a *parametric model*. This means that we fix the formula of the training function ahead of the actual training, and all the training does is solve for the parameters that are involved in the formula. Fixing the formula ahead of time is analogous to specifying the *family* that a training function belongs to, and finding the parameter values specifies the exact member of that family that best explains the data.

Nonparametric models, such as decision trees and random forests that we will discuss later in this chapter, do not specify the formula for the training function with its parameters ahead of time. So when we train a nonparametric model, we do not know how many parameters the trained model will end up having. The model *adapts* to the data and determines the required amount of parameters depending on the data. Careful here, the bells of overfitting are ringing! Recall that we don't want our models to adapt to the data too much, because they might not generalize well to unseen data. These models are usually accompanied with techniques that help them avoid overfitting.

Both parametric and nonparametric models have other parameters called *hyperparameters* that also need to be tuned during the training process. These, however, are not built into the formula of the training function (and don't end up in the formula of a nonparametric model either). We will encounter plenty of hyperparameters throughout the book.

Loss Function

We have convinced ourselves that the next logical step is finding suitable values for the ω 's that appear in the training function (of our linear parametric model), using the data that we have. To do that, we need to *optimize an appropriate loss function*.

The predicted value versus the true value

Suppose we assign some random numerical values for each of our unknown ω_0 , ω_1 , ω_2 , ω_3 , ω_4 , and ω_5 —say, for example, $\omega_0 = -3$, $\omega_1 = 4$, $\omega_2 = 0.2$, $\omega_3 = 0.03$, $\omega_4 = 0.4$, and $\omega_5 = 0.5$. Then the formula for the linear training function $y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \omega_4 x_4 + \omega_5 x_5$ becomes:

$$y = -3 + 4x_1 + 0.2x_2 + 0.03x_3 + 0.4x_4 + 0.5x_5$$

and is ready to make predictions. Plug in numerical values for the length features of the i^{th} fish, then obtain a predicted value for the weight of this fish. For example, the first fish in our data set is a bream and has length measurements $x_1^1 = 23.2$, $x_2^1 = 25.4$, $x_3^1 = 30$, $x_4^1 = 11.52$, and $x_5^1 = 4.02$. Plugging these into the training function, we get the prediction for the weight of this fish:

$$\begin{aligned} y_{predict}^1 &= \omega_0 + \omega_1 x_1^1 + \omega_2 x_2^1 + \omega_3 x_3^1 + \omega_4 x_4^1 + \omega_5 x_5^1 \\ &= -3 + 4(23.2) + 0.2(25.4) + 0.03(30) + 0.4(11.52) + 0.5(4.02) \\ &= 102.398 \text{ grams} \end{aligned}$$

In general, for the i^{th} fish, we have:

$$y_{predict}^i = \omega_0 + \omega_1 x_1^i + \omega_2 x_2^i + \omega_3 x_3^i + \omega_4 x_4^i + \omega_5 x_5^i$$

The fish under consideration, however, has a certain *true* weight, y_{true}^i , which is its label if it belongs in the labeled data set. For the first fish in our data set, the true weight is $y_{true}^1 = 242$ grams. Our linear model with randomly chosen ω values predicted 102.398 grams. This is of course pretty far off, since we did not calibrate the ω values at all. In any case, we can measure the error between the weight predicted by our model and the true weight, then find ways to do better in terms of our choices for the ω 's.

The absolute value distance versus the squared distance

One of the nice things about mathematics is that it has multiple ways to measure how far things are from each other, using different distance metrics. For example, we can naively measure the distance between two quantities as being 1 if they are different, and 0 if they are the same, encoding the words: different-1, similar-0. Of course, using such a naive metric, we lose a ton of information, since the distance between quantities such as 2 and 10 will be equal to the distance between 2 and 1 million, namely 1.

There are some distance metrics that are popular in machine learning. We first introduce the two most commonly used:

- The absolute value distance: $|y_{predict} - y_{true}|$, stemming from the calculus function $|x|$.
- The squared distance: $|y_{predict} - y_{true}|^2$, stemming from the calculus function $|x|^2$ (which is the same as x^2 for scalar quantities). Of course, this will square the units as well.

Inspecting the graphs of the functions $|x|$ and x^2 in Figure 3-4, we notice a great difference in function smoothness at the point $(0,0)$. The function $|x|$ has a corner at that point, rendering it undifferentiable at $x = 0$. This *singularity* of $|x|$ at $x = 0$ turns many practitioners (and mathematicians!) away from incorporating this function, or functions with similar singularities, into their models. However, let's engrave the following into our brains.



Mathematical models are flexible. When we encounter a hurdle we dig deeper, understand what's going on, then we work around the hurdle.

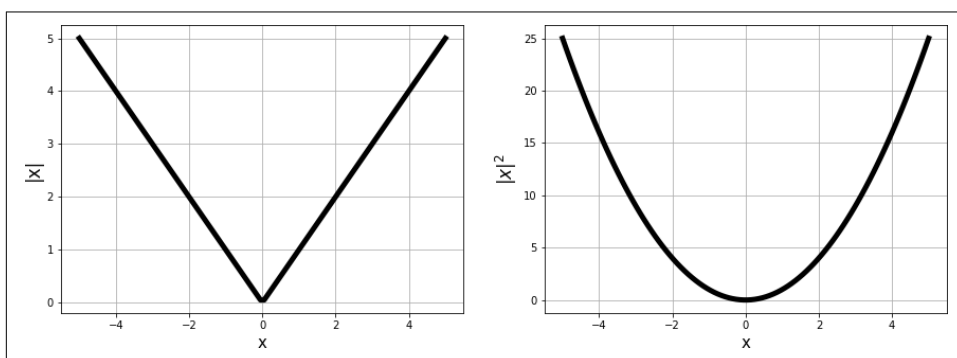


Figure 3-4. Left: graph of $|x|$ has a corner at $x = 0$, rendering its derivative undefined at that point. Right: graph of $|x|^2$ is smooth at $x = 0$, so its derivative has no problems there.

Other than the difference in the *regularity* of the functions $|x|$ and $|x|^2$ (meaning whether they have derivatives at all points or not), there is one more point that we need to pay attention to before deciding to incorporate either function into our error formula: *if a number is large, then its square is even larger*. This simple observation means that if we decide to measure the error using squared distances between true values and predicted values, then our method will be *more sensitive to the outliers* in the data. One messed-up outlier might skew our whole prediction function toward it, and hence away from the more prevalent patterns in the data. Ideally, we would have taken care of outliers and decided whether we should keep them or not during the data preparation step, before feeding the data into any machine learning model.

One last difference between $|x|$ (and similar piecewise linear functions) and x^2 (and similar nonlinear but differentiable functions) is that the derivative of $|x|$ is very easy:

1 if $x > 0$, -1 if $x < 0$ (and undefined if $x = 0$)

In a model that involves billions of computational steps, this property where there is *no need to evaluate anything* when using the derivative of $|x|$ proves to be extremely valuable. Derivatives of functions that are neither linear nor piecewise linear usually involve evaluations (because they also have x 's in their formulas and not only constants, like in the piecewise linear case), which can be expensive in big data settings.

Functions with singularities

In general, graphs of *differentiable* functions do not have cusps, kinks, corners, or anything pointy. If they do have such *singularities*, then the function has no derivative at these points. The reason is that at a pointy point, you can draw two different tangent lines to the graph of the function, depending on whether you decide to draw the tangent line to the left or to the right of the point (see Figure 3-5). Recall that the derivative of a function at a point is the slope of the tangent line to the graph of the function at that point. If there are two *different* slopes, then we cannot define the derivative at the point.

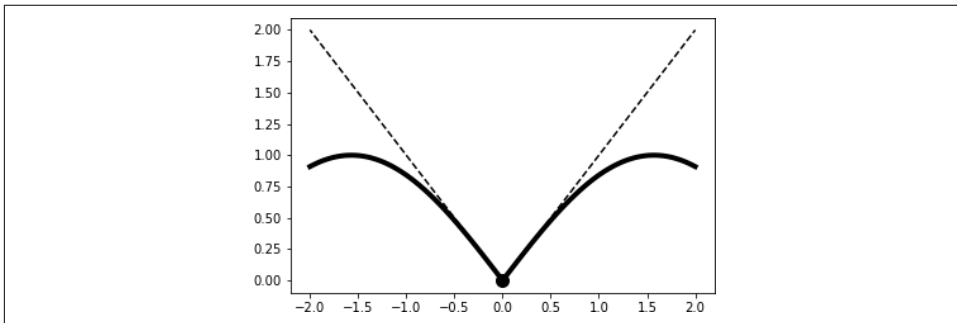


Figure 3-5. At singular points, the derivative does not exist. There is more than one possible slope of the tangent at such points.

This discontinuity in the slope of the tangent creates a problem for methods that rely on evaluating the derivative of the function, such as the gradient descent method. The problem here is twofold:

Undefined derivative

What derivative value should you use? If you happen to land at a quirky pointy point, then the method doesn't know what to do, since there is no defined derivative there. Some people assign a value for the derivative at that point (called the *subgradient* or the *subdifferential*) and move on. In reality, what are the odds that we will be unlucky enough to land exactly at that one horrible point? Unless the landscape of the function looks like the rough terrain of the Alps (actually, many do), the numerical method might manage to avoid them.

Instability

The other problem is instability. Since the value of the derivative jumps so abruptly as you traverse the landscape of the function across this point, a method using this derivative will abruptly change value as well, creating instabilities if you are trying to converge somewhere. Imagine you are hiking down the Swiss Alps in [Figure 3-6](#) (the landscape of the loss function) and your destination is that pretty little town that you can see down in the valley (the place with the lowest error value). Then suddenly you get carried away by some alien (the alien is the mathematical search method relying on this abruptly changing derivative) to the *other* side of the mountain, where you cannot see your destination anymore. In fact, now all you can see in the valley are some ugly shrubs and an extremely narrow canyon that can trap you if your method carries you there. Your convergence to your original destination is now unstable, if not totally lost.



Figure 3-6. Swiss Alps: optimization is similar to hiking the landscape of a function

Nevertheless, functions with such singularities are used all the time in machine learning. We will encounter them in the formulas of some neural network training functions (rectified linear unit function—who names these?), in some loss functions (absolute value distance), and in some regularizing terms (lasso regression—who names these too?).

For linear regression, the loss function is the mean squared error

Back to the main goal for this section: constructing an error function, also called the *loss function*, which encodes how much error our model commits when making its predictions, and must be made small.

For linear regression, we use the *mean squared error function*. This function averages over the squared distance errors between the prediction and the true value for m data points (we will mention which data points to include here shortly):

$$\text{Mean Squared Error} = \frac{1}{m} \left(\left| y_{\text{predict}}^1 - y_{\text{true}}^1 \right|^2 + \left| y_{\text{predict}}^2 - y_{\text{true}}^2 \right|^2 + \cdots + \left| y_{\text{predict}}^m - y_{\text{true}}^m \right|^2 \right)$$

Let's write the above expression more compactly using the sum notation:

$$\text{Mean Squared Error} = \frac{1}{m} \sum_{i=1}^m \left| y_{\text{predict}}^i - y_{\text{true}}^i \right|^2$$

Now we get into the great habit of using the even more compact linear algebra notation of vectors and matrices. This habit proves extremely handy in the field, as we don't want to drown while trying to keep track of indices. Indices can sneak up into our rosy dreams of understanding everything and quickly transform them into very scary nightmares. Another important reason to use the compact linear algebra notation is that both the software and the hardware built for machine learning models are optimized for matrix and *tensor* (think of an object made of layered matrices, like a three-dimensional box instead of a flat square) computations. Moreover, the beautiful field of numerical linear algebra has worked through many potential problems and paved the way for us to enjoy the fast methods to perform all kinds of matrix computations.

Using linear algebra notation, we can write the mean squared error as:

$$\begin{aligned} \text{Mean Squared Error} &= \frac{1}{m} \left(\vec{y}_{\text{predict}} - \vec{y}_{\text{true}} \right)^t \left(\vec{y}_{\text{predict}} - \vec{y}_{\text{true}} \right) \\ &= \frac{1}{m} \left\| \vec{y}_{\text{predict}} - \vec{y}_{\text{true}} \right\|_2^2 \end{aligned}$$

The last equality introduces the ℓ^2 norm of a vector, which by definition is just the $\sqrt{\text{sum of squares of its components}}$.

Take-home idea: *the loss function that we constructed encodes the difference between the predictions and the ground truths for the data points involved the training process, measured in some norm—a mathematical entity that acts as a distance*. There are many other norms that we could have used, but the ℓ^2 norm is pretty popular.

Notation: Vectors in this book are always column vectors

To be consistent in notation throughout the book, *all* vectors are column vectors. So

if a vector $\vec{v}$ has four components, the symbol $\vec{v}$ stands for $\begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix}$.

The transpose of a vector $\vec{v}$ is, then, always a row vector. The transpose of the above vector with four components is $\vec{v}^t = (v_1 \ v_2 \ v_3 \ v_4)$.

We might also use the dot product notation (also called the scalar product because we multiply two vectors but our answer is a scalar number). The dot product of two vectors $\vec{a} \cdot \vec{b}$ is the same thing as $\vec{a}^t \vec{b}$. In essence, $\vec{a}^t \vec{b}$ thinks of a column vector as a matrix of shape, *length of the vector by 1*, and its transpose as a matrix of shape, *1 by length of the vector*.

Suppose now that $\vec{a}$ and $\vec{b}$ have four components, then:

$$\vec{a}^t \vec{b} = (a_1 \ a_2 \ a_3 \ a_4) \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3 + a_4 b_4 = \sum_{i=1}^4 a_i b_i$$

Moreover,

$$\|\vec{a}\|_l^2 = \vec{a}^t \vec{a} = a_1^2 + a_2^2 + a_3^2 + a_4^2$$

Similarly,

$$\|\vec{b}\|_l^2 = \vec{b}^t \vec{b} = b_1^2 + b_2^2 + b_3^2 + b_4^2$$

This way, we use matrix notation throughout, and we only put an arrow above a letter to indicate that we are dealing with a column vector.

The training, validation, and test subsets

Which data points do we include in our loss function? Do we include the whole data set, some small batches of it, or even only one point? Are we measuring this mean

squared error for the data points in the *training subset*, the *validation subset*, or the *test subset*? And what are these subsets anyway?

In practice, we split a data set into three subsets:

Training subset

This is the subset of the data that we use to fit our training function. This means that the data points in this subset are the ones that get incorporated into our loss function (by plugging their feature values and label into the $y_{predict}$ and the y_{true} of the loss function).

Validation subset

The data points in this subset are used in multiple ways:

- The common description is that we use this subset to *tune the hyperparameters of the machine learning model*. The hyperparameters are any parameters in the machine learning model that *are not* the ω 's of the training function that we are trying to solve for. In machine learning, there are many of these, and their values affect the results and the performance of the model. Examples of hyperparameters include (you don't have to know what these are yet): the learning rate that appears in the gradient descent method; the hyperparameter that determines the width of the margin in support vector machine methods; the percentage of original data split into training, validation, and test subsets; the batch size when doing randomized batch gradient descent; weight decay hyperparameters such as those used in ridge, lasso, and elastic net regression; hyperparameters that come with momentum methods such as gradient descent with momentum and ADAM (these have terms that accelerate the convergence of the method toward the minimum, and these terms are multiplied by hyperparameters that need to be tuned before testing and deployment); the number of *epochs* during the optimization process (the number of passes over the entire training subset that the optimizer has seen); and the architecture of a neural network (such as the number of layers, the width of each layer, etc.).
- The validation subset also helps us know when to stop optimizing *before* overfitting our training subset.
- It also serves as a test set to compare the performance of different machine learning models on the same data set, for example, comparing the performance of a linear regression model to a random forest to a neural network.

Test subset

After deciding on the best model to use (or averaging or aggregating the results of multiple models) and training the model, we use this untouched subset of the data as a last-stage test for our model before deployment into the real world. Since the model has not seen any of the data points in this subset before (which

means it has not included any of them in the optimization process), it can be considered as the closest analog to a real-world situation. This allows us to judge the performance of our model before we start applying it to completely new real-world data.

Recap

Let's recap a little before moving forward:

- Our current machine learning model is called *linear regression*.
- Our training function is linear with the formula:

$$y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \omega_4 x_4 + \omega_5 x_5$$

The x 's are the features, and the ω 's are the unknown weights or parameters.

- If we plug in the feature values of a particular data point—for example, the tenth data point—into the formula of the training function, we get our model's prediction for this point:

$$y_{predict}^{10} = \omega_0 + \omega_1 x_1^{10} + \omega_2 x_2^{10} + \omega_3 x_3^{10} + \omega_4 x_4^{10} + \omega_5 x_5^{10}$$

The superscript 10 indicates that these are values corresponding to the tenth data point.

- Our loss function is the mean squared error function with the formula:

$$\begin{aligned} \text{Mean Squared Error} &= \frac{1}{m} \left(\vec{y}_{predict} - \vec{y}_{true} \right)^t \left(\vec{y}_{predict} - \vec{y}_{true} \right) \\ &= \frac{1}{m} \left\| \vec{y}_{predict} - \vec{y}_{true} \right\|_2^2 \end{aligned}$$

- We want to find the values of the ω 's that minimize this loss function. So the next step must be solving a minimization (optimization) problem.

To make our optimization life much easier, we will once again employ the convenient notation of linear algebra (vectors and matrices). This allows us to include the entire training subset of the data as a matrix in the formula of the loss function, and do our computations immediately on the training subset, as opposed to computing on each data point separately. This little notation maneuver saves us from a lot of mistakes, pain, and tedious calculations with many components that are difficult to keep track of on very large data sets.

First, write the prediction of our model corresponding to each data point of the training subset:

$$\begin{aligned} y_{predict}^1 &= 1\omega_0 + \omega_1 x_1^1 + \omega_2 x_2^1 + \omega_3 x_3^1 + \omega_4 x_4^1 + \omega_5 x_5^1 \\ y_{predict}^2 &= 1\omega_0 + \omega_1 x_1^2 + \omega_2 x_2^2 + \omega_3 x_3^2 + \omega_4 x_4^2 + \omega_5 x_5^2 \\ &\vdots \\ y_{predict}^m &= 1\omega_0 + \omega_1 x_1^m + \omega_2 x_2^m + \omega_3 x_3^m + \omega_4 x_4^m + \omega_5 x_5^m \end{aligned}$$

We can easily arrange this system as:

$$\begin{pmatrix} y_{predict}^1 \\ y_{predict}^2 \\ \vdots \\ y_{predict}^m \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \omega_0 + \begin{pmatrix} x_1^1 \\ x_1^2 \\ \vdots \\ x_1^m \end{pmatrix} \omega_1 + \begin{pmatrix} x_2^1 \\ x_2^2 \\ \vdots \\ x_2^m \end{pmatrix} \omega_2 + \begin{pmatrix} x_3^1 \\ x_3^2 \\ \vdots \\ x_3^m \end{pmatrix} \omega_3 + \begin{pmatrix} x_4^1 \\ x_4^2 \\ \vdots \\ x_4^m \end{pmatrix} \omega_4 + \begin{pmatrix} x_5^1 \\ x_5^2 \\ \vdots \\ x_5^m \end{pmatrix} \omega_5$$

Or even better:

$$\begin{pmatrix} y_{predict}^1 \\ y_{predict}^2 \\ \vdots \\ y_{predict}^m \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_2^1 & x_3^1 & x_4^1 & x_5^1 \\ 1 & x_1^2 & x_2^2 & x_3^2 & x_4^2 & x_5^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^m & x_2^m & x_3^m & x_4^m & x_5^m \end{pmatrix} \begin{pmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \\ \omega_5 \end{pmatrix}$$

The vector on the lefthand side of that equation is $\vec{y}_{predict}$, the matrix on the righthand side is the training subset X augmented with the vector of ones, and the last vector on the righthand side has all the unknown weights packed neatly into it. Call this vector $\vec{\omega}$, then write $\vec{y}_{predict}$ compactly in terms of the training subset and $\vec{\omega}$ as:

$$\vec{y}_{predict} = X\vec{\omega}$$

Now the formula of the mean squared error loss function, which we wrote before as:

$$\begin{aligned} \text{Mean Squared Error} &= \frac{1}{m} \left(\vec{y}_{predict} - \vec{y}_{true} \right)^t \left(\vec{y}_{predict} - \vec{y}_{true} \right) \\ &= \frac{1}{m} \left\| \vec{y}_{predict} - \vec{y}_{true} \right\|_2^2 \end{aligned}$$

becomes:

$$\text{Mean Squared Error} = \frac{1}{m} \left(X\vec{\omega} - \vec{y}_{true} \right)^t \left(X\vec{\omega} - \vec{y}_{true} \right) = \frac{1}{m} \| X\vec{\omega} - \vec{y}_{true} \|^2$$

We are now ready to find the $\vec{\omega}$ that minimizes the neatly written loss function. For that, we have to visit the rich and beautiful mathematical field of optimization.

When the training data has highly correlated features

Inspecting the training matrix (augmented with the vector of ones):

$$X = \begin{pmatrix} 1 & x_1^1 & x_2^1 & x_3^1 & x_4^1 & x_5^1 \\ 1 & x_1^2 & x_2^2 & x_3^2 & x_4^2 & x_5^2 \\ \vdots & & & & & \\ 1 & x_1^m & x_2^m & x_3^m & x_4^m & x_5^m \end{pmatrix}$$

that appears in the vector $\vec{y}_{predict} = X\vec{\omega}$, the formula of the mean squared error loss function, and later the formula that determines the unknown $\vec{\omega}$ (also called *the normal equation*):

$$\vec{\omega} = (X^t X)^{-1} X^t \vec{y}_{true}$$

we can see how our model might have a problem if two or more features (x columns) of the data are highly correlated. This means that there is a strong linear relationship between the features, so one of these features can be determined (or nearly determined) using a linear combination of the others. Thus, the corresponding feature columns are *not linearly independent* (or close to not being linearly independent). For matrices, this is a problem, since it indicates that the matrix either cannot be inverted or is *ill conditioned*. Ill-conditioned matrices produce large instabilities in computations, since slight variations in the training data (which must be assumed) produce large variations in the model parameters and hence render its predictions unreliable.

We desire well-conditioned matrices in our computations, so we must get rid of the sources of ill conditioning. When we have highly correlated features, one possible avenue is to include only one of them in our model, as the others do not add much information. Another solution is to apply dimension reduction techniques such as principal component analysis, which we will encounter in [Chapter 11](#). The

Fish Market data set has highly correlated features, and the accompanying Jupyter Notebook addresses those.

That said, it is important to note that some machine learning models, such as decision trees and random forests (discussed soon) are not affected by correlated features, while others, such as the current linear regression model, and the next logistic regression and support vector machine models, are negatively affected by them. As for neural network models, even though they can *learn* the correlations involved in the data features during training, they perform better when these redundancies are taken care of ahead of time, in addition to saving computation cost and time.

Optimization

Optimization means finding the optimal, best, maximal, minimal, or extreme solution.

We wrote a linear training function:

$$y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \omega_4 x_4 + \omega_5 x_5$$

and we left the values of its six parameters ω_0 , ω_1 , ω_2 , ω_3 , ω_4 , and ω_5 unknown. The goal is to find the values that make our training function *best fit the training data subset*, where the word *best* is quantified using the loss function. This function provides a measure of how far the prediction made by the model's training function is from the ground truth. We want this loss function to be small, so we solve a minimization problem.

We are not going to sit there and try out every possible ω value until we find the combination that gives the least loss. Even if we did, we wouldn't know when to stop, since we wouldn't know whether there are other *better* values. We must have prior knowledge about the landscape of the loss function and take advantage of its mathematical properties. The analogy is hiking down the Swiss Alps with a blindfold versus hiking with no blindfold and a detailed map (Figure 3-7 shows the rough terrain of the Swiss Alps). Instead of searching the landscape of the loss function for minimizers with a blindfold, we tap into the field of *optimization*. Optimization is a beautiful branch of mathematics that provides various methods to efficiently search for and locate optimizers of functions and their corresponding optimal values.

The optimization problem in this chapter and in the next few looks like:

$$\min_{\vec{\omega}} \text{loss function}$$



Figure 3-7. Swiss Alps: optimization is similar to hiking the landscape of a function. The destination is the bottom of the lowest valley (minimization) or the top of the highest peak (maximization). We need two things: the coordinates of the minimizing or maximizing points, and the height of the landscape at those points.

For the current linear regression model, this is:

$$\min_{\vec{\omega}} \frac{1}{m} \left(X\vec{\omega} - \vec{y}_{true} \right)^t \left(X\vec{\omega} - \vec{y}_{true} \right) = \min_{\vec{\omega}} \frac{1}{m} \| X\vec{\omega} - \vec{y}_{true} \|^2$$

When we do math, we should never lose track of what it is that we know and what it is that we are looking for. Otherwise we would run the risk of getting trapped in a circular logic. In the formula just mentioned, we know:

m

The number of instances in the training subset

X

The training subset augmented with a vector of ones

$\vec{y}_{true}$

The vector of labels corresponding to the training subset

And we are looking for:

- The minimizing $\vec{\omega}$
- The minimum value of the loss function at the minimizing $\vec{\omega}$

Convex landscapes versus nonconvex landscapes

The *easiest* functions to deal with and the easiest equations to solve are linear. Unfortunately, most of the functions (and equations) that we deal with are nonlinear. At the same time, this is not too unfortunate since linear life is flat, boring, unstimulating, and uneventful. When the function we have at hand is full-blown nonlinear, we sometimes *linearize* it near certain points that we care for. The idea here is that even though the full picture of the function might be nonlinear, we may be able to approximate it with a linear function in the locality that we focus on. In other words, in a very small neighborhood, the nonlinear function might look and behave linearly, albeit the said neighborhood might be infinitesimally small. For an analogy, think about how Earth looks (and behaves in terms of calculating distances, etc.) flatly from our own locality, and how we can only see its nonlinear shape from high up. When we want to linearize a function near a point, we approximate it by its tangent space near that point (this is its tangent line if it is a function of one variable, tangent plane if it is a function of two variables, and tangent hyperplane if it is a function of three or more variables). For this, we need to calculate one derivative of the function with respect to all of its variables, since this gives us the slope (which measures the inclination) of the approximating flat space.

The sad news is that linearizing near one point may not be enough, and we may want to use linear approximations at multiple locations. Thankfully, that is doable, since all we have to do computationally is to evaluate one derivative at several points. This leads us to the *next easiest* functions to deal with (after linear functions): piecewise linear functions, which are linear but only in piecewise structures, or linear except at isolated points or locations. The field of *linear programming* deals with such functions, where the functions to optimize are linear, and the boundaries of the domains where the optimization happens are piecewise linear (they are intersections of half spaces).

When our goal is optimization, the best functions to deal with are either linear (where the field of linear programming helps us) or convex (where we do not worry about getting stuck at local minima, and where we have good inequalities that help us with the analysis).

One important type of function to keep in mind, which appears in machine learning, is a function that is the maximum of two or more convex functions. These functions are always convex. Recall that linear functions are flat, so they are at the same time convex and concave. This is useful since some functions are defined as the maxima of linear functions: those are not guaranteed to be linear (they are piecewise linear), but are guaranteed to be convex. That is, even though we lose linearity when we take the maximum of linear functions, we are compensated with convexity.

The Rectified Linear Unit function (ReLU) that is used as a nonlinear activation function in neural networks is an example of a function defined as the maximum

of two linear functions: $\text{ReLU}(x) = \max(0, x)$. Another example is the hinge loss function used for support vector machines: $H(x) = \max(0, 1 - tx)$ where t is either 1 or -1 .

Note that the minimum of a family of convex functions is not guaranteed to be convex; it can have a double well. However, their maximum is definitely convex.

There is one more relationship between linearity and convexity. If we have a convex function (nonlinear since linear would be trivial), then the maximum of all the linear functions that stay below our function is exactly equal to it. In other words, convexity replaces linearity, in the sense that when linearity is not available, but convexity is, we can replace our convex function with the maximum of all the linear functions whose graph lies below our function's graph (see [Figure 3-8](#)). Recall that the graph of a convex function lies above the graph of its tangent at any point, and that the tangents are linear. This gives us a direct path to exploiting the simplicity of linear functions when we have convex functions. We have equality when we consider the maximum of *all* the tangents, and only approximation when we consider the maximum of the tangents at a few points.

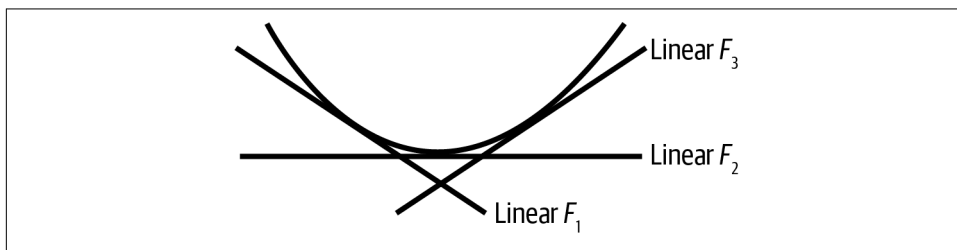


Figure 3-8. A convex function is equal to the maximum of all of its tangents

Figures [3-9](#) and [3-10](#) show the general landscapes of nonlinear convex and nonconvex functions, respectively. Overall, the landscape of a convex function is good for minimization problems. We have no fear of getting stuck at local minima since any local minimum is also a global minimum for a convex function. The landscape of a nonconvex function has peaks, valleys, and saddle points. A minimization problem on such a landscape runs the risk of getting stuck at the local minima and never finding the global minima.

Finally, make sure you know the distinction among a convex function, a convex set, and a convex optimization problem, which optimizes a convex function over a convex set.

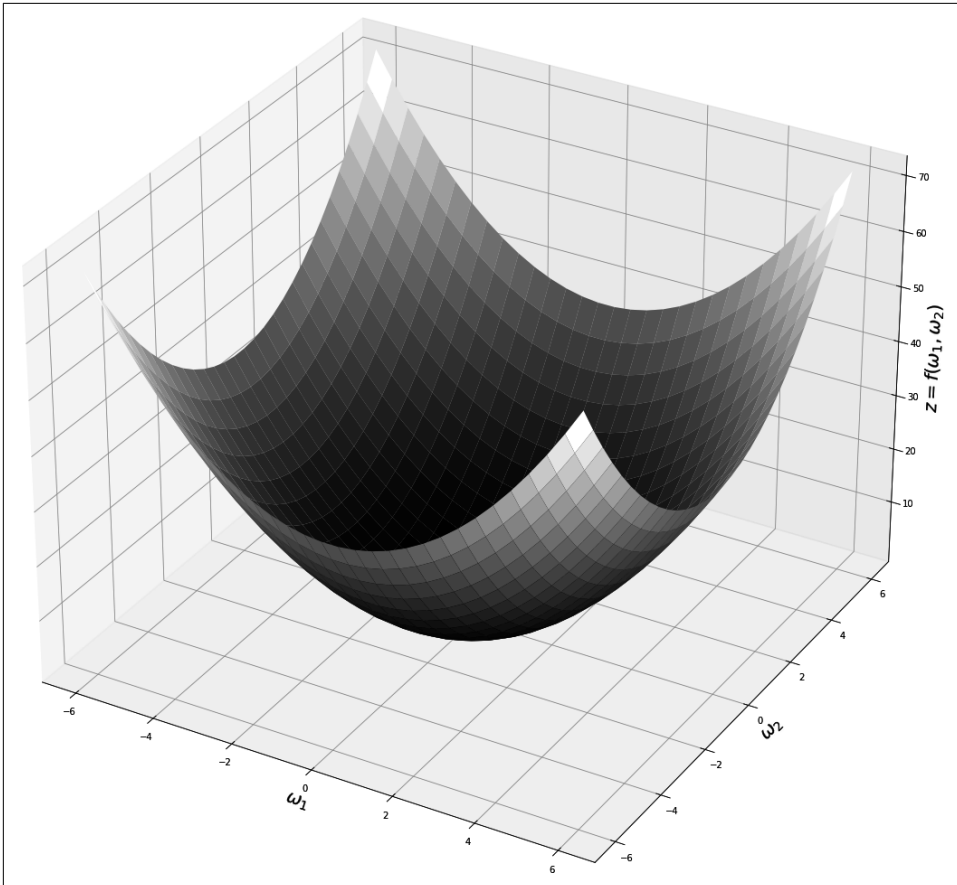


Figure 3-9. The landscape of a convex function is good for minimization problems. We have no fear of getting stuck at the local minima since any local minimum is also a global minimum for a convex function.

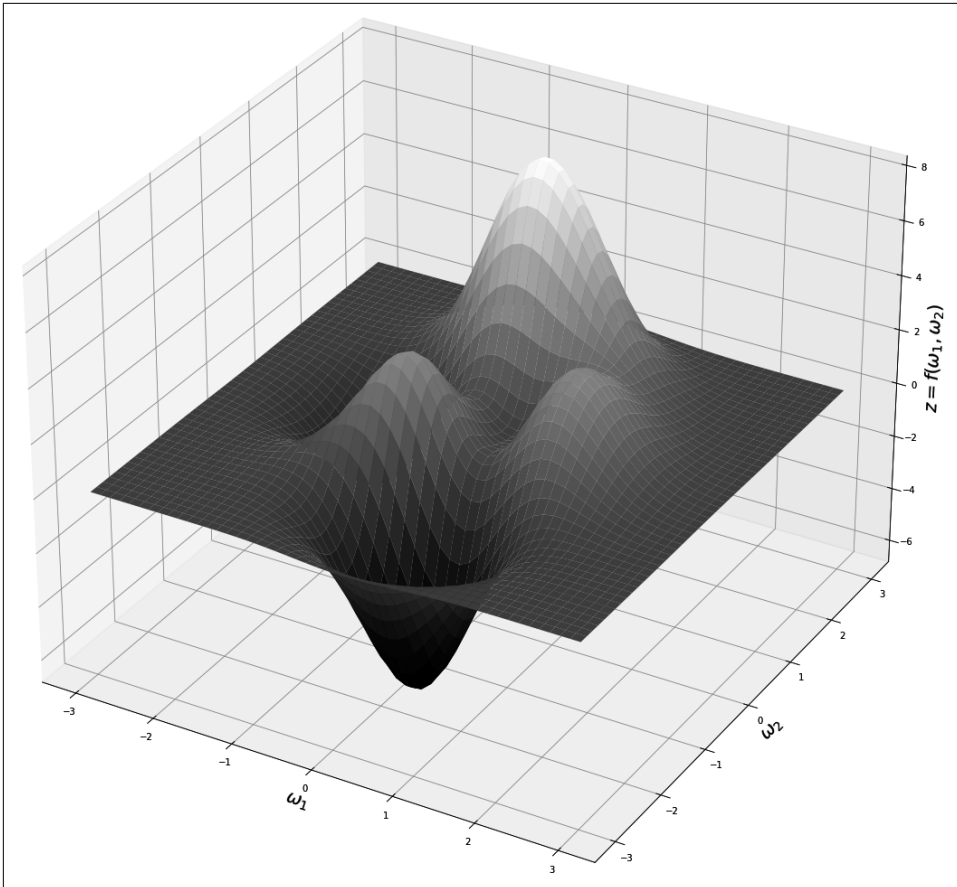


Figure 3-10. The landscape of a nonconvex function has peaks, valleys, and saddle points. A minimization problem on such a landscape runs the risk of getting stuck at a local minima and never finding the global minima.

How do we locate minimizers of functions?

In general, there are two approaches to locating minimizers (and/or maximizers) of functions. The trade-off is usually between:

1. Calculating only one derivative and converging to the minimum slowly (though there are acceleration methods to speed up the convergence). These are called *gradient* methods. The gradient is one derivative of a function of several variables. For example, our loss function is a function of several ω 's (or of one vector $\vec{\omega}$).
2. Calculating two derivatives (computationally much more expensive, which is a big turnoff, especially when we have thousands of parameters) and converging to

the minimum faster. Computation costs can be saved a little by approximating the second derivative instead of computing it exactly. Second derivative methods are called *Newton's* methods. The *Hessian* (the matrix of second derivatives) or an approximation of the Hessian appears in these methods.

We never need to go beyond calculating two derivatives.

But why are the first and second derivatives of a function so important for locating its optimizers? The concise answer is that the first derivative contains information on how fast a function increases or decreases at a point (so if you follow its direction, you might ascend to the maximum or descend to the minimum), and the second derivative contains information on the shape of the *bowl* of the function—if it curves up or curves down.

One key idea from calculus remains fundamental: minimizers (and/or maximizers) happen at *critical points* (defined as the points where one derivative of our function is either equal to zero or does not exist) or at boundary points. So to locate these optimizers, we must search through *both* the boundary points (if our search space has a boundary) *and* the interior critical points.

How do we locate the critical points in the interior of our search space?

Approach 1

We follow these steps:

- Find one derivative of our function (not too bad, we all did it in calculus).
- Set it equal to zero (we can all write the symbols equal and zero).
- Solve for the ω 's that make our derivative zero (this is the bad step!).

For functions whose derivatives are linear, such as our mean squared error loss function, it is sort of easy to solve for these ω 's. The field of linear algebra was especially built to help solve linear systems of equations. The field of numerical linear algebra was built to help solve realistic and large systems of linear equations where ill conditioning is prevalent. We have many tools at our disposal (and software packages) when our systems are linear.

On the other hand, when our equations are nonlinear, finding solutions is an entirely different story. It becomes a hit-or-miss game, with mostly misses! Here's a short example that illustrates the difference between solving a linear and a nonlinear equation:

Solving a linear equation

Find ω such that $0.002\omega - 5 = 0$.

Solution: Moving the 5 over to the other side, then dividing by 0.002, we get $\omega = 5/0.002 = 2500$. Done.

Solving a nonlinear equation

Find ω such that $0.002 \sin(\omega) - 5\omega^2 + e^\omega = 0$.

Solution: Yes, I am out of here. We need a numerical method! (See [Figure 3-11](#) for a graphical approximation of the solution of this nonlinear equation.)

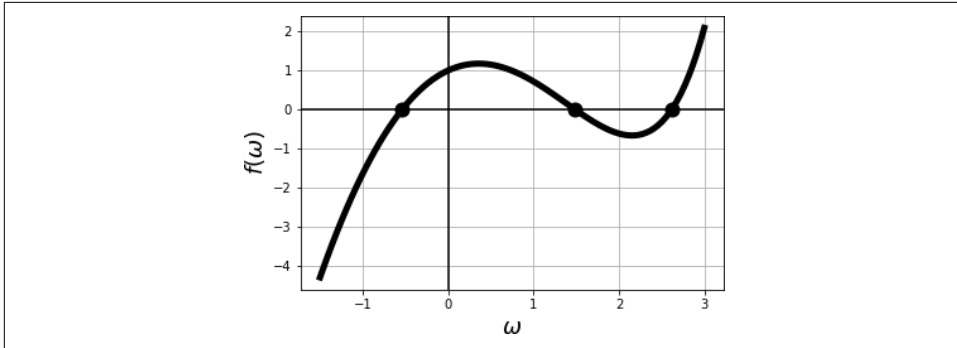


Figure 3-11. It is difficult to solve nonlinear equations. Here, we plot $f(\omega) = 0.002 \sin(\omega) - 5\omega^2 + e^\omega$ and approximate its three roots (points where $f(\omega) = 0$) on the graph.

There are many numerical techniques devoted solely to finding solutions of nonlinear equations (and entire fields devoted to numerically solving nonlinear ordinary and partial differential equations). These methods find approximate solutions, then provide bounds on how far off the numerical solutions are from the exact analytical solutions. They usually construct a sequence that converges to the analytical solution under certain conditions. Some methods converge faster than others, and are better suited for certain problems than others.

Approach 2

Another option is to follow the gradient direction to descend toward the minimum or ascend toward the maximum.

To understand these gradient-type methods, think of hiking down a mountain (or skiing down the mountain if the method is accelerated or has momentum). We start at a random point in our search space, and that sets us at an initial height level on the landscape of the function. Now the method moves us to a new point in the search space, and hopefully, at this new location, we end up at a new height level that is *lower* than the height level we came from. Hence, we would have *descended*. We repeat this and ideally, if the terrain of the function cooperates, this sequence of points will converge toward the minimizer of the function that we are looking for.

Of course, for functions with landscapes that have many peaks and valleys, where we start—or in other words, how to initialize—matters, since we could descend down an

entirely different valley than the one where we want to end up. We might end up at a local minimum instead of a global minimum.

Functions that are convex and bounded below are shaped like a salad bowl, so with those we do not worry about getting stuck at local minima and away from global minima. There could be another source of worry with convex functions: when the shape of the bowl of the function is too narrow, our method might become painfully slow to converge. We will go over this in [Chapter 4](#).

Both Approach 1 and Approach 2 are useful and popular. Sometimes, we have no option but to use one or the other, depending on how fast each method converges for our particular setting, how *regular* the function we are trying to optimize is (how many well-behaved derivatives it has), etc. Other times, it is just a matter of taste. For linear regression's mean squared error loss function, both types of methods work, so we will use Approach 1, only because we will use gradient descent methods for *all* the other loss functions in this book.

We must mention that the hiking-down-the-mountain analogy for descent methods is excellent but a tad bit misleading. When we humans hike down a mountain, we physically belong in the same three-dimensional space that our mountain landscape exists in, meaning we are at a certain elevation and we are able to descend to a location at a lower elevation, even with a blindfold, and even when it is too foggy and we can only descend one tiny step at a time. We sense the elevation, then move downhill. Numerical descent methods, on the other hand, do not search for the minimum in the same space dimension as the one the landscape of the function is embedded in. Instead, they search on the ground level, one dimension *below* the landscape (see [Figure 3-12](#)). This makes descending toward the minimum much more difficult, since at ground level we can move from any point to any other without knowing what height level exists above us, until we evaluate the function itself at the point and find the height. So our method might accidentally move us from one ground point with a certain elevation above us to another ground point with a *higher* elevation, hence farther away from the minimum. This is why it is important to locate, on ground level, a direction that quickly *decreases* the function height, and how far we can move in that direction on ground level (step size) while *still decreasing the function height above us*. The step size is also called the *learning rate hyperparameter*, which we will encounter every time we use a descent method.

Back to our main goal: we want to find the best $\vec{\omega}$ for our training function, so we must minimize our mean squared error loss function using Approach 1: take one derivative of the loss function and set it equal to zero, then solve for the vector $\vec{\omega}$. For this, we need to master doing calculus on linear algebra expressions. Let's revisit our calculus class first.

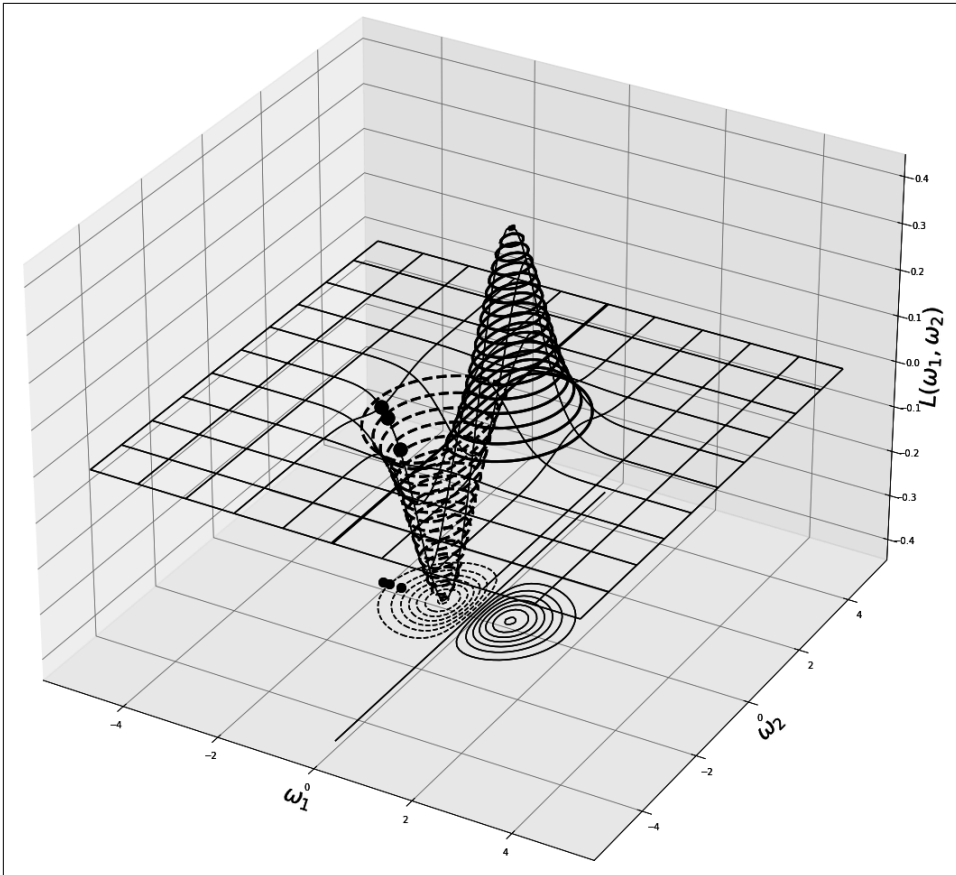


Figure 3-12. Searching for the minimum happens on ground level and not directly on the landscape of the function

Calculus in a nutshell

In a first course on calculus, we learn about functions of single variables ($f(\omega)$), their graphs, and how to evaluate them at certain points. Then we learn about the most important operation in mathematical analysis: the limit. From the limit concept, we define continuity and discontinuity of functions, derivative at a point $f'(\omega)$ (limit of slopes of secants through a point), and integral over a domain (limit of sums of mini regions determined by the function over the domain). We end the class with the fundamental theorem of calculus, relating integration and differentiation as inverse operations. One of the key properties of the derivative is that it determines how fast a function increases or decreases at a certain point; hence, it plays a crucial role in locating a minimum and/or maximum of a function in the interior of its domain (boundary points are separate).

In a multivariable calculus course, which is usually a third course in calculus, many ideas transfer from single-variable calculus, including the importance of the derivative, now called the *gradient* because we have several variables, in locating any interior minima and/or maxima. The gradient $\nabla(f(\vec{\omega}))$ of $f(\vec{\omega})$ is the derivative of the function with respect to the vector $\vec{\omega}$ of variables.

In deep learning, the unknown weights are organized in matrices, not in vectors, so we need to take the derivative of a function $f(W)$ with respect to a matrix W of variables.

For our purposes in AI, the function whose derivative we need to calculate is the loss function, which has the training function built into it. By the *chain rule for derivatives*, we would also need to calculate the derivative of the training function with respect to the ω 's.

Let's demonstrate using one simple example from single-variable calculus, then immediately transition to taking derivatives of linear algebra expressions.

A one-dimensional optimization example

Problem: Find the minimizer(s) and the minimum value (if any) of the function $f(\omega) = 3 + (0.5\omega - 2)^2$ on the interval $[-1, 6]$.

One impossibly long way to go about this is to try out the *infinitely many* values of ω between -1 and 6 , and choose the ω 's that give the lowest f value. Another way is to use our calculus knowledge that optimizers (minimizers and/or maximizers) happen either at critical points (where the derivative is either nonexistent or zero) or at boundary points. For reference, see [Figure 3-13](#).

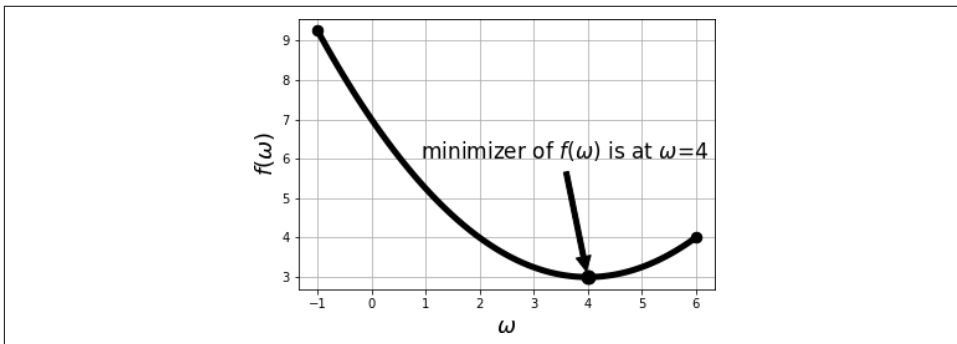


Figure 3-13. The minimum value of the function $f(\omega) = 3 + (0.5\omega - 2)^2$ on the interval $[-1, 6]$ is 3 and happens at the critical point $\omega = 4$. At this critical point, the derivative of the function is zero, meaning that if we draw a tangent line, it will be horizontal.

Our boundary points are -1 and 6 , so we evaluate our function at these points first: $f(-1) = 3 + (0.5(-1) - 2)^2 = 9.25$ and $f(6) = 3 + (0.5(6) - 2)^2 = 4$. Obviously, -1 is not a minimizer since $f(6) < f(-1)$, so this boundary point gets out of the competition and now only boundary point 6 is competing with interior critical point(s). In order to find our critical points, we inspect the derivative of the function in the interior of the interval $[-1, 6]$: $f'(\omega) = 0 + 2(0.5\omega - 2) * 0.5 = 0.25(0.5\omega - 2)$. Setting this derivative equal to zero, we have $0.25(0.5\omega - 2) = 0$, implying that $\omega = 4$. Thus, we only found one critical point $\omega = 4$ in the interior of the interval $[-1, 6]$. At this special point, the value of the function is $f(4) = 3 + (0.5(4) - 2)^2 = 3$. Since the value of f is the lowest here, we have obviously found the winner of our minimization competition, namely, $\omega = 4$ with the minimum f value equal to 3 .

Derivatives of linear algebra expressions that we use all the time

It is efficient to calculate derivatives directly on expressions involving vectors and matrices, without having to resolve them into their components. The following two are popular:

1. When a and ω are scalars and a is constant, the derivative of $f(\omega) = a\omega$ is $f'(\omega) = a$. When $\vec{a}$ and $\vec{\omega}$ are vectors (of the same length) and the entries of $\vec{a}$ are constant, then the gradient of $f(\vec{\omega}) = \vec{a}^t \vec{\omega}$ is $\nabla f(\vec{\omega}) = \vec{a}$. Similarly, the gradient of $f(\vec{\omega}) = \vec{\omega}^t \vec{a}$ is $\nabla f(\vec{\omega}) = \vec{a}$.
2. When s is scalar and constant and ω is scalar, then the derivative of the quadratic function $f(\omega) = s\omega^2$ is $f'(\omega) = 2s\omega$. The analogous high-dimensional case is when S is a symmetric matrix with constant entries, then the function $f(\vec{\omega}) = \vec{\omega}^t S \vec{\omega}$ is quadratic and its gradient is $\nabla f(\vec{\omega}) = 2S\vec{\omega}$.

Minimizing the mean squared error loss function

We are finally ready to minimize the mean squared error loss function:

$$L(\vec{\omega}) = \frac{1}{m} (X\vec{\omega} - \vec{y}_{true})^t (X\vec{\omega} - \vec{y}_{true})$$

Let's open that expression up before taking its gradient and setting it equal to zero:

$$\begin{aligned}
L(\vec{\omega}) &= \frac{1}{m} \left((X\vec{\omega})^t - \vec{y}_{true}^t \right) (X\vec{\omega} - \vec{y}_{true}) \\
&= \frac{1}{m} \left(\vec{\omega}^t X^t - \vec{y}_{true}^t \right) (X\vec{\omega} - \vec{y}_{true}) \\
&= \frac{1}{m} \left(\vec{\omega}^t X^t X\vec{\omega} - \vec{\omega}^t X^t \vec{y}_{true} - \vec{y}_{true}^t X\vec{\omega} + \vec{y}_{true}^t \vec{y}_{true} \right) \\
&= \frac{1}{m} \left(\vec{\omega}^t S \vec{\omega} - \vec{\omega}^t \vec{a} - \vec{a}^t \vec{\omega} + \vec{y}_{true}^t \vec{y}_{true} \right),
\end{aligned}$$

where in the last step we set $X^t X = S$ and $X^t \vec{y}_{true} = \vec{a}$. Next, take the gradient of the last expression with respect to $\vec{\omega}$ and set it equal to zero. When calculating the gradient, we use what we just learned about differentiating linear algebra expressions:

$$\begin{aligned}
\nabla L(\vec{\omega}) &= \frac{1}{m} (2S\vec{\omega} - \vec{a} - \vec{a} + 0) \\
&= \vec{0}
\end{aligned}$$

Now it is easy to solve for $\vec{\omega}$:

$$\frac{1}{m} (2S\vec{\omega} - 2\vec{a}) = \vec{0}$$

so:

$$2S\vec{\omega} = 2\vec{a}$$

which gives:

$$\vec{\omega} = S^{-1} \vec{a}$$

Now recall that we set $S = X^t X$ and $\vec{a} = X^t \vec{y}_{true}$, so let's rewrite our minimizing $\vec{\omega}$ in terms of the training set X (augmented with ones) and the corresponding labels vector $\vec{y}_{true}$:

$$\vec{\omega} = (X^t X)^{-1} X^t \vec{y}_{true}$$

For the Fish Market data set, this ends up being (see the accompanying Jupyter notebook):

$$\vec{\omega} = \begin{pmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \\ \omega_5 \end{pmatrix} = \begin{pmatrix} -475.19929130109716 \\ 82.84970118 \\ -28.85952426 \\ -28.50769512 \\ 29.82981435 \\ 30.97250278 \end{pmatrix}$$



Multiplying Large Matrices by Each Other Is Very Expensive; Multiply Matrices by Vectors Instead

Try to avoid multiplying matrices by each other at all costs; instead, multiply your matrices with *vectors*. For example, in the normal equation $\vec{\omega} = (X^t X)^{-1} X^t \vec{y}_{true}$, compute $X^t \vec{y}_{true}$ first, and avoid computing $(X^t X)^{-1}$ altogether. The way around this is to solve instead the linear system $X \vec{\omega} = \vec{y}_{true}$ using the pseudoinverse of X (check the accompanying Jupyter notebook). We will discuss the pseudoinverse in [Chapter 11](#), but for now, it allows us to invert (which is equivalent to divide by) matrices that do not have an inverse.

We just located the weights vector $\vec{\omega}$ that gives the best fit between our training data and the linear regression training function:

$$f(\vec{\omega}; \vec{x}) = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \omega_4 x_4 + \omega_5 x_5$$

We used an analytical method (compute the gradient of the loss function and set it equal to zero) to derive the solution given by the normal equation. This is one of the very rare instances where we are able to derive an analytical solution. All other methods for finding the minimizing $\vec{\omega}$ will be numerical.



We Never Want to Fit the Training Data Too Well

The $\vec{\omega} = (X^t X)^{-1} X^t \vec{y}_{true}$ that we calculated gives the ω values that make the training function *best fit* the training data, but too good of a fit means that the training function might also be picking up on the noise and not only on the signal in the data. So the solution just mentioned, or even the minimization problem itself, needs to be modified in order to *not get too good of a fit*. *Regularization* or *early stopping* are helpful here. We will spend some time on those in [Chapter 4](#).

This was the long way to regression. We had to pass through calculus and linear algebra on the way, because we are just starting. Presenting the upcoming machine learning models—logistic regression, support vector machines, decision trees, and random forests—will be faster, since all we do is apply the exact same ideas to different functions.

Logistic Regression: Classify into Two Classes

Logistic regression is mainly used for classification tasks. We first explain how we can use this model for binary classification tasks (classify into two classes, such as cancer/not cancer, safe for children/not safe, likely to pay back a loan/unlikely, etc.). Then we will generalize the model into classifying into multiple classes (for example, classify handwritten images of digits into 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9). Again, we have the same mathematical setup:

1. Training function
2. Loss function
3. Optimization

Training Function

Similar to linear regression, the training function for logistic regression computes a linear combination of the features and adds a constant bias term, but instead of outputting the result as is, it passes it through the *logistic function*, whose graph is plotted in [Figure 3-14](#), and whose formula is:

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

This is a function that only takes values between 0 and 1, so its output can be interpreted as a probability of a data point belonging to a certain class. If the output

is less than 0.5, then classify the data point as belonging to the first class, and if the output is greater than 0.5, then classify the data point in the other class. The number 0.5 is the *threshold* where the decision to classify the data point is made.

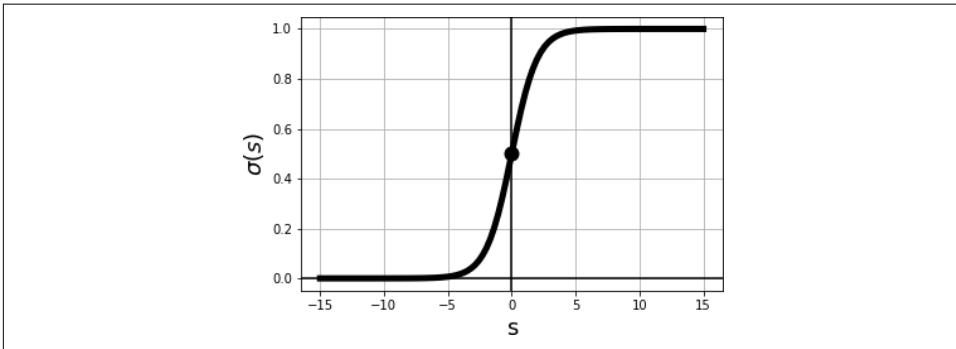


Figure 3-14. Graph of the logistic function $\sigma(s) = \frac{1}{1 + e^{-s}}$. Note that this function can be evaluated at any s and always outputs a number between 0 and 1, hence its output can be interpreted as a probability.

Therefore, the training function here ends up being a linear combination of features, plus bias, composed first with the logistic function, then finally composed with a thresholding function:

$$y = \text{Thresh}(\sigma(\omega_0 + \omega_1 x_1 + \cdots + \omega_n x_n))$$

Similar to the linear regression case, the ω 's are the unknowns for which we need to optimize our loss function. Just like linear regression, the number of these unknowns is equal to the number of data features, plus one for the bias term. For tasks like classifying images, each pixel is a feature, so we could have thousands of those.

Loss Function

Let's design a good *loss function* for classification. We are the engineers and we want to penalize wrongly classified training data points. In our labeled data set, if an instance belongs in a class, then its $y_{true} = 1$, and if it doesn't, then its $y_{true} = 0$.

We want our training function to output $y_{predict} = 1$ for training instances that belong in the positive class (whose y_{true} is also 1). Successful ω values should give a high value of t (result of the linear combination step) to go into the logistic function, hence assigning high probability for positive instances and passing the 0.5 threshold to obtain $y_{predict} = 1$. Therefore, if the linear combination plus bias step gives a low t value while $y_{true} = 1$, penalize it.

Similarly, successful weight values should give a low t value to go into the logistic function for training instances that do not belong in the class (their true $y_{true} = 0$). Therefore, if the linear combination plus bias step gives a high t value while $y_{true} = 0$, penalize it.

So how do we find a loss function that penalizes a wrongly classified training data point? Both false positives and false negatives should be penalized. Recall that the outputs of this classification model are either 1 or 0:

- Think of a calculus function that rewards 1 and penalizes 0: $-\log(s)$ (see [Figure 3-15](#)).
- Think of a calculus function that penalizes 1 and rewards 0: $-\log(1 - s)$ (see [Figure 3-15](#)).

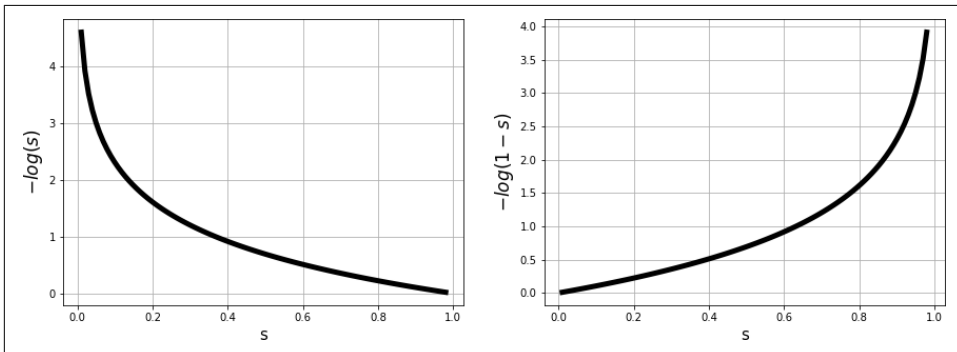


Figure 3-15. Left: graph of the function $f(s) = -\log(s)$. This function assigns high values for numbers near 0 and low values for numbers near 1. Right: graph of the function $f(s) = -\log(1 - s)$. This function assigns high values for numbers near 1 and low values for numbers near 0.

Now focus on the output of the logistic function $\sigma(s)$ for the current choice of ω 's:

- If $\sigma(s)$ is less than 0.5 (model prediction is $y_{predict} = 0$) but the true $y_{true} = 1$ (a false negative), make the model pay by penalizing $-\log(\sigma(s))$. If instead $\sigma(s) > 0.5$, the model prediction is $y_{predict} = 1$ (a true positive), $-\log(\sigma(s))$ is small, so no high penalty is paid.
- Similarly, if $\sigma(s)$ is more than 0.5, but the true $y_{true} = 0$ (a false positive), make the model pay by penalizing $-\log(1 - \sigma(s))$. Again, for a true negative no high penalty is paid either.

Therefore, we can write the cost for misclassifying one training instance $(x_1^i, x_2^i, \dots, x_n^i; y_{true})$ as:

$$\begin{aligned} cost &= \begin{cases} -\log(\sigma(s)) & \text{if } y_{true} = 1 \\ -\log(1 - \sigma(s)) & \text{if } y_{true} = 0 \end{cases} \\ &= -y_{true} \log(\sigma(s)) - (1 - y_{true}) \log(1 - \sigma(s)) \end{aligned}$$

Finally, the loss function is the average cost over m training instances, giving us the formula for the popular *cross-entropy loss function*:

$$\begin{aligned} L(\vec{\omega}) &= -\frac{1}{m} \sum_{i=1}^m y_{true}^i \log(\sigma(\omega_0 + \omega_1 x_1^i + \dots + \omega_n x_n^i)) + \\ &\quad (1 - y_{true}^i) \log(1 - \sigma(\omega_0 + \omega_1 x_1^i + \dots + \omega_n x_n^i)) \end{aligned}$$

Optimization

Unlike the linear regression case, if we decide to minimize the loss function by setting $\nabla L(\omega) = 0$, there is no closed form solution formula for the ω 's. The good news is that this function is convex, so the gradient descent in [Chapter 4](#) (or stochastic or mini-batch gradient descents) is guaranteed to find a minimum (if the *learning rate* is not too large and if we wait long enough).

Softmax Regression: Classify into Multiple Classes

We can easily generalize the logistic regression idea to classify into multiple classes. A famous example for such a nonbinary classification task is classifying images of the 10 handwritten numerals 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9 using the [MNIST data set](#).¹ This data set contains 70,000 images of handwritten numerals (see [Figure 3-16](#) for samples of these images), split into a 60,000-image training subset and a 10,000-image test subset. Each image is labeled with the class it belongs to, which is one of the 10 numerals.

This data set also contains results from many classifying models, including linear classifiers, *k-nearest neighbors*, *decision trees*, *support vector machines* with various *kernels*, and neural networks with various architectures, along with references for the corresponding papers and their years of publication. It is interesting to see the progress in performance as the years go by and the methods evolve.

¹ At the time of writing, this site may request credentials, but you can access it using these instructions from [Hacker News](#).



Figure 3-16. Sample images from the MNIST data set (*image source*)



Do Not Confuse Classifying into Multiple Classes with Multioutput Models

Softmax regression predicts one class at a time, so we cannot use it to classify, for example, five people in the same image. Instead, we can use it to check whether a given Facebook image is a picture of me, my sister, my brother, my husband, or my daughter. An image passed into the softmax regression model can have only one of the five of us, or the model's classification would be less obvious. This means that our classes have to be mutually exclusive. So when Facebook automatically tags five people in the same image, they are using a multioutput model, not a softmax regression model.

Suppose we have the features of a data point and we want to use this information to classify the data point into one of k possible classes. The following training function, loss function, and optimization process should be clear by now.



Features of Image Data

For grayscale images, each pixel intensity is a feature, so images usually have thousands of features. Grayscale images are usually represented as two-dimensional matrices of numbers, with pixel intensities as the matrix entries. Color images come in three channels, red, green, and blue, where each channel is again represented as a two-dimensional matrix of numbers, and the channels are stacked on top of each other, forming three layers of two-dimensional matrices. This structure is called a *tensor*. Check out the notebook on processing images at this [book's GitHub page](#) that illustrates how we can work with grayscale and color images in Python.

Training Function

The first step is always the same: linearly combine the features and add a constant bias term. In logistic regression, when we only had two classes, we passed the result into the logistic function of the formula:

$$\sigma(s) = \frac{1}{1 + e^{-s}} = \frac{1}{1 + \frac{1}{e^s}} = \frac{e^s}{1 + e^s} = \frac{e^s}{e^0 + e^s}$$

which we interpreted as the probability of the data point belonging in the class of interest or not. Note that we rewrote the formula for the logistic function as

$\sigma(s) = \frac{e^s}{e^0 + e^s}$ to highlight the fact that it captures two probabilities, one for each class.

In other words, $\sigma(s)$ gives the probability that a data point is in the class of interest, and $1 - \sigma(s) = \frac{e^0}{e^0 + e^s}$ gives the probability that the data point is not in the class.

When we have multiple classes instead of only two, then for the same data point, we repeat same process multiple times: one time for each class. Each class has its own bias and set of weights that linearly combine the features, thus given a data point with feature values $x_1, x_2, \dots$, and x_n , we compute k different linear combinations plus biases:

$$s^1 = \omega_0^1 + \omega_1^1 x_1 + \omega_2^1 x_2 + \dots + \omega_n^1 x_n$$

$$s^2 = \omega_0^2 + \omega_1^2 x_1 + \omega_2^2 x_2 + \dots + \omega_n^2 x_n$$

$$\vdots$$

$$s^k = \omega_0^k + \omega_1^k x_1 + \omega_2^k x_2 + \dots + \omega_n^k x_n$$



Get into Good Habits

You want to get into the good habit of keeping track of how many unknown ω 's end up in the formula for your training function. Recall that these are the ω 's that we find via minimizing a loss function. The other good habit is having an efficient and consistent way to organize them throughout your model (in a vector, matrix, etc.). In the softmax case, when we have k classes, and n features for each data point, we end up with $k \times n$ ω 's for the linear combinations, then k biases, for a total of $k \times n + k$ unknown ω 's. For example, if we use a softmax regression model to classify images in the **MNIST data set** of handwritten numerals, each image has 28×28 pixels, meaning 784 features, and we want to classify them into 10 classes, so we end up having to optimize for 7850 ω 's. For both the linear and logistic regression models, we only had $n + 1$ unknown ω 's that we needed to optimize for.

Next we pass each of these k results into the *softmax function*, which generalizes the logistic function from two to multiple classes, and we also interpret it as a probability. The formula for the softmax function looks like:

$$\sigma(s^j) = \frac{e^{s^j}}{e^{s^1} + e^{s^2} + \dots + e^{s^k}}$$

This way, the same data point will get k probability scores, one score corresponding to each class. Finally, we classify the data point as belonging to the class where it obtained the largest probability score.

Aggregating all of the above, we obtain the final formula of the training function that we can now use for classification (that is, after we find the optimal ω values by minimizing an appropriate loss function):

$$y = j \text{ such that } \sigma(\omega_0^j + \omega_1^j x_1 + \dots + \omega_n^j x_n) \text{ is maximal}$$

Note that for this training function, all we have to do is input the data features (the x values), and it returns one class number: j .



The Logistic and Softmax Functions and Statistical Mechanics

If you are familiar with statistical mechanics, you might have noticed that the logistic and softmax functions calculate probabilities in the same way the *partition function* from the field of statistical mechanics calculates the probability of finding a system in a certain state.

Loss Function

We derived the cross-entropy loss function in the case of logistic regression:

$$L(\vec{\omega}) = -\frac{1}{m} \sum_{i=1}^m y_{true}^i \log(\sigma(\omega_0 + \omega_1 x_1^i + \dots + \omega_n x_n^i)) + (1 - y_{true}^i) \log(1 - \sigma(\omega_0 + \omega_1 x_1^i + \dots + \omega_n x_n^i))$$

using:

$$\begin{aligned} cost &= \begin{cases} -\log(\sigma(s)) & \text{if } y_{true} = 1 \\ -\log(1 - \sigma(s)) & \text{if } y_{true} = 0 \end{cases} \\ &= -y_{true} \log(\sigma(s)) - (1 - y_{true}) \log(1 - \sigma(s)) \end{aligned}$$

Now we generalize the same logic to multiple classes. Let's use the notation that $y_{true,i} = 1$ if a certain data point belongs in the i th class, and is zero otherwise. Then we have the cost associated with misclassifying a certain data point as:

$$\begin{aligned} cost &= \begin{cases} -\log(\sigma(s^1)) & \text{if } y_{true,1} = 1 \\ -\log(\sigma(s^2)) & \text{if } y_{true,2} = 1 \\ -\log(\sigma(s^3)) & \text{if } y_{true,3} = 1 \\ \vdots \\ -\log(\sigma(s^k)) & \text{if } y_{true,k} = 1 \end{cases} \\ &= -y_{true,1} \log(\sigma(s^1)) - \dots - y_{true,k} \log(\sigma(s^k)) \end{aligned}$$

Averaging over all the m data points in the training set, we obtain the *generalized cross-entropy loss function*, generalizing it from the case of only two classes to the case of multiple classes:

$$L(\vec{\omega}) = -\frac{1}{m} \sum_{i=1}^m y_{true,1}^i \log(\sigma(\omega_0^1 + \omega_1^1 x_1^i + \dots + \omega_n^1 x_n^i)) + y_{true,2}^i \log(\sigma(\omega_0^2 + \omega_1^2 x_1^i + \dots + \omega_n^2 x_n^i)) + \dots + y_{true,k}^i \log(\sigma(\omega_0^k + \omega_1^k x_1^i + \dots + \omega_n^k x_n^i))$$

Optimization

Now that we have a formula for the loss function, we can search for its minimizing ω 's. As with most of the loss functions that we will encounter, there is no explicit formula for the minimizers of this loss function in terms of the training set and their

target labels, so we settle for finding the minimizers using numerical methods, in particular: gradient descent, stochastic gradient descent, or mini-batch gradient descent (see [Chapter 4](#)). Again, the generalized cross-entropy loss function has its convexity working to our advantage in the minimization process, so we are guaranteed to find our sought-after ω 's.



Cross Entropy and Information Theory

The cross-entropy concept is borrowed from information theory. We will elaborate on this when discussing decision trees later in this chapter. For now, keep the following quantity in mind, where p is the probability of an event occurring:

$$\log \left(\frac{1}{p} \right) = - \log (p)$$

That quantity is large when p is small, therefore, it quantifies bigger *surprise* for *less probable* events.

Incorporating These Models into the Last Layer of a Neural Network

The linear regression model makes its predictions by appropriately linearly combining data features, then adding bias. The logistic regression and the softmax regression models make their classifications by appropriately linearly combining data features, adding bias, then passing the result into a probability scoring function. In these simple models, the features of the data are only linearly combined, hence, these models are weak in terms of picking up on potentially important nonlinear interactions among the data features. Neural network models incorporate nonlinear *activation functions* into their training functions, do this over multiple layers, and hence are better equipped to detect nonlinear and more complex relationships. The last layer of a neural network is its output layer. The layer right before the last layer spits out some higher-order features and inputs them into the last layer. If we want our network to classify data into multiple classes, then we can make our last layer a softmax layer; if we want it to classify into two classes, then our last layer can be a logistic regression layer; and if we want the network to predict numerical values, then we can make its last layer a regression layer. We will see examples of these in [Chapter 5](#).

Other Popular Machine Learning Techniques and Ensembles of Techniques

After regression and logistic regression, it is important to branch out into the machine learning community and learn the ideas behind some of the most popular techniques for classification and regression tasks. *Support vector machines*, *decision trees*, and *random forests* are very powerful and popular, and are able to perform both classification and regression tasks. The natural question is then, when do we use a specific machine learning method, including linear and logistic regression, and later neural networks? How do we know which method to use and base our conclusions and predictions on? These are the types of questions where the mathematical analysis of the machine learning models helps.

Since the mathematical analysis of each method, including the types of data sets it is usually best suited for, is only now gaining serious attention after the recent increase in resource allocation for research in AI, machine learning, and data science, the current practice is to try out each method on the same data set and use the one with the best results. That is, assuming we have the required computational and time resources to try out different machine learning techniques. Even better, if you do have the time and resources to train various machine learning models (parallel computing is perfect here), then it is good to utilize *ensemble methods*. These combine the results of different machine learning models, either by averaging or by voting, ironically, yet mathematically sound, giving better results than the best individual performers, *and even when the best performers are weak performers!*

One example of an ensemble is a random forest: it is an ensemble of decision trees.

When basing our predictions on ensembles, industry terms like *bagging* (or *bootstrap aggregating*), *pasting*, *boosting* (such as *ADA boost* and *gradient boosting*), *stacking*, and *random patches* appear. Bagging and pasting train *the same* machine learning model on different random subsets of the training set. Bagging samples instances from the training set with replacement, and pasting samples instances from the training set without replacement. *Random patches* sample from the feature space as well, training a machine learning model on a random subset of the features at a time. This is very helpful when the data set has many, many features, such as images (where each pixel is a feature). *Stacking* learns the prediction mechanism of the ensemble instead of simple voting or averaging.

Support Vector Machines

Support vector machine is an extremely popular machine learning method that's able to perform classification and regression tasks with both linear (flat) and nonlinear (curved) decision boundaries.

For classification, this method seeks to separate the labeled data using a widest possible margin, resulting in an optimal *highway* of separation as opposed to a thin line of separation. Let's explain how support vector machines classify labeled data instances in the context of this chapter's structure of training function, loss function, and optimization.

Training function

Once again we linearly combine the features of a data point with unknown weights ω 's and add bias ω_0 . We then pass the answer through the *sign* function. If the linear combination of features plus bias is a positive number, return 1 (or classify in the first class), and if it is negative, return -1 (or classify in the other). So the formula for the training function becomes:

$$f(\vec{\omega}; \vec{x}) = \text{sign}(\vec{\omega}^t \vec{x} + \omega_0)$$

Loss function

We must design a loss function that penalizes misclassified points. For logistic regression, we used the cross-entropy loss function. For support vector machines, our loss function is based on a function called the *hinge loss function*:

$$\max \left(0, 1 - y_{\text{true}} (\vec{\omega}^t \vec{x} + \omega_0) \right)$$

Let's see how the hinge loss function penalizes errors in classification. First, recall that y_{true} is either 1 or -1, depending on whether the data point belongs in the positive or the negative class.

- If for a certain data point y_{true} is 1 but $\vec{\omega}^t \vec{x} + \omega_0 < 0$, the training function will misclassify it and give us $y_{\text{predict}} = -1$, and the hinge loss function's value will be $1 - (1)(\vec{\omega}^t \vec{x} + \omega_0) > 1$, which is a high penalty when your goal is to minimize.
- If, on the other hand, y_{true} is 1 and $\vec{\omega}^t \vec{x} + \omega_0 > 0$, the training function will correctly classify it and give us $y_{\text{predict}} = 1$. The hinge loss function, however, is designed in such a way that it would still penalize us if $\vec{\omega}^t \vec{x} + \omega_0 < 1$, and its value will be $1 - (1)(\vec{\omega}^t \vec{x} + \omega_0)$, which is now less than 1 but still bigger than 0.
- Only when y_{true} is 1 and $\vec{\omega}^t \vec{x} + \omega_0 > 1$ (the training function will still correctly classify this point and give $y_{\text{predict}} = 1$) will the hinge loss function value be 0, since it will be the maximum between 0 and a negative quantity.

- The same logic applies when y_{true} is -1 . The hinge loss function will penalize a lot for a wrong prediction, and a little for a right prediction when it doesn't have far enough *margin* from the *zero divider* (a margin bigger than 1). The hinge loss function will return 0 only when the prediction is right and the point is at a distance larger than 1 from the 0 divider.
- Note that the 0 divider has the equation $\vec{\omega}^T x + \omega_0 = 0$, and the margin edges have equations $\vec{\omega}^T x + \omega_0 = -1$ and $\vec{\omega}^T x + \omega_0 = 1$. The distance between the margin edges is easy to calculate as $\frac{2}{\|\omega\|_2}$. So if we want to increase this margin width, we have to decrease $\|\omega\|_2$; thus, this term must enter the loss function, along with the hinge loss function, which penalizes both misclassified points and points within the margin boundaries.

Now if we average the hinge loss over all the m data points in the training set, and add $\|\omega\|_2^2$, we obtain the formula for the loss function that is commonly used for support vector machines:

$$L(\vec{\omega}) = \frac{1}{m} \sum_{i=1}^m \max \left(0, 1 - y_{true}^i (\vec{\omega}^T x^i + \omega_0) \right) + \lambda \|\omega\|_2^2$$

Optimization

Our goal now is to search for the $\vec{\omega}$ that minimizes the loss function. Let's observe this loss function for a minute:

- It has two terms: $\frac{1}{m} \sum_{i=1}^m \max \left(0, 1 - y_{true}^i (\vec{\omega}^T x^i + \omega_0) \right)$ and $\lambda \|\vec{\omega}\|_2^2$. Whenever we have more than one term in an optimization problem, it is most likely that they are competing terms, in the sense that the same ω values that make the first term small and thus happy might make the second term big and thus sad. So it is a push-and-pull game between the two terms as we search for the $\vec{\omega}$ that optimizes their sum.
- The λ that appears with the $\lambda \|\vec{\omega}\|_2^2$ term is an example of a model hyperparameter that we can tune during the validation stage of the training process. Note that controlling the value of λ helps us control the width of the margin this way: if we choose a large λ value, the optimizer will get busy choosing $\vec{\omega}$ with very low $\|\omega\|_2^2$, to compensate for that large λ , and the first term of the loss function will get less attention. But recall that a smaller $\|\omega\|_2$ means a larger margin!
- The $\lambda \|\vec{\omega}\|_2^2$ term can also be thought of as a *regularization term*, which we will discuss in [Chapter 4](#).

- This loss function is convex and bounded below by 0, so its minimization problem is not too bad: we don't have to worry about getting stuck at local minima. The first term has a singularity, but as mentioned before, we can define its subgradient at the singular point, then apply a descent method.

Some optimization problems can be reformulated, and instead of solving the original *primal* problem, we end up solving its *dual* problem! Usually, one is easier to solve than the other. We can think of the dual problem as another optimization problem living in a parallel universe of the primal problem. The universes meet at the optimizer. Hence solving one problem automatically gives the solution of the other. We study duality when we study optimization. Of particular interest and huge application are linear and quadratic optimization, also known as linear and quadratic programming. The minimization problem that we currently have:

$$\min_{\vec{\omega}} \frac{1}{m} \sum_{i=1}^m \max \left(0, 1 - y_{true}^i \left(\vec{\omega}^t \vec{x}^i + \omega_0 \right) \right) + \lambda \|\vec{\omega}\|_2^2$$

is an example of quadratic programming, and it has a dual problem formulation that turns out to be easier to optimize than the primal (especially when the number of features is high):

$$\max_{\vec{\alpha}} \sum_{j=1}^m \alpha_j - \frac{1}{2} \sum_{j=1}^m \sum_{k=1}^m \alpha_j \alpha_k y_{true}^j y_{true}^k \left(\left(\vec{x}^j \right)^t \vec{x}^k \right)$$

subject to the constraints $\alpha_j \geq 0$ and $\sum_{j=1}^m \alpha_j y_{true}^j = 0$. Writing that formula is usually straightforward when we learn about primal and dual problems, so we skip the derivation in favor of not interrupting our flow.

Quadratic programming is a very well-developed field, and there are many software packages that can solve this problem. Once we find the maximizing $\vec{\alpha}$, we can find the vector $\vec{\omega}$ that minimizes the primal problem using $\vec{\omega} = \sum_{j=1}^m \alpha_j y_{true}^j \vec{x}^j$. Once we have our $\vec{\omega}$, we can classify new data points using our now trained function:

$$\begin{aligned} f(\vec{x}_{new}) &= \text{sign}(\vec{\omega}^t \vec{x}_{new} + \omega_0) \\ &= \text{sign}\left(\sum_j \alpha_j y^j \left(\vec{x}^j\right)^t \vec{x}_{new} + \omega_0\right) \end{aligned}$$

If you want to avoid quadratic programming, there is another very fast method called *coordinate descent* that solves the dual problem and works very well with large data sets with a high number of features.

The kernel trick

We can now transition the same ideas to nonlinear classification. Let's first observe this important note about the dual problem: the data points appear only in pairs, more specifically, only in a scalar product, namely, $\left(\vec{x}^j, \vec{x}^k\right)$. Similarly, they only appear as a scalar product in the trained function. This simple observation allows for magic:

- If we find a function $K\left(\vec{x}^j, \vec{x}^k\right)$ that acts on pairs of data points, and it happens to output the scalar product of a *transformation* of the data points into some higher dimensional space, namely, $K\left(\vec{x}^j, \vec{x}^k\right) = \overrightarrow{\phi\left(\vec{x}^j\right)} \cdot \overrightarrow{\phi\left(\vec{x}^k\right)}$ (we don't even have to know what the actual transformation $\vec{\phi}$ into higher dimensions is), then we can solve the same exact dual problem in higher dimensions by replacing the scalar product in the formula of the dual problem with $K\left(\vec{x}^j, \vec{x}^k\right)$.
- The intuition here is that data that is nonlinearly separable in lower dimensions is almost always linearly separable in higher dimensions. So transform all the data points to higher dimensions, then separate. The kernel trick solves the linear classification problem in higher dimensions *without* transforming each point. The kernel itself evaluates the dot product of transformed data without transforming the data. Pretty cool stuff.

Examples of kernel functions include (note that the transformation $\vec{\phi}$ is nowhere to be seen):

- $K\left(\vec{x}^j, \vec{x}^k\right) = \left(\left(\vec{x}^j\right)^t \vec{x}^k\right)^2$
- The polynomial kernel: $K\left(\vec{x}^j, \vec{x}^k\right) = \left(1 + \left(\vec{x}^j\right)^t \vec{x}^k\right)^d$
- The Gaussian kernel: $K\left(\vec{x}^j, \vec{x}^k\right) = e^{-\gamma\left|x_j - x_k\right|^2}$

Decision Trees

Staying with our driving theme for this chapter that everything is a function, a decision tree, in essence, is a function that takes Boolean variables as an input (these are variables that can only assume *true* [or 1] or *false* [or 0] values) such as: is the feature > 5, is the feature = sunny, is the feature = man, etc. It outputs a *decision* such as: approve the loan, classify as covid19, return 25, etc. Instead of adding or multiplying boolean variables, we use the logical *or*, *and*, and *not* operators.

But what if our features are not given in the original data set as Boolean variables? Then we must transform them to Boolean variables before feeding them into the model to make predictions. For example, the decision tree in [Figure 3-17](#) was trained on the Fish Market data set. It is a regression tree. The tree takes raw data, but the function representing the tree actually operates on new variables, which are the original data features transformed into Boolean variables:

1. $a_1 = (\text{Width} \leq 5.117)$
2. $a_2 = (\text{Length3} \leq 59.55)$
3. $a_3 = (\text{Length3} \leq 41.1)$
4. $a_4 = (\text{Length3} \leq 34.9)$
5. $a_5 = (\text{Length3} \leq 27.95)$
6. $a_6 = (\text{Length3} \leq 21.25)$

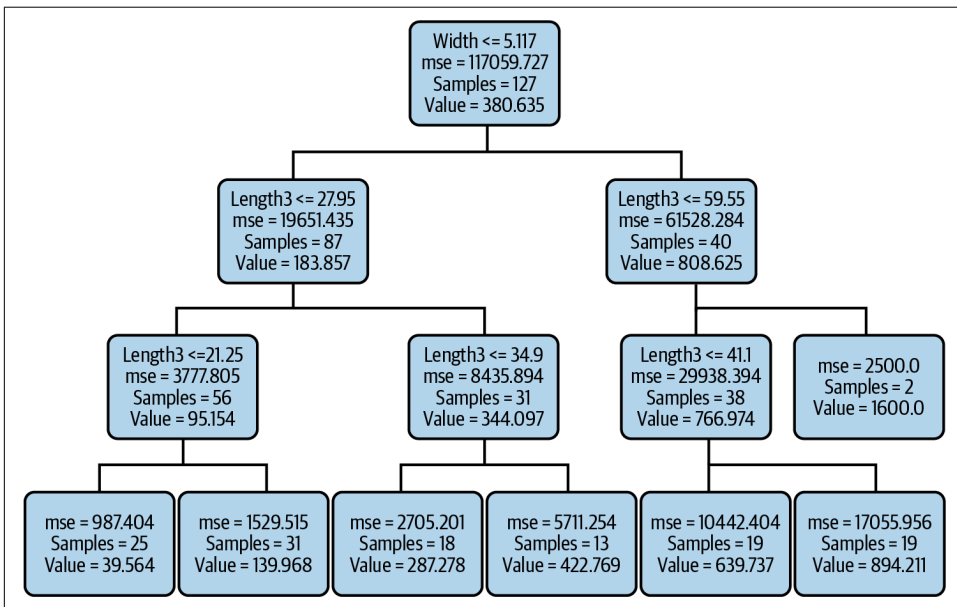


Figure 3-17. A regression decision tree constructed on the Fish Market data set. See the accompanying Jupyter notebook for details.

Now the function representing the decision tree in [Figure 3-17](#) is:

$$\begin{aligned}
 f(a_1, a_2, a_3, a_4, a_5, a_6) = & (a_1 \text{ and } a_5 \text{ and } a_6) \times 39.584 + (a_1 \text{ and } a_5 \text{ and not } a_6) \\
 & \times 139.968 + (a_1 \text{ and not } a_5 \text{ and } a_4) \times 287.278 + (a_1 \text{ and not } a_5 \text{ and not } a_4) \\
 & \times 422.769 + (\text{not } a_1 \text{ and } a_2 \text{ and } a_3) \times 639.737 + (\text{not } a_1 \text{ and } a_2 \text{ and not } a_3) \\
 & \times 824.211 + (\text{not } a_1 \text{ and not } a_2) \times 1600
 \end{aligned}$$

Note that unlike the training functions that we've encountered in this chapter so far, this function has no parameters ω 's that we need to solve for. This is called a *nonparametric model*, and it doesn't fix the shape of the function ahead of time. This gives it the flexibility to *grow* with the data, or in other words, adapt to the data. Of course, with this high adaptability to the data comes the high risk of overfitting the data. Thankfully there are ways around this, some of which we list here without any elaboration: pruning the tree after growing it, restricting the number of layers, setting a minimum number of data instances per node, or using an ensemble of trees instead of one tree, called a *random forest*, discussed later.

One very important observation: the decision tree *decided* to split over only two features of the original data set, namely the Width and Length3 features. Decision trees are designed to keep the more important features (those providing the most information that contribute to our prediction) closer to the root. Therefore, decision trees can help in feature selection, where we select the most important features to contribute to our final model's predictions.

It is no wonder that the Width and Length3 features ended up being the most important for predicting the weight of the fish. The correlation matrix in [Figure 3-18](#) and the scatterplots in [Figure 3-3](#) show extremely strong correlation between all the length features. This means that the information they provide is redundant, and including all of them in our prediction models will increase computation costs and lower performance.

	Weight	Length1	Length2	Length3	Height	Width
Weight	1.000000	0.908678	0.911888	0.917883	0.747700	0.896036
Length1	0.908678	1.000000	0.999493	0.991731	0.637844	0.870414
Length2	0.911888	0.999493	1.000000	0.993869	0.653291	0.877268
Length3	0.917883	0.991731	0.993869	1.000000	0.716450	0.882716
Height	0.747700	0.637844	0.653291	0.716450	1.000000	0.802115
Width	0.896036	0.870414	0.877268	0.882716	0.802115	1.000000

Figure 3-18. Correlation matrix for the Fish Market data set. There is an extremely strong correlation between all the length features.



Feature Selection

We just introduced the very important topic of feature selection. Real-world data sets come with many features, and some of them may provide redundant information, while others are not important at all for predicting our target label. Including irrelevant and redundant features in a machine learning model increases computational cost and lowers its performance. We just saw that decision trees are one way to help select the important features. Another way is a regularization technique called lasso regression, which we will introduce in [Chapter 4](#). There are statistical tests that test for feature dependencies on each other. The *F-test* tests for linear dependencies (this gives higher scores for correlated features, but correlations alone are deceptive), and *mutual information* tests for nonlinear dependencies. These provide a measure of how much a feature contributes to determining the target label, and hence aid in feature selection by keeping the most promising features. We can also test for feature dependencies on each other, as well as their correlations and scatterplots. *Variance* thresholding removes features with little to no variance, on the premise that if a feature does not vary much within itself, it has little predictive power.

How do we train a decision tree on a data set? What function do we optimize? There are two functions that are usually optimized when *growing* decision trees: the entropy and the Gini impurity. Using one or the other doesn't make much difference in the resultant trees. We develop these next.

Entropy and Gini impurity

Here, we decide to split a node of the tree on the feature that is evaluated as the *most important*. Entropy and Gini impurity are two popular ways to measure importance of a feature. They are not mathematically equivalent, but they both work and provide reasonable decision trees. Gini impurity is usually less expensive to compute, so it is the default in software packages, but you have the option to change from the default setting to entropy. Using Gini impurity tends to produce less-balanced trees when there are classes with much higher frequency than others. These classes end up isolated in their own branches. However, in many cases, using either entropy or Gini impurity does not provide much difference in the resulting decision trees.

With the *entropy* approach, we look for the feature split that provides the *maximal information gain* (we'll give its formula shortly). Information gain is borrowed from information theory, and it has to do with the concept of entropy. Entropy, in turn, is borrowed from thermodynamics and statistical physics, and it quantifies the amount of disorder in a certain system.

With the *Gini impurity* approach, we look for the feature split that provides children nodes with lowest average Gini impurity (we'll also give its formula shortly).

To maximize information gain (or minimize Gini impurity), the algorithm that grows a decision tree has to go over each feature of the training data subset and calculate the information gain (or Gini impurity), accomplished whether the tree uses that particular feature as a node to split on, then choose the feature that provides the highest information gain (or children nodes with lowest average Gini impurity). Moreover, if the feature has real numerical values, the algorithm has to decide on *what question to ask at the node*, meaning what feature value to split on; for example, is $x_5 < 0.1$? The algorithm has to do this sequentially at each layer of the tree, calculating information gain (or Gini impurities) over the features of the data instances in each node, and sometimes on each split value possibility. This is easier to understand with examples. But first, we write the formulas for the entropy, information gain, and Gini impurity.

Entropy and information gain

The easiest way to understand the entropy formula is to rely on the intuition that if an event is highly probable, then there is little surprise associated with it happening. So when $p(event)$ is large, its surprise is low. We can mathematically encode this with a function that decreases when its probability increases. The calculus function $\log \frac{1}{x}$ works and has the additional property that the surprises of independent events add up. Therefore, we can define:

$$Surprise(event) = \log \frac{1}{p(event)} = -\log(p(event))$$

Now the entropy of a random variable (which in our case is a particular feature in our training data set) is defined as the *expected surprise* associated with the random variable, so we must add up the surprises of each possible outcome of the random variable (surprise of each value of the feature in question) multiplied by their respective probabilities, obtaining:

$$Entropy(X) = -p(outcome_1) \log(p(outcome_1)) - p(outcome_2) \log(p(outcome_2)) - \dots - p(outcome_n) \log(p(outcome_n))$$

The entropy for one feature of our training data that assumes a bunch of values is:

$$Entropy(Feature) = -p(value_1) \log(p(value_1)) - p(value_2) \log(p(value_2)) - \dots - p(value_n) \log(p(value_n))$$

Since our goal is to select to split on a feature that provides large information gain about the outcome (the label or the target feature), let's first calculate the entropy of the outcome feature.

Binary output. Assume for simplicity that this is a binary classification problem, so the outcome feature only has two values: positive (in the class) and negative (not in the class).

If we let p be the number of positive instances in the target feature and n be the number of negative ones, then $p + n = m$ will be the number of instances in the training data subset. Now the probability to select a positive instance from that target column will be $\frac{p}{m} = \frac{p}{p+n}$, and the probability to select a negative instance is similarly $\frac{n}{m} = \frac{n}{p+n}$.

Thus, the entropy of the outcome feature (without leveraging any information from the other features) is:

$$\begin{aligned} & \text{Entropy}(\text{Outcome Feature}) \\ &= -p(\text{positive}) \log(p(\text{positive})) - p(\text{negative}) \log(p(\text{negative})) \\ &= -\frac{p}{p+n} \log\left(\frac{p}{p+n}\right) - \frac{n}{p+n} \log\left(\frac{n}{p+n}\right) \end{aligned}$$

Next, we leverage information from one other feature and calculate the difference in entropy of the outcome feature, which we expect to decrease as we gain more information (more information generally results in less surprise).

Suppose we choose Feature A to split a node of our decision tree on. Suppose Feature A assumes four values, and has k_1 instances with $value_1$, of these p_1 are labeled positive as their target outcome, and n_1 are labeled negative as their target outcome, so $p_1 + n_1 = k_1$. Similarly, Feature A has k_2 instances with $value_2$, of these p_2 are labeled positive as their target outcome, and n_2 are labeled negative as their target outcome, so $p_2 + n_2 = k_2$. The same applies for $value_3$ and $value_4$ of Feature A. Note that $k_1 + k_2 + k_3 + k_4 = m$, the total number of instances in the training subset of the data set.

We can think of each value $value_k$ of Feature A as a random variable in its own respect, with p_k positive outcomes and n_k negative outcomes, so we can calculate its entropy (expected surprise):

$$\text{Entropy}(value_1) = -\frac{p_1}{p_1 + n_1} \log \left(\frac{p_1}{p_1 + n_1} \right) - \frac{n_1}{p_1 + n_1} \log \left(\frac{n_1}{p_1 + n_1} \right)$$

$$\text{Entropy}(value_2) = -\frac{p_2}{p_2 + n_2} \log \left(\frac{p_2}{p_2 + n_2} \right) - \frac{n_2}{p_2 + n_2} \log \left(\frac{n_2}{p_2 + n_2} \right)$$

$$\text{Entropy}(value_3) = -\frac{p_3}{p_3 + n_3} \log \left(\frac{p_3}{p_3 + n_3} \right) - \frac{n_3}{p_3 + n_3} \log \left(\frac{n_3}{p_3 + n_3} \right)$$

$$\text{Entropy}(value_4) = -\frac{p_4}{p_4 + n_4} \log \left(\frac{p_4}{p_4 + n_4} \right) - \frac{n_4}{p_4 + n_4} \log \left(\frac{n_4}{p_4 + n_4} \right)$$

Now that we have this information, we can calculate the *expected entropy* after splitting on Feature A, by adding the four entropies just mentioned, each multiplied by its respective probability: $p(value_1) = \frac{k_1}{m}$, $p(value_2) = \frac{k_2}{m}$, $p(value_3) = \frac{k_3}{m}$, and $p(value_4) = \frac{k_4}{m}$.

Therefore, the expected entropy after splitting on Feature A would be:

$$\begin{aligned} &\text{Expected Entropy(Feature A)} \\ &= p(value_1)\text{Entropy}(value_1) + p(value_2)\text{Entropy}(value_2) \\ &+ p(value_3)\text{Entropy}(value_3) + p(value_4)\text{Entropy}(value_4) \\ &= \frac{k_1}{m}\text{Entropy}(value_1) + \frac{k_2}{m}\text{Entropy}(value_2) + \frac{k_3}{m}\text{Entropy}(value_3) + \frac{k_4}{m}\text{Entropy}(value_4) \end{aligned}$$

So what would be the information gained from using Feature A to split on? It would be the difference between the entropy of the outcome feature without any information from Feature A and the expected entropy of Feature A. That is, we have a formula for *information gain* given that we decide to split on Feature A:

Information Gain=

$$\begin{aligned} &\text{Entropy(Outcome Feature)} - \text{Expected Entropy(Feature A)} = \\ &-\frac{p}{p+n} \log \left(\frac{p}{p+n} \right) - \frac{n}{p+n} \log \left(\frac{n}{p+n} \right) - \text{Expected Entropy(Feature A)} \end{aligned}$$

Now it is easy to go through each feature of the training data subset and calculate the information gain resulting from using that feature to split on. Ultimately, the decision tree algorithm decides to split on the feature with highest information gain. The algorithm does this recursively for each node and at each layer of the tree, until it runs out of features to split on or data instances. This is how we obtain our entropy-based decision tree.

Multi-class output. It is not too difficult to generalize this logic to the case where we have a multiclass output, for example, a classification problem with three or more target labels. The classical **Iris data set** from the **UCI Machine Learning Repository** is a great example with three target labels. This data set has four features for a given Iris flower: its sepal length and width, and its petal length and width. Note that each of these features is a continuous random variable, not discrete. So we have to devise a test to split on the values of each feature, *before* applying the previously mentioned logic. This is part of the feature engineering stage of a data science project. The engineering step here is: transform a continuous valued feature into a Boolean feature; for example, is the petal length > 2.45 ? We will not go over how to choose the number 2.45, but by now you probably can guess that there is an optimization process that should go on here as well.

Gini impurity

Each decision tree is characterized by its nodes, branches, and leaves. A node is considered *pure* if it only contains data instances from the training data subset that have the same target label (this means they belong to the same class). Note that a pure node is a desired node, since we know its class. Therefore, an algorithm would want to grow a tree in a way that minimizes the impurity of the nodes: if the data instances in a node do not all belong in the same class, then the node is *impure*. Gini impurity quantifies this impurity the following way.

Suppose that our classification problem has three classes, like the **Iris data set**. Suppose also that a certain node in a decision tree grown to fit this data set has n training instances, with n_1 of these belonging in the first class, n_2 in the second class, and n_3 in the third class (so $n_1 + n_2 + n_3 = n$). Then the Gini impurity of this node is given by:

$$\text{Gini impurity} = 1 - \left(\frac{n_1}{n}\right)^2 - \left(\frac{n_2}{n}\right)^2 - \left(\frac{n_3}{n}\right)^2$$

So for each node, the fraction of the data instances belonging to each class is calculated, squared, then the sum of those is subtracted from 1. Note that if all the data instances of a node belong in the same class, then that formula gives a Gini impurity equal to 0.

The decision tree growing algorithm now looks for the feature and split point in each feature that produce children nodes with the lowest Gini impurity, on average. This means the children of a node must on average be purer than the parent node. Thus, the algorithm tries to minimize a weighted average of the Gini impurities of two of the children nodes (of a binary tree). Each child's Gini impurity is weighted by its relative size, which is the ratio between its number of instances relative to the total

number of instances in that tree layer (which is the same as the number of instances as its parent's). Thus, we end up having to search for the feature and the split point (for each feature) combination that solve the following minimization problem:

$$\min_{Feature, FeatureSplitValue} \frac{n_{left}}{n} Gini(\text{Left Node}) + \frac{n_{right}}{n} Gini(\text{Right Node})$$

where n_{left} and n_{right} are the number of data instances that end up being in the left and right children nodes, and n is the number of data instances that are in the parent node (note that n_{left} and n_{right} must add up to n).

Regression decision trees

It is important to point out that decision trees can be used for both regression and classification. A regression decision tree returns a predicted value rather than a class, but a similar process to a classification tree applies.

Instead of splitting a node by selecting a feature and a feature value (for example, is height > 3 feet?) that maximize information gain or minimize Gini impurity, we select a feature and a feature value that minimize a mean squared distance between the true labels and the average of the labels of all the instances in each of the left and right children nodes. That is, the algorithm chooses a feature and feature value to split on, then looks at the left and right children nodes resulting from that split, and calculates:

- The average value of all the labels of the training data instances in the left node. This average will be the left node value y_{left} , and is the value predicted by the decision tree if this node ends up being a leaf node.
- The average value of all the labels of the training data instances in the right node. This average will be the right node value y_{right} . Similarly, this is the value predicted by the decision tree if this node ends up being a leaf node.
- The sum of the squared distance between the left node value and the true label of each instance in the left node $\sum_{LeftNodeInstances} |y_{true}^i - y_{left}|^2$.
- The sum of the squared distance between the right node value and the true label of each instance in the right node $\sum_{RightNodeInstances} |y_{true}^i - y_{right}|^2$.
- A weighted average of the just-mentioned two sums, where each node is weighted by its size relative to the parent node, just like we did for the Gini impurity:

$$\frac{n_{left}}{n} \sum_{LeftNodeInstances} |y_{true}^i - y_{left}|^2 + \frac{n_{right}}{n} \sum_{RightNodeInstances} |y_{true}^i - y_{right}|^2$$

That algorithm is *greedy* and computation-heavy, in the sense that it has to do this for *each feature and each possible feature split value*, then choose the feature and feature split that provide the smallest weighted squared error average between the left and right children nodes.

The famous CART (Classification and Regression Tree) algorithm is used by software packages, including Python's scikit-learn, which we use in the Jupyter notebooks supplementing this book. This algorithm produces trees with nodes that only have two children (binary trees), where the test at each node only has Yes or No answers. Other algorithms such as ID3 can produce trees with nodes that have two or more children.

Shortcomings of decision trees

Decision trees are very easy to interpret and are popular for many good reasons: they adapt to large data sets, different data types (discrete and continuous features, no scaling of data needed), and can perform both regression and classification tasks. However, they can be unstable, in the sense that adding just one instance to the data set can change the tree at its root and hence result in a very different decision tree.

They are also sensitive to rotations in the data, since their decision boundaries are usually horizontal and vertical (not slanted like support vector machines). This is because splits usually happen at specific feature values, so the decision boundaries end up parallel to the feature axes. One fix is to transform the data set to match its *principal axes*, using the *singular value decomposition method* presented in [Chapter 6](#). Decision trees tend to overfit the data, so they need pruning. This is usually done using statistical tests. The greedy algorithms involved in constructing the trees, where the search happens over all features and their values, makes them computationally expensive and less accurate. Random forests, discussed next, address some of these shortcomings.

Random Forests

When I first learned about decision trees, the most perplexing aspects for me were:

- How do we start the tree, meaning how do we decide which data feature is the root feature?
- At what particular feature value do we decide to split a node?

- When do we stop?
- In essence, how do we grow a tree?

(Note that we answered some of these questions in the previous subsection.) It didn't make matters any easier that I would surf the internet looking for answers, only to encounter declarations that decision trees are so easy to build and understand, so it felt like I was the only one deeply confused by decision trees.

My puzzlement instantly disappeared when I learned about *random forests*. The amazing thing about random forests is that we can get incredibly good regression or classification results *without* answering any of my bewildering questions. Randomizing the whole process means building many decision trees while answering all my questions with two words: *choose randomly*. Then aggregating the predictions in an ensemble produces very good results, even better than one carefully crafted decision tree. It has been said that *randomization often produces reliability*!

Another very useful property of random forests is that they give a measure of *feature importance*, helping us pinpoint which features significantly affect our predictions, and aid in feature selection as well.

k-means Clustering

One common goal of data analysts is to partition data into *clusters*, each cluster highlighting certain common traits. *k-means clustering* is a common machine learning method that partitions n data points (vectors) into k clusters, where each data point gets assigned to the cluster with the nearest mean. The mean of each cluster, or its centroid, serves as the prototype of the cluster. Overall, *k-means* clustering minimizes the variance (the squared Euclidean distances to the mean) within each cluster.

The most common algorithm for *k-means* clustering is iterative:

1. Start with an initial set of k means. This means that we specify the number of clusters ahead of time, which raises the question: How to initialize it? How to select the locations of the first k centroids? There is literature on that.
2. Assign each data point to the cluster with the nearest mean in terms of squared Euclidean distance.
3. Recalculate the means of each cluster.

The algorithm converges when the data point assignments to each cluster do not change anymore.

Performance Measures for Classification Models

It is relatively easy to develop mathematical models that compute things and produce outputs. It is a completely different story to develop models that perform well for our desired tasks. Furthermore, models that perform well according to some metrics behave badly according to some other metrics. We need extra care in developing performance metrics and deciding which ones to rely on, depending on our specific use cases.

Measuring the performance of models that predict numerical values, such as regression models, is easier than classification models, since we have many ways to compute distances between numbers (good predictions and bad predictions). On the other hand, when our task is classification (we can use models such as logistic regression, softmax regression, support vector machines, decision trees, random forests, or neural networks), we have to put some extra thought into evaluating performance. Moreover, there are usually trade-offs. For example, if our task is to classify YouTube videos as being safe for kids (positive) or not safe for kids (negative), do we tweak our model so as to reduce the number of false positives or false negatives? It is obviously more problematic if a video is classified as safe while in reality it is unsafe (false positive) than the other way around, so our performance metric needs to reflect that.

The following are the performance measures commonly used for classification models. Do not worry about memorizing their names, as the way they are named does not make logical sense. Instead, spend your time understanding their meanings:

Accuracy

Percentage of times the prediction model got the classification right:

$$Accuracy = \frac{\text{true positives} + \text{true negatives}}{\text{all predicted positives} + \text{all predicted negatives}}$$

Confusion matrix

Counting all true positives, false positives, true negatives, and false negatives:

True negative	False positive
False negative	True positive

Precision score

Accuracy of the positive predictions:

$$Precision = \frac{\text{true positives}}{\text{all predicted positives}} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

Recall score

Ratio of the positive instances that are correctly classified:

$$Recall = \frac{\text{true positives}}{\text{all positive labels}} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Specificity

Ratio of the negative instances that are correctly classified:

$$Specificity = \frac{\text{true negatives}}{\text{all negative labels}} = \frac{\text{true negatives}}{\text{true negatives} + \text{false positives}}$$

F₁ score

This quantity is only high when both precision and recall scores are high:

$$F_1 = \frac{2}{\frac{1}{precision} + \frac{1}{recall}}$$

AUC (area under the curve) and ROC (receiver operating characteristics) curves

These curves provide a performance measure for a classification model at various threshold values. We can use these curves to measure how well a certain variable predicts a certain outcome; for example, how well does the GRE subject test score predict passing a graduate school's qualifying exam in the first year?

Andrew Ng's book, *Machine Learning Yearning* (self-published), provides an excellent guide for performance metrics best practices. Please read carefully before diving into real AI applications, since the book's recipes are based on many trials, successes, and failures.

Summary and Looking Ahead

In this chapter, we surveyed some of the most popular machine learning models, emphasizing a particular mathematical structure that appears throughout the book: training function, loss function, and optimization. We discussed linear, logistic, and softmax regression, then breezed over support vector machines, decision trees, ensembles, random forests, and k -means clustering.

Moreover, we made a decent case for studying these topics from mathematics:

Calculus

The minimum and maximum happen at the boundary or at points where one derivative is zero or does not exist.

Linear algebra

- Linearly combining features: $\omega_1 x_1 + \omega_2 x_2 + \cdots + \omega_n x_n$.
- Writing various mathematical expressions using matrix and vector notation.
- The scalar product of two vectors $\vec{a} \cdot \vec{b}$.
- The l^2 norm of a vector.
- Avoid working with ill-conditioned matrices. Get rid of linearly dependent features. This also has to do with feature selection.
- Avoid multiplying matrices by each other; this is too expensive. Multiply matrices by vectors instead.

Optimization

- For convex functions, we do not worry about getting stuck at local minima since local minima are also global minima. We do worry about narrow valleys (see [Chapter 4](#)).
- Gradient descent methods need only one derivative (see [Chapter 4](#)).
- Newton's methods need two derivatives or an approximation of two derivatives (inconvenient for large data).
- Quadratic programming, the dual problem, and coordinate descent (all appear in support vector machines).

Statistics

- Correlation matrix and scatterplots
- The F-test and mutual information test for feature selection
- Standardizing the data features (subtracting the mean and dividing by the standard deviation)

More steps we did not and will not go over (yet):

- Validating our models: tune the weight values and the hyperparameters so as not to overfit.
- Test the trained model on the testing subset of the data, which our model had not used (or seen) during the training and validation steps (we do this in the accompanying Jupyter notebook on the [book's GitHub page](#)).
- Deploy and monitor the finalized model.
- Never stop thinking on how to improve our models and how to better integrate them into the whole production pipeline.

In [Chapter 4](#), we step into the new and exciting era of neural networks.

Optimization for Neural Networks

I have lived each and every day of my life optimizing....My first aha moment was when I learned that our brain, too, learns a model of the world.

—H.

Various artificial neural networks have fully connected layers in their architecture. In this chapter, we explain how the mathematics of a fully connected neural network works. We design and experiment with various training and loss functions. We also explain that the optimization and backpropagation steps used when training neural networks are similar to how learning happens in our brains. The brain learns by reinforcing neuron connections when faced with a concept it has seen before, and weakening connections if it learns new information that contradicts previously learned concepts. Machines only understand numbers. Mathematically, stronger connections correspond to larger numbers, and weaker connections correspond to smaller numbers.

Finally, we walk through various regularization techniques, explaining their advantages, disadvantages, and use cases.

The Brain Cortex and Artificial Neural Networks

Neural networks are modeled after the brain cortex, which involves billions of neurons arranged in a layered structure. [Figure 4-1](#) shows an image of three vertical cross-sections of the brain neocortex, and [Figure 4-2](#) shows a diagram of a fully connected artificial neural network.

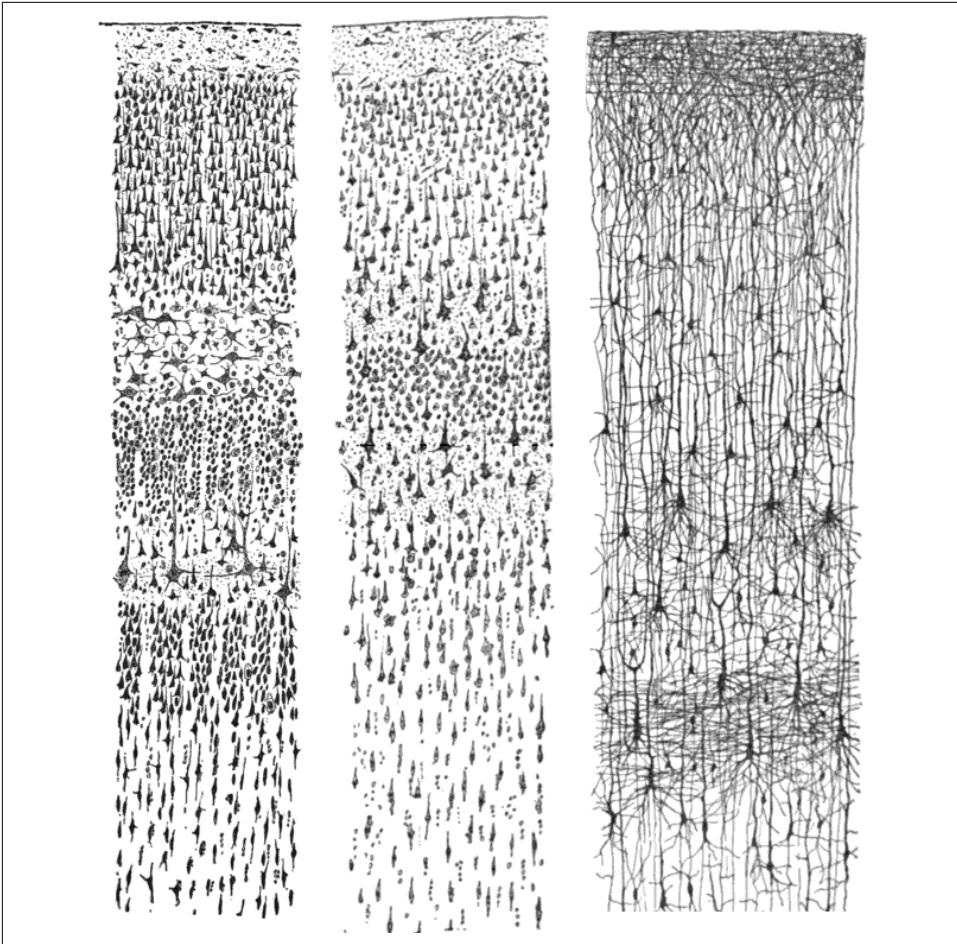


Figure 4-1. Three drawings of cortical lamination by Santiago Ramón y Cajal, taken from the book Comparative Study of the Sensory Areas of the Human Cortex (Andersite Press) (image source: [Wikipedia](#))

In [Figure 4-1](#), each drawing shows a vertical cross-section of the cortex, with the surface (outermost side closest to the skull) of the cortex at the top. On the left is a Nissl-stained visual cortex of a human adult. In the middle is a Nissl-stained motor cortex of a human adult. On the right is a Golgi-stained cortex of a month-and-a-half-old infant. The Nissl stain shows the cell bodies of neurons. The Golgi stain shows the dendrites and axons of a random subset of neurons. The layered structure of the neurons in the cortex is evident in all three cross-sections.

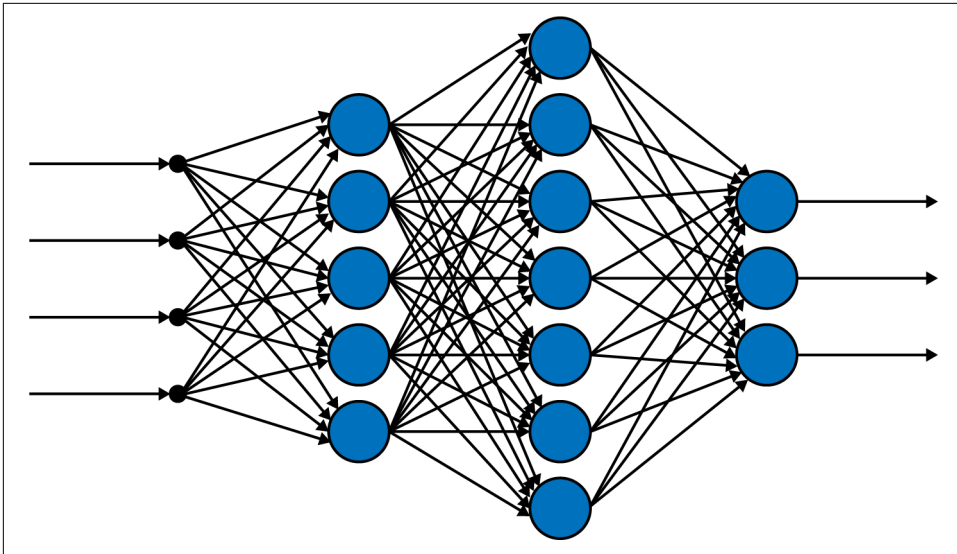


Figure 4-2. A fully connected or dense artificial neural network with four layers

Even though different regions of the cortex are responsible for different functions, such as vision, auditory perception, logical thinking, language, speech, etc., what actually determines the function of a specific region are its *connections*: which sensory and motor skills input and output regions it connects to. This means that if a cortical region is connected to a different sensory input/output region—for example, a vision locality instead of an auditory one—then it will perform vision tasks (computations), not auditory tasks. In a very simplified sense, the cortex performs one basic function at the neuron level. In an artificial neural network, the basic computation unit is *the perceptron*, and it functions in the same way across the whole network. The various connections, layers, and architecture of the neural network (both the brain cortex and artificial neural networks) are what allow these computational structures to do very impressive things.

Training Function: Fully Connected, or Dense, Feed Forward Neural Networks

In a *fully connected* or *dense* artificial neural network (see [Figure 4-2](#)), every neuron, represented by a node (the circles), in every layer is connected to all the neurons in the next layer. The first layer is the input layer, the last layer is the output layer, and the intermediate layers are called *hidden layers*. The neural network itself, whether fully connected or not (the networks that we will encounter in the new few chapters are *convolutional* and are not fully connected), is a computational graph representing

the formula of the training function. Recall that we use this function to make predictions after training.

Training in the neural networks context means finding the parameter values, or weights, that enter into the formula of the training function via minimizing a loss function. This is similar to training linear regression, logistic regression, softmax regression, and support vector machine models, which we discussed in [Chapter 3](#). The mathematical structure here remains the same:

1. Training function
2. Loss function
3. Optimization

The only difference is that for the simple models of [Chapter 3](#), the formulas of the training functions are very uncomplicated. They linearly combine the data features, add a bias term (ω_0), and pass the result into at most one nonlinear function (for example, the logistic function in logistic regression). As a consequence, the results of these models are also simple: a linear (flat) function for linear regression, and a linear division boundary between different classes in logistic regression, softmax regression, and support vector machines. Even when we use these simple models to represent nonlinear data, such as in the cases of polynomial regression (fitting the data into polynomial functions of the features) or support vector machines with the kernel trick, we still end up with linear functions or division boundaries, but these will either be in higher dimensions (for polynomial regression, the dimensions are the feature and its powers) or in transformed dimensions (such as when we use the kernel trick with support vector machines).

For neural network models, on the other hand, the process of linearly combining the features, adding a bias term, then passing the result through a nonlinear function (now called *activation function*) is the computation that happens *only in one neuron*. This simple process happens over and over again in dozens, hundreds, thousands, or sometimes millions of neurons, arranged in layers, where the output of one layer acts as the input of the next layer. Similar to the brain cortex, the aggregation of simple and similar processes over many neurons and layers produces, or allows for the representation of, much more complex functionalities. This is sort of miraculous. Thankfully, we are able to understand much more about artificial neural networks than our brain's neural networks, mainly because we design them, and after all, an artificial neural network is just one mathematical function. No black box remains dark once we dissect it under the lens of mathematics. That said, the mathematical analysis of artificial neural networks is a relatively new field. There are still many questions to be answered and a lot to be discovered.

A Neural Network Is a Computational Graph Representation of the Training Function

Even for a network with only five neurons, such as the one in [Figure 4-3](#), it is pretty messy to write the formula of the training function. This justifies the use of computational graphs to represent neural networks in an organized and easy way. Graphs are characterized by two things: nodes and edges (congratulations, this was lesson one in graph theory). In a neural network, an edge connecting node i in layer m to node j in layer n is assigned a weight $\omega_{mn,ij}$. That is four indices for only one edge! At the risk of drowning in a deep ocean of indices, we must organize a neural network's weights in matrices.

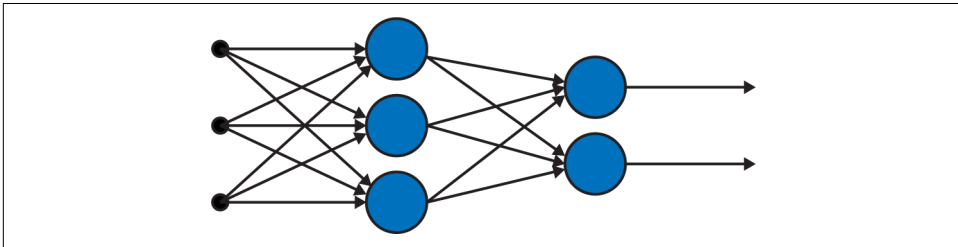


Figure 4-3. A fully connected (or dense) feed forward neural network with only five neurons arranged in three layers. The first layer (the three black dots on the very left) is the input layer, the second layer is the only hidden layer with three neurons, and the last layer is the output layer with two neurons.

Let's model the training function of a *feed forward* fully connected neural network. Feed forward means that the information flows forward through the computational graph representing the network's training function.

Linearly Combine, Add Bias, Then Activate

What kind of computations happen within a neuron when it receives input from other neurons? Linearly combine the input information using different weights, add a bias term, then use a nonlinear function to *activate* the neuron. We will go through this process one step at a time.

The weights

Let the matrix W^1 contain the weights of the edges *incident to* hidden layer 1, W^2 contain the weights of the edges incident to hidden layer 2, and so on, until we reach the output layer.

So for the small neural network represented in [Figure 4-3](#), we only have $h = 1$ hidden layer, obtaining two matrices of weights:

$$W^1 = \begin{pmatrix} \omega_{11}^1 & \omega_{12}^1 & \omega_{13}^1 \\ \omega_{21}^1 & \omega_{22}^1 & \omega_{23}^1 \\ \omega_{31}^1 & \omega_{32}^1 & \omega_{33}^1 \end{pmatrix} \text{ and } W^{h+1} = W^2 = W^{output} = \begin{pmatrix} \omega_{11}^2 & \omega_{12}^2 & \omega_{13}^2 \\ \omega_{21}^2 & \omega_{22}^2 & \omega_{23}^2 \end{pmatrix},$$

where the superscripts indicate the layer to which the edges point. Note that if we only had one node at the output layer instead of two, then the last matrix of weights $W^{h+1} = W^{output}$ would only be a row vector:

$$W^{h+1} = W^2 = W^{output} = (\omega_{11}^2 \ \omega_{12}^2 \ \omega_{13}^2)$$

Now at one node of this neural network, two computations take place:

1. A linear combination plus bias
2. Passing the result through a nonlinear activation function (the composition operation from calculus)

We elaborate on these two, then ultimately construct the training function of the fully connected feed forward neural network represented in [Figure 4-3](#).

A linear combination plus bias

At the first node in the first hidden layer (the only hidden layer for this small network), we linearly combine the inputs:

$$z_1^1 = \omega_{11}^1 x_1 + \omega_{12}^1 x_2 + \omega_{13}^1 x_3 + \omega_{01}^1$$

At the second node in the first hidden layer, we linearly combine the inputs using different weights than the previous linear combination:

$$z_2^1 = \omega_{21}^1 x_1 + \omega_{22}^1 x_2 + \omega_{23}^1 x_3 + \omega_{02}^1$$

At the third node in the first hidden layer, we linearly combine the inputs using different weights than the previous two linear combinations:

$$z_3^1 = \omega_{31}^1 x_1 + \omega_{32}^1 x_2 + \omega_{33}^1 x_3 + \omega_{03}^1$$

Let's express the three equations above using vector and matrix notation. This will be extremely convenient for our optimization task later, and of course it will preserve our sanity:

$$\begin{pmatrix} z_1^1 \\ z_2^1 \\ z_3^1 \end{pmatrix} = \begin{pmatrix} \omega_{11}^1 \\ \omega_{21}^1 \\ \omega_{31}^1 \end{pmatrix} x_1 + \begin{pmatrix} \omega_{12}^1 \\ \omega_{22}^1 \\ \omega_{32}^1 \end{pmatrix} x_2 + \begin{pmatrix} \omega_{13}^1 \\ \omega_{23}^1 \\ \omega_{33}^1 \end{pmatrix} x_3 + \begin{pmatrix} \omega_{01}^1 \\ \omega_{02}^1 \\ \omega_{03}^1 \end{pmatrix} = \begin{pmatrix} \omega_{11}^1 & \omega_{12}^1 & \omega_{13}^1 \\ \omega_{21}^1 & \omega_{22}^1 & \omega_{23}^1 \\ \omega_{31}^1 & \omega_{32}^1 & \omega_{33}^1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} \omega_{01}^1 \\ \omega_{02}^1 \\ \omega_{03}^1 \end{pmatrix}$$

We can now summarize the above expression compactly as:

$$\vec{z}^1 = W^1 \vec{x} + \vec{\omega}_0^1$$

Pass the result through a nonlinear activation function

Linearly combining the features and adding bias are not enough to pick up on more complex information in the data, and neural networks would have never been successful without this crucial but very simple step: compose with a *nonlinear* function at each node of the hidden layers.

Linear Combination of a Linear Combination Is Still a Linear Combination

If we skip the step of composing with a nonlinear function, and pass the information from the first layer to the next layer using only linear combinations, then our network will not learn anything new in the next layer. It will not be able to pick up on more complex features from one layer to the next. The math of why this is the case is straightforward. Suppose for simplicity that we only have two input features, the first hidden layer has only two nodes, and the second one has two nodes as well. Then the outputs of the first hidden layer without a nonlinear activation function would be:

$$\begin{aligned} z_1^1 &= \omega_{11}^1 x_1 + \omega_{12}^1 x_2 + \omega_{01}^1 \\ z_2^1 &= \omega_{21}^1 x_1 + \omega_{22}^1 x_2 + \omega_{02}^1 \end{aligned}$$

At the second hidden layer, these will be linearly combined again, so the output of the first node of this layer would be:

$$\begin{aligned} z_1^2 &= \omega_{11}^2 z_1^1 + \omega_{12}^2 z_2^1 + \omega_{01}^2 \\ &= \omega_{11}^2 (\omega_{11}^1 x_1 + \omega_{12}^1 x_2 + \omega_{01}^1) + \omega_{12}^2 (\omega_{21}^1 x_1 + \omega_{22}^1 x_2 + \omega_{02}^1) + \omega_{01}^2 \\ &= (\omega_{11}^2 \omega_{11}^1 + \omega_{12}^2 \omega_{21}^1) x_1 + (\omega_{11}^2 \omega_{12}^1 + \omega_{12}^2 \omega_{22}^1) x_2 + (\omega_{11}^2 \omega_{01}^1 + \omega_{12}^2 \omega_{02}^1 + \omega_{01}^2) \\ &= \omega_1 x_1 + \omega_2 x_2 + \omega_3 \end{aligned}$$

This output is nothing but a simple linear combination of the original features plus bias. Hence, adding a layer without any nonlinear activation contributes nothing new. In other words, the training function would remain linear and would lack the ability to pick up on any nonlinear relationships in the data.

We are the ones who decide on the formula for the nonlinear activation function, and different nodes can have different activation functions, even though it is rare to do this in practice. Let f be this activation function, then the output of the first hidden layer will be:

$$\vec{s}^1 = f(\vec{z}^1) = f(W^1 \vec{x} + \vec{\omega}_0^1)$$

It is now straightforward to see that if we had more hidden layers, their outputs will be *chained* with those of previous layers, making writing the training function a bit tedious:

$$\begin{aligned}\vec{s}^2 &= f(\vec{z}^2) = f(W^2 \vec{s}^1 + \vec{\omega}_0^2) = f\left(W^2 \left(f\left(W^1 \vec{x} + \vec{\omega}_0^1\right)\right) + \vec{\omega}_0^2\right) \\ \vec{s}^3 &= f(\vec{z}^3) = f(W^3 \vec{s}^2 + \vec{\omega}_0^3) = f\left(W^3 \left(f\left(W^2 \left(f\left(W^1 \vec{x} + \vec{\omega}_0^1\right)\right) + \vec{\omega}_0^2\right) + \vec{\omega}_0^3\right)\end{aligned}$$

This chaining goes on until we reach the output layer. What happens at this very last layer depends on the task of the network. If the goal is regression (predict one numerical value) or binary classification (classify into two classes), then we only have one output node (see [Figure 4-4](#)).

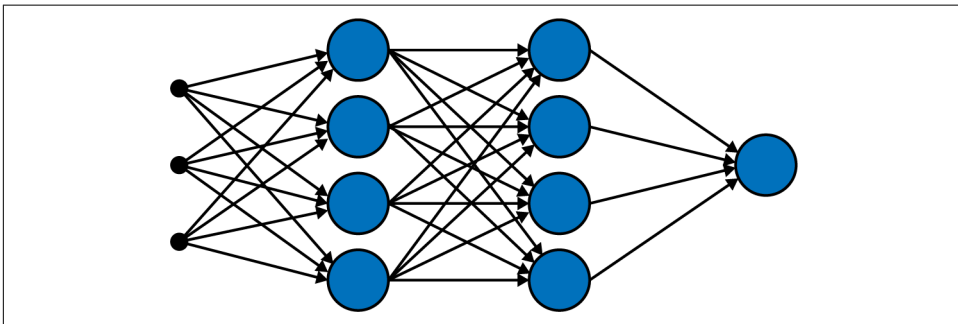


Figure 4-4. A fully connected (or dense) feed forward neural network with only nine neurons arranged in four layers. The first layer on the very left is the input layer, the second and third layers are the two hidden layers with four neurons each, and the last layer is the output layer with only one neuron (this network performs either a regression task or a binary classification task).

- If the task is regression, we linearly combine the outputs of the previous layer at the final output node, add bias, and go home (we *do not* pass the result through a nonlinear function in this case). Since the output layer only has one node, the output matrix is just a row vector $W^{output} = W^{h+1}$, and one bias ω_0^{h+1} . The prediction of the network will now be:

$$y_{predict} = W^{h+1} \vec{s}^h + \omega_0^{h+1}$$

where h is the total number of hidden layers in the network (this does not include the input and output layers).

- If, on the other hand, the task is binary classification, then again we have only one output node, where we linearly combine the outputs of the previous layer, add bias, then pass the result through the logistic function $\sigma(s) = \frac{1}{1 + e^{-s}}$, resulting in the network's prediction:

$$y_{predict} = \sigma\left(W^{h+1} \vec{s}^h + \omega_0^{h+1}\right)$$

- If the task is to classify into multiple classes, say, five classes, then the output layer would include five nodes. At each of these nodes, we linearly combine the outputs of the previous layer, add bias, then pass the result through the softmax function:

$$\sigma(z^1) = \frac{e^{z^1}}{e^{z^1} + e^{z^2} + e^{z^3} + e^{z^4} + e^{z^5}}$$

$$\sigma(z^2) = \frac{e^{z^2}}{e^{z^1} + e^{z^2} + e^{z^3} + e^{z^4} + e^{z^5}}$$

$$\sigma(z^3) = \frac{e^{z^3}}{e^{z^1} + e^{z^2} + e^{z^3} + e^{z^4} + e^{z^5}}$$

$$\sigma(z^4) = \frac{e^{z^4}}{e^{z^1} + e^{z^2} + e^{z^3} + e^{z^4} + e^{z^5}}$$

$$\sigma(z^5) = \frac{e^{z^5}}{e^{z^1} + e^{z^2} + e^{z^3} + e^{z^4} + e^{z^5}}$$

Group those into a vector function $\vec{\sigma}$ that also takes vectors as input: $\vec{\sigma}(\vec{z})$, then the final prediction of the neural network is a vector of five probability scores where a data instance belongs to each of the five classes:

$$\begin{aligned}\vec{y}_{predict} &= \vec{\sigma}(\vec{z}) \\ &= \vec{\sigma}\left(W^{output}_s \vec{z}^h + \vec{\omega}_0^{h+1}\right)\end{aligned}$$

Notation overview

We will try to remain consistent with notation throughout our discussion of neural networks. The x 's are the input features, the W 's are the matrices or column vectors containing the weights that we use for linear combinations, the ω_0 's are the biases that are sometimes grouped into a vector, the z 's are the results of linear combinations plus biases, and the s 's are the results of passing the z 's into the nonlinear activation functions.

Common Activation Functions

In theory, we can use any nonlinear function to *activate our nodes* (think of all the calculus functions we've ever encountered). In practice, there are some popular ones, listed next and graphed in [Figure 4-5](#).

By far the Rectified Linear Unit function (ReLU) is the most commonly used in today's networks, and the success of [AlexNet](#) in 2012 is partially attributed to the use of this activation function, as opposed to the hyperbolic tangent and logistic functions (sigmoid) that were commonly used in neural networks at the time (and are still in use).

The first four functions in the following list and in [Figure 4-5](#) are all inspired by computational neuroscience, where they attempt to model a threshold for the activation (firing) of one neuron cell. Their graphs look similar to each other: some are smoother variants of others, some output only positive numbers, others output more balanced numbers between -1 and 1 , or between $-\frac{\pi}{2}$ and $\frac{\pi}{2}$. They all *saturate* for small or large inputs, meaning their graphs become flat for inputs large in magnitude. This creates a problem for *learning*, since if these functions output the same numbers over and over again, there will not be much learning happening.

Mathematically, this phenomenon manifests itself as *the vanishing gradient problem*. The second set of activation functions attempts to rectify this saturation problem, which it does, as we see in the graphs of the second row in [Figure 4-5](#). This, however, introduces another problem, called *the exploding gradient problem*, since

these activation functions are unbounded and can now output big numbers, and if these numbers grow over multiple layers, we have a problem.

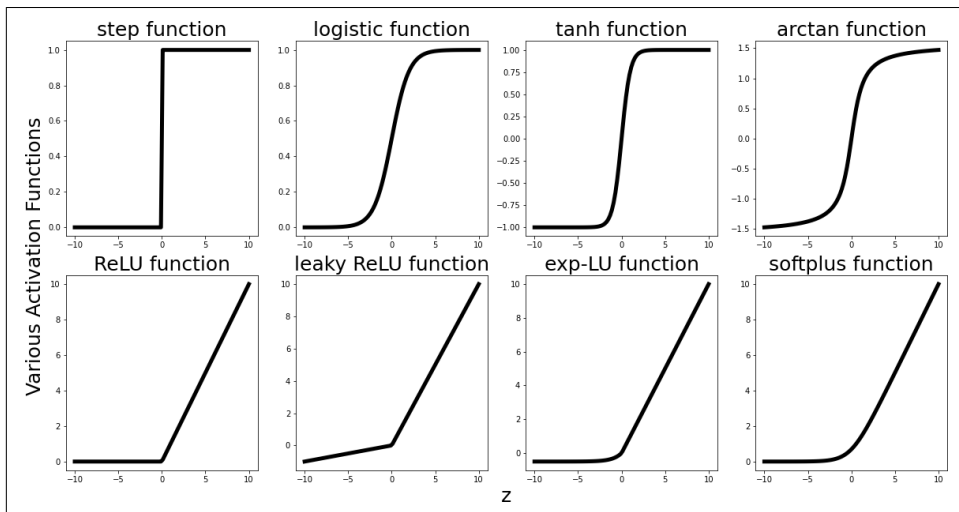


Figure 4-5. Various activation functions for neural networks. The first row consists of sigmoidal-type activation functions, shaped like the letter S. These saturate (become flat and output the same values) for inputs large in magnitude. The second row consists of ReLU-type activation functions, which do not saturate. An engineer pointed out the analogy of these activation functions to the physical function of a transistor.

Every new set of problems that gets introduced comes with its own set of techniques attempting to fix it, such as *gradient clipping*, normalizing the outputs after each layer, etc. The take-home lesson is that none of this is magic. A lot of it is trial and error, and new methods emerge to fix problems that other new methods introduced. We only need to understand the principles, the why and the how, and get a decent exposure to what is popular in the field, while keeping an open mind for improving things, or doing things entirely differently.

Let's state the formulas of common activation functions, as well as their derivatives. We need to calculate one derivative of the training function when we optimize the loss function in our search for the best weights of the neural network:

- Step function: $f(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$
- Its derivative: $f'(z) = \begin{cases} 0 & \text{if } z \neq 0 \\ \text{undefined} & \text{if } z = 0 \end{cases}$

- Logistic function: $\sigma(z) = \frac{1}{1 + e^{-z}}$.

Its derivative: $\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$.

- Hyperbolic tangent function: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{2}{1 + e^{-2z}} - 1$

Its derivative: $\tanh'(z) = \frac{4}{(e^z + e^{-z})^2} = 1 - f(z)^2$

- Inverse tangent function: $f(z) = \arctan(z)$.

Its derivative: $f'(z) = \frac{1}{1 + z^2}$.

- Rectified Linear Unit function or ReLU(z): $f(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$

Its derivative: $f'(z) = \begin{cases} 0 & \text{if } z < 0 \\ \text{undefined} & \text{if } z = 0 \\ 1 & \text{if } z > 0 \end{cases}$

- Leaky Rectified Linear Unit function (or parametric linear unit):

$$f(z) = \begin{cases} \alpha z & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

Its derivative: $f'(z) = \begin{cases} \alpha & \text{if } z < 0 \\ \text{undefined} & \text{if } z = 0 \\ 1 & \text{if } z > 0 \end{cases}$

- Exponential Linear Unit function: $f(z) = \begin{cases} \alpha(e^z - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$

Its derivative: $f'(z) = \begin{cases} f(z) + \alpha & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$

- Softplus function: $f(z) = \ln(1 + e^z)$

Its derivative: $f'(z) = \frac{1}{1 + e^{-z}} = \sigma(z)$

Note that all of these activations are rather elementary functions. This is a good thing, since both they and their derivatives are usually involved in massive computations with thousands of parameters (weights) and data instances during training, testing, and deployment of neural networks, so better keep them elementary. The other reason is that in theory, it doesn't really matter what activation function we end up choosing because of the *universal function approximation theorems*, discussed next. Careful here: operationally, it definitely matters what activation function we choose for our neural network nodes. As we mentioned earlier in this section, AlexNet's

success in image classification tasks is partly due to its use of the Rectified Linear Unit function, $\text{ReLU}(z)$. Theory and practice do not contradict each other in this case, even though it seems so on the surface. We explain this in the next subsection.

Universal Function Approximation

Approximation theorems, when available, are awesome because they tell us, with mathematical confidence and authority, that if we have a function that we do not know, or that we know but that is difficult to include in our computations, then we do not have to deal with this unknown or difficult function altogether. We can, instead, approximate it using known functions that are much easier to compute, to a great degree of precision. This means that under certain conditions on both the unknown or complicated function, and the known and simple (sometimes elementary) functions, we can use the simple functions and be confident that our computations are doing the right thing. These types of approximation theorems quantify how far off the true function is from its approximation, so we know exactly how much error we are committing when substituting the true function with this approximation.

The fact that neural networks, even sometimes *nondeep* neural networks with only one hidden layer, have proved so successful for accomplishing various tasks in vision, speech recognition, classification, regression, and others means that they have some universal approximation property going on for them. The training function that a neural network represents (built from elementary linear combinations, biases, and very simple activation functions) approximates the underlying unknown function that truly represents or generates the data rather well.

The natural questions that mathematicians must now answer with a theorem, or a bunch of theorems, are:

Given some function that we don't know but we really care about (because we think it is the true function underlying or generating our data), is there a neural network that can approximate it to a good degree of precision (without ever having to know this true function)?

Practice using neural networks successfully suggests that the answer is yes, and universal approximation theorems for neural networks *prove* that the answer is yes for a certain class of functions and networks.

If there is a neural network that approximates this true and elusive data generating function, how do we construct it? How many layers should it have? How many nodes in each layer? What type of activation function should it include?

In other words, what is the *architecture* of this network? Sadly, as of now, little is known on how to construct these networks, and experimentation with various architectures and activations is the only way forward until more mathematicians get on this.

Are there multiple neural network architecture that work well? Are there some that are better than others?

Experiments suggest that the answer is yes, given the comparable performance of various architectures on the same tasks and data sets.

Note that having definite answers for these questions is very useful. An affirmative answer to the first question tells us: hey, there is no magic here, neural networks do approximate a *wide class of functions* rather well! This *wide coverage*, or universality, is crucial, because recall that we do not know the underlying generating function of the data, but if the approximation theorem covers a wide class of functions, our unknown and elusive function might as well be included, hence the success of the neural network. Answering the second and third sets of questions is even more useful for practical applications, because if we know which architecture works best for each task type and data set, then we would be saved from so much experimentation, and we'd immediately choose an architecture that performs well.

Before stating the universal approximation theorems for neural networks and discussing their proofs, let's go over two examples where we already encountered approximation type theorems, even when we were in middle school. The same principle applies for all examples: we have an unruly quantity that for whatever reason is difficult to deal with or is unknown, and we want to approximate it using another quantity that is easier to deal with. If we want universal results, we need to specify three things:

1. What class or what kind of *space* does the unruly quantity or function belong to? Is it the set of real numbers $\mathbb{R}$? The set of irrational numbers? The space of continuous functions on an interval? The space of compactly supported functions on $\mathbb{R}$? The space of Lebesgue measurable functions (I did slide in some *measure theory* stuff in here, hoping that no one notices or runs away)? Etc.
2. What kind of easier quantities or functions are we using to approximate the unruly entities, and how does using these quantities instead of the true function benefit us? How do these approximations fare against *other approximations*, if there are already some other popular approximations?
3. In what sense is the approximation happening, meaning that when we say we can approximate f_{true} using $f_{approximate}$, how exactly are we measuring the distance between f_{true} and $f_{approximate}$? Recall that in math we can measure sizes of objects, including distances, in many ways. So exactly which way are we using for our particular approximations? This is where we hear about the Euclidean norm, uniform norm, supremum norm, L^2 norm, etc. What do norms (sizes) have to do with distances? A norm induces a distance. This is intuitive: if our space allows us to talk about sizes of objects, then it better allow us talk about distances as well.

Example 1: Approximating irrational numbers with rational numbers

Any irrational number can be approximated by a rational number, up to any precision that we desire. Rational numbers are so well-behaved and useful, since they are just pairs of whole numbers. Our minds can easily intuit about whole numbers and fractions. Irrational numbers are quite the opposite. Have you ever been asked, maybe in grade 6, to calculate $\sqrt{47} = 6.8556546\dots$ without a calculator, and stay at it until you had a definite answer? I have. Pretty mean! Even calculators and computers approximate irrational numbers with rationals. But I had to sit there thinking I could keep writing digits until I either found a pattern or the computation terminated. Of course neither happened, and around 30 digits later, I learned that some numbers are just irrational.

There is more than one way to write a mathematical statement quantifying this approximation. They are all equivalent and useful:

The approximating entity can be made arbitrarily close to the true quantity

This is the most intuitive way.

Given an irrational number s and any precision ϵ , no matter how small, we can find a rational number q within a distance ϵ from s :

$$|s - q| < \epsilon$$

This means that rational and irrational numbers live arbitrarily close to each other on the real line $\mathbb{R}$. This introduces the idea of denseness.

Denseness and closure

Approximating entities are *dense* in the space where the true quantities live.

This means that if we focus only on the space of approximating members, then add in all the limits of all their sequences, we get the whole space of the true members. Adding in all the limiting points of a certain space S is called *closing* the space, or taking its *closure*, $\bar{S}$. For example, when we add to the open interval (a,b) its limit points a and b , we get the *closed* interval $[a,b]$. Thus the *closure* of (a,b) is $[a,b]$. We write $\overline{(a,b)} = [a,b]$.

The set of rational numbers $\mathbb{Q}$ is *dense* in the real line $\mathbb{R}$. In other words, the *closure* of $\mathbb{Q}$ is $\mathbb{R}$. We write $\overline{\mathbb{Q}} = \mathbb{R}$.

Limits of sequences

The true quantity is the limit of a sequence of the approximating entities.

The idea of adding in the limit points in the previous bullet introduces approximation using the terminology of sequences and their limits.

In the context of rational numbers approximating irrational numbers, we can therefore write: for any irrational number s , there is a sequence q_n of rational numbers such that $\lim_{n \rightarrow \infty} q_n = s$. This gives us the chance to write as an example one of the favorite definitions of the most famous irrational number: $e = 2.71828182\dots$

$$\lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = e$$

That is, the *irrational* number e is the limit of the sequence of *rational* numbers $\left(1 + \frac{1}{1}\right)^1, \left(1 + \frac{1}{2}\right)^2, \left(1 + \frac{1}{3}\right)^3, \dots$ which is equivalent to $2, 2.25, 2.370370\dots, \dots$.

Whether we approximate an irrational number with a rational number using the *arbitrarily close* concept, the *denseness and closure* concepts, or the *limits of sequences* concept, any distance involved in the mathematical statements is measured using the usual Euclidean norm: $d(s, q) = |s - q|$, which is the normal distance between two numbers.



Closeness Statements Need to Be Accompanied by a Specific Norm

We might wonder: what if we change the norm? Would the approximation property still hold? Can we still approximate irrationals using rationals if we measure the distance between them using some other definition of distance than the usual Euclidean norm? Welcome to *mathematical analysis*. In general, the answer is *no*. Quantities can be close to each other using some norm and very far using another norm. So in mathematics, when we say that quantities are close to each other, approximate others, or converge somewhere, we need to mention the accompanying norm in order to pinpoint in what sense these closeness statements are happening.

Example 2: Approximating continuous functions with polynomials

Continuous functions can be anything. A child can draw a wiggly line on a piece of paper and that would be a continuous function that no one knows the formula of. Polynomials, on the other hand, are a special type of continuous function that are extremely easy to evaluate, differentiate, integrate, explain, and do computations with. The only operations involved in polynomial functions are powers, scalar multiplication, addition, and subtraction. A polynomial of degree n has a simple formula:

$$p_n(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_nx^n$$

where the a_i 's are scalar numbers. Naturally, it is extremely desirable to be able to approximate nonpolynomial continuous functions using polynomial functions. The wonderful news is that we can, up to any precision ϵ . This is a classical result in mathematical analysis, called the Weierstrass Approximation Theorem:

Suppose f is a continuous real-valued function defined on a real interval $[a,b]$. For any precision $\epsilon > 0$, there exists a polynomial p_n such that for all x in $[a,b]$, we have $|f(x) - p_n(x)| < \epsilon$, or equivalently, the supremum norm $\|f - p_n\| < \epsilon$.

Note that the same principle as the one that we discussed for using rational numbers to approximate irrationals applies here. The theorem asserts that we can always find polynomials that are *arbitrarily close* to a continuous function, which means that the set of polynomials is *dense* in the space of continuous functions over the interval $[a,b]$; or equivalently, for any continuous function f , we can find a sequence of polynomial functions that converge to f (so f is the limit of a sequence of polynomials). In all of these variations of the same fact, the distances are measured with respect to the supremum norm. In [Figure 4-6](#), we verify that the continuous function $\sin x$ is the limit of the sequence of the polynomial functions $\left\{x, x - \frac{x^3}{3!}, x - \frac{x^3}{3!} + \frac{x^5}{5!}, \dots\right\}$.

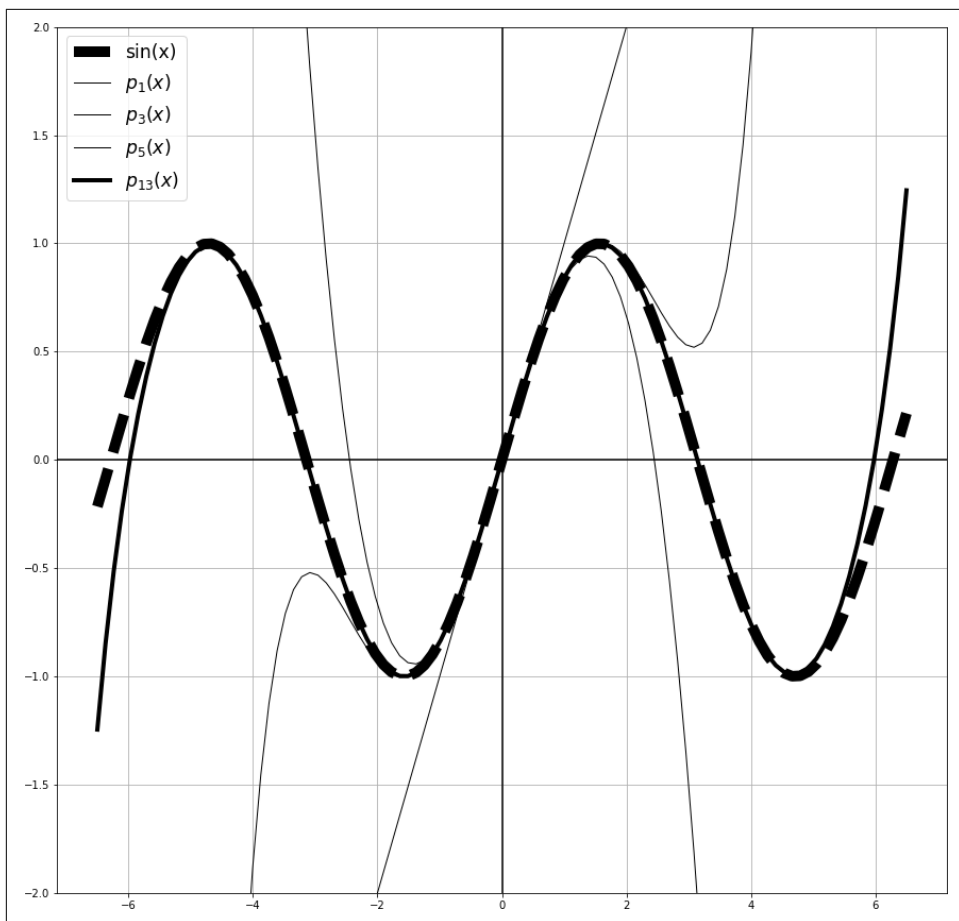


Figure 4-6. Approximation of the continuous function $\sin x$ by a sequence of polynomials

Statement of the universal approximation theorem for neural networks

Now that we understand the principles of approximation, let's state the most recent approximation theorems for neural networks.

Recall that a neural network is the representation of the training function as a computational graph. We want this training function to approximate the unknown function that generates the data well. This allows us to use the training function instead of the underlying true function, which we do not know and probably will never know, to make predictions. The following approximation theorems assert that neural networks can approximate the underlying functions up to any precision. When we compare the statements of these theorems to the two previous examples on irrational numbers

and continuous functions, we notice that they are the same kind of mathematical statements.

The following result is from [Hornik, Stinchcombe, and White \(1989\)](#): let f be a continuous function on a compact set K (this is the true but unknown function underlying the data) whose outputs are in $\mathbb{R}^d$. Then:

Arbitrarily close

There exists a feed forward neural network, having only a single hidden layer, which uniformly approximates f to within an arbitrary $\epsilon > 0$ on K .

Denseness

The set of neural networks, with prescribed nonlinear activations and bounds on the number of neurons and layers depending on d , is dense in the uniform topology of $C(K, \mathbb{R}^d)$.

In both variations of the same fact, the distances are measured with respect to the supremum norm on continuous functions.

The proof needs mathematical concepts from *measure theory* and *functional analysis*. We will introduce measure theory in [Chapter 11](#) on probability. For now we only list what is needed for the proof without any details: Borel and Radon measures, Hahn-Banach theorem, and Riesz representation theorem.

Approximation Theory for Deep Learning

We only motivated approximation theory and stated one of its main results for deep learning. For more information and a deeper discussion, we point to the state-of-the-art results such as [the ability of neural networks to learn probability distributions](#), Barron's theorem, the neural tangent kernel, [and others](#).

Loss Functions

Even though in this chapter we transitioned from [Chapter 3](#)'s traditional machine learning to the era of deep learning, the structure of the training function, loss function, and optimization is still exactly the same. The loss functions used for neural networks are not different from those discussed in [Chapter 3](#), since the goal of a loss function has not changed: to capture the error between the ground truth and prediction made by the training function. In deep learning, the neural network represents the training function, and for feed forward neural networks, we saw that this is nothing more than a sequence of linear combinations followed by compositions with nonlinear activation functions.

The most popular loss functions used in deep learning are still the mean squared error for regression tasks and the cross-entropy function for classification tasks. Go back to [Chapter 3](#) for a thorough explanation of these functions.

There are other loss functions that we sometimes come across in the field. When we encounter a new loss function, usually the designers of the model have a certain reason to prefer it over the other more popular ones, so make sure you go through their rationale for using that specific loss function for their particular setup. Ideally, a good loss function penalizes bad predictions, is not expensive to compute, and has one derivative that is easy to compute. We need this derivative so that our optimization method behaves well. As we discussed in [Chapter 3](#), functions with one good derivative have smoother terrains than functions with discontinuous derivatives, and hence are easier to navigate during the optimization process when searching for minimizers of the loss function.



Cross-Entropy Function, log-likelihood Function, and KL Divergence

Minimizing the cross-entropy loss function is the same as maximizing the log-likelihood function; KL divergence for probability distributions is closely related. Recall that the cross-entropy function is borrowed from information theory and statistical mechanics, and it quantifies the cross entropy between the true (empirical) distribution of the data and the distribution (of predictions) produced by the neural network's training function. The cross-entropy function has a negative sign and a log function in its formula. Minimizing the minus of a function is the same as maximizing the same function without the minus sign, so sometimes you encounter the following statement in the field: *maximizing the log likelihood function*, which for us is equivalent to *minimizing the cross-entropy loss function*. A closely related concept is the *Kullback-Leibler divergence*, also called *KL divergence*. Sometimes, as in the cases where we generate images or machine audio, we need to learn a probability distribution, not a deterministic function. Our loss function in this case should capture the *difference* (I will not say distance since its mathematical formula is not a distance metric) between the true probability distribution of the data and the learned probability distribution. KL divergence is an example of such a loss function that quantifies the amount of information lost when the learned distribution is used to approximate the true distribution, or the relative entropy of the true distribution with respect to the learned distribution.

Optimization

Faithful to our *training function*, *loss function*, and *optimization* mathematical structure, we now discuss the optimization step. Our goal is to perform an efficient search of the landscape of the loss function $L(\vec{\omega})$ to find the minimizing ω 's. Note that when we previously explicitly wrote formulas for the training functions of the neural network, we bundled up the ω weights in matrices W , and the biases in vectors $\vec{\omega}_0$. In this section, for the sake of simplifying notation and to keep the focus on the mathematics, we put all the weights and biases in one very long vector $\vec{\omega}$. That is, we write the loss function as $L(\vec{\omega})$, while in reality, for a fully connected neural network with h hidden layers, it is:

$$\text{Loss function} = L\left(W^1, \vec{\omega}_0^1, W^2, \vec{\omega}_0^2, \dots, W^{h+1}, \vec{\omega}_0^{h+1}\right)$$

We only need that representation when we explicitly compute the derivative of the loss function using *backpropagation*, which we will cover later in this chapter.

For deep learning, the number of ω 's in the vector $\vec{\omega}$ can be extremely high, as in tens of thousands, millions, or even billions. **OpenAI's GPT-2 for natural language** has 1.5 billion parameters, and was trained on a data set of eight million web pages. We need to solve for these many unknowns! Think parallel computing, or mathematical and algorithmic pipelining.

Using optimization methods, such as Newton-type methods that require computing matrices of second derivatives of the loss function in that many unknowns, is simply unfeasible even with our current powerful computational abilities. This is a great example where the mathematical theory of a numerical method works perfectly fine but is impractical for computational and real-life implementation. The sad part here is that numerical optimization methods that use the second derivative usually converge faster than those that use only the first derivative, because they take advantage of the extra knowledge about the concavity of the function (the shape of its bowl), as opposed to only using the information on whether the function is increasing or decreasing that the first derivative provides. Until we invent even more powerful computers, we have to satisfy ourselves with first-order methods that use only one derivative of the loss function with respect to the unknown ω 's. These are the *gradient-descent-type* methods, and luckily, they perform extremely well for many real-life AI systems that are currently deployed for use in our everyday life, such as Amazon's Alexa.

Mathematics and the Mysterious Success of Neural Networks

It is worth pausing here to reflect on the success of neural networks, which in the context of this section translates to: our ability to locate a minimizer for the loss function $L(\vec{\omega})$ that makes the training function generalize to new and unseen data really well. I do not have a North American accent, and Amazon's Alexa understands me perfectly fine. Mathematically, this success of neural networks is still puzzling for various reasons:

- The loss function's $\vec{\omega}$ -domain $L(\vec{\omega})$, where the search for the minimum is happening, is very high-dimensional (can reach billions of dimensions). We have billions or even trillions of options. How are we finding the right one?
- The landscape of the loss function itself is nonconvex, so it has a bunch of local minima and saddle points where optimization methods can get stuck or converge to the wrong local minimum. Again, how are we finding the right one?
- In some AI applications, such as computer vision, there are much more ω 's than data points (images). Recall that for images, each pixel is a feature, so that is already a lot of ω 's only at the input level. For such applications, there are much more unknowns (the ω 's) than information required to determine them (the data points). Mathematically, this is an *under-determined* system, and such systems have infinitely many possible solutions! So exactly how is the optimization method for our network picking up on the good solutions? The ones that generalize well?

Some of this mysterious success is attributed to techniques that have become a staple during the training process, such as regularization (discussed later in this chapter), validation, testing, etc. However, deep learning still lacks a solid theoretical foundation. This is why a lot of mathematicians have recently converged to answer such questions. The efforts of the National Science Foundation (NSF) in this direction, and the quotes that we copy next from its announcements are quite informative and give great insight on how mathematics is intertwined with advancing AI:

The NSF has recently established 11 new artificial intelligence research institutes to advance AI in various fields, such as human-AI interaction and collaboration, AI for advances in optimization, AI and advanced cyberinfrastructure, AI in computer and network systems, AI in dynamic systems, AI-augmented learning, and AI-driven innovation in agriculture and the food system. The NSF's ability to bring together numerous fields of scientific inquiry, including computer and information science and engineering, along with cognitive science and psychology, economics and game theory, engineering and control theory, ethics, linguistics, mathematics, and philosophy, uniquely positions the agency to lead the nation in expanding the frontiers of AI. NSF funding will help the U.S. capitalize on the full potential of AI to strengthen the economy, advance job growth, and bring benefits to society for decades to come.

The following is quoted from the NSF's [Mathematical and Scientific Foundations of Deep Learning \(SCALE MoDL\) webcast](#).

Deep learning has met with impressive empirical success that has fueled fundamental scientific discoveries and transformed numerous application domains of artificial intelligence. Our incomplete theoretical understanding of the field, however, impedes accessibility to deep learning technology by a wider range of participants. Confronting our incomplete understanding of the mechanisms underlying the success of deep learning should serve to overcome its limitations and expand its applicability. The SCALE MoDL program will sponsor new research collaborations consisting of mathematicians, statisticians, electrical engineers, and computer scientists. Research activities should be focused on explicit topics involving some of the most challenging theoretical questions in the general area of Mathematical and Scientific Foundations of Deep Learning. Each collaboration should conduct training through research involvement of recent doctoral degree recipients, graduate students, and/or undergraduate students from across this multidisciplinary spectrum. A wide range of scientific themes on theoretical foundations of deep learning may be addressed in these proposals. Likely topics include but are not limited to geometric, topological, Bayesian, or game-theoretic formulations, to analysis approaches exploiting optimal transport theory, optimization theory, approximation theory, information theory, dynamical systems, partial differential equations, or mean field theory, to application-inspired viewpoints exploring efficient training with small data sets, adversarial learning, and closing the decision-action loop, not to mention foundational work on understanding success metrics, privacy safeguards, causal inference, and algorithmic fairness.

Gradient Descent
$$\vec{\omega}^{i+1} = \vec{\omega}^i - \eta \nabla L(\vec{\omega}^i)$$

The widely used gradient descent method for optimization in deep learning is so simple that we could fit its formula in this subsection's title. This is how gradient descent searches the landscape of the loss function $L(\vec{\omega})$ for a local minimum:

Initialize somewhere at $\vec{\omega}^0$

Randomly pick starting numerical values for $\vec{\omega}^0 = (\omega_0, \omega_1, \dots, \omega_n)$. This choice places us somewhere in the search space and at the landscape of $L(\vec{\omega})$. One big warning here: *where we start matters!* Do not initialize with all zeros or all equal numbers. This will diminish the network's ability to learn different features, since different nodes will output exactly the same numbers. We will discuss initialization shortly.

Move to a new point $\vec{\omega}^1$

The gradient descent moves in the direction opposite to the gradient vector of the loss function $-\nabla L(\vec{\omega}^0)$. This is guaranteed to decrease the gradient if the step size η , also called the *learning rate*, is not too large:

$$\vec{\omega}^1 = \vec{\omega}^0 - \eta \nabla L(\vec{\omega}^0)$$

Move to a new point $\vec{\omega}^2$

Again, the gradient descent moves in the direction opposite to the gradient vector of the loss function $-\nabla L(\vec{\omega}^1)$. This is guaranteed to decrease the gradient if the learning η is not too large:

$$\vec{\omega}^2 = \vec{\omega}^1 - \eta \nabla L(\vec{\omega}^1)$$

Keep going until the sequence of points $\{\vec{\omega}^0, \vec{\omega}^1, \vec{\omega}^2, \dots\}$ converges

Note that in practice, we sometimes have to stop before it is clear that this sequence has converged—for example, when it becomes painfully slow due to a flattening landscape.

Figure 4-7 shows minimizing a certain loss function $L(\omega_1, \omega_2)$ using gradient descent. We as humans are limited to the three-dimensional space that we exist in, so we cannot visualize beyond three dimensions. This is a severe limitation for us in terms of visualization since our loss functions usually act on very high-dimensional spaces. They are functions of many ω 's, but we can only visualize them accurately if they depend on at most two ω 's. That is, we can visualize a loss function $L(\omega_1, \omega_2)$ depending on two ω 's, but not a loss function $L(\omega_1, \omega_2, \omega_3)$ depending on three (or more) ω 's. Even with this severe limitation on our capacity to visualize loss functions acting on high-dimensional spaces, Figure 4-7 gives an accurate picture of how the gradient descent method operates in general. In Figure 4-7, the search happens in the two-dimensional (ω_1, ω_2) plane (the flat ground in Figure 4-7), and we track the progress on the landscape of the function $L(\omega_1, \omega_2)$ that is embedded in $\mathbb{R}^3$. The search space always has one dimension less than the dimension of the space in which the landscape of the loss function is embedded. This makes the optimization process harder, since we are looking for a minimizer of a busy landscape in a flattened or squished version of its terrain (the ground level in Figure 4-7).

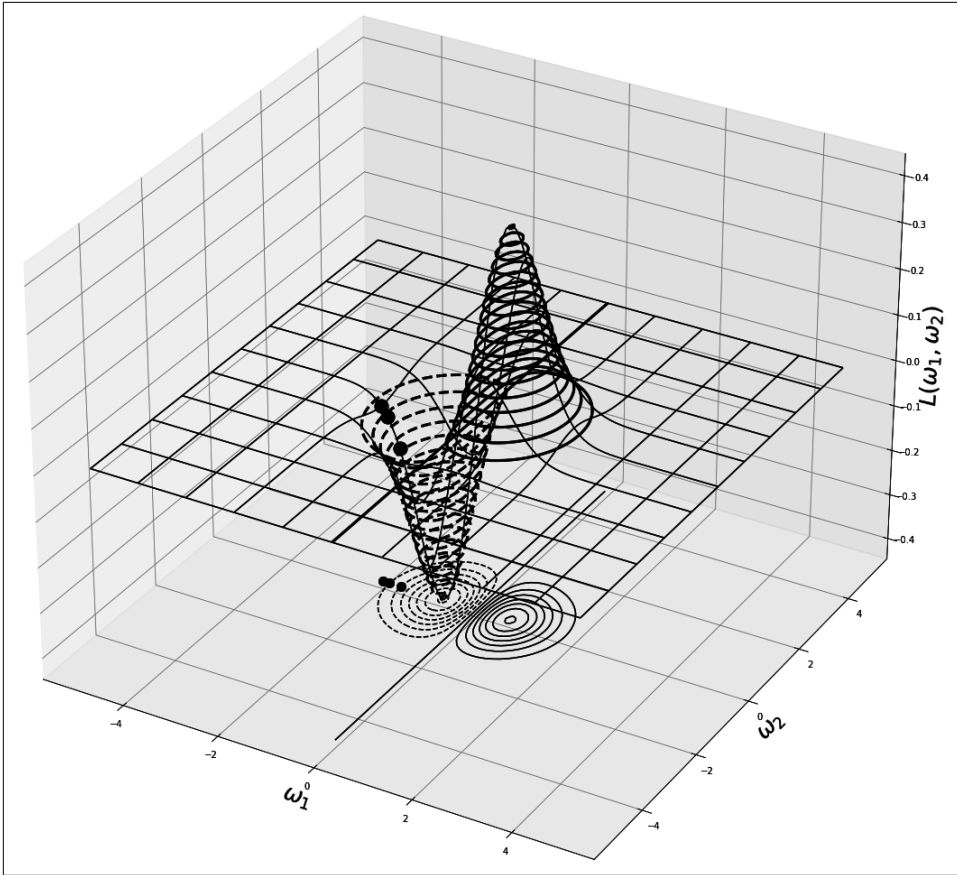


Figure 4-7. Two gradient descent steps. Note that if we start on the other side of the mountain, we wouldn't converge to the minimum. So when we are searching for the minimum of a nonconvex function, where we start or how we initiate the ω 's matters a lot.

Explaining the Role of the Learning Rate Hyperparameter η

At each iteration, the gradient descent method $\vec{\omega}^{i+1} = \vec{\omega}^i - \eta \nabla L(\vec{\omega}^i)$ moves us from the point $\vec{\omega}^i$ in the search space to another point $\vec{\omega}^{i+1}$. The gradient descent adds $-\eta \nabla L(\vec{\omega}^i)$ to the current $\vec{\omega}^i$ to obtain $\vec{\omega}^{i+1}$. The quantity $-\eta \nabla L(\vec{\omega}^i)$ is made up of a scalar number η multiplied by the negative of the gradient vector $-\nabla L(\vec{\omega}^i)$, which points in the direction of the steepest decrease of the loss function from the point $\vec{\omega}^i$. Thus, the scaled $-\eta \nabla L(\vec{\omega}^i)$ tells us how far in the search space we are going to go

along the steepest descent direction in order to choose the next point $\vec{\omega}^{i+1}$. In other words, the vector $-\nabla L(\vec{\omega}^i)$ specifies in which direction we will move away from our current point, and the scalar number η , called the *learning rate*, controls how far we are going to step along that direction. Figure 4-8 shows one step of gradient descent with two different learning rates η . Too large of a learning rate might overshoot the minimum and cross to the other side of the valley. On the other hand, too small of a learning rate takes a while to get to the minimum. So the trade-off is between choosing a large learning rate and risking overshooting the minimum, and choosing a small learning rate and increasing computational cost and time for convergence.

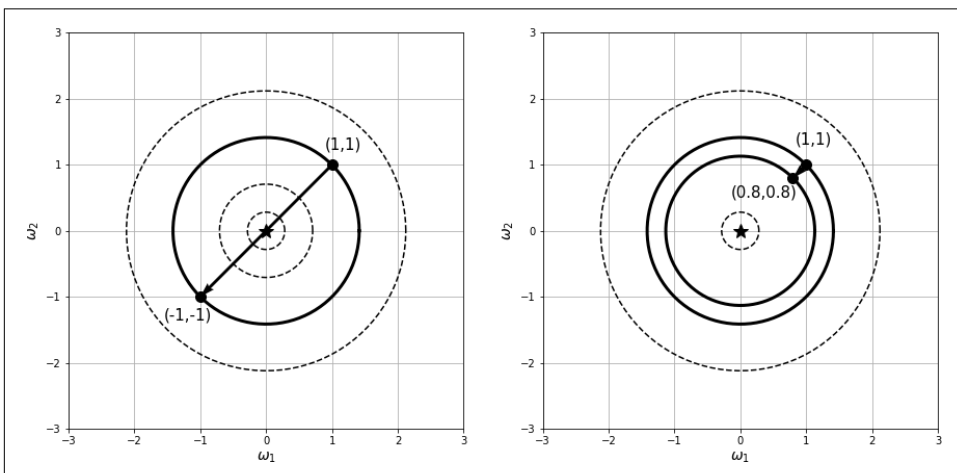


Figure 4-8. One-step gradient descent with two different learning rates. On the left, the learning rate is too large, so the gradient descent overshoots the minimum (the star point) and lands on the other side of the valley. On the right, the learning rate is small, however, it will take a while to get to the minimum (the star point). Note how the gradient vector at a point is perpendicular to the level set at that point.

The learning rate η is another example of a hyperparameter of a machine learning model. It is not one of the weights that goes into the formula of the training function. It is a parameter that is intrinsic to the algorithm that we employ to estimate the weights of the training function.

The scale of the features affects the performance of the gradient descent

This is one of the reasons to standardize the features ahead of time. Standardizing a feature means subtracting from each data instance the mean and dividing by the standard deviation. This forces all the data values to have the same scale, with mean zero and standard deviation one, as opposed to having vastly different scales, such as

a feature measured in the millions and another measured in 0.001. But why does this affect the performance of the gradient descent method? Read on.

Recall that the values of the input features get multiplied by the weights in the training function, and the training function in turn enters into the formula of the loss function. Very different scales of the input features change the shape of the bowl of the loss function, making the minimization process harder. Figure 4-9 shows the level sets of the function $L(\omega_1, \omega_2) = \omega_1^2 + a\omega_2^2$ with different values of a , mimicking different scales of input features. Note how the level sets of loss function become much more narrow and elongated as the value of a increases. This means that the shape of the bowl of the loss function is a long, narrow valley.

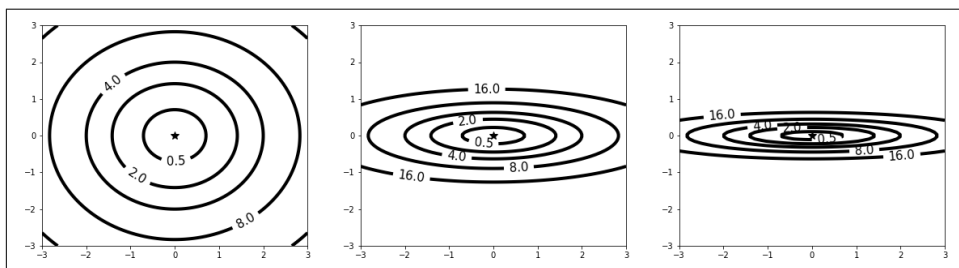


Figure 4-9. The level sets of the loss function $L(\omega_1, \omega_2) = \omega_1^2 + a\omega_2^2$ become much more narrow and elongated as the value of a increases from 1 to 20 to 40

When the gradient descent method tries to operate in such a narrow valley, its points hop from one side of the valley to the other, zigzagging as it tries to locate the minimum, and slowing down the convergence considerably. Imagine zigzagging along all the streets of Rome before arriving at the Vatican, as opposed to taking a helicopter straight to the Vatican.

But why does this zigzagging behavior happen? One hallmark of the gradient vector of a function is that it is perpendicular to the level sets of that function. So if the valley of the loss function is so long and narrow, its level sets almost look like lines that are parallel to each other, and with a large enough step size (learning rate), we can literally cross from one side of the valley to the other since it is so narrow. Google *gradient descent zigzag* and you will see many images illustrating this behavior.

One fix for zigzagging, even with a narrow, long valley (assuming we did not scale the input feature values ahead of time), is to choose a very small learning rate, preventing the gradient descent method from stepping from one side of the valley to the other. However, that slows down the arrival to the minimum in its own way, since the method will step only incrementally at each iteration. We will eventually arrive at the Vatican from Rome, but at a turtle's pace.

Near the minima (local and/or global), flat regions, or saddle points of the loss function's landscape, the gradient descent method crawls

The gradient descent method updates the current point $\vec{w}^i$ by adding the vector $-\eta \nabla L(\vec{w}^i)$. Therefore, the exact length of the step from the point $\vec{w}^i$ in the direction of the negative gradient vector is η multiplied by the length of the gradient vector $\nabla L(\vec{w}^i)$. At a minimum, maximum, saddle point, or any flat region of the landscape of the loss function, the gradient vector is zero, hence its length is zero as well. This means that near a minimum, maximum, saddle point, or any flat region, the step size of the gradient descent method becomes very small, and the method slows down significantly. If this happens near a minimum, then there is not much worry since this can be used as a stopping criterion, unless this minimum is a local minimum very far from the global minimum. If, on the other hand, it happens in a flat region or near a saddle point, then the method will get stuck there for a while, and that is undesirable. Some practitioners put the learning rate η on a schedule, changing its value during the optimization process. When we look into these, we notice that the goals are to avoid crawling, save computational time, and speed up convergence.

We will discuss stochastic (random) gradient descent later in this chapter. Due to the random nature of this method, the points hop around a lot, as opposed to following a more consistent route toward the minimum. This works to our advantage in situations where we are stuck, such as saddle points or local minima, since we might get randomly propelled out of the local minimum or away from the saddle point into a part of the landscape with a better route toward the minimum.

Convex Versus Nonconvex Landscapes

We cannot have an optimization chapter without discussing *convexity*. In fact, entire mathematical fields are dedicated solely to *convex optimization*. It is equally important to immediately note that optimization for neural networks is in general, *nonconvex*.

When we use nonconvex activation functions, such as the sigmoid-type functions in the first row of [Figure 4-5](#), the landscapes of the loss functions involved in the resulting neural networks are not convex. This is why we spend a good amount of time talking about getting stuck at local minima, flat regions, and saddle points, which we wouldn't worry about for convex landscapes. The contrast between convex and nonconvex landscapes is obvious in [Figure 4-10](#), which shows a convex loss function and its level sets, and [Figures 4-11](#) and [4-12](#), which show nonconvex functions and their level sets.

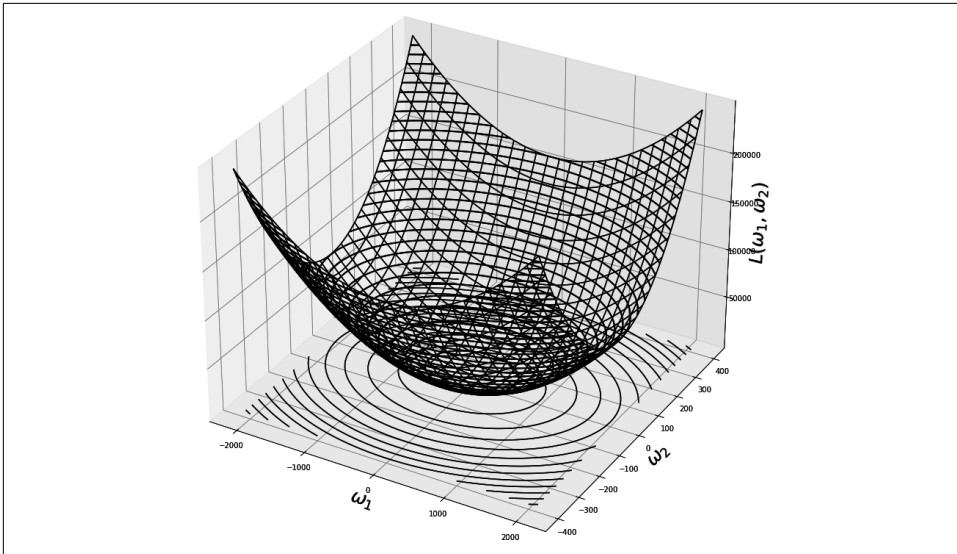


Figure 4-10. Plot of three-dimensional convex function and its level sets. Gradient vectors live in the same space ($\mathbb{R}^2$) as the level sets, not in $\mathbb{R}^3$.

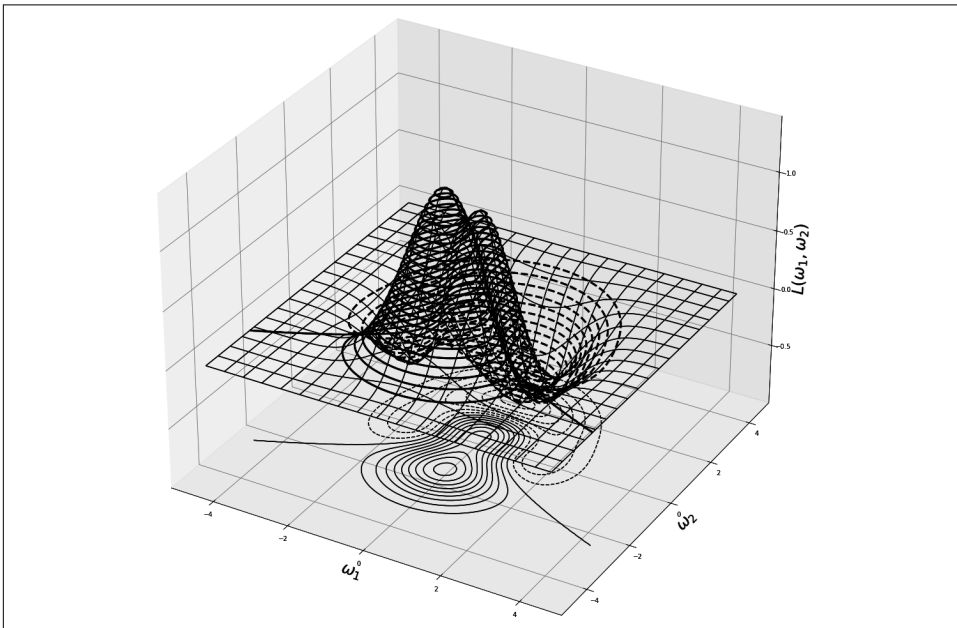


Figure 4-11. Plots of three-dimensional nonconvex functions and their level sets. Gradient vectors live in the same space ($\mathbb{R}^2$) as the level sets, not in $\mathbb{R}^3$.

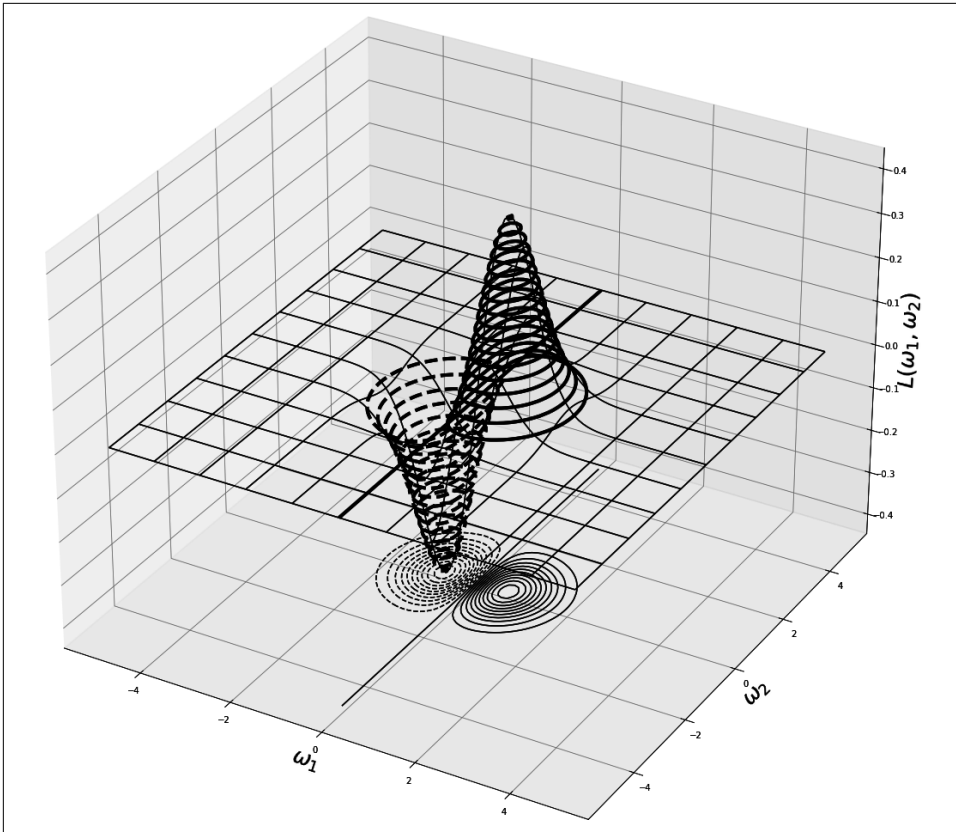


Figure 4-12. Plots of three-dimensional nonconvex functions and their level sets. Gradient vectors live in the same space ($\mathbb{R}^2$) as the level sets, not in $\mathbb{R}^3$.

When we use convex activation functions throughout the network, such as the ReLU-type functions in the second row of [Figure 4-5](#), and convex loss functions, we can still end up with a *nonconvex* optimization problem, because the composition of two convex functions is not necessarily convex. If the loss function happens to be nondecreasing and convex, then its composition with a convex function is convex. The loss functions that are popular for neural networks, such as mean squared error, cross-entropy, and hinge loss, are all convex but not nondecreasing.

It is important to become familiar with central concepts from convex optimization. If you do not know where to start, keep in mind that convexity replaces linearity when linearity is too simplistic or unavailable, then learn everything about the following (which will be tied to AI, deep learning, and reinforcement learning when we discuss operations research in [Chapter 10](#)):

- Max of linear functions is convex
- Max-min and min-max
- Saddle points
- Two-player zero-sum games
- Duality

Since convex optimization is such a well-developed and understood field (at least more than the mathematical foundations for neural networks), and neural networks still have a long way to go mathematically, it would be nice if we could exploit our knowledge about convexity in order to gain a deeper understanding of neural networks. Research in this area is ongoing. For example, in a [2020 paper](#) titled “Convex Geometry of Two-Layer ReLU Networks: Implicit Autoencoding and Interpretable Models,” Tolga Ergen and Mert Pilanci frame the problem of training two layered ReLU networks as a convex analytic optimization problem. The following is the abstract of the paper:

We develop a convex analytic framework for ReLU neural networks which elucidates the inner workings of hidden neurons and their function space characteristics. We show that rectified linear units in neural networks act as convex regularizers, where simple solutions are encouraged via extreme points of a certain convex set. For one dimensional regression and classification, we prove that finite two-layer ReLU networks with norm regularization yield linear spline interpolation. In the more general higher dimensional case, we show that the training problem for two-layer networks can be cast as a convex optimization problem with infinitely many constraints. We then provide a family of convex relaxations to approximate the solution, and a cutting-plane algorithm to improve the relaxations. We derive conditions for the exactness of the relaxations and provide simple closed form formulas for the optimal neural network weights in certain cases. Our results show that the hidden neurons of a ReLU network can be interpreted as convex autoencoders of the input layer. We also establish a connection to $l_0 - l_1$ equivalence for neural networks analogous to the minimal cardinality solutions in compressed sensing. Extensive experimental results show that the proposed approach yields interpretable and accurate models.

Stochastic Gradient Descent

So far, training a feed forward neural network has progressed as follows:

1. Fix an initial set of weights $\vec{w}^0$ for the training function.
2. Evaluate this training function at *all* the data points in the training subset.
3. Calculate the individual losses at *all* the data points in the training subset by comparing their true labels to the predictions made by the training function.
4. Do this for *all* the data in the training subset.

5. Average *all* these individual losses. This average is the loss function.
6. Evaluate the gradient of this loss function at this initial set of weights.
7. Choose the next set of weights according to the steepest descent rule.
8. Repeat until you converge somewhere, or stop after a certain number of iterations determined by the performance of the training function on the validation set.

The problem with this process is that when we have a large training subset with thousands of points, and a neural network with thousands of weights, it gets too expensive to evaluate the training function, the loss function, and the gradient of the loss function on *all* the data points in the training subset. The remedy is to randomize the process: randomly choose a very small portion of the training subset to evaluate the training function, loss function, and gradient of this loss function at each step. This slashes the computational cost dramatically.

Keep repeating this random selection (in principle with replacement but in practice without replacement) of small portions of the training subset until you converge somewhere, or stop after a certain number of iterations determined by the performance of the training function on the validation set. One pass through the whole training subset is called *one epoch*.

Stochastic gradient descent performs remarkably well, and it has become a staple in training neural networks.

Initializing the Weights $\vec{\omega}^0$ for the Optimization Process

We have already established that initializing with all zero weights or all the same weights is a really bad idea. The next logical step, and what was the traditional practice (before 2010), would be to choose the weights in the initial $\vec{\omega}^0$ randomly, sampled either from the uniform distribution over small intervals, such as $[-1,1]$, $[0,1]$, or $[-0.3,0.3]$, or from the Gaussian distribution with a preselected mean and variance. Even though this has not been studied in depth, it seems from empirical evidence that it doesn't matter whether the initial weights are sampled from the uniform distribution or Gaussian distribution, but it does seem that the scale of the initial weights matters when it comes to both the progress of the optimization process and the ability of the network to generalize well to unseen data. It turns out that some choices are better than others in this respect. Currently, the two state-of-the-art choices depend on the choice of the activation function: whether it is sigmoid-type or ReLU-type.

Xavier Glorot initialization

Here, initial weights are sampled from uniform distribution over the interval $[-\frac{\sqrt{6}}{\sqrt{n+m}}, \frac{\sqrt{6}}{\sqrt{n+m}}]$, where n is the number of inputs to the node (e.g., the number of nodes in the previous layer), and m is the number of outputs from the layer (e.g., the number of nodes in the current layer).

Kaiming He initialization

Here, the initial weights are sampled from the Gaussian distribution with zero mean and variance $2/n$, where n is the number of inputs to the node.

Regularization Techniques

Regularization helps us arrive at a good choice for the weights of the training function while at the same time avoiding overfitting the data. We want our trained function to follow the signal in the data rather than the noise, so it can generalize well to unseen data. Here we include four simple yet popular regularization techniques that are used while training a neural network: dropout, early stopping, batch normalization, and weight decay (ridge, lasso, and elastic net) regularizations.

Dropout

Drop some randomly selected neurons from each layer during training. Usually, about 20% of the input layer's nodes and about half of each of the hidden layers' nodes are randomly dropped. No nodes from the output layer are dropped. Dropout is partially inspired by genetic reproduction, where half a parent's genes are dropped and there is a small random mutation. This has the effect of training different networks at once (with a different number of nodes at each layer) and averaging their results, which typically produces more reliable results.

One way to implement dropout is by introducing a hyperparameter p for each layer that specifies the probability at which each node in that layer will be dropped. Recall the basic operations that take place at each node: linearly combine the outputs of the nodes of the previous layer, then activate. With dropout, each output of the nodes of the previous layer (starting with the input layer), is multiplied by a random number r , which can be either 0 or 1 with probability p . Thus when a node's r takes the value 0, that node is essentially dropped from the network, which now forces the other retained nodes to pick up the slack when adjusting the weights in one gradient descent step. We will explain this further in [“Backpropagation in Detail” on page 155](#), and this [link](#) provides a step-by-step route to implementing dropout.

For a deeper mathematical exploration, a [2015 paper](#) connects dropout to Bayesian approximations of model uncertainty.

Early Stopping

As we update the weights during training, in particular during gradient descent, after each epoch, we evaluate the error made by the training function at the current weights on the validation subset of the data.

This error should be decreasing as the model learns the training data; however, after a certain number of epochs, this error will start increasing, indicating that the training function has now started overfitting the training data and is failing to generalize well to the validation data. Once we observe this increase in the model's prediction over the validation subset, we stop training and go back to the set of weights where that error was lowest, right before we started observing the increase.

Batch Normalization of Each Layer

The main idea here is to normalize the inputs to each layer of the network. This means that the inputs to each layer will have mean 0 and variance 1. This is usually accomplished by subtracting the mean and dividing by the variance for each of the layer's inputs. We will detail this in a moment. The reason this is good to do at each hidden layer is similar to why it is good at the original input layer.

Applying *batch normalization* often eliminates the need for dropout, and allows us to be less particular about initialization. It makes the training faster and safer from vanishing and exploding gradients. It also has the added advantage of regularization. The cost for all of these gains is not too high, as it usually involves training only two additional parameters, one for scaling and one for shifting, at each layer.

The [2015 paper by Ioffe and Szegedy](#) introduced the method. The abstract of their paper describes the batch normalization process and the problems it addresses (the brackets are my own comments):

Training Deep Neural Networks is complicated by the fact that the distribution of each layer's inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models with saturating nonlinearities [such as the sigmoid type activation functions, in [Figure 4-5](#), which become almost constant, outputting the same value when the input is large in magnitude. This renders the nonlinearity useless in the training process, and the network stops learning at subsequent layers]. We refer to this phenomenon [the change in the distribution of the inputs to each layer] as internal covariate shift, and address the problem by normalizing layer inputs. Our method draws its strength from making normalization a part of the model architecture and performing the normalization for each training mini-batch. Batch Normalization allows us to use much higher learning rates and be less careful about initialization, and in some cases eliminates the need for Dropout. Applied to a state-of-the-art image classification model, Batch Normalization achieves the same accuracy with 14 times fewer training steps, and beats the original model by a significant margin. Using an ensemble of batch-normalized networks, we improve

upon the best published result on ImageNet classification: reaching 4.82% top-5 test error, exceeding the accuracy of human raters.

Batch normalization is often implemented in the architecture of a network either in its own layer before the activation step, or after activation. The process, *during training*, usually follows these steps:

1. Choose a batch from the training data of size b . Each data point in this has feature vector $\vec{x}_i$, so the whole batch has feature vectors $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_b$.
2. Calculate the vector whose entries are the means of each feature in this particular batch: $\vec{\mu} = \frac{\vec{x}_1 + \vec{x}_2 + \dots + \vec{x}_b}{b}$.
3. Calculate the variance across the batch: subtract $\vec{\mu}$ from each $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_b$, calculate the result's l^2 norm, add, and divide by b .
4. Normalize each of $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_b$ by subtracting the mean and dividing by the square root of the variance:
5. Scale and shift by trainable parameters that can be initialized and learned by gradient descent, the same way the weights of the training function are learned. This becomes the input to the first hidden layer.
6. Do the same for the input of each of the subsequent layers.
7. Repeat for the next batch.

During testing and prediction, there is no batch of data to train on, and the parameters at each layer are already learned. The batch normalization step, however, is already incorporated into the formula of the training function. During training, we were changing these *per batch* of training data. This in turn was changing the formula of the loss function slightly per batch. However, the point of normalization was partly not to change the formula of the loss function too much, because that in turn would change the locations of its minima, and that would cause us to forever chase a moving target. Alright, we fixed that with batch normalization during training, and now we want to validate, test, and predict. Which mean vector and variance do we use for a particular data point that we are testing/predicting at? Do we use the means and variances of the features of the original data set? We have to make such decisions.

Control the Size of the Weights by Penalizing Their Norm

Another way to regularize the training function to avoid overfitting the data is to introduce a *competing term* into the minimization problem. Instead of solving for the set of weights $\vec{\omega}$ that minimizes *only* the loss function:

$$\min_{\vec{\omega}} L(\vec{\omega})$$

we introduce a new term $\alpha \|\vec{\omega}\|$ and solve for the set of weights $\vec{\omega}$ that minimizes:

$$\min_{\vec{\omega}} L(\vec{\omega}) + \alpha \|\vec{\omega}\|$$

For example, for the mean squared error loss function usually used for regression problems, the minimization problem looks like:

$$\min_{\vec{\omega}} \frac{1}{m} \sum_{i=1}^m \left| y_{\text{predict}}(\vec{\omega}) - y_{\text{true}} \right|^2 + \alpha \|\vec{\omega}\|$$

Recall that so far we have established two ways to solve that minimization problem:

The minimum happens at points where the derivative (gradient) is equal to zero

So the minimizing $\vec{\omega}$ must satisfy $\nabla L(\vec{\omega}) + \alpha \nabla(\|\vec{\omega}\|) = 0$. Then we solve this equation for $\vec{\omega}$ if we have the luxury to get a *closed form* for the solution. In the case of linear regression (which we can think about as an extremely simplified neural network, with only one layer and zero nonlinear activation function), we do have this luxury, and for this *regularized* case, the formula for the minimizing $\vec{\omega}$ is:

$$\vec{\omega} = (X^t X + \alpha B)^{-1} X^t \vec{y}_{\text{true}}$$

where the columns of X are the feature columns of the data augmented with a vector of ones, and B is the identity matrix (if we use ridge regression, discussed later). The closed form solution for the extremely simple linear regression problem with regularization helps us appreciate weight decay type regularization and see the important role it plays. Instead of inverting the matrix $(X^t X)$ in the *unregularized* solution and worrying about its ill-conditioning (for example, from highly correlated input features) and the resulting instabilities, we invert $(X^t X + \alpha B)$ in the *regularized* solution. Adding this αB term is equivalent to adding a small positive term to the denominator of a scalar number that helps us avoid division by zero. Instead of using $1/x$ where x runs the risk of being zero, we use $1/(x + \alpha)$ where α is a positive constant. Recall that matrix inversion is the analog of scalar number division.

Gradient descent

We use gradient descent or any of its variations, such as stochastic gradient descent, when we do not have the luxury of obtaining closed form solutions for the derivative equals zero equation, and when our problem is very large, that computing second-order derivatives is extremely expensive.

Commonly used weight decay regularizations

There are three popular regularizations that control the size of the weights that we are forever searching for in this book:

Ridge regression

Penalize the l^2 norm of $\vec{\omega}$. In this case, we add the term $\alpha \sum_{i=1}^n |\omega_i|^2$ to the loss function, then we minimize.

Lasso regression

Penalize the l^1 norm of the ω 's. In this case, we add the term $\alpha \sum_{i=1}^n |\omega_i|$ to the loss function, then we minimize.

Elastic net

This is a middle-ground case between ridge and lasso regressions. We introduce one additional hyperparameter γ , which can take any value between zero and one, and add a term to the loss function that combines both ridge and lasso regressions through $\gamma: \gamma \alpha \sum_{i=1}^n |\omega_i|^2 + (1 - \gamma) \alpha \sum_{i=1}^n |\omega_i|$. When $\gamma = 0$, this becomes lasso regression; when it is equal to one, it is ridge regression; and when it is between zero and one, it is some sort of a middle ground.

When do we use plain linear regression, ridge, lasso, or elastic net?

If you are already confused and slightly overwhelmed by the multitude of choices that are available for building machine learning models, join the club, but do not get frustrated. Until the mathematical analysis that tells us exactly which choices are better than others and under what circumstances becomes available (or catches us with mathematical computation and experimentation), think about the enormity of available choices the same way you think about a home renovation: we have to choose from many available materials, designs, and architectures to produce a final product. This is a home renovation, not a home decoration, so our decisions are fateful and more consequential than a mere home decoration. They do affect the *quality* and the *function* of our final product, but they are choices nevertheless. Rest easy, there is more than one way to skin AI:

- Some regularization is always good. Adding a term that controls the sizes of the weights and competes with minimizing the loss function is good in general.
- Ridge regression is usually a good choice because the l^2 norm is differentiable. Minimizing this is more stable than minimizing the l^1 norm.
- If we decide to go with the l^1 norm, even though it is not differentiable at 0, we can still define its *subdifferential* or *subgradient* at 0. For example, we can set that to be zero. Note that $f(x) = |x|$ is differentiable when $x \neq 0$: it has derivative 1 when $x > 0$ and -1 when $x < 0$; the only problematic point is $x=0$.
- If we suspect only a few features are useful, then it is good to use either lasso or elastic net as a data preprocessing step to kill off the less important features.
- Elastic net is usually preferred over lasso because lasso might behave badly when the number of features is greater than the number of training instances or when several features are strongly correlated.

Penalizing the l^2 Norm Versus Penalizing the l^1 Norm

Our goal is to find $\vec{\omega}$ that solves the minimization:

$$\min_{\vec{\omega}} L(\vec{\omega}, \vec{\omega}_0) + \alpha \|\vec{\omega}\|$$

The first term wants to decrease the loss $L(\vec{\omega}, \vec{\omega}_0)$. The other term wants to decrease the values of the coordinates of $\vec{\omega}$ all the way to zeros. The type of the norm that we choose for $\|\vec{\omega}\|$ determines the path $\vec{\omega}$ follows on its way to $\vec{0}$.

If we use the l^1 norm, the coordinates of $\vec{\omega}$ will decrease; however, a lot of them might encounter premature death, hitting zero before others. That is, the l^1 norm encourages sparsity: when a weight dies, it kills the contribution of the associated feature to the training function.

The plot on the right in [Figure 4-13](#) shows the diamond-shaped level sets of $\|\vec{\omega}\|_{l^1} = |\omega_1| + |\omega_2|$ in two dimensions (if we only had two features), namely, $|\omega_1| + |\omega_2| = c$ for various values of c . If a minimization algorithm follows the path of steepest descent, such as the gradient descent, then we must travel in the direction perpendicular to the level sets, and as the arrow shows in the plot, ω_2 becomes zero pretty fast since going perpendicular to the diamond-shaped level sets is bound to hit one of the coordinate axes, effectively killing the respective feature. ω_1 then travels to zero along the horizontal axis.

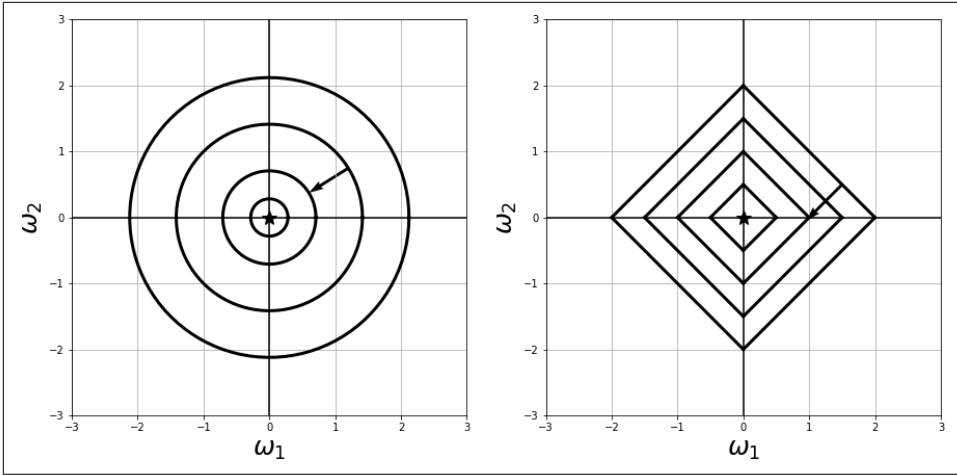


Figure 4-13. The plot on the left shows the circular level sets of the l^2 norm of $\vec{\omega}$, along with the direction the gradient descent follows toward the minimum at (0,0). The plot on the right shows the diamond-shaped level sets of the l^1 norm of $\vec{\omega}$, along with the direction the gradient descent follows toward the minimum at (0,0).

- If we use the l^2 norm, the weight sizes get smaller without necessarily killing them. The plot on the left in [Figure 4-13](#) shows the circular-shaped level sets of $\|\vec{\omega}\|_{l^2}^2 = \omega_1^2 + \omega_2^2$ in two dimensions, namely, $\omega_1^2 + \omega_2^2 = c$ for various values of c . We see that following the path perpendicular to the circular level sets toward the minimum at (0,0) decreases the values of both ω_1 and ω_2 without either of them becoming zero before the other.

Which norm to choose depends on our use cases. Note that in all cases, we do not regularize the bias weights $\vec{\omega}_0$. This is why in this section we wrote them separately in the loss function $L(\vec{\omega}, \vec{\omega}_0)$.

Explaining the Role of the Regularization Hyperparameter α

The minimization problem with weight decay regularization looks like:

$$\min_{\vec{\omega}} L(\vec{\omega}) + \alpha \|\vec{\omega}\|$$

To understand the role of the regularization hyperparameter α , we observe the following:

- There is a competition between the first term, where the loss function $L(\vec{\omega})$ chooses ω 's that fit the training function to the training data, and the second term that just cares about making the ω values small. These two objectives are not necessarily in sync. The values of ω 's that make the first term smaller might make the second term bigger and vice versa.
- If α is big, then the minimization process will compensate by making values of ω 's very small, regardless of whether these small values of ω 's will make the first term small as well. So the more we increase α , the more important minimizing the second term becomes than the first term, so our ultimate model might end up not fitting the data perfectly (high bias), but this is sometimes desired (low variance) so that it generalizes well to unseen data.
- If, on the other hand, α is small (say, close to zero), then we can choose larger ω values, and minimizing the first term becomes more important. Here, the minimization process will result in ω values that make the first term happy, so the data will fit into the model nicely (low bias) but the variance might be high. In this case, our model would work well on seen data (it is designed to fit it nicely through minimizing $L(\vec{\omega})$), but might not generalize well to unseen data.
- As $\alpha \rightarrow 0$, we can prove mathematically that the solution of the regularized problem converges to the solution of the unregularized problem.

Hyperparameter Examples That Appear in Machine Learning

We have now encountered many hyperparameters that enter machine learning models. It is good practice to list the ones that enter our particular model along with their values. Let's list the ones we have come across and recall that tuning these enhances the performance of our models. Most of the time, there are recommended values for us to use. These are usually implemented as default values in machine learning libraries and software packages. However, it is always good to experiment with different values during the validation stage of our modeling process, given that we have the available time and resources. The hyperparameters include:

- The learning rate in gradient descent.
- Weight decay coefficients, such as the ones that appear in ridge, lasso, and elastic net regularizations.
- The number of epochs before we stop training.
- The sizes of data split into training, validation, and testing subsets.

- The sizes of mini-batches during stochastic gradient descent and its variants.
- The acceleration coefficients in momentum methods.
- The architecture of a neural network: number of layers, number of neurons in each layer, what happens at each layer (batch normalization, type of activation function), type of regularization (dropout, ridge, lasso), type of network (feed forward, dense, convolutional, adversarial, recurrent), type of loss functions, etc.

Chain Rule and Backpropagation: Calculating $\nabla L(\vec{\omega}^i)$

It is time to get our hands dirty and compute something important: the gradient of the loss function, namely, $\nabla L(\vec{\omega}^i)$. Whether we decide to find our optimal weights using gradient descent, stochastic gradient descent, mini-batch gradient descent, or any other variant of gradient descent, there is no escape from calculating this quantity. Recall that the loss function includes in its formula the neural network's training function, which in turn is made up of subsequent linear combinations and compositions with activation functions. This means that we have to cleverly use the chain rule. Back in calculus, we only used the single variable chain rule for derivatives, but now we somehow have to transition to a chain rule of several variables: several, as in, sometimes billions.

It is the layered architecture of a neural network that forces us to pause and think: how exactly are we going to compute this one derivative of the loss function? The workhorse here is the *backpropagation* algorithm (also called *backward mode automatic differentiation*), and it is a powerful one.

Before writing formulas, let's summarize the steps that we follow as we train a neural network:

- The training function is a function of $\vec{\omega}$, so the outcome of the neural network after a data point passes through it, which is the same as evaluating the training function at the data point, is: $outcome = function(\vec{\omega})$. This is made up of linear combinations of node outputs, followed by compositions with activation functions, repeated over all of the network's layers. The output layer might or might not have an activation function and could have one node or multiple nodes, depending on the ultimate task of the network.
- The loss function provides a measure of how badly the outcome of the training function diverged from what is true.
- We initialize our learning function with a *random* set of weights $\vec{\omega}^0$, according to preferred initialization rules prescribed in the previous sections. Then we

compute the loss, or error, that we committed because of using these particular weight values. This is the forward pass of the data point through the net.

- We want to move to the next set of weights $\vec{\omega}^1$ that gives a lower error. We move in the direction opposite to the gradient vector of the loss function.
- But: the training function is built into the loss function, and given the layered structure of this function, which comes from the architecture of the neural network, along with its high dimensionality, how do we efficiently perform the multivariable chain rule to find the gradient and evaluate it at the current set of weights?

The answer is that we send the data point back through the network, computing the gradient *backward* from the output layer all the way to the input layer, evaluating along the way how each node contributed to the error. In essence, we compute $\frac{\partial L}{\partial \text{node functions}}$, then we tweak the weights accordingly, updating them from $\vec{\omega}^0$ to $\vec{\omega}^1$. The process continues as we pass more data points into the network, usually in batches. One *epoch* is then counted each time the network has seen the full training set.

Backpropagation Is Not Too Different from How Our Brain Learns

When we encounter a new math concept, the neurons in our brain make certain connections. The next time we see the same concept, the same neurons connect better. The analogy for our neural network is that the value ω of the edge connecting the neurons increases. When we see the same concept again and again, it becomes part of our brain's model. This model will not change, unless we learn new information that undoes the previous information. In that case, the connection between the neurons weakens. For our neural network, the ω value connecting the neurons decreases. Tweaking the ω 's via minimizing the loss function accomplishes exactly that: establishing the correct connections between the neurons.

The neuroscientist Donald Hebb mentions in his 1949 book *The Organization of Behavior: A Neuropsychological Theory* (paraphrased): *When a biological neuron triggers another neuron often, the connection between these two neurons grows stronger. In other words, cells that fire together, wire together.*

Similarly, a neural network's computational model takes into account the error made by the network when it produces an outcome. Since computers only understand numbers, the ω of an edge increases if the node contributes to lowering the error, and decreases if the node contributes to increasing the error function. So a neural network's learning rule reinforces the connections that reduce the error by increasing the corresponding ω 's, and weakens the connections that increase the error by decreasing the corresponding ω 's.

Why Is It Better to Backpropagate?

Backpropagation computes the derivative of the training function with respect to each node, moving *backward* through the network. This measures the contribution of each node to both the training function and the loss function $L(\vec{\omega})$.

The most important formula to recall here is: the chain rule from calculus. This calculates the derivatives of *chained* functions (or function compositions). The calculus chain rule mostly deals with functions depending only on one variable ω ; for example, for three chained functions, the derivative with respect to ω is:

$$\frac{d}{d\omega}f_3(f_2(f_1(\omega))) = \left\{ \frac{d}{d\omega}f_1(\omega) \right\} \left\{ \frac{d}{df_1}f_2(f_1(\omega)) \right\} \left\{ \frac{d}{df_2}f_3(f_2(f_1(\omega))) \right\}$$

For neural networks, we must apply the chain rule to the loss function that depends on the matrices and vectors of variables W and $\vec{\omega}_0$. So we have to generalize the above rule to a *many variables chain rule*. The easiest way to do this is to follow the structure of the network computing the derivatives backward, from the outcome layer all the way back to the input layer.

If instead we decide to compute the derivatives forward through the network, we would not know whether these derivatives with respect to each variable will ultimately contribute to our final outcome, because we do not know if they will connect through the graph of the network. Even when the graph is fully connected, the weights for deeper layers are not present in earlier layers, so it is a big waste to compute for their derivatives in the early layers.

When we compute the derivatives backward through the network, we start with the output and follow the edges of the graph of the network back, computing the derivatives at each node. Each node's contribution is calculated only from the edges leading to it and edges going out of it. This is computationally much cheaper because now we are sure of how and when these nodes contribute to the network's outcome.

In linear algebra, it is much cheaper to compute the multiplication of a matrix with a vector than to compute the multiplication of two matrices together. We must always avoid multiplying two matrices with each other: computing $A(B\mathbf{v})$ is cheaper than computing $(AB)\mathbf{v}$, even though in theory, these two are exactly the same. Over large matrices and vectors, this simple observation provides enormous cost savings.

Backpropagation in Detail

Let's pause and be thankful that software packages exist so that we never have to implement the following computation ourselves. Let's also not forget to be grateful to the creators of these software packages. Now we compute.

For a neural network with h hidden layers, we can write the loss function as a function of the training function, which in turn is a function of all the weights that appear in the network:

$$L = L\left(g\left(W^1, \vec{s}^1, W^2, \vec{s}^2, \dots, W^h, \vec{s}^h, W^{h+1}, \vec{s}^{h+1}\right)\right)$$

We will compute the partial derivatives of L backward, starting with $\frac{\partial L}{\partial W^{h+1}}$ and $\frac{\partial L}{\partial \vec{s}^{h+1}}$, and working our way back to $\frac{\partial L}{\partial W^1}$ and $\frac{\partial L}{\partial \vec{s}^1}$. Their derivatives are taken with respect to each entry in the corresponding matrix or vector.

Suppose for simplicity, but without loss of generality, that the network is a regression network predicting a single numerical value, so that the training function g is scalar (not a vector). Suppose also that we use the same activation function f for each neuron throughout the network. The output neuron has no activation since this is a regression.

Let's find the derivatives with respect to the weights pointing to the output layer. The loss function is:

$$L = L\left(W^{h+1} \vec{s}^h + \omega_0^{h+1}\right)$$

so that

$$\frac{\partial L}{\partial \omega_0^{h+1}} = 1 \times L'\left(W^{h+1} \vec{s}^h + \omega_0^{h+1}\right)$$

and

$$\frac{\partial L}{\partial W^{h+1}} = \left(\vec{s}^h\right)^t L'\left(W^{h+1} \vec{s}^h + \omega_0^{h+1}\right)$$

Recall that $\vec{s}^h$ is the output of the last layer, so it depends on all the weights of the previous layers, namely, $\left(W^1, \vec{s}^1, W^2, \vec{s}^2, \dots, W^h, \vec{s}^h\right)$.

To compute derivatives with respect to the weights pointing to the last hidden layer, we show them explicitly in the formula of the loss function:

$$L = L\left(W^{h+1}\left(f\left(W^h \vec{s}^{h-1} + \vec{\omega}_0^h\right)\right) + \omega_0^{h+1}\right)$$

so that

$$\frac{\partial L}{\partial \vec{\omega}_0^h} = \vec{1}\left(W^{h+1}f'\left(W^h \vec{s}^{h-1} + \vec{\omega}_0^h\right)\right)L'\left(W^{h+1}\left(f\left(W^h \vec{s}^{h-1} + \vec{\omega}_0^h\right)\right) + \omega_0^{h+1}\right)$$

and

$$\frac{\partial L}{\partial W^h} = \vec{s}^{h-1}\left(W^{h+1}f'\left(W^h \vec{s}^{h-1} + \vec{\omega}_0^h\right)\right)L'\left(W^{h+1}\left(f\left(W^h \vec{s}^{h-1} + \vec{\omega}_0^h\right)\right) + \omega_0^{h+1}\right)$$

Recall that $\vec{s}^{h-1}$ is the output of the hidden layer before the last hidden layer, so it depends on all the weights of the previous layers, namely, $\left(W^1, \vec{\omega}_0^1, W^2, \vec{\omega}_0^2, \dots, W^{h-1}, \vec{\omega}_0^{h-1}\right)$.

We continue the process systematically until we reach the input layer.

Assessing the Significance of the Input Data Features

One goal of data analysts is to assess the significance of the input variables (data features) with respect to the output or target variable.

The main question to answer here is: if we tweak the value of a certain input variable, what is the relative change of the output?

For example, if we add one more bus on a given bus route, would that affect the overall bus ridership?

The math question that we are asking is a *derivative* question: find the partial derivative of the output with respect to the input variable in question.

We have plenty of literature in statistics on variable significance when the models are linear (sensitivity analysis). When the models are nonlinear, such as our neural network models, there isn't as much literature. We cannot make our predictions based on nonlinear models, then employ variable significance analysis that is built for linear models. Many data analysts who use built-in software packages for their analysis fall into this trap. This is another reason to seek to understand deeply the assumptions of the models on which we base our business decisions.

Summary and Looking Ahead

This chapter represents our official transition to the deep learning era in the AI field. While [Chapter 3](#) presented traditional yet still very useful machine learning models, [Chapter 4](#) adds neural networks to our arsenal of machine learning models. Both chapters built the models with the general mathematical structure of: training function, loss function, and optimization, where each was tailored to the particular task and model at hand.

By employing nonlinear activation functions at each neuron of a neural network, over multiple layers, our training function is able to pick up on complex features in the data that are otherwise hard to describe using an explicit formula of a nonlinear function. Mathematical analysis—in particular, universal approximation theorems for neural networks—back up this intuition and provide a theoretical background that justifies the wild success of neural networks. These theorems, however, still lack the ability to provide us with a map to construct special networks tailored to specific tasks and data sets, so we must experiment with various architectures, regularizations, and hyperparameters until we obtain a neural network model that performs well on new and unseen data.

Neural networks are well tailored to large problems with large data sets. The optimization task for such large problems requires efficient and computationally inexpensive methods, though all computations at that scale can be considered expensive. Stochastic gradient descent is the popular optimization method of choice, and the backpropagation algorithm is the workhorse of this method. More specifically, the backpropagation algorithm computes the gradient of the loss function (or the objective function when we add weight decay regularization) at the current weight choice. Understanding the landscape of the objective function remains central for any optimization task, and as a rule of thumb, convex problems are easier to optimize than nonconvex ones. Loss functions involved in neural network models are generally nonconvex.

[Chapter 4](#) is the last foundational (and long) chapter in this book. We can finally discuss more specialized AI models, as well as deeper mathematics, when needed. The next chapters are independent from each other, so read them in the order that feels most relevant to your immediate application area.

Finally, let's summarize the mathematics that appeared in this chapter, which we must elaborate more on as we progress in the field:

Probability and measure

This is needed to prove universal approximation-type theorems, and will be discussed in [Chapter 11](#). It is also related to uncertainty analysis for dropout.

Statistics

Input standardizing steps during batch normalization at each layer of the neural network, and the resulting reshaping of the related distributions.

Optimization

Gradient descent, stochastic gradient descent, convex, and nonconvex landscapes.

Calculus on linear algebra

Backpropagation algorithm: this is the chain rule from calculus applied on functions of matrices of variables.

Convolutional Neural Networks and Computer Vision

They. Can. See.

—H.

Convolutional neural networks have revolutionized the computer vision and the natural language processing fields. Application areas, irrespective of the ethical questions associated with them (such as surveillance, automated weapons, etc.), are limitless: self-driving cars, smart drones, facial recognition, speech recognition, medical imaging, generating audio, generating images, robotics, etc.

In this chapter, we start with the simple definitions and interpretations of *convolution* and *cross-correlation*, and highlight the fact that these two slightly different mathematical operations are conflated in machine learning terminology. We perpetrate the same sin and conflate them as well, but with a good reason.

We then apply the convolution operation to filtering grid-like signals, which it is perfectly suited for, such as time series data (one-dimensional), audio data (one-dimensional), and images (two-dimensional if the images are grayscale, and three-dimensional if they are color images, with the extra dimension corresponding to the red, green, and blue channels). When data is one-dimensional, we use one-dimensional convolutions, and when it is two-dimensional, we use two-dimensional convolutions (for the sake of simplicity and conciseness, we will not do three-dimensional convolutions in this chapter, corresponding to three-dimensional color images, called *tensors*). In other words, we adapt our network to the shape of the data, a process that has wildly contributed to the success of convolutional neural networks. This is in contrast to forcing the data to adapt to the shape of the network's input, such as flattening a two-dimensional image into one long vector to make it fit a

network that only takes one-dimensional data as its input. In later chapters, we see that the same applies for the success of graph neural networks for graph-type data.

Next, we incorporate convolution into the architecture of a feed forward neural network. The convolution operation has the effect of making the network *locally connected* as opposed to fully connected. Mathematically, the matrix containing the weights for each layer is not *dense* (so most of the weights for convolutional layers are zero). Moreover, the weights have similar values (*weight sharing*), unlike the case of fully connected neural networks, where a different weight is assigned to each input. Therefore, the matrix containing the weights of a convolutional layer is mostly zeros, and the nonzero parts are localized and share similar values. For image data or audio data, this is great, since most of the information is contained locally. Moreover, this dramatically reduces the number of weights that we need to store and compute during the optimization step, rendering convolutional neural networks ideal for data with a massive amount of input features (recall that for images, each pixel is a feature).

We then discuss *pooling*, which is another layer that is common to the architecture of convolutional neural networks. As in the previous chapter, both the multilayer structure and the nonlinearity at each layer enable us to extract increasingly complex features from images, significantly enhancing computer vision tasks.

Once we comprehend the basic anatomy of a convolutional neural network, it is straightforward to apply the same mathematics to tasks involving natural language processing, such as sentiment analysis, speech recognition, audio generation, and others. The fact that the same mathematics works for both computer vision and natural language processing is akin to the impressive ability of our brain to physically change in response to circumstances, experiences, and thoughts (the brain's version of virtual simulations). Even when some portions of the brain are damaged, other parts can take over and perform new functions. For example, the parts of the brain devoted to sight can start performing hearing or remembering tasks when those are impaired. In neuroscience, this is called *neuroplasticity*. We are very far from having a complete understanding of the brain and how it works, but the simplest explanation to this phenomenon is that each neuron in the brain performs the same basic function, similar to how neurons in a neural network perform one basic mathematical calculation (actually two: linearly combine, then activate), and the various neural connections over multiple layers produce the observed complexity in perception and behavior.

Convolutional neural networks are in fact inspired by the brain's visual neocortex. The success in 2012 for image classification ([AlexNet2012](#)) propelled AI back into the mainstream, inspiring many and bringing us here. If you happen to have some extra time, a good bedtime read accompanying this chapter would be about the function

of the brain's visual neocortex, and its analogy to convolutional neural networks designed for computer vision.

Convolution and Cross-Correlation

Convolution and cross-correlation are slightly different operations and measure different things in a signal, which can be a digital image, a digital audio signal, or others. They are exactly the same if we use a symmetric function k , called a *filter* or a *kernel*. Briefly speaking, convolution *flips* the filter, then slides it across the function, while cross-correlation slides the filter across the function *without flipping* it. Naturally, if the filter happens to be symmetric, then convolution and cross-correlation are exactly the same. Flipping the kernel has the advantage of making the convolution operation commutative, which in turn is good for writing theoretical proofs. That said, from a neural networks perspective, commutativity is not important, for three reasons:

- First, the convolution operation does not usually appear alone in a neural network, but is composed with other nonlinear functions, so we lose commutativity irrespective of whether we flip the kernel or not.
- Second, a neural network usually *learns* the values of the entries in the kernel during the training process; thus, it learns the correct values in the correct locations, and flipping becomes immaterial.
- Third, and this is important for practical implementation, convolutional networks usually use *multichannel convolution*; for example, the input can be a color image with red-green-blue channels, or even a video, with red-green-blue space channels and one time channel. Furthermore, they use batch mode convolution, meaning they take input vectors, images, videos, or other data types, in batches and apply parallel convolution operations simultaneously. Even with kernel flipping, these operations are not guaranteed to be commutative unless each operation has the same number of output channels as input channels. This is usually not the case, since the outputs of multiple channels are usually summed together as a whole or partially, producing a different number of output channels than input channels.

For all these reasons, many machine learning libraries do not flip the kernel when implementing convolution, in essence implementing cross-correlation and calling it convolution. We do the same here.

The convolution operation between two real valued functions k (the filter) and f is defined as:

$$\begin{aligned}
(k \star f)(t) &= \int_{-\infty}^{\infty} f(s)k(-s+t)ds \\
&= \int_{-\infty}^{\infty} f(-s+t)k(s)ds
\end{aligned}$$

and the discrete analog for discrete functions is:

$$\begin{aligned}
(k \star f)(n) &= \sum_{s=-\infty}^{\infty} f(s)k(-s+n) \\
&= \sum_{s=-\infty}^{\infty} f(-s+n)k(s)
\end{aligned}$$

The cross-correlation operation between two real valued functions k (the filter) and f is defined as:

$$\begin{aligned}
(k^* f)(t) &= \int_{-\infty}^{\infty} f(s)k(s+t)ds \\
&= \int_{-\infty}^{\infty} f(s-t)k(s)ds
\end{aligned}$$

and the discrete analog for discrete functions is:

$$\begin{aligned}
(k^* f)(n) &= \sum_{s=-\infty}^{\infty} f(s)k(s+n) \\
&= \sum_{s=-\infty}^{\infty} f(s-n)k(s)
\end{aligned}$$

Note that the formulas defining convolution and cross-correlation look exactly the same, except that for convolution we use $-s$ instead of s . This corresponds to flipping the involved function (before shifting). Note also that the indices involved in the convolution integral and sum add up to t or n , which is not the case for cross-correlation. This makes convolution commutative, in the sense that $(f \star k)(n) = (k \star f)(n)$, and cross-correlation not necessarily commutative. We comment further on commutativity later.

Every time we encounter a new mathematical object, it is good to pause and ask ourselves a few questions before diving in. This way we can build a solid mathematical foundation while managing to avoid the vast and dark ocean of technical and complicated mathematics.

What kind of mathematical object do we have on our hands?

For convolution and cross-correlation, we are looking at infinite sums for the discrete case, and integrals over infinite domains for the continuum case. This guides our next question.

Since we are summing over infinite domains, what kind of functions can we allow without our computations blowing up to infinity?

In other words, for what kind of functions do these infinite sums and integrals exist and are well-defined? Here, start with the easy answers, such as when f and k are compactly supported (are zeros except in a finite portion of the domain), or when f and k decay rapidly enough to allow the infinite sum or integral to converge. Most of the time, these simple cases are enough for our applications, such as image and audio data, and the filters that we accompany them with. Only when we find that our simple answer does not apply to our particular use case do we seek more general answers. These come in the form of theorems and proofs. Do not seek the most general answers first, as these are usually built on top of a large mathematical theory that took centuries, and countless questions and searches for answers, to take form and materialize. Taking the most general road first is overwhelming, counterintuitive, counter-historical, and time- and resource-draining. It is not how mathematics and analysis naturally evolve. Moreover, if we happen to encounter someone talking in the most general and technical language without providing any context or motivation for why this level of generality and complexity is needed, we just tune them out and move on peacefully with our lives; otherwise they might confuse us beyond repair.

What is this mathematical object used for?

The convolution operation has far-reaching applications that range from purely theoretical mathematics to applied sciences to the engineering and systems design fields. In mathematics, it appears in many fields, such as differential equations, measure theory, probability, statistics, analysis, and numerical linear algebra. In the applied sciences and engineering, it is used in acoustics, spectroscopy, image processing and computer vision, and in the design and implementation of finite impulse response filters in signal processing.

How is this mathematical object useful in my particular field of study and how does it apply to my particular interest or use case?

For our AI purposes, we will use the convolution operation to construct convolutional neural networks for both one-dimensional text and audio data and two-dimensional image data. The same ideas generalize to any type of high-dimensional data where most of the information is contained locally. We use convolutional neural networks in two contexts: *understanding* image, text, and audio data; and *generating* image, text, and audio data.

Additionally, in the context of data and distributions of data, we use the following result that has to do with the probability distribution of the sum of two independent random variables. If μ and ν are probability measures on the topological group $(\mathbb{R}, +)$, and if X and Y are two independent random variables whose respective distributions are μ and ν , then the convolution $\mu \star \nu$ is the probability distribution of the sum random variable $X + Y$. We elaborate on this in [Chapter 11](#) on probability.

How did this mathematical object come to be?

It is worth investing a tad bit of extra time to learn some of the history and the chronological order of when, how, and why the object that we care about first appeared, along with the main results associated with it. In other words, rather than studying our mathematical object through a series of dry lemmas, propositions, and theorems, which are usually deprived of all context, we learn through its own story, and through the ups and downs that mathematicians encountered while attempting to develop it. One of the most valuable insights we learn here is that mathematics develops organically with our quest to answer various questions, establish connections, and gain a deeper understanding of something that we need to use. Modern mathematical analysis developed while attempting to answer very simple questions related to Fourier sine and cosine series (decomposing a function into its component frequencies), which turned out to be not so simple for many types of functions. For example: when can we interchange the integral and the infinite sum, what is an integral, and what is dx anyway?

What comes as a surprise to many, especially to people who feel intimidated or scared of mathematics, is that during the quest to gain understanding, some of the biggest names in mathematics, including the fathers and mothers of certain fields, made multiple mistakes along the way and corrected them at later times, or were corrected by others until the theory finally took shape.

One of the earliest uses of the convolution integral appeared in 1754, in d'Alembert's derivation of Taylor's theorem. Later, between 1797 and 1800, it was used by Sylvestre François Lacroix in his book, *Treatise on Differences and Series*, which is part of his encyclopedic series, *An Elementary Treatise on Differential Calculus and Integral Calculus*. Soon thereafter, convolution operations appear in the works of very famous names in mathematics, such as Laplace, Fourier, Poisson, and Volterra. What is common here is that all of these studies have to do with integrals, derivatives, and series of functions. In other words, calculus, and again, decomposing functions into their component frequencies (Fourier series and transform).

What are the most important operations, manipulations, and/or theorems related to this mathematical object that we must be aware of before diving in?

Things do not spring into fame or viral success without being immensely beneficial to many, many people. The convolution operation is so simple yet so useful and generalizes neatly to more involved mathematical entities, such as measures and distributions. It is commutative, associative, distributive over addition and scalar multiplication; its integral becomes a product, and its derivative transforms to differentiating only one of its component functions.

Translation Invariance and Translation Equivariance

These properties of convolutional neural networks enable us to detect similar features in different parts of an image. That is, a pattern that occurs in one location of an image is easily recognized in other locations of the image. The main reason is that at a convolutional layer of a neural network, we convolute with the same filter (also called a kernel, or template) throughout the image, picking up on the same patterns (such as edges, or horizontal, vertical, and diagonal orientations). This filter has a fixed set of weights. Recall that this is not the case for fully connected neural networks, where we would have to use different weights for different pixels of an image. Since we use convolution instead of matrix multiplication to perform image filtering, we have the benefit of *translation invariance*, since usually all we care for is the presence of a pattern, irrespective of its location. Mathematically, translation invariance looks like:

$$\text{trans}_a(k) \star f = k \star \text{trans}_a(f) = \text{trans}_a(k \star f)(t)$$

where trans_a is the translation of a function by a . For our AI purposes, this implies that given a filter designed to pick up on a certain feature in an image, convolving it with a translated image (in the horizontal or vertical directions), is the same as filtering the image *then* translating it. This property is sometimes called *translation equivariance*, and translation invariance is instead attributed to the pooling layer that is often built into the architecture of convolutional neural networks. We discuss this later in this chapter. In any case, the fact that we use one filter (one set of weights) throughout the whole image at each layer means that we detect one pattern at various locations of the image, when present. The same applies to audio data or any other type of grid-like data.

Convolution in Usual Space Is a Product in Frequency Space

The Fourier transform of the convolution of two functions is the product of the Fourier transform of each function, up to a scaling. In short, the Fourier transform resolves a function into its frequency components (we elaborate on the Fourier transform in [Chapter 13](#)). Therefore, the convolution operation does not create new

frequencies, and the frequencies present in the convolution function are simply the product of the frequencies of the component functions.

In mathematics, we are forever updating our arsenal of useful tools for tackling different problems, so depending on our domain of expertise, it is at this point that we branch out and study related but more involved results, such as *circular convolutions for periodic functions*, preferred algorithms for computing convolutions, and others. The best route for us to dive into convolution in a way that is useful for our AI purposes is through signal and system design, which is the topic of the next section.

Convolution from a Systems Design Perspective

We are surrounded by systems, each interacting with its environment and designed to accomplish a certain task. Examples include HVAC systems in our buildings, adaptive cruise control systems in our cars, city transportation systems, irrigation systems, various communication systems, security systems, navigation systems, data centers, etc. Some systems interact with each other, others do not. Some are very large, others are as small and simple as a single device, such as a thermostat, receiving signals from its environment through sensors, processing them, and outputting other signals, for example, to actuators.

The convolution operation appears when designing and analyzing such simple systems, which process an input signal and produce an output signal, if we impose two special constraints: *linearity* and *time invariance*. Linearity and time invariance become linearity and *translation or shift invariance* if we are dealing with *space-dependent signals*, such as images, instead of *time-dependent signals*, such as electrical signals or audio signals. Note that a video is both space- (two or three spatial dimensions) and time-dependent. Linearity in this context has to do with the output of a scaled signal (amplified or reduced), and the output of two superimposed signals. Time and translation invariance have to do with the output of a delayed (time-dependent) signal, or the output of a translated or shifted (space-dependent) signal. We will detail these in the next section.

Together, linearity and time or translation invariance are very powerful. They allow us to find the system's output of *any signal* only if we know the output of a simple *impulse signal*, called the system's *impulse response*. The system's output to *any input signal* is obtained by merely convolving the signal with the system's impulse response. Therefore, imposing the conditions of linearity and time or translation invariance dramatically simplifies the analysis of signal processing systems.

The nagging question is then: how realistic are these wonder-making conditions? In other words, how prevalent are linear and time/translation invariant systems, or even only approximately linear and approximately time/translation invariant systems?

Aren't most realistic systems nonlinear and complex? Thankfully, we have control over system designs, so we can just decide to design systems with these properties. One example is any electrical circuit consisting of capacitors, resistors, inductors, and linear amplifiers. This is in fact mathematically equivalent to an ideal mechanical spring, mass, and damper system. Other examples that are relevant for us include processing and filtering various types of signals and images. We discuss these in the next few sections.

Convolution and Impulse Response for Linear and Translation Invariant Systems

Let's formalize the concepts of a linear system and a time/translation invariant system, then understand how the convolution operation naturally arises when attempting to quantify the response of a system possessing these properties to *any* signal. From a math perspective, a system is a function H that takes an input signal x and produces an output signal y . The signals x and y can depend on time, space (single or multiple dimensions), or both. If we enforce linearity on such a function, then we are claiming two things:

1. Output of a scaled input signal is nothing but a scaling of the original output: $H(ax) = aH(x) = ay$.
2. Output of two superimposed signals is nothing but the superposition of the two original outputs: $H(x_1 + x_2) = H(x_1) + H(x_2) = y_1 + y_2$.

If we enforce time/translation invariance, then we are claiming that the output of a delayed/translated/shifted signal is nothing but a delayed/translated/shifted original output: $H(x(t - t_0)) = y(t - t_0)$.

We can leverage these conditions when we think of any arbitrary signal, whether discrete or continuous, in terms of a superposition of a bunch of *impulse signals* of various amplitudes. This way, if we are able to measure the system's output to a single impulse signal, called the system's *impulse response*, then that is enough information to measure the system's response to any other signal. This becomes the basis of a very rich theory. We only walk through a discrete case in this chapter, since the signals we care for in AI (for example, for natural language processing, human machine interaction and computer vision), whether one-dimensional audio signals, or two- or three-dimensional images, are discrete. The continuous case is analogous except that we consider infinitesimal steps instead of discrete steps, we use integrals instead of sums, and we enforce continuity conditions (or whatever conditions we need to make the involved integrals well-defined). Actually, the continuous case brings a little extra complication: having to properly define an *impulse* in a mathematically sound way, for it is not a function in the usual sense. Thankfully, there are multiple mathematical ways to make it well-defined, such as the theory of distributions, or defining it as an

operator acting on usual functions, or as a measure and using the help of Lebesgue integrals. However, I have been avoiding measures, Lebesgue integrals, and any deep mathematical theory until we truly need them with their added functionality, so for now, the discrete case is very sufficient.

We define a unit impulse $\delta(k)$ to be zero for each nonzero k , and one when $k = 0$, and define its response as $H(\delta(k)) = h(k)$. Then $\delta(n - k)$ would be zero for each k , and 1 for $k = n$. This represents a unit impulse located at $k = n$. Therefore, $x(k)\delta(n - k)$ is an impulse of amplitude $x(k)$ located at $k = n$. Now we can write the input signal $x(n)$ as:

$$x(n) = \sum_{k=-\infty}^{\infty} x(k)\delta(n - k)$$

The above sum might seem like such a convoluted way to write a signal, and it is, but it is very helpful, since it says that any discrete signal can be expressed as an infinite sum of unit impulses that are scaled correctly at the right locations. Now, using the linearity and translation invariance assumptions on H , it is straightforward to see that the system's response to the signal $x(n)$ is:

$$\begin{aligned} H(x(n)) &= H\left(\sum_{k=-\infty}^{\infty} x(k)\delta(n - k)\right) \\ &= \sum_{k=-\infty}^{\infty} x(k)H(\delta(n - k)) \\ &= \sum_{k=-\infty}^{\infty} x(k)h(n - k) \\ &= (x \star h)(n) \\ &= y(n) \end{aligned}$$

Therefore, a linear and translation invariant system is completely described by its impulse response $h(n)$. But there is another way to look at this, which is independent of linear and translation invariant systems, and which is very useful for our purposes in the next few sections. The statement:

$$y(n) = (x \star h)(n)$$

says that the signal $x(n)$ can be transformed into the signal $y(n)$ after we convolve it with the filter $h(n)$. Hence, designing the filter $h(n)$ carefully can produce a $y(n)$ with some desired characteristics, or can *extract certain features* from the signal $x(n)$, for example, all the edges. Moreover, using different filters $h(n)$ extracts different features from the same signal $x(n)$. We elaborate more on these concepts in the next few

sections; meanwhile, keep the following in mind: as information, such as signals or images, flows through a convolutional neural network, different features get extracted (mapped) at each convolutional layer.

Before leaving linear and time/translation invariant systems, we must mention that such systems have a very simple response for sinusoidal inputs. If the input to the system is a sine wave with a given frequency, then the output is also a sine wave with the same frequency, but possibly with a different amplitude and phase. Moreover, knowing the impulse response of the system allows us to compute its *frequency response*, which is the system's response for sine waves at all frequencies, and vice versa. That is, determining the system's impulse response allows us to compute its frequency response, and determining the frequency response allows us to compute its impulse response, which in turn completely determines the system's response to any arbitrary signal. This connection is very useful from both theoretical and applications perspectives, and is closely connected to Fourier transforms and frequency domain representations of signals. In short, the frequency response of a linear and translation invariant system is simply the Fourier transform of its impulse response. We do not go over the computational details here, since these concepts are not essential for the rest of this book; nevertheless, it is important to be aware of these connections and understand how things from seemingly different fields come together and relate to each other.

Convolution and One-Dimensional Discrete Signals

Let's dissect the convolution operation and understand how it creates a new signal from an input signal by sliding a filter (kernel) against it. We will not flip the kernel here, as we established the fact that flipping the kernel is irrelevant from a neural network perspective. We start with a one-dimensional discrete signal $x(n)$, then we convolve it with a filter (kernel) $k(n)$, and produce a new signal $z(n)$. We show how we obtain $z(n)$ one entry at a time by sliding $k(n)$ along $x(n)$. For simplicity, let $x(n) = (x_0, x_1, x_2, x_3, x_4)$ and $k(n) = (k_0, k_1, k_2)$. In this example the input signal $x(n)$ only has five entries, and the kernel has three entries. In practice, such as in signal processing, image filtering, or AI's neural networks, the input signal $x(n)$ is orders of magnitude larger than the filter $k(n)$. We will see this very soon, but the example here is only for illustration. Recall the formula for discrete cross-correlation, which is the convolution without flipping the kernel:

$$\begin{aligned}(k \star x)(n) &= \sum_{s=-\infty}^{\infty} x(s)k(s+n) \\ &= \cdots + x(-1)k(-1+n) + x(0)k(n) + x(1)k(1+n) + x(2)k(2+n) + \cdots\end{aligned}$$

Since neither $x(n)$ nor $k(n)$ has infinitely many entries, and the sum is infinite, we pretend that the entries are zero whenever the indices are not defined. The new filtered signal resulting from the convolution will only have nontrivial entries with indices: -4, -3, -2, -1, 0, 1, 2. Let's write each of these entries:

$$\begin{aligned}
(k * x)(-4) &= x_4 k_0 \\
(k * x)(-3) &= x_3 k_0 + x_4 k_1 \\
(k * x)(-2) &= x_2 k_0 + x_3 k_1 + x_4 k_2 \\
(k * x)(-1) &= x_1 k_0 + x_2 k_1 + x_3 k_2 \\
(k * x)(0) &= x_0 k_0 + x_1 k_1 + x_2 k_2 \\
(k * x)(1) &= x_0 k_1 + x_1 k_2 \\
(k * x)(2) &= x_0 k_2
\end{aligned}$$

That operation is easier to understand with a mental picture: fix the signal $x(n) = (x_0, x_1, x_2, x_3, x_4)$ and slide the filter $k(n) = (k_0, k_1, k_2)$ against it from right to left. This can also be summarized concisely using linear algebra notation, where we multiply the input vector $x(n)$ by a special kind of matrix, called the *Toeplitz matrix*, containing the filter weights. We will elaborate on this later in this chapter.

When we slide a filter across the signal, the output signal will peak at the indices where $x(n)$ and $k(n)$ match, so we can design the filter in a way that picks up on certain patterns in $x(n)$. This way, convolution (unflipped) provides a measure of similarity between the signal and the kernel.

We can also view cross-correlation, or unflipped convolution, in a different way: each entry of the output signal is a *weighted average* of the entries of the input signal. This way, we emphasize both the linearity of this transformation and the fact that we can choose the weights in the kernel in way that emphasizes certain features over others. We see this more clearly with image processing, discussed next, but for that we have to write a formula for convolution (unflipped) in two dimensions. In linear algebra notation, discrete two-dimensional convolution amounts to multiplication of the two-dimensional signal by another special kind of matrix, called the *doubly block circulant matrix*, also appearing later in this chapter. Keep in mind that multiplication by a matrix is a linear transformation.

Convolution and Two-Dimensional Discrete Signals

The convolution (unflipped) operation in two dimensions looks like:

$$(k * x)(m, n) = \sum_{q=-\infty}^{\infty} \sum_{s=-\infty}^{\infty} x(m+q, n+s) k(q, s)$$

For example, the (2,1) entry of the convolution (unflipped) between the following 4×4 matrix A and the 3×3 kernel K :

$$A * K = \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & \boxed{a_{21}} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix} * \begin{pmatrix} k_{00} & k_{01} & k_{02} \\ k_{10} & \boxed{k_{11}} & k_{12} \\ k_{20} & k_{21} & k_{22} \end{pmatrix}$$

is $z_{21} = a_{10}k_{00} + a_{11}k_{01} + a_{12}k_{02} + a_{20}k_{10} + a_{21}k_{11} + a_{22}k_{12} + a_{30}k_{20} + a_{31}k_{21} + a_{32}k_{22}$. To see this, imagine placing the kernel K exactly on top of the matrix A with its center k_{11} on top of a_{21} , meaning the entry of A with the required index, then multiply the entries that are on top of each other and add all the results together. Note that here we only computed one entry of the output signal, meaning if we were working with images, it would be the value of only one pixel of the filtered image. We need all the others! For this, we have to know which indices are valid convolutions. By valid we mean *full*, as in they take all of the kernel entries into account during the computation. The word *valid* is a bit misleading since *all* entries are valid if we allow ourselves to *pad the boundaries of matrix A by zeros*. To find the indices that take the full kernel into account, recall our mental image of placing K exactly on top of A , with K 's center at the index that we want to compute. With this placement, the rest of K should not exceed the boundaries of A , so for our example, the good indices will be (1,1), (1,2), (2,1), and (2,2), producing the output:

$$Z = \begin{pmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{pmatrix}$$

This means that if we were working with images, the filtered image Z would be smaller in size than the original image A . If we want to produce an image with the same size as the original image, then we must pad the original image with zeros before applying the filter. For our example, we would need to pad with one layer of zeros around the full boundary of A , but if K was larger, then we would need more layers of zeros. The following is A padded with one layer of zeros:

$$A_{padded} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & a_{00} & a_{01} & a_{02} & a_{03} & 0 \\ 0 & a_{10} & a_{11} & a_{12} & a_{13} & 0 \\ 0 & a_{20} & a_{21} & a_{22} & a_{23} & 0 \\ 0 & a_{30} & a_{31} & a_{32} & a_{33} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

There are other ways to retain the size of the output image than zero padding, even though zero padding is the simplest and most popular approach. These include:

Reflection

Instead of adding a layer or multiple layers of zeros, add a layer or multiple layers of the same values as the boundary pixels of the image, and the ones below them, and so on if we need more. That is, instead of turning off the pixels outside an image boundary, extend the image using the same pixels that are already on or near the boundary.

Wraparound

This is used for periodic signals. Mathematically, this is cyclic convolution, circulant matrix, discrete Fourier transform producing the eigenvalues of the circulant matrix, and the Fourier matrix containing its eigenvectors as columns. We will not dive into that here, but recall that periodicity usually simplifies things, and makes us think of periodic waves, which in turn makes us think of Fourier stuff.

Multiple channels

Apply more than one *channel* of independent filters (weight matrices), each sampling the original input, then combine their outputs.

Filtering Images

Figure 5-1 shows an example of how convolving the same image with various kernels extracts various features of the image.

For example, the third kernel in the table has 8 at its center and the rest of its entries are -1 . This means that this kernel makes the current pixel 8 times as intense, then subtracts from that the values of all the pixels surrounding it. If we are in a uniform region of the image, meaning all the pixels are equal or very close in value, then this process will give zero, returning a black or turned-off pixel. If, on the other hand, this pixel lays on an edge boundary, for example, the boundary of the eye or the boundary of the face of the deer, then the output of the convolution will have a nonzero value, so it will be a bright pixel. When we apply this process to the whole image, the result is a new image with many edges traced with bright pixels and the rest of the image dark.

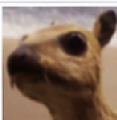
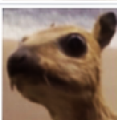
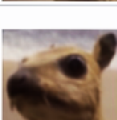
Operation	Kernel ω	Image result $g(x,y)$	
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$		
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$		Replace pixel, say, a_{22}, with $a_{11}-a_{13}-a_{31}+a_{33}$ → If there are no edges, this process will turn a lot of pixels off (black).
	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$		
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$		
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$		→ Make pixel very intense, and then subtract the sum of those vertically and horizontally around it.
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$		→ Each pixel becomes the average of all those around it, hence a blurry image.
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$		
Gaussian blur 5 × 5 (approximation)	$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$		
Unsharp masking 5 × 5 Based on Gaussian blur with amount as 1 and threshold as 0 (with no image mask)	$\frac{-1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$		

Figure 5-1. Applying various filters to an image (*image source*)

As we see in the table, the choice of the kernel that is able to detect edges, blur, etc., is not unique. The same table includes two-dimensional discrete Gaussian filters for blurring. When we discretize a one-dimensional Gaussian function, we do not lose its symmetry, but when we discretize a two-dimensional Gaussian function, we lose its radial symmetry since we have to approximate its natural circular or elliptical shape by a square matrix. Note that a Gaussian peaks at the center and decays as it spreads away from the center. Moreover, the area under its curve (surface for two dimensions) is one. This has the overall effect of averaging and smoothing (removing noise) when we convolve this with another signal. The price we pay is removing sharp edges, which is exactly what blurring is (think that a sharp edge is replaced by a smooth decaying average of itself and all the surrounding pixels within a distance of a few standard deviations from the center). The smaller the standard deviation (or the variance, which is the square of the standard deviation), the more detail we can retain from the image.

Another great example is shown in [Figure 5-2](#). Here, each image on the right is the result of the convolution between the image on the left and the filter (kernel) portrayed in the middle of each row. These filters are called [Gabor filters](#). They are designed to pick up on certain patterns/textures/features in an image, and they work in a similar way to the filters found in the human visual system.

Our eyes detect the parts of an image where things change, i.e., contrasts. They pick up on edges (horizontal, vertical, diagonal) and gradients (measuring the steepness of a change). We design filters that do the same things mathematically, sliding them via convolution across a signal. These produce smooth and uneventful results (zeros or numbers close to each other in value) when nothing is changing in the signal, and spiking when an edge or a gradient that lines up with the kernel is detected.

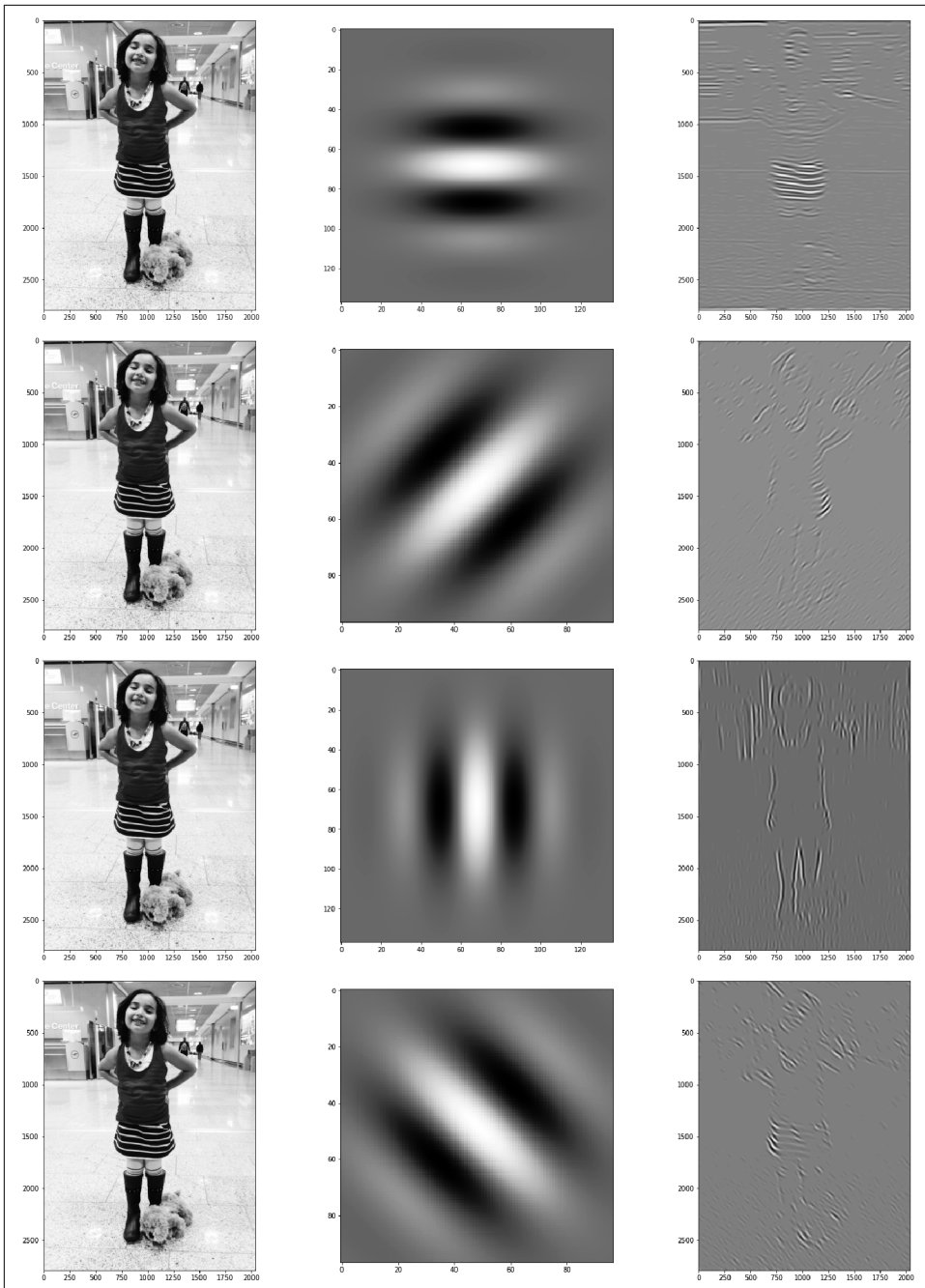


Figure 5-2. Gabor filters applied to the same image; we design the filters to pick up on different textures and orientations in the image. Visit [the book's GitHub page](#) to reproduce these images.

Feature Maps

A convolutional neural network *learns* the kernels from the data by optimizing a loss function in the same way a fully connected network does. The unknown weights that enter the formula of the training function are the entries of each kernel at each convolutional layer, the biases, and the weights related to any fully connected layer involved in the network's architecture. The output of a convolutional layer (which includes the nonlinear activation function) is called a *feature map*, and the learned kernels are feature detectors. A common observation in neural networks is that earlier layers in the network (close to the input layer) learn low-level features such as edges, and later layers learn higher-level features such as shapes. This is naturally expected since at each new layer we compose with a nonlinear activation function, so complexity increases over multiple layers and so does the network's ability to express more elaborate features.

How do we plot feature maps?

Feature maps help us open the black box and directly observe what a trained network detects at each of its convolutional layers. If the network is still in the training process, feature maps allow us to pinpoint the sources of error, then tweak the model accordingly. Suppose we input an image to a trained convolutional neural network. At the first layer, using the convolution operation, a kernel slides through the whole image and produces a new filtered image. This filtered image then gets passed through a nonlinear activation function, producing yet another image. Finally, this gets passed through a pooling layer, explained soon, and produces the final output of the convolutional layer. This output is a different image than the one we started with, and possibly of different dimensions, but if the network was trained well, the output image would highlight some important features of the original image, such as edges, texture, etc. This output image is usually a matrix of numbers or a tensor of numbers (three-dimensional for color images, or four-dimensional if we are working in batches of images or with video data, where there is one extra dimension for the time series). It is easy to visualize these as feature maps using the matplotlib library in Python, where each entry in the matrix or tensor is mapped to the intensity of a pixel in the same location as the matrix entry. **Figure 5-3** shows various feature maps at various convolutional layers of a convolutional neural network.

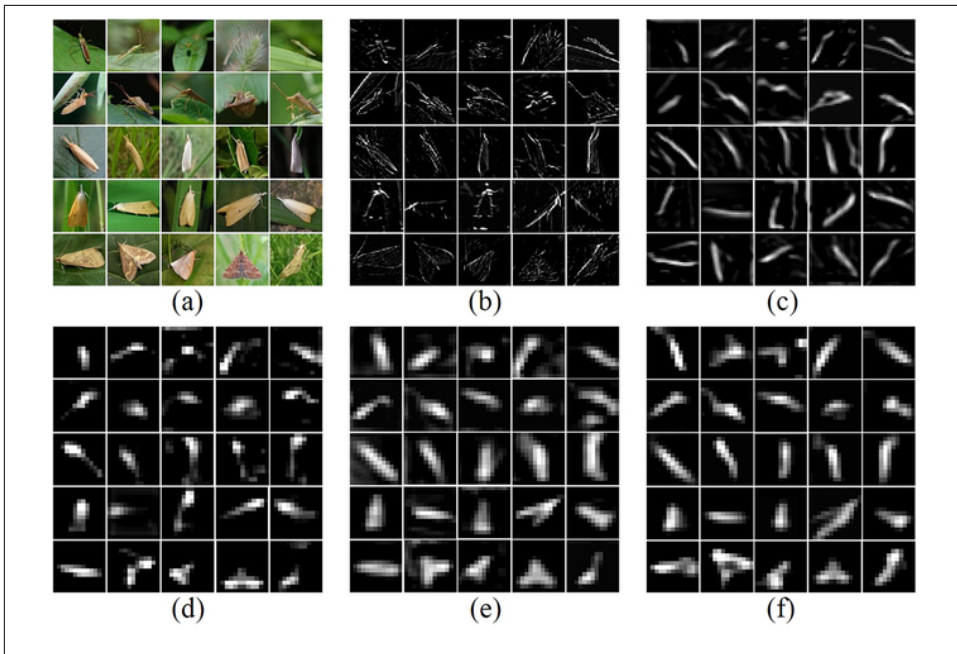


Figure 5-3. Feature maps at various convolutional layers of a convolutional neural network (*image source*)

Linear Algebra Notation

We now know that the most fundamental operation inside a convolutional neural network is the convolution. Given a filter k (could be one-dimensional, two-dimensional, three-dimensional, or higher), a convolution operation takes an input signal and applies k to it by sliding it across the signal. This operation is linear, meaning each output is a linear combination (by the weights in k) of the input components, thus it can be efficiently represented as a matrix multiplication. We need this efficiency, since we must still write the training function representing the convolutional neural network in a way that is easy to evaluate and differentiate. The mathematical structure of the problem is still the same as for many machine learning models, which we are very familiar with now:

Training function

For convolutional neural networks, this usually includes linear combinations of components of the input, composed with activation functions, then a pooling function (discussed soon), over multiple layers of various sizes and connections, and finally topped with a logistic function, a support vector machine function, or other functions depending on the ultimate purpose of the network (classification, image segmentation, data generation, etc.).

The difference from fully connected neural networks is that the linear combination happens now with the weights associated with the filter, which means they are not all different from each other (unless we are only doing locally connected layers instead of convolutional layers).

Moreover, the sizes of the filters are usually orders of magnitude smaller than the input signals, so when we express the convolution operation in matrix notation, most of the weights in the matrix will actually be zero. Recall that there is a weight per each input feature at each layer of a network. For example, if the input is a color image, there is a different weight for each pixel in each channel, unless we decide to implement a convolutional layer—then there would only be a few unique weights, and many, many zeros. This simplifies storage requirements and computation time tremendously, while at the same time capturing the important local interactions.

That said, it is common in the architecture of a convolutional neural network to have fully connected layers (with a different weight assigned to each connection) near the output layer. One can rationalize this as a distillation of features: after important and locally dependent features, which increase in complexity with each layer, are captured, these are combined to make a prediction. That is, when information arrives to a fully connected layer, it would have been distilled to its most important components, which in turn act as unique features contributing to a final prediction.

Loss function

This is similar to all the loss functions we went over in the previous chapters, always providing a measure of the error made by the network's predictions and ground truths.

Optimization

This again uses stochastic gradient descent, due to the enormity of the size of the problem and the number of variables that go into it. Here, as usual, we need to evaluate one derivative of the loss function, which includes one derivative of the training function. We compute the derivative with respect to all the unknown weights involved in the filters of all the network's layers and channels, and the associated biases. Computationally, the backpropagation algorithm is still the workhorse of the differentiation process.

Linear algebra and computational linear algebra have all the tools we need to produce trainable convolutional neural networks. In general, the worst kind of matrix to be involved in our computations is a dense (mostly nonzero entries) matrix with no obvious structure or pattern to its entries (even worse if it is nondiagonalizable, etc.). But when we have a sparse matrix (mostly zeros), or a matrix with a certain pattern to its entries (such as diagonal, tridiagonal, circular, etc.), or both, then we are in a computationally friendly world, given that we learn how to exploit the special matrix

structure to our advantage. Researchers who study large matrix computations and algorithms are the kings and queens of such necessary exploitation, and without their work we'd be left with theory that is very hard to implement in practice and at scale.

In one dimension, the convolution operation can be represented using a special kind of matrix, called the *Toeplitz* matrix; and in two dimensions, it can be represented using another special kind of matrix, called a *doubly block circulant* matrix. Let's only focus on these two, but with the take-home lesson that matrix notation, in general, is the best way to go, and we would be fools not to discover and make the most use of the inherent structure within our matrices. In other words, attacking the most general cases first might just be a sad waste of time, which happens to be a rare commodity in this life. A good compromise between working with the most general matrix and the most specific matrix is to accompany results with complexity analysis, such as computing the order of a method ($O(n^3)$ or $O(n \log n)$, etc.), so that stakeholders are made aware of the trade-offs between implementing certain methods versus others.

We borrow a simple example from the free ebook *Deep Learning* (Chapter 9, page 334) by Ian Goodfellow et al. (MIT Press), on the efficiency of using either convolution or exploiting the many zeros in a matrix versus using the usual matrix multiplication to detect vertical edges within a certain image. Convolution is an extremely efficient way of describing transformations that apply the same linear transformation of a small local region across the entire input.

The image on the right of Figure 5-4 was formed by taking each pixel in the original image and subtracting the value of its neighboring pixel on the left. This shows the strength of all the vertically oriented edges in the input image, which can be a useful operation for object detection.

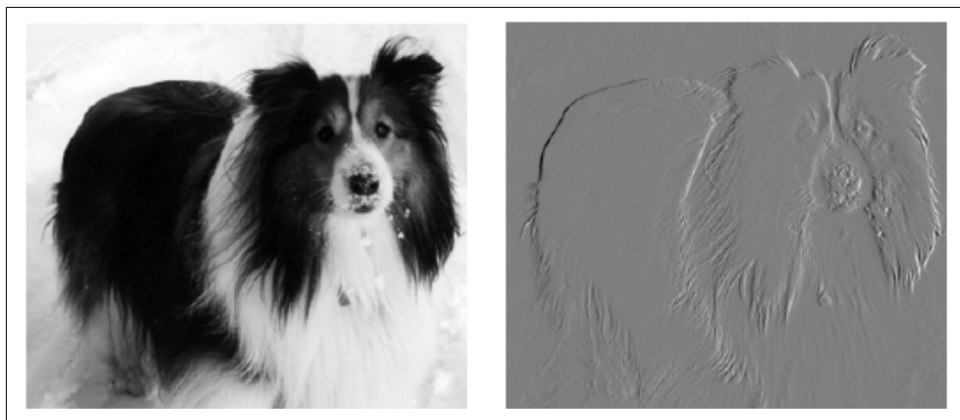


Figure 5-4. Detecting the vertical edges in an image (*image source*)

Both images are 280 pixels tall. The input image is 320 pixels wide, while the output image is 319 pixels wide. This transformation can be described by a convolution kernel containing two elements, and requires $319 \times 280 \times 3 = 267,960$ floating-point operations (two multiplications and one addition per output pixel) to compute using convolution.

To describe the same transformation with a matrix multiplication would take $320 \times 280 \times 319 \times 280$, or over eight billion entries in the matrix, making convolution four billion times more efficient for representing this transformation. The straightforward matrix multiplication algorithm performs over 16 billion floating-point operations, making convolution roughly 60,000 times more efficient computationally. Of course, most of the entries of the matrix would be zero. If we stored only the nonzero entries of the matrix, then both matrix multiplication and convolution would require the same number of floating-point operations to compute. The matrix would still need to contain $2 \times 319 \times 280 = 178,640$ entries.

—*Deep Learning*, by Ian Goodfellow et al.

The One-Dimensional Case: Multiplication by a Toeplitz Matrix

A *banded* Toeplitz matrix looks like:

$$Toeplitz = \begin{pmatrix} k_0 & k_1 & k_2 & 0 & 0 & 0 & 0 \\ 0 & k_0 & k_1 & k_2 & 0 & 0 & 0 \\ 0 & 0 & k_0 & k_1 & k_2 & 0 & 0 \\ 0 & 0 & 0 & k_0 & k_1 & k_2 & 0 \\ 0 & 0 & 0 & 0 & k_0 & k_1 & k_2 \end{pmatrix}$$

Multiplying this Toeplitz matrix by a one-dimensional signal $x = (x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7)$ yields the exact result of the convolution of a one-dimensional filter $k = (k_1, k_2, k_3)$ with the signal x , namely, $(Toeplitz)x^t = k * x$. Performing the multiplication, we see the *sliding effect* of the filter across the signal.

The Two-Dimensional Case: Multiplication by a Doubly Block Circulant Matrix

The two-dimensional analog involves the two-dimensional convolution operation and filtering images. Here, instead of multiplying with a Toeplitz matrix, we end up multiplying with a *doubly block circulant matrix*, where each row is a circular shift of a given vector. It is a nice exercise in linear algebra to write this matrix down along with its equivalence to two-dimensional convolution. In deep learning, we end up learning the weights, which are the entries of these matrices. This linear algebra notation (in terms of Toeplitz or circulant matrices) helps us find compact formulas for the derivatives of the loss function with respect to these weights.

Pooling

One step common to almost all convolutional neural networks is *pooling*. This is typically implemented after the input gets filtered via convolution, then passed through the nonlinear activation function. There is more than one type of pooling, but the idea is the same: replace the current output at a certain location with a summary statistic of the nearby outputs. An example for images is to replace four pixels with one pixel containing the maximum value of the original four (*max pooling*), or with their average value, or a weighted average, or the square root of the sum of their squares, etc.

Figure 5-5 shows how max pooling works.

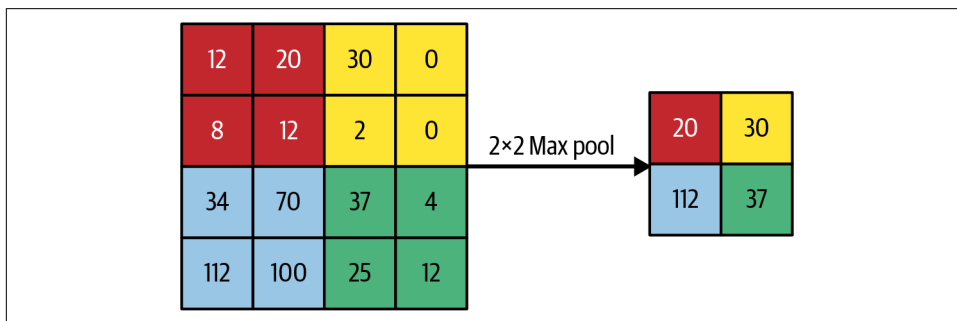


Figure 5-5. Max pooling (*image source*)

In effect, this reduces the dimension and summarizes whole neighborhoods of outputs, at the expense of sacrificing fine detail. So pooling is not very good for use cases where fine detail is essential for making predictions. Nevertheless, pooling has many advantages:

- It provides approximate invariance to small spatial translations of the input. This is useful if we care more about whether some feature is present than its exact location.
- It can greatly improve the statistical efficiency of the network.
- It improves the computational efficiency and memory requirements of the network because it reduces the number of inputs to the next layer.
- It helps with handling inputs of varying sizes, because we can control the size of the pooled neighborhoods and the size of the output after pooling.

A Convolutional Neural Network for Image Classification

It is impossible to have an exhaustive list of all the different architectures and variations that are involved in neural networks without diverting from the main purpose of the book: understanding the mathematics that underlies the different models. It is possible, however, to go over the essential components and how they all come together to accomplish an AI task, such as image classification for computer vision. During the training process, the steps of [Chapter 4](#) still apply:

1. Initialize random weights (according to the initialization processes we described in [Chapter 4](#)).
2. Forward pass a batch of images through the convolutional network and output a class for the image.
3. Evaluate the loss function for this particular choice of weights.
4. Backpropagate the error through the network.
5. Adjust the weights that contributed to the error (stochastic gradient descent).
6. Repeat for a certain number of iterations, or until you converge.

Thankfully, we do not have to do any of this on our own. Python's Keras library has many pre-trained models, which means that their weights have already been fixed, and all we have to do is evaluate the trained model on our particular data set.

What we can and should do is observe and learn the architecture of successful and winning networks. [Figure 5-6](#) shows the simple architecture of [LeNet1](#) by LeCun et al. (1989), and [Figure 5-7](#) shows AlexNet's (2012) architecture.

A good exercise is to count the number of the weights that go into the training function of LeNet1 and AlexNet. Note that more units in each layer (feature maps) means more weights. When I tried to count the weights involved in LeNet1 based on the architecture in [figure Figure 5-6](#), I ended up with 9,484 weights, but the original paper mentions 9,760 weights, so I do not know where the rest of the weights are. If you find them please let me know. Either way, the point is we need to solve an optimization problem in $\mathbb{R}^{9760}$. Now do the same computation for AlexNet in [Figure 5-7](#): we have around 62.3 million weights, so the optimization problem ends up in $\mathbb{R}^{62.3million}$. Another startling number: we need 1.1 billion computation units for one forward pass through the net.

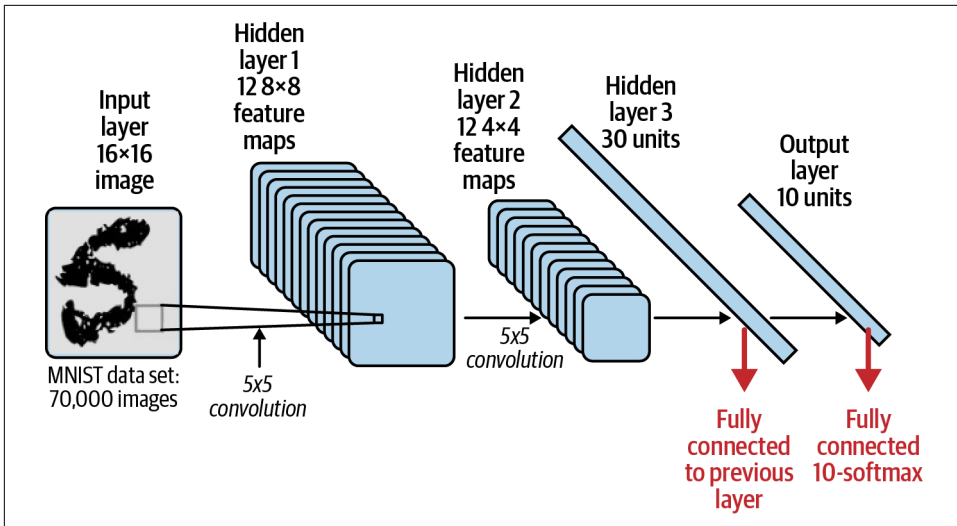


Figure 5-6. The architecture of LeNet1 (1989)

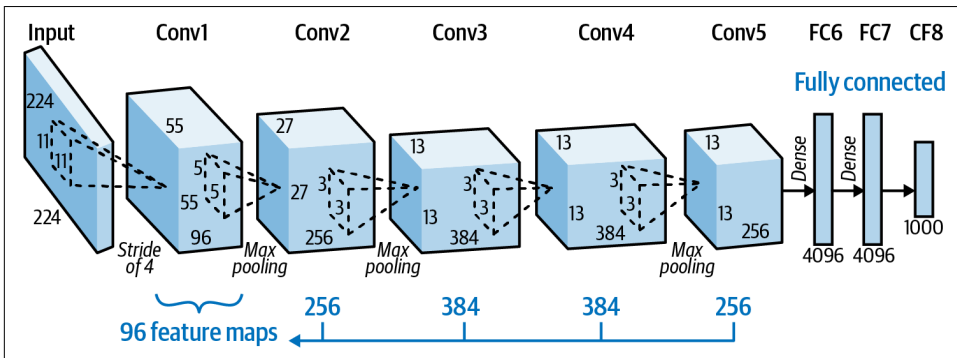


Figure 5-7. The architecture of AlexNet with a whopping 62.3 million weights (2012); adapted from <https://oreil.ly/eEWgJ>

Figure 5-8 shows a wonderful illustration of an image of the handwritten digit 8 passing through a pre-trained LeNet1 and ultimately getting correctly classified as 8.

Finally, if the choice of architecture seems arbitrary to you, meaning that if you are wondering whether we could accomplish similar performance with simpler architecture, join the club. The whole community is wondering the same thing.

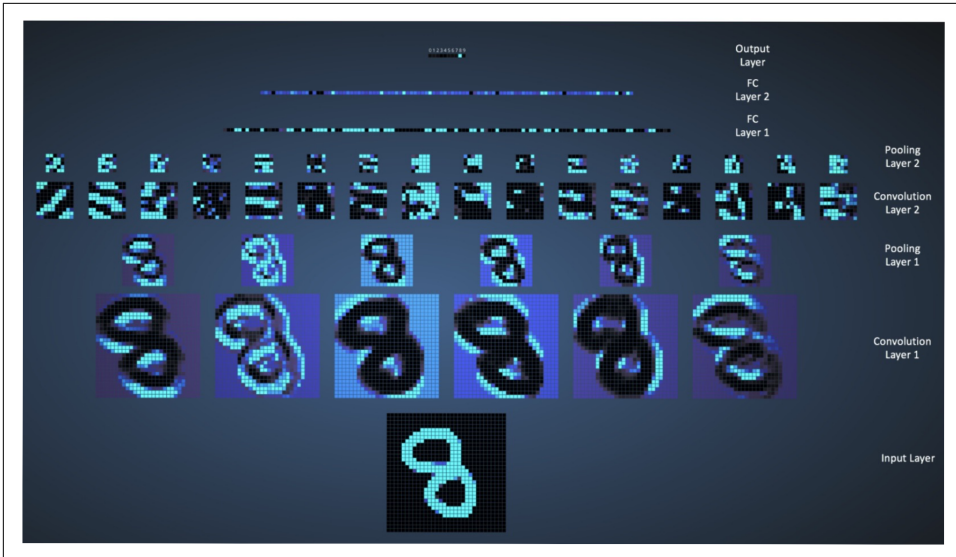


Figure 5-8. Passing an image of a handwritten 8 through a pretrained LeNet1 (*image source*)

Summary and Looking Ahead

In this chapter, we defined the convolution operation: the most significant component of convolutional neural networks. Convolutional neural networks are essential for computer vision, machine audio processing, and other AI applications.

We presented convolution from a systems design perspective, then through filtering one-dimensional and two-dimensional signals. We highlighted the linear algebra equivalent of the convolution operation (multiplying by matrices of special structure), and ended with an example of image classification.

We will encounter convolutional neural networks frequently in this book, as they have become a staple in many AI systems that include vision and/or natural language.

Singular Value Decomposition: Image Processing, Natural Language Processing, and Social Media

Show me the essential, and only the essential.

—H.

The singular value decomposition is a mathematical operation from linear algebra that is widely applicable in the fields of data science, machine learning, and artificial intelligence. It is the mathematics behind principal component analysis (in data analysis) and latent semantic analysis (in natural language processing). This operation transforms a dense matrix into a diagonal matrix. In linear algebra, diagonal matrices are very special and highly desirable. They behave like scalar numbers when we multiply by them, only stretching or squeezing in certain directions.

When computing the singular value decomposition of a matrix, we get the extra bonus of revealing and quantifying the action of the matrix on space itself: rotating, reflecting, stretching, and/or squeezing. There is no warping (bending) of space, since this operation is linear (after all, it is called linear algebra). Extreme stretching or squeezing in one direction versus the others affects the stability of any computations involving our matrix, so having a measure of that allows us direct control over the sensitivity of our computations to various perturbations, for example, noisy measurements.

The power of the singular value decomposition lies in the fact that it can be applied to any matrix. That and its wide use in the field of AI earns it its own chapter in this book. In the following sections, we explore the singular value decomposition, focusing on the big picture rather than the tiny details, and on applications to image processing, natural language processing, and social media.

Given a matrix C (an image, a data matrix, etc.), we omit the details of computing its singular value decomposition. Most linear algebra books do that, presenting a theoretical method based on computing the eigenvectors and eigenvalues of the symmetric matrices $C^t C$ and $C C^t$, which for us are the covariance matrices of the data (if the data is centered). While it is still very important to understand the theory, the method it provides for computing the singular value decomposition is not useful for efficient computations, and is especially impossible for the large matrices involved in many realistic problems. Moreover, we live in an era where software packages help us compute it so easily. In Python, all we have to do is call the `numpy.linalg.svd` method from the *numpy* library. We peek briefly into the numerical algorithms that go into these software packages later in this chapter. However, our main focus is on understanding how the singular value decomposition works and why this decomposition is important for reducing the storage and computational requirements of a given problem without losing its essential information. We will also understand the role it plays in clustering data.

Matrix Factorization

We can factorize a scalar number in multiple ways; for instance, we can write the number $12 = 4 \times 3$, $12 = 2 \times 2 \times 3$, or $12 = 0.5 \times 24$. Which factorization is better depends on our use case. The same can be done for matrices of numbers. Linear algebra provides us with a variety of useful matrix factorizations. The idea is that we want to break down an object into its smaller components, and these components give us insight about the function and the action of the object itself. This breakdown also gives us a good idea about which components contain the most information, and thus are more important than others. In this case, we might benefit from throwing away the less important components, and building a smaller object with a similar function. The smaller object might not be as detailed as the object that we started with, as that contained all of its components; however, it contains enough significant information from the original object that using it with its smaller size provides benefits. The singular value decomposition is a matrix factorization that does exactly that. Its formula looks like:

$$C_{m \times n} = U_{m \times m} \Sigma_{m \times n} V_{n \times n}^t$$

where we break down the matrix C into three component matrices: U , Σ , and V^t . U and V are square matrices that have orthonormal rows and columns. Σ is a diagonal matrix that has the same shape as C (see [Figure 6-1](#)).

Let's start with matrix multiplication. Suppose A is a matrix with 3 rows and 3 columns:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}_{3 \times 3}$$

and B is a matrix with 3 rows and 2 columns:

$$B = \begin{pmatrix} 1 & 3 \\ 4 & -2 \\ 0 & 1 \end{pmatrix}_{3 \times 2}$$

Then $C = AB$ is a matrix with 3 rows and 2 columns:

$$C_{3 \times 2} = A_{3 \times 3} B_{3 \times 2} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}_{3 \times 3} \begin{pmatrix} 1 & 3 \\ 4 & -2 \\ 0 & 1 \end{pmatrix}_{3 \times 2} = \begin{pmatrix} 9 & 2 \\ 24 & 8 \\ 39 & 14 \end{pmatrix}_{3 \times 2}$$

We can think of C as being factorized into the product of A and B , in the same way as the number $12 = 4 \times 3$. The previous factorization of C has no significance, since neither A nor B is a special type of matrix. A very significant factorization of C is its singular value decomposition. Any matrix has a singular value decomposition. We calculate it using Python (see the associated Jupyter notebook for the code):

$$C_{3 \times 2} = U_{3 \times 3} \Sigma_{3 \times 2} V_{2 \times 2}^t = \begin{pmatrix} -0.1853757 & 0.8938507 & 0.4082482 \\ -0.5120459 & 0.2667251 & -0.8164965 \\ -0.8387161 & -0.3604005 & 0.4082482 \end{pmatrix} \begin{pmatrix} 49.402266 & 0 \\ 0 & 1.189980 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} -0.9446411 & -0.3281052 \\ 0.3281052 & -0.9446411 \end{pmatrix}$$

Observe the following in this decomposition: the rows of V^t are the *right singular vectors* (these are exactly the columns of V), the columns of U are the *left singular vectors*, and the diagonal entries of Σ are the *singular values*. The singular values are always positive and always arranged in decreasing order along the diagonal of Σ . The ratio of largest singular value to the smallest singular value is the *condition number* κ of a matrix. In our case there are only two singular values, and $\kappa = \frac{49.402266}{1.189980} = 41.515207$. This number plays an important role in the stability of computations involving our matrix. Well-conditioned matrices are those with condition numbers that are not very large.

The left singular vectors are orthonormal (orthogonal to each other and have length 1). Similarly, the right singular vectors are also orthonormal.

For qualitative properties, images are faster to assess than endless arrays of numbers. It is easy to visualize matrices as images using Python (and vice versa, images are stored as matrices of numbers): the value of an entry of the matrix corresponds to the intensity of the corresponding pixel. The higher the number, the brighter the pixel. Smaller numbers in the matrix show up as darker pixels, and larger numbers show up as brighter pixels. **Figure 6-1** shows the previously mentioned singular value decomposition. We observe that the diagonal matrix Σ has the same shape as C , and its diagonal entries are arranged in decreasing order, with the brightest pixel, corresponding to the largest *singular value*, at the top-left corner.

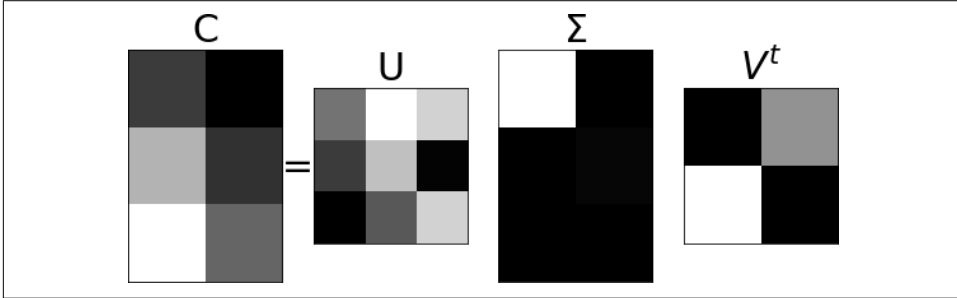


Figure 6-1. Visualizing the singular value decomposition

Figures 6-2 and 6-3 visualize the singular value decompositions of two rectangular matrices A and B , where A is wide and B is tall:

$$A = \begin{pmatrix} -1 & 3 & -5 & 4 & 18 \\ 1 & -2 & 4 & 0 & -7 \\ 2 & 0 & 4 & -3 & -8 \end{pmatrix}_{3 \times 5} = U_{3 \times 3} \Sigma_{3 \times 5} V_{5 \times 5}^t$$

$$B = \begin{pmatrix} 5 & 4 \\ 4 & 0 \\ 7 & 10 \\ -1 & 8 \end{pmatrix}_{4 \times 2} = U_{4 \times 4} \Sigma_{4 \times 2} V_{2 \times 2}^t$$

In **Figure 6-2**, we note that the last two columns of Σ are all zeros (black pixels), hence we can economize in storage and throw away these two columns along with the last two rows of V^t (see the next section for multiplying by a diagonal matrix from the left). Similarly, in **Figure 6-3**, we note that the last two rows of Σ are all zeros (black pixels), hence we can economize in storage and throw away these two rows along with the last two columns of U (see the next section for multiplying by a diagonal matrix from the right). The singular value decomposition is already saving

us some space (note that we usually only store the diagonal entries of Σ as opposed to the whole matrix with all its zeros).

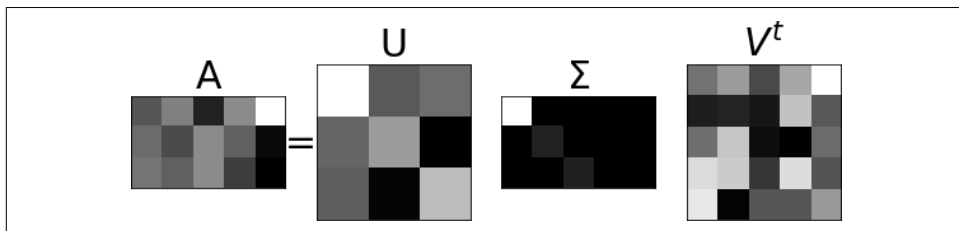


Figure 6-2. Visualizing the singular value decomposition of a wide rectangular matrix. The last two columns of Σ are all zeros (black pixels), allowing for storage reduction: throw away the last two columns of Σ along with the last two rows of V^t .

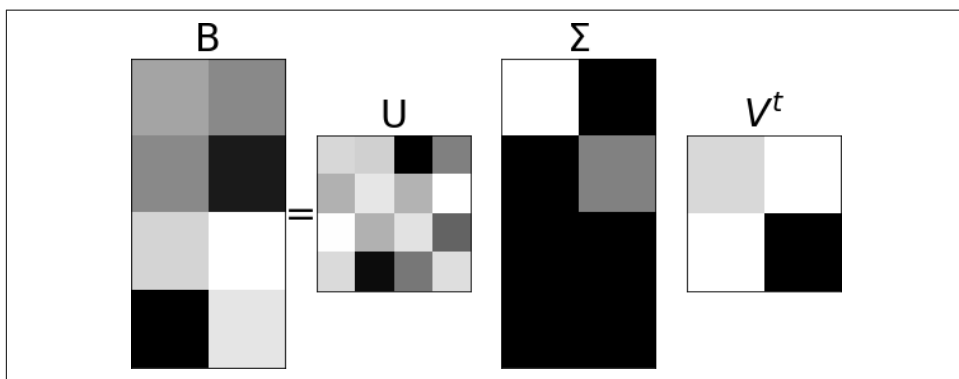


Figure 6-3. Visualizing the singular value decomposition of a tall rectangular matrix. The last two rows of Σ are all zeros (black pixels), allowing for storage reduction: throw away the last two rows of Σ along with the last two columns of U .

Diagonal Matrices

When we multiply a vector by a scalar number, say 3, we obtain a new vector along the same direction with the same orientation, but whose length is stretched three times. When we multiply the same vector by another scalar number, say -0.5 , we get another vector, again along the same direction, but this time its length is halved and its orientation is flipped. Multiplying by a scalar number is such a simple operation, and it would be nice if we had matrices that behaved equally easily when we applied them to (in other words, multiplied them by) vectors. If our life was one-dimensional we would only have to deal with scalar numbers, but since our life and applications of interest are higher dimensional, then we have to satisfy ourselves with diagonal matrices (Figure 6-4). These are the good ones.

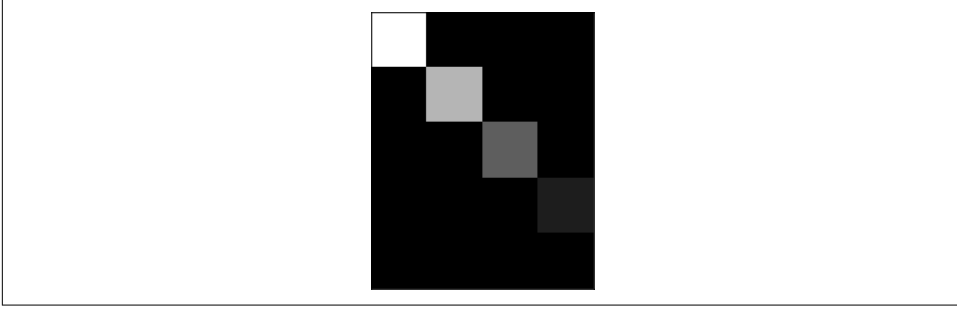


Figure 6-4. An image of a 5×4 diagonal matrix with diagonal entries: 10 (brightest pixel), 6, 3, and 1 (darkest pixel other than the zeros)

Multiplying by a diagonal matrix corresponds to stretching or squeezing in certain directions in space, with orientation flipping corresponding to any negative numbers on the diagonal. As we very well know, most matrices are very far from being diagonal. The power of the singular value decomposition is that it provides us with the directions in space along which the matrix *behaves like* (albeit in a broad sense) a diagonal matrix. A diagonal matrix usually stretches/squeezes in the same directions as those for the vector coordinates. If, on the other hand, the matrix is not diagonal, it generally does not stretch/squeeze in the same directions as the coordinates. It does so in *other* directions, *after* a change of coordinates. The singular value decomposition gives us the required coordinate change (*right singular vectors*), the directions along which vectors will be stretched/squeezed (*left singular vectors*), as well as the magnitude of the stretch/squeeze (*singular values*). We detail this in the next section, but we first clarify multiplication by diagonal matrices from the left and from the right.

If we multiply a matrix A by a diagonal matrix Σ from the right, $A\Sigma$, then we scale the columns of A by the σ 's, for example:

$$A\Sigma = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} \begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{pmatrix} = \begin{pmatrix} \sigma_1 a_{11} & \sigma_2 a_{12} \\ \sigma_1 a_{21} & \sigma_2 a_{22} \\ \sigma_1 a_{31} & \sigma_2 a_{32} \end{pmatrix}$$

If we multiply A by Σ from the left ΣA , then we scale the rows of A by the σ 's, for example:

$$\Sigma A = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} = \begin{pmatrix} \sigma_1 a_{11} & \sigma_1 a_{12} \\ \sigma_2 a_{21} & \sigma_2 a_{22} \\ \sigma_3 a_{31} & \sigma_3 a_{32} \end{pmatrix}$$

Matrices as Linear Transformations Acting on Space

One way we can view matrices is as linear transformations (no warping) that act on vectors in space, and on space itself. If no warping is allowed because it would render an operation nonlinear, then what actions are allowed? The answers are rotation, reflection, stretching, and/or squeezing, which are all nonwarping operations. The singular value decomposition $A = U\Sigma V^t$ captures this concept. When A acts on a vector $\vec{v}$, let's go over the multiplication $A\vec{v} = U\Sigma V^t\vec{v}$ step-by-step:

1. First $\vec{v}$ gets rotated/reflected because of the orthogonal matrix V^t .
2. Then it gets stretched/squeezed along special directions because of the diagonal matrix Σ .
3. Finally, it gets rotated/reflected again because of the other orthogonal matrix U .

Reflections and rotations do not really change space, as they preserve size and symmetries (think of rotating an object or looking at its reflection in a mirror). The amount of stretch and/or squeeze encoded in the diagonal matrix Σ (via its singular values on the diagonal) is very informative regarding the action of A .



Orthogonal Matrix

An orthogonal matrix has orthonormal rows and orthonormal columns. It never stretches or squeezes, it only rotates and/or reflects, meaning that it does not change the size and shape of objects when acting on them, only their direction and/or orientation. As with many things in mathematics, these names are confusing. It is called an *orthogonal* matrix even though its rows and columns are an *orthonormal*, which means orthogonal *and* of length equal to one. One more useful fact: if C is an orthogonal matrix, then $CC^t = C^tC = I$, that is, the inverse of this matrix is its transpose. Computing the inverse of a matrix is usually a very costly operation, but for orthogonal matrices, all we have to do is exchange its rows for its columns.

We illustrate these concepts using two-dimensional matrices since they are easy to visualize. In the following subsections, we explore:

- The action of a matrix A on the right singular vectors, which are the columns $\vec{v}_1$ and $\vec{v}_2$ of the matrix V . These get sent to multiples of the left singular vectors $\vec{u}_1$ and $\vec{u}_2$, which are the columns of U .

- The action of A on the standard unit vectors $\vec{e}_1$ and $\vec{e}_2$. We also notice that the unit square gets transformed to a parallelogram.
- The action of A on a general vector $\vec{x}$. This will help us understand the matrices U and V as rotations or reflections in space.
- The action of A on the unit circle. We see that A transforms the unit circle to an ellipse, with its principal axes along the left singular vectors (the $\vec{u}$'s), and the lengths of its principal axes are the singular values (the σ 's). Since the singular values are ordered from largest to smallest, then $\vec{u}_1$ defines the direction with the most variation, and $\vec{u}_2$ defines the direction with the second most variation, and so on.

Action of A on the Right Singular Vectors

Let A be the 2×2 matrix:

$$A = \begin{pmatrix} 1 & 5 \\ -1 & 2 \end{pmatrix}$$

Its singular value decomposition $A = U\Sigma V^t$ is given by:

$$A = \begin{pmatrix} 0.93788501 & 0.34694625 \\ 0.34694625 & -0.93788501 \end{pmatrix} \begin{pmatrix} 5.41565478 & 0 \\ 0 & 1.29254915 \end{pmatrix} \begin{pmatrix} 0.10911677 & 0.99402894 \\ 0.99402894 & -0.10911677 \end{pmatrix}$$

The expression $A = U\Sigma V^t$ is equivalent to:

$$AV = U\Sigma$$

since all we have to do is multiply $A = U\Sigma V^t$ by V from the right and exploit the fact that $V^t V = I$ due to the orthogonality of V .

We can think of AV as the matrix A acting on each column of the matrix V . Since $AV = U\Sigma$, then the action of A on the orthonormal columns of V is the same as stretching/squeezing the columns of U by the singular values. That is:

$$A\vec{v}_1 = \sigma_1 \vec{u}_1$$

and

$$A\vec{v}_2 = \sigma_2\vec{u}_2$$

This is demonstrated in **Figure 6-5**.

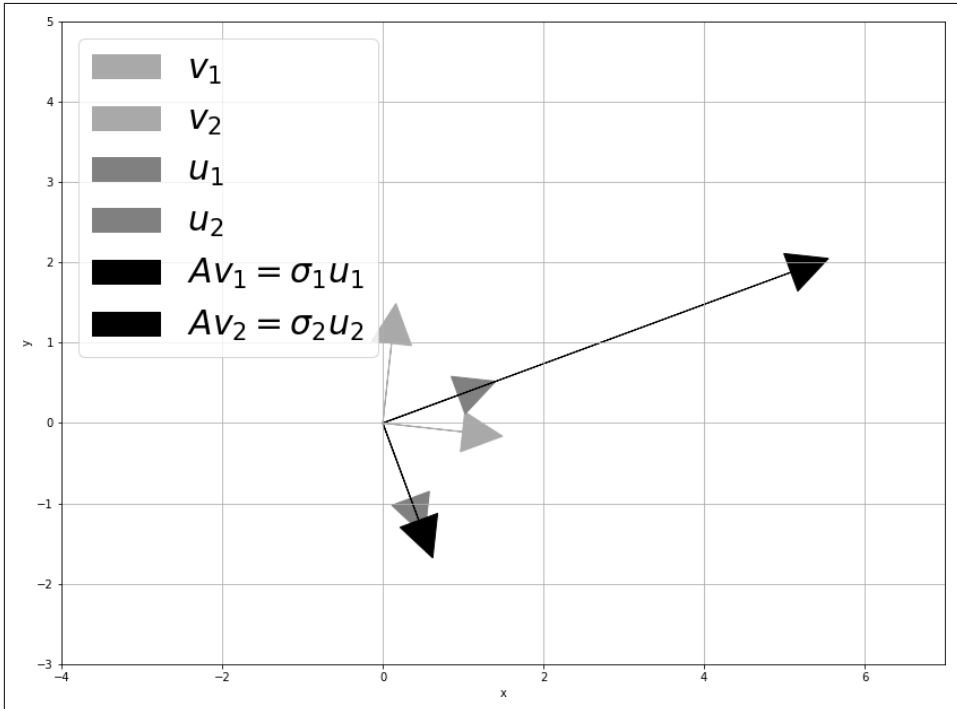


Figure 6-5. Matrix A sends the right singular vectors to multiples of the left singular vectors: $Av_1 = \sigma_1 u_1$ and $Av_2 = \sigma_2 u_2$

Action of A on the Standard Unit Vectors and the Unit Square Determined by Them

The matrix A sends the standard unit vectors to its own columns and transforms the unit square into a parallelogram. There is no warping (bending) of space. **Figure 6-6** shows this transformation.

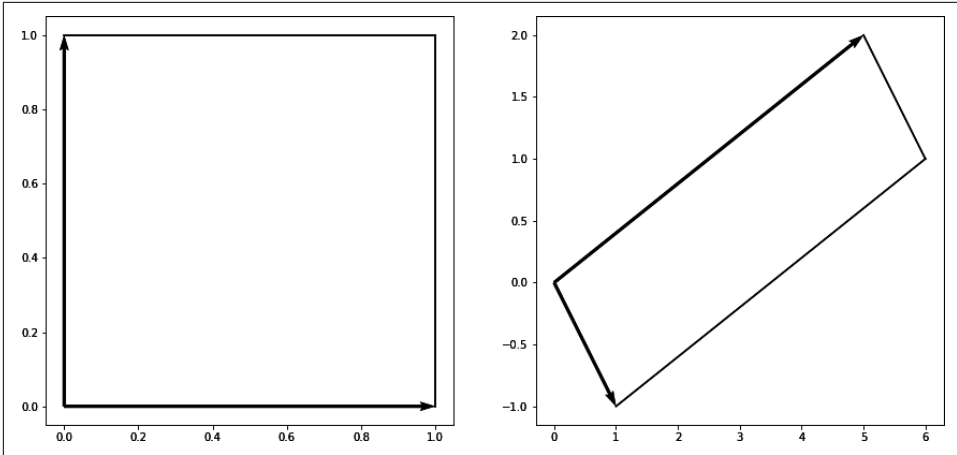


Figure 6-6. Transforming the standard unit vectors

Action of A on the Unit Circle

Figure 6-7 shows that the matrix A sends the unit circle to an ellipse. The principal axes along the u 's and lengths of the principal axes are equal to the σ 's. Again, since matrices represent linear transformations, there is reflection/rotation and stretching/squeezing of space, but no warping.

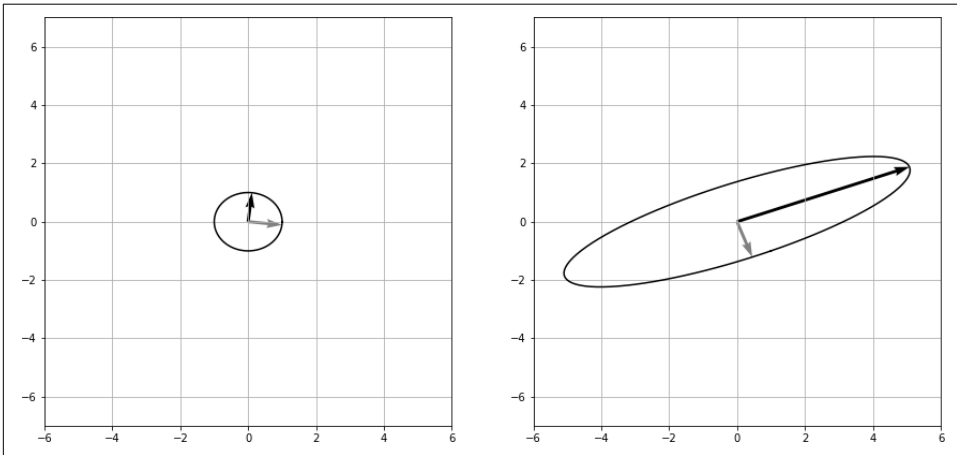


Figure 6-7. Matrix A sends the unit circle to an ellipse with principal axes along the left singular vectors and lengths of principal axes equal to the singular values

We can easily see the described action from the singular value decomposition.

The polar decomposition:

$$A = QS$$

is a very easy way that geometrically shows how a circle gets transformed into an ellipse.

Breaking Down the Circle-to-Ellipse Transformation According to the Singular Value Decomposition

Figure 6-8 shows four subplots that break down the steps of the circle-to-ellipse transformation illustrated previously:

1. First we multiply the unit circle and the vectors $\vec{v}_1$ and $\vec{v}_2$ by V^t . Since $V^t V = I$, we have $V^t \vec{v}_1 = \vec{e}_1$ and $V^t \vec{v}_2 = \vec{e}_2$. So, in the beginning, the right singular vectors get *straightened out*, aligning correctly with the standard unit vectors.
2. Then we multiply by Σ . All that happens here is stretching/squeezing the standard unit vectors by σ_1 and σ_2 (the stretch or squeeze depend on whether the magnitude of the singular value is greater or smaller than one).
3. Finally we multiply by U . This either reflects the ellipse across a line or rotates it a certain amount clockwise or counterclockwise. The next subsection explains this in detail.

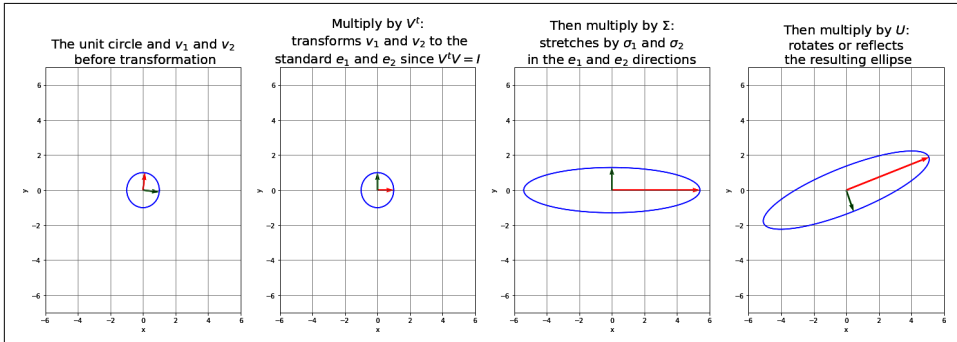


Figure 6-8. Steps of the unit circle-to-ellipse transformation using the singular value decomposition

Rotation and Reflection Matrices

The matrices U and V^t that appear in the singular value decomposition $A = U\Sigma V^t$ are orthogonal matrices. Their rows and columns are orthonormal, and their inverse is the same as their transpose. In two dimensions, the U and V could either be rotation or reflection (about a line) matrices.

Rotation matrix

A matrix that rotates clockwise by an angle θ is given by:

$$\begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix}$$

The transpose of a rotation matrix is a rotation in the opposite direction. So if a matrix rotates clockwise by an angle θ , then its transpose rotates counterclockwise by θ and is given by:

$$\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

Reflection matrix

A reflection matrix about a line L making an angle θ with the x -axis is:

$$\begin{pmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{pmatrix}$$

The slope of the straight line L is $\tan \theta$ and it passes through the origin, so its equation is $y = (\tan \theta)x$. This line acts like a mirror for the reflection operation.

Figure 6-9 shows the two straight lines about which the matrices V^t and U reflect, together with a vector $\vec{x}$ and its subsequent transformation.

The determinant of a rotation matrix is 1, and the determinant of the reflection matrix is -1 .

In higher dimensions, reflection and rotation matrices look different. Always make sure you understand the object you are dealing with. If we have a rotation in a three-dimensional space, then about what axis? If we have a reflection, then about what plane? If you want to dive deeper, this is a good time to read about orthogonal matrices and their properties.

Action of A on a General Vector $\vec{x}$

We have explored the action of A on the right singular vectors (they get mapped to the left singular vectors), the standard unit vectors (they get mapped to the columns of A), the unit square (it gets mapped to a parallelogram), and the unit circle (it gets mapped to an ellipse with principal axes along the left singular vectors and whose lengths are equal to the singular values). Finally, we explore the action of A on a general, nonspecial, vector $\vec{x}$. This gets mapped to another nonspecial vector $A\vec{x}$. However, breaking down this transformation into steps using the singular value decomposition is informative.

Recall our matrix A and its singular value decomposition:

$$\begin{aligned} A &= \begin{pmatrix} 1 & 5 \\ -1 & 2 \end{pmatrix} \\ &= U\Sigma V^t \\ &= \begin{pmatrix} 0.93788501 & 0.34694625 \\ 0.34694625 & -0.93788501 \end{pmatrix} \begin{pmatrix} 5.41565478 & 0 \\ 0 & 1.29254915 \end{pmatrix} \begin{pmatrix} 0.10911677 & 0.99402894 \\ 0.99402894 & -0.10911677 \end{pmatrix} \end{aligned}$$

Both U and V^t in this singular value decomposition happen to be reflection matrices. The straight lines L_U and L_{V^t} that act as mirrors for these reflections are plotted in **Figure 6-9**, and their equations are easy to find from their respective matrices: $\cos(2\theta)$ and $\sin(2\theta)$ are on the first row, so we can use those to find the slope $\tan(\theta)$. The equation of the line along which V^t reflects is then $y = (\tan \theta_{V^t})x = 0.8962347008436108x$, and that of the line along which U reflects is $y = (\tan \theta_U)x = 0.17903345403184898x$. Since $A\vec{x} = U\Sigma V^t\vec{x}$, first $\vec{x}$ gets reflected across the line L_{V^t} , arriving at $V^t\vec{x}$. Then, when we multiply by Σ from the left, the first coordinate of $V^t\vec{x}$ gets stretched horizontally by the first singular value, and the second coordinate gets stretched by the second singular value, obtaining $\Sigma V^t\vec{x}$. Finally, when we multiply by U , the vector $\Sigma V^t\vec{x}$ gets reflected across the line L_U , arriving at $A\vec{x} = U\Sigma V^t\vec{x}$. **Figure 6-9** illustrates this process.

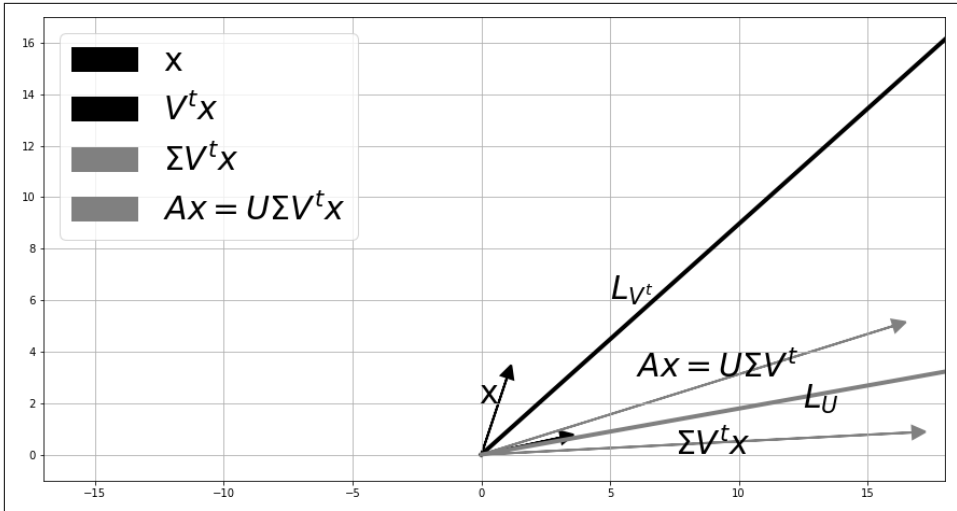


Figure 6-9. Action of a matrix A on a general vector $\vec{x}$. The transformation is done in steps using the singular value decomposition.

Three Ways to Multiply Matrices

Efficient algorithms for matrix multiplication are so desirable in the age of big data. In theory, there are three ways to multiply two matrices $A_{m \times n}$ and $B_{n \times s}$:

Row-column approach

Produce one entry $(ab)_{ij}$ at a time by taking the dot product of the i th row of A with the j th column of B :

$$(ab)_{ij} = A_{row_i} B_{col_j} = \sum_{k=1}^n a_{ik} b_{kj}$$

Column-columns approach

Produce one column $(AB)_{col_i}$ at a time by linearly combining the columns of A using the entries of the i th column of B :

$$(AB)_{col_i} = b_{1i} A_{col_1} + b_{2i} A_{col_2} + \cdots + b_{ni} A_{col_n}$$

Column-row approach

Produce *rank one pieces* of the product, one piece at a time, by multiplying the first column of A with the first row of B , the second column of A with the second row of B , and so on. Then add all these rank one matrices together to get the final product AB :

$$AB = A_{col_1}B_{row_1} + A_{col_2}B_{row_2} + \cdots + A_{col_n}B_{row_n}$$

How does this help us understand the usefulness of the singular value decomposition? We can expand the product $A = U\Sigma V^t$ of the singular value decomposition as a sum of rank one matrices, using the *column-row* approach for matrix multiplication. Here, we multiply the matrix $U\Sigma$ (which scales each column U_{col_i} of U by σ_i) with V^t :

$$A = U\Sigma V^t = \sigma_1 U_{col_1} V_{row_1}^t + \sigma_2 U_{col_2} V_{row_2}^t + \cdots + \sigma_r U_{col_r} V_{row_r}^t$$

where r is the number of nonzero singular values of A (also called the *rank* of A).

The great thing about this expression is that it splits A into a sum of rank one matrices arranged according to their order of importance, since the σ 's are arranged in decreasing order. Moreover, it provides a straightforward way to approximate A by lower rank matrices: throw away lower singular values. The *Eckart-Young-Mirsky theorem* asserts that this is in fact the *best way* to find low rank approximation of A , when the *closeness* of approximation is measured using the *Frobenius norm* (which is the square root of the sum of squares of the singular values) for matrices. Later in this chapter, we take advantage of this rank one decomposition of A for digital image compression.



Algorithms for Matrix Multiplication

Finding efficient algorithms for matrix multiplication is an essential, yet surprisingly difficult, goal. In matrix multiplication algorithms, saving on even one multiplication operation is worthy (saving on addition is not as much of a big deal). Recently, DeepMind developed *AlphaTensor* (2022) to automatically discover more efficient algorithms for matrix multiplication. This is a milestone because matrix multiplication is a fundamental part of a vast array of technologies, including neural networks, computer graphics, and scientific computing.

The Big Picture

So far we have focused on the singular value decomposition of a matrix $A = U\Sigma V^t$ in terms of A 's action on space and in terms of approximating A using lower rank matrices. Before moving to applications relevant to AI, let's have an eagle-eye perspective and address the big picture.

Given a matrix of real numbers, we want to understand the following, depending on our use case:

- If the matrix represents data that we care about, like images or tabular data, what are the most important components of this matrix (data)?
- Along what important directions is the data mostly spread (directions with most variation in the data)?
- If I think of a matrix $A_{m \times n}$ as a transformation from the initial space $\mathbb{R}^n$ to the target space $\mathbb{R}^m$, what is the effect of this matrix on vectors in $\mathbb{R}^n$? To which vectors do they get sent in $\mathbb{R}^m$?
- What is the effect of this matrix on space itself? Since this is a linear transformation, we know there is no space warping, but there is space stretching, squeezing, rotating, and reflecting.
- Many physical systems can be represented as a system of linear equations $A\vec{x} = \vec{b}$. How can we solve this system (find $\vec{x}$)? What is the most efficient way to go about this, depending on the properties of A ? If there is no solution, is there an approximate solution that satisfies our purposes? Note that here we are looking for the unknown vector $\vec{x}$ that gets transformed to $\vec{b}$ when A acts on it.

The singular value decomposition can be used to answer all these questions. The first two are intrinsic to the matrix itself, while the second two have to do with the effect of multiplying the matrix with vectors (the matrix acts on space and the vectors in this space). The last question has to do with the very important problem of solving systems of linear equations and appears in all kinds of applications.

Therefore, we can investigate a matrix of numbers in two ways:

- What are its intrinsic properties?
- What are its properties when viewed as a transformation?

These two are related because the matrix's intrinsic properties affect how it acts on vectors and on space.

The following are properties to keep in mind:

- A sends the orthonormal vectors v_i (right singular vectors) of its initial space to scalar multiples of the orthonormal vectors u_i (left singular vectors) of its target space:

$$Av_i = \sigma_i u_i$$

- If our matrix is square, then the absolute value of its determinant is equal to the product of all its singular values: $\sigma_1\sigma_2\cdots\sigma_r$.
- The condition number of the matrix, with respect to the l^2 norm, which is the usual distance in Euclidean space, is the ratio of the largest singular value to the smallest singular value:

$$\kappa = \frac{\sigma_1}{\sigma_r}$$

The Condition Number and Computational Stability

The condition number is very important for computational stability:

- The condition number measures how much A stretches space. If the condition number is too large, then it stretches space too much in one direction relative to another direction, and it could be dangerous to do computations in such an extremely stretched space. Solving $A\vec{x} = \vec{b}$ when A has a large condition number makes the solution $\vec{x}$ unstable in the sense that it is extremely sensitive to perturbations in $\vec{b}$. A small error in $\vec{b}$ will result in a solution $\vec{x}$ that is wildly different from the solution without the error in $\vec{b}$. It is easy to envision this instability geometrically.
- Numerically solving $A\vec{x} = \vec{b}$ (say, by Gaussian elimination) and iterative methods works fine when the involved matrices have reasonable (not very large) condition numbers.
- One thing about a matrix with an especially large condition number: it stretches space so much that it almost collapses into a space of lower dimension. The interesting part is that if we decide to throw away that very small singular value and hence work in the collapsed space of lower dimension, our computations become perfectly fine. So at the boundaries of extremeness lies normalcy, except that this normalcy now lies in a lower dimension.
- Many iterative numerical methods, including the very useful gradient descent, have matrices involved in their analysis. If the condition number of these matrices is too large, then the iterative method might not converge to a solution. The condition number controls how fast these iterative methods converge.

The Ingredients of the Singular Value Decomposition

In this chapter we have been dissecting only one formula: $A = U\Sigma V^t$. We used Python to compute the entries of U , Σ , and V , but what exactly are these entries? The answer is short, if we happen to know what eigenvectors and eigenvalues are, which we clarify in the next section. For now, we list the ingredients of U , Σ , and V :

- The columns of V (the right singular vectors) are the orthonormal eigenvectors of the symmetric matrix $A^t A$.
- The columns of U (the left singular vectors) are the orthonormal eigenvectors of the symmetric matrix AA^t .
- The singular values $\sigma_1, \sigma_2, \dots, \sigma_r$ are the square roots of the eigenvalues of $A^t A$ or AA^t . The singular values are nonnegative and arranged in decreasing order. The singular values can be zero.
- $Av_i = \sigma_i u_i$.



Every real symmetric positive semi-definite (nonnegative eigenvalues) matrix is diagonalizable $S = PDP^{-1}$, which means that it is similar to a diagonal matrix D when viewed in a different set of coordinates (the columns of P). $A^t A$ and AA^t both happen to be symmetric positive semi-definite, so they are diagonalizable.

Singular Value Decomposition Versus the Eigenvalue Decomposition

It is important to learn more about symmetric matrices if we want to understand the ingredients of the singular value decomposition. This will also help us discern the difference between the singular value decomposition $A = U\Sigma V^t$ and the eigenvalue decomposition $A = PDP^{-1}$ when the latter exists.



The singular value decomposition (SVD) always exists, but the eigenvalue decomposition exists only for special matrices, called diagonalizable. Rectangular matrices are never diagonalizable. Square matrices may or may not be diagonalizable. When the square matrix is diagonalizable, the SVD and the eigenvalue decomposition are *not equal*, unless the matrix is symmetric and has nonnegative eigenvalues.

We can think of a hierarchy in terms of the desirability of matrices:

1. The best and easiest matrices are square diagonal matrices with the same number along the diagonal.
2. The second best ones are square diagonal matrices D that don't necessarily have the same numbers along the diagonal.
3. The third best matrices are symmetric matrices. These have real eigenvalues and orthogonal eigenvectors. They are the next closest type of matrices to diagonal matrices, in the sense that they are diagonalizable $S = PDP^{-1}$, or similar to a diagonal matrix after a change in basis. The columns of P (eigenvectors) are orthogonal.
4. The fourth best matrices are square matrices that are diagonalizable $A = PDP^{-1}$. These are similar to a diagonal matrix after a change of basis; however, the columns of P (eigenvectors) need not be orthogonal.
5. The fifth best matrices are all the rest. These are not diagonalizable, meaning there is no change of basis that can turn them diagonal; however, there is the next closest approach to making them similar to a diagonal matrix via the singular value decomposition $A = U\Sigma V^t$. Here U and V are different from each other, and they have orthonormal columns and rows. Their inverse is very easy, since it is the same as their transpose. The singular value decomposition works for both square and nonsquare matrices.

Given a matrix A , both $A^t A$ and AA^t happen to be symmetric and positive semi-definite (meaning their eigenvalues are nonnegative); thus, they are diagonalizable with two bases of orthogonal eigenvectors. When we divide by the norm of these orthogonal eigenvectors, they become orthonormal. These are the columns of V and of U , respectively.

$A^t A$ and AA^t have exactly the same nonnegative eigenvalues, $\lambda_i = \sigma_i^2$. Arrange the square root of these in decreasing order (keeping the corresponding eigenvector order in U and V), and we get the diagonal matrix Σ in the singular value decomposition.

What if the matrix we start with is symmetric? How is its singular value decomposition $A = U\Sigma V^t$ related to its diagonalization $A = PDP^{-1}$? The columns of P , which are the eigenvectors of symmetric A , are orthogonal. When we divide by their lengths, they become orthonormal. Stack these orthonormal eigenvectors in a matrix in the order corresponding to the decreasing absolute value of the eigenvalues and we get both the U and the V for the singular value decomposition. Now if all the eigenvalues of symmetric A happen to be nonnegative, the singular value decomposition of this positive semi-definite symmetric matrix will be the same as its eigenvalue decomposition, provided you normalize the orthogonal eigenvectors in P , ordering them with respect to the nonnegative eigenvalues in decreasing order.

So $U = V$ in this case. What if some (or all) of the eigenvalues are negative? Then $\sigma_i = |\lambda_i| = -\lambda_i$, but now we have to be careful with the corresponding eigenvectors $Av_i = -\lambda_i v_i = \lambda_i(-v_i) = \sigma_i u_i$. This makes U and V in the singular value decomposition unequal. So the singular value decomposition of a symmetric matrix that has some negative eigenvalues can be easily extracted from its eigenvalue decomposition, but it is not exactly the same.

What if the matrix we start with is not symmetric but diagonalizable? How is its singular value decomposition $A = U\Sigma V^t$ related to its diagonalization $A = PDP^{-1}$? In this case, the eigenvectors of A , which are the columns of P , are in general not orthogonal, so the singular value decomposition and the eigenvalue decomposition of such a matrix are not related.

Computation of the Singular Value Decomposition

How do Python and others numerically calculate the singular value decomposition of a matrix? What numerical algorithms lie under the hood? The fast answer is: *QR decomposition*, *Householder reflections*, and *iterative algorithms* for eigenvalues and eigenvectors.

In theory, calculating the singular value decomposition for a general matrix, or the eigenvalues and the eigenvectors for a square matrix, requires setting a polynomial equal to 0 to solve for the eigenvalues, then setting up a linear system of equations to solve for the eigenvectors. This is far from being practical for applications. The problem of finding the zeros of a polynomial is very sensitive to any variations in the coefficients of the polynomials, so the computational problem becomes prone to roundoff errors that are present in the coefficients. We need stable numerical methods that find the eigenvectors and eigenvalues without having to numerically compute the zeros of a polynomial. Moreover, we need to make sure that the matrices involved in linear systems of equations are well conditioned, otherwise popular methods like Gaussian elimination (the *LU* decomposition) do not work.

Most numerical implementations of the singular value decomposition try to avoid computing AA^t and A^tA . This is consistent with one of the themes of this book: avoid multiplying matrices; instead, multiply a matrix with vectors. The popular numerical method for the singular value decomposition uses an algorithm called *Householder reflections* to transform the matrix to a bidiagonal matrix (sometimes preceded by a *QR* decomposition), then uses iterative algorithms to find the eigenvalues and eigenvectors. The field of numerical linear algebra develops such methods and adapts them to the types and sizes of matrices that appear in applications. In the next subsection, we present an iterative method to compute one eigenvalue and its corresponding eigenvector for a given matrix.

Computing an Eigenvector Numerically

An eigenvector of a square matrix A is a nonzero vector that does not change its direction when multiplied by A ; instead, it only gets scaled by an eigenvalue λ :

$$Av = \lambda v$$

The following iterative algorithm is an easy numerical method that finds an eigenvector of a matrix corresponding to its largest eigenvalue:

1. Start at a random unit vector (of length 1) v_0 .
2. Multiply by A : $v_{i+1} = Av_i$.
3. Divide by the length of v_{i+1} to avoid the size of our vectors growing too large.
4. Stop when you converge.

This iterative method is very simple but has a drawback: it only finds one eigenvector of the matrix—the eigenvector corresponding to its largest eigenvalue. So it finds the direction that gets stretched the most when we apply A .

For example, consider the matrix $A = \begin{pmatrix} 1 & 2 \\ 2 & -3 \end{pmatrix}$. We start with the vector $\vec{v}_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and apply the above algorithm. We note the algorithm after 28 iterations to the vector $\vec{v} = \begin{pmatrix} -0.38268343 \\ 0.92387953 \end{pmatrix}$. The code is in the linked Jupyter notebook and the output is shown here:

```
[1, 0]
[0.4472136  0.89442719]
[ 0.78086881 -0.62469505]
[-0.1351132  0.99083017]
[ 0.49483862 -0.86898489]
[-0.3266748  0.9451368]
[ 0.40898444 -0.91254136]
[-0.37000749  0.92902877]
[ 0.38871252 -0.92135909]
[-0.37979817  0.92506937]
[ 0.3840601 -0.9233081]
[-0.38202565  0.92415172]
[ 0.38299752 -0.92374937]
[-0.38253341  0.92394166]
[ 0.38275508 -0.92384985]
[-0.38264921  0.92389371]
[ 0.38269977 -0.92387276]
[-0.38267563  0.92388277]
[ 0.38268716 -0.92387799]
[-0.38268165  0.92388027]
[ 0.38268428 -0.92387918]
```

```

[-0.38268303  0.9238797 ]
[ 0.38268363 -0.92387945]
[-0.38268334  0.92387957]
[ 0.38268348 -0.92387951]
[-0.38268341  0.92387954]
[ 0.38268344 -0.92387953]
[-0.38268343  0.92387953]
[ 0.38268343 -0.92387953]

v= [-0.38268343  0.92387953]
Av= [ 1.46507563 -3.53700546]
 $\lambda = -3.828427140993716$ 

```

Figure 6-10 shows this iteration. Note that all the vectors have length 1 and that the direction of the vector does not change when the algorithm converges, hence capturing an eigenvector of A . For the last few iterations, the sign keeps oscillating, so the vector keeps flipping orientation, and the eigenvalue must be negative. Indeed, we find it to be $\lambda = -3.828427140993716$.

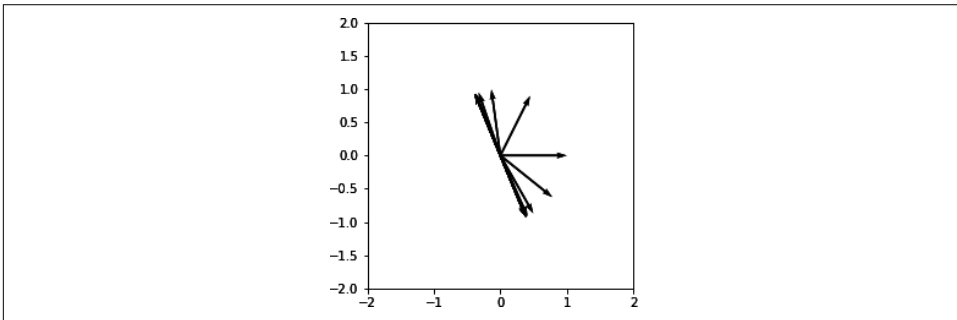


Figure 6-10. We start at $\vec{v}_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, then multiply by A and normalize until we converge to an eigenvector

The Pseudoinverse

Many physical systems can be represented by (or approximated by) a linear system of equations $A\vec{x} = \vec{b}$. If $\vec{x}$ is an unknown vector that we care for, then we need to divide by matrix A in order to find $\vec{x}$. The matrix equivalent of division is finding the inverse A^{-1} , so that the solution $\vec{x} = A^{-1}\vec{b}$. Matrices that have an inverse are called *invertible*. These are square matrices with a nonzero determinant (the determinant is the product of the eigenvalues; the product of the singular values and the determinant will have the same absolute value). But what about all the systems whose matrices are rectangular? How about those with noninvertible matrices? And those whose matrices are square and invertible, but are almost noninvertible (their determinant is very close to zero)? We still care about finding solutions to such systems. The power

of the singular value decomposition is that it exists for any matrix, including those mentioned above, and it can help us invert any matrix.

Given any matrix and its singular value decomposition $A = U\Sigma V^t$, we can define its *pseudoinverse* as:

$$A^+ = V\Sigma^+U^t$$

where Σ^+ is obtained from Σ by inverting all its diagonal entries except for the ones that are zero (or very close to zero if the matrix happens to be ill-conditioned).

This allows us to find solutions to any system of linear equation $Ax = b$, namely $x = A^+b$.



The pseudoinverse of a matrix coincides with its inverse when the latter exists.

Applying the Singular Value Decomposition to Images

We are finally ready for real-world applications of the singular value decomposition. We start with image compression. Digital images are stored as matrices of numbers, where each number corresponds to the intensity of a pixel. We will use the singular value decomposition to reduce the storage requirements of an image without losing its most essential information. All we have to do is throw away the insignificant singular values, along with the corresponding columns of U and rows of V^t . The mathematical expression that helps us here is:

$$A = U\Sigma V^t = \sigma_1 U_{col_1} V_{row_1}^t + \sigma_2 U_{col_2} V_{row_2}^t + \cdots + \sigma_r U_{col_r} V_{row_r}^t$$

Recall that the σ 's are arranged from the largest value to the smallest value, so the idea is that we can keep the first few large σ 's and throw away the rest of the σ 's, which are small anyway.

Let's work with the image in [Figure 6-11](#). The code and details are in the [book's GitHub page](#). Each color image has three channels: red, green, and blue (see [Figures 6-12](#) and [6-13](#)). Each channel is a matrix of numbers, just like the ones we have been working with in this chapter.

Each channel of the image in **Figure 6-11** is a $size = 960 \times 714$ matrix, so to store the full image, we need $size = 960 \times 714 \times 3 = 2,056,320$ numbers. Imagine the storage requirements for a streaming video, which contains many image frames. We need a compression mechanism, so as not to run out of memory. We compute the singular value decomposition for each channel (see **Figure 6-14** for an image representation of the singular value decomposition for the red channel). We then perform a massive reduction, retaining for each channel only the first 25 singular values (out of 714), 25 columns of U (out of 960), and 25 rows of V^t (out of 714). The storage reduction for each channel is substantial: U is now 960×25 , V^t is 25×714 , and we only need to store 25 singular values (no need to store the zeros of the diagonal matrix Σ). This adds up 41,875 numbers for each channel, so for all 3 channels, we need to store $41,875 \times 3 = 125,625$ numbers, a whopping 93% storage requirement reduction.

We put the image back together, one channel at a time, by multiplying the reduced U , reduced Σ , and reduced V^t together:

$$Channel_{reduced} = U_{960 \times 25} \Sigma_{25 \times 25} (V^t)_{25 \times 714}$$

Figure 6-15 shows the result of this multiplication for each of the red, green, and blue channels.

Finally, we layer the reduced channels to produce the reduced image (**Figure 6-16**). It is obvious that we lost a lot of detail in the process, but that is a trade-off we have to live with.

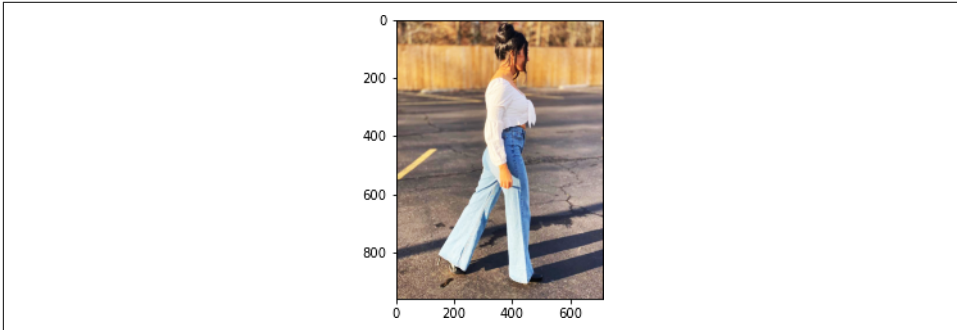


Figure 6-11. A digital color image of $size = 960 \times 714 \times 3$

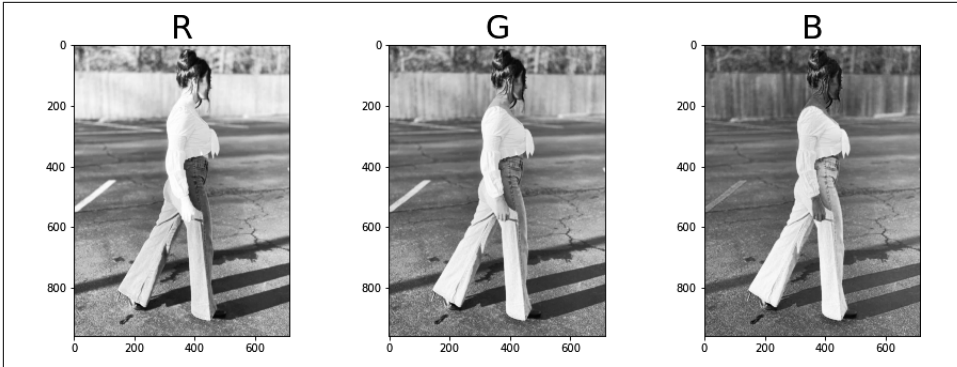


Figure 6-12. The red, green, and blue channels of the digital image. Each has $size = 960 \times 714$.

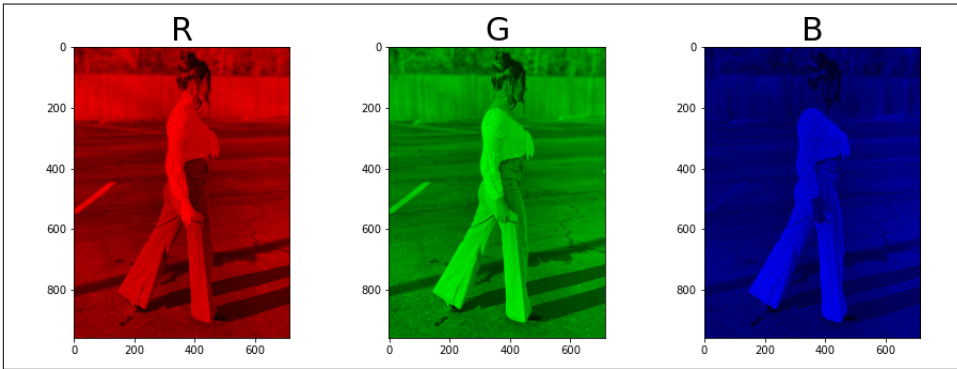


Figure 6-13. Showing the red, green, and blue tints for the three channels of the digital image. Each has $size = 960 \times 714 \times 3$. See the [color image on GitHub](#).

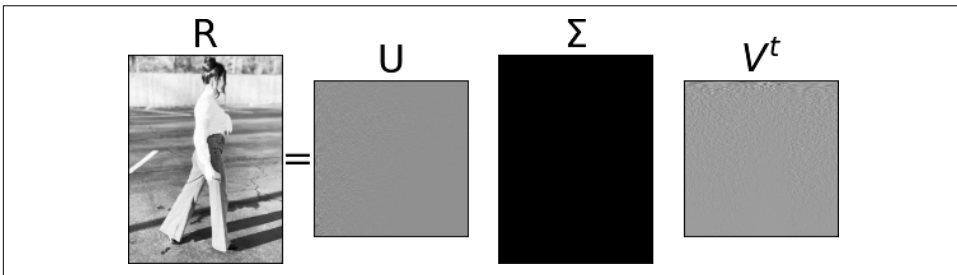


Figure 6-14. The singular value decomposition of the red channel. We have 714 nonzero singular values, but few significant ones. Even though the diagonal matrix Σ appears all black, it had nonzero singular on its diagonal. The pixels are not bright enough to appear clearly at this resolution level.

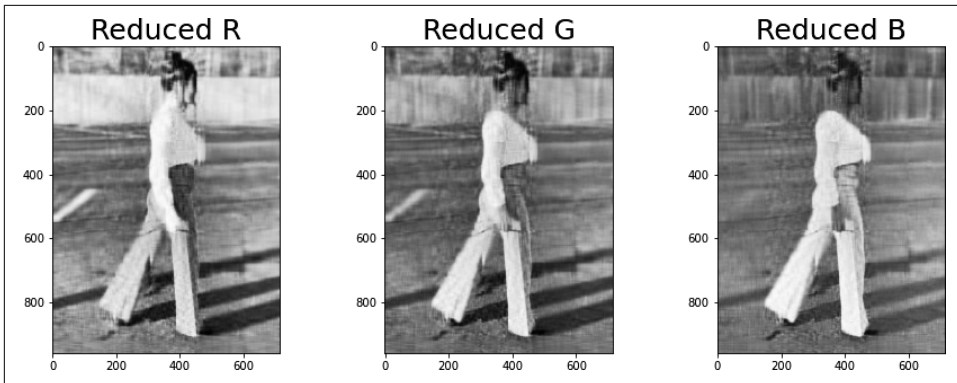


Figure 6-15. The red, green, and blue channels after rank reduction. For each channel we have retained only the first 25 singular values, the first 25 columns of U , and the first 25 rows of V^t .

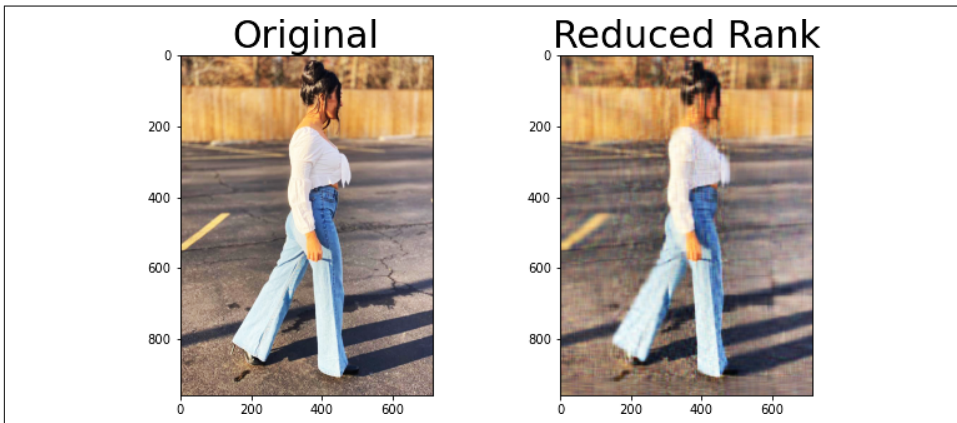


Figure 6-16. Original image with 714 singular values versus reduced rank image with only 25 singular values. Both still have $size = 960 \times 714 \times 3$ but require different storage space.

For advanced image compression techniques, [check this article](#).

Principal Component Analysis and Dimension Reduction

Principal component analysis is widely popular for data analysis. It is used for dimension reduction and clustering in unsupervised machine learning. In a nutshell, it is the singular value decomposition performed on the data matrix X , after *centering* the data, which means subtracting the average value of each feature from each feature

column (each column of X). The principal components are then the right singular vectors, which are the rows of V^t in the now familiar decomposition $X = U\Sigma V^t$.

Statisticians like to describe the **principal component analysis** using the language of variance, or variation in the data, and *uncorrelating* the data. They end up working with the eigenvectors of the *covariance matrix* of the data. This is a familiar description of principal component analysis in statistics:

It is a method that reduces the dimensionality of a dataset, while preserving as much variability, or statistical information, as possible. Preserving as much variability as possible translates into finding new features that are linear combinations of those of the dataset, that successively maximize variance and that are uncorrelated with each other.

The two descriptions (the right singular vectors of the centered data, and the eigenvectors of the covariance matrix) are exactly the same, since the rows of V^t are the eigenvectors of $X_{centered}^t X_{centered}$, which in turn is the covariance matrix of the data. Moreover, the term *uncorrelating* in statistics corresponds to *diagonalizing* in mathematics and linear algebra, and the singular value decomposition says that any matrix *acts like* a diagonal matrix, namely Σ , when written in a new set of coordinates, namely the rows of V^t , which are the columns of V .

Let's explain this in detail. Suppose X is a centered data matrix and its singular value decomposition is $X = U\Sigma V^t$. This is the same as $XV = U\Sigma$, or, when we resolve the expression by column, $XV_{col_i} = \sigma_i U_{col_i}$. Note that XV_{col_i} is just a linear combination of the features of the data using the entries of that particular column of V . Now, faithful to what we have been doing all along in this chapter, we can throw away the less significant components, meaning the columns of V and U corresponding to the lower singular values.

Suppose now that our data has 200 features, but only 2 singular values are significant, so we decide to only keep the first 2 columns of V and the first 2 columns of U . Thus, we have reduced the dimension of the features from 200 to 2. The first new feature is a linear combination of all the original 200 features using the entries of the first column of V , but that is exactly $\sigma_1 U_{col_1}$, and the second new feature is a linear combination of all the original 200 features using the entries of the second column of V , but that is exactly $\sigma_2 U_{col_2}$.

Now let's think of individual data points. A data point in the data matrix X has 200 features. This means that we need 200 axes to plot this data point. However, taking the dimension reduction we performed previously using only the first two principal components, this data point will now have only two coordinates, which are the corresponding entries of $\sigma_1 U_{col_1}$ and $\sigma_2 U_{col_2}$. So if this was the third data point in the data set, then its new coordinates will be the third entry of $\sigma_1 U_{col_1}$ and the third

entry of $\sigma_2 U_{col_2}$. Now it is easy to plot this data point in a two-dimensional space, as opposed to plotting it in the original 200-dimensional space.

We choose how many singular values (and thus principal components) to retain. The more we keep, the more faithful to the original data set we would be, but of course the dimension will be higher. This *truncation* decision (finding the optimal threshold for singular value truncation) is the subject of ongoing research. The common method is determining the desired rank ahead of time, or keeping a certain amount of variance in the original data. Other techniques plot all the singular values, observe an obvious change in the graph, and decide to truncate at that location, hopefully separating the essential patterns in the data from the noise.

It is important not to only center the data, but to also standardize it: subtract the mean of each feature and divide by the standard deviation. The reason is that the singular value decomposition is sensitive to the scale of the feature measurements. When we standardize the data, we end up working with the correlation matrix instead of the covariance matrix. To not confuse ourselves, the main point to keep in mind is that we perform the singular value decomposition on the standardized data set, then the principal components are the columns of V , and the new coordinates of the data points are the entries of $\sigma_i U_{col_i}$.

Principal Component Analysis and Clustering

We saw in the previous section how we can use principal component analysis to reduce the number of features of the data, providing a new set of features in hierarchical order in terms of variation in the data. This is incredibly useful for visualizing data, since we can only visualize in two or three dimensions. It is important to be able to visualize patterns and correlations in high-dimensional data, for example, in genetic data. Sometimes, in the reduced dimensional space determined by the principal components, there is an inherent clustering of the data by category. For example, if the data set contains both patients with cancer and patients without cancer, along with their genetic expression (usually in the thousands), we might notice that plotting the data in the first three principal components space, patients with cancer cluster separately from patients without cancer.

A Social Media Application

In the same essence of principal component analysis and clustering, a recent [publication](#) (Dec 2020) by Dan Vilenchik presents a wonderful application from social media: an unsupervised approach to characterizing users in online social media platforms. Here's the abstract from a talk he gave on the subject, along with the abstract from his publication:

Making sense of data that is automatically collected from online platforms such as online social media or e-learning platforms is a challenging task: the data is massive, multidimensional, noisy, and heterogeneous (composed of differently behaving individuals). In this talk we focus on a central task common to all on-line social platforms and that is the task of user characterization. For example, automatically identify a spammer or a bot on Twitter, or a disengaged student in an e-learning platform.

Online social media channels play a central role in our lives. Characterizing users in social networks is a long-standing question, dating back to the 50's when Katz and Lazarsfeld studied influence in “Mass Communication”. In the era of Machine Learning, this task is typically cast as a supervised learning problem, where a target variable is to be predicted: age, gender, political incline, income, etc. In this talk we explore what can be achieved in an unsupervised manner. Specifically, we harness principal component analysis to understand what underlying patterns and structures are inherent to some social media platforms, but not to others, and why. We arrive at a Simpson-like paradox that may give us a deeper understanding of the data-driven process of user characterization in such platforms.

The idea of principal component analysis for creating clusters with maximal variance in the data will appear multiple times throughout this book.

Latent Semantic Analysis

Latent semantic analysis for natural language data (documents) is similar to principal component analysis for numerical data.

Here, we want to analyze the relationships between a set of documents and the words they contain. The distributional hypothesis for latent semantic analysis states that words that have similar meaning occur in similar pieces of text, and hence in similar documents. Computers only understand numbers, so we have to come up with a numerical representation of our word documents before doing any analysis on them. One such representation is the *word count* matrix X : the columns represent unique words (such as apple, orange, dog, city, intelligence, etc.) and the rows represent each document. Such a matrix is very large but very sparse (has many zeros). There are too many words (these are the features), so we need to reduce the dimension of the features while preserving the similarity structure among the documents (the data points). By now we know what to do: perform the singular value decomposition on the word count matrix, $X = U\Sigma V^t$, then throw away the smaller singular values along with the corresponding columns from U and rows from V^t . We can now represent each document in the lower-dimensional space (of linear combinations of words) in exactly the same way principal component analysis allows for data representation in lower-dimensional feature space.

Once we have reduced the dimension, we can finally compare the documents using *cosine similarity*: compute the cosine of the angle between the two vectors representing the documents. If the cosine is close to 1, then the documents point in the same

direction in *word space* and hence represent very similar documents. If the cosine is close to 0, then the vectors representing the documents are orthogonal to each other and hence are very different from each other.

In its early days, Google search was more like an index, then it evolved to accept more natural language searches. The same is true for smartphone autocomplete. Latent semantic analysis compresses the meaning of a sentence or a document into a vector, and when this is integrated into a search engine, it dramatically improves the quality of the engine, retrieving the exact documents we are searching for.

Randomized Singular Value Decomposition

In this chapter, we have avoided computing the singular value decomposition on purpose because it is expensive. We did mention, however, that common algorithms use a matrix decomposition called QR decomposition (which obtains an orthonormal basis for the columns of the data matrix), then Householder reflections to transform to a bidiagonal matrix, and finally iterative methods to compute the required eigenvectors and eigenvalues. Sadly, for the ever-growing data sets, the matrices involved are too large even for these efficient algorithms. Our only salvation is through *randomized linear algebra*. This field provides extremely efficient methods for matrix decomposition, relying on *the theory of random sampling*. Randomized numerical methods work wonders, providing accurate matrix decompositions while at the same time being much cheaper than deterministic methods. Randomized singular value decomposition samples the column space of the large data matrix X , computes the QR decomposition of the sampled (much smaller) matrix, projects X onto the smaller space ($Y = Q^t X$, so $X \approx QY$), then computes the singular value decomposition of Y ($Y = U\Sigma V^t$). The matrix Q is orthonormal and approximates the column space of X , so the matrices Σ and V are the same for X and Y . To find the U for X , we can compute it from the U for Y and Q $U_X = QU_Y$.

Like all randomized methods, they have to be accompanied by error bounds, in terms of the expectation of how far off the original matrix X is from the sampled QY . We do have such error bounds, but we postpone them until [Chapter 7](#), which discusses large random matrices.

Summary and Looking Ahead

The star of the show in this chapter was one formula:

$$X = U\Sigma V^t = \sigma_1 U_{col_1} V_{row_1}^t + \sigma_2 U_{col_2} V_{row_2}^t + \cdots + \sigma_r U_{col_r} V_{row_r}^t$$

This is equivalent to $XV = U\Sigma$ and $XV_{col_i} = \sigma_i U_{col_i}$.

The power of the singular value decomposition is that it allows for rank reduction without losing essential information. This enables us to compress images, reduce the dimension of the feature space of a data set, and compute document similarity in natural language processing.

We discussed principal component analysis, latent semantic analysis, and the clustering structure inherent in the principal component space. We discussed examples where principal component analysis was used as an unsupervised clustering technique for cancer patients according to their gene expression, as well as characterizing social media users.

We ended the chapter with randomized singular value decomposition, highlighting a recurring theme for this book: when things are too large, sample them. *Randomness is pretty much dependable!*

If you are interested in diving deeper, you can read about tensor decompositions and N-way data arrays, and the importance of data alignment for the singular value decomposition to work properly. I learned this stuff from the book *Deep Learning*. If you are interested in other popular examples, you can read about eigenfaces from a modern perspective.

Natural Language and Finance AI: Vectorization and Time Series

They. Can. Read.

—H.

One of the hallmarks of human intelligence is our mastery of language at a very early age: comprehension of written and spoken language, written and spoken expression of thoughts, conversation between two or more people, translation from one language to another, and the use of language to express empathy, convey emotions, and process visual and audio data perceived from our surroundings. Leaving the philosophical question of consciousness aside, if machines acquire the ability to perform these language tasks, deciphering the intent of words, at a level similar to humans, or above humans, then it is a major propeller toward general artificial intelligence. These tasks fall under the umbrellas of *natural language processing*, *computational linguistics*, *machine learning*, and/or *probabilistic language modeling*. These fields are vast and it is easy to find interested people wandering aimlessly in a haze of various models with big promises. We should not get lost. The aim of this chapter is to lay out the natural processing field all at once so we can have a bird's-eye view without getting into the weeds.

The following questions guide us at all times:

- What type of task is at hand? In other words, what is our goal?
- What type of data is at hand? What type of data do we need to collect?
- What state-of-the-art models are out there that deal with similar tasks and similar types of data? If there are none, then we have to come up with the models ourselves.

- How do we train these models? In what formats do they consume their data? In what formats do they produce their outputs? Do they have a training function, loss function (or objective function), and optimization structure?
- What are the advantages and disadvantages of various models versus others?
- Are there Python packages or libraries available for model implementation? Luckily, nowadays, most models come out accompanied with Python implementations and very simple APIs. Even better, there are many pre-trained models available to download and ready for use in applications.
- How much computational infrastructure do we need to train and/or deploy these models?
- Can we do better? There is always room for improvement.

We also need to *extract the math from the best-performing models*. Thankfully, this is the easy part, since similar mathematics underlies many models, even when relating to different types of tasks or from dissimilar application areas, such as predicting the next word in a sentence or predicting stock market behavior.

The state-of-the-art models that we intend to cover in this chapter are:

- Transformers or attention models (since 2017). The important math here is extremely simple: the dot product between two vectors.
- Recurrent long short-term memory neural networks (since 1995). The important math here is *backpropagation in time*. We covered backpropagation in [Chapter 4](#), but for recurrent nets, we take the derivatives with respect to time.
- Convolutional neural networks (since 1989) for time series data. The important math is the *convolution* operation, which we covered in [Chapter 5](#).

These models are very well suited for *time series data*, that is, data that appears sequentially with time. Examples of time series data are movies, audio files such as music and voice recordings, financial markets data, climate data, dynamic systems data, documents, and books.

We might wonder why documents and books can be considered as time dependent, even though they have already been written and are just *there*. How come an image is not time dependent but a book and, in general, reading and writing are? The answer is simple:

- When we read a book, we comprehend what we read one word at a time, then one phrase at a time, then one sentence at a time, then one paragraph at a time, and so on. This is how we grasp the concepts and topics of the book.

- The same is true when we write a document, outputting one word at time, even though the whole idea we are trying to express is already there, *encoded*, before we output the words, *sequentially*, on paper.
- When we caption an image, the image itself is not time dependent, but our captioning (the output) is.
- When we summarize an article, answer a question, or translate from one language to another, the output text is time dependent. The input text could be time dependent if processed using a recurrent neural network, or stationary if processed all at once using a transformer or a convolutional model.

Until 2017, the most popular machine learning models to process time series data were based either on *convolutional neural networks* or on *recurrent neural networks with long short-term memory*. In 2017, *transformers* took over, abandoning recurrence altogether in certain application areas. The question of whether recurrent neural networks are obsolete is out there, but with things changing every day in the AI field, who knows which models will die and which will survive the test of time. Moreover, recurrent neural networks power many AI engines and are still subjects of active research.

In this chapter, we answer the following questions:

- How do we transform natural language text to numerical quantities that retain meaning? Our machines only understand numbers, and we need to process natural language using these machines. We must *vectorize* our samples of text data, or *embed* them into finite dimensional vector spaces.
- How do we lower the dimension of the vectors from the enormous ones initially required to represent natural language? For example, the French language has around 135,000 distinct words, so how do we get around having to one-hot code words in a French sentence using vectors of 135,000 entries each?
- Does the model at hand consider (as its input and/or output) our natural language data as a time dependent sequence fed into it one term at a time, or a stationary vector consumed all at once?
- How exactly do various models for natural language processing work?
- Why is there finance in this chapter as well?

Along the way, we discuss the types of natural language and finance applications that our models are well suited for. We keep the focus on the mathematics and not the programming, since such models (especially for language applications) require substantive computational infrastructures. For example, the **DeepL Translator** generates its translations using a supercomputer operated with hydropower from Iceland, which reaches 5.1 petaflops. We also note that the AI-specialized chip industry is booming, led by NVIDIA, Google's Tensor Processing Unit, AWS Inferentia, AMD's

Instinct GPU, and startups like Cerebras and Graphcore. While conventional chips have struggled to keep pace with Moore's law, which predicted a doubling of processing power every 18 months, AI-specialized chips have outpaced this law by a wide margin.

Even though we do not write code for this chapter, we note that most programming can be accomplished using Python's TensorFlow and Keras libraries.

Throughout our discussion, we have to be mindful of whether we are in the *training* phase of a model or in the *prediction* phase (using the pre-trained model to do tasks). Moreover, it is important to differentiate whether our model needs labeled data to be trained, such as English sentences along with their French translations as labels, or can learn from unlabeled data, such as computing the meanings of words from their contexts.

Natural Language AI

Natural language processing applications are ubiquitous. This technology has been integrated into so many aspects of our lives that we just take it for granted: when using apps on our smartphones, digital calendars, digital home assistants, Siri, Alexa, and others. The following list is partially adapted from the excellent book *Natural Language Processing in Action* by Hobson Lane, Hannes Hapke, and Cole Howard (Manning 2019), demonstrating how indispensable natural language processing has become:

- Search and information retrieval: web, documents, autocomplete, chatbots
- Email: spam filter, email classification, email prioritization
- Editing: spelling check, grammar check, style recommendation
- Sentiment analysis: product reviews, customer care, monitoring of community morale
- Dialog: chatbots, digital assistants such as Amazon's Alexa, scheduling
- Writing: indexing, concordance, table of contents
- Text mining: summarization, knowledge extraction such as mining election campaigns' finance and natural language data (finding connections between political donors), résumé-to-job matching, medical diagnosis
- Law: legal inference, precedent search, subpoena classification
- News: event detection, fact checking, headline composition
- Attribution: plagiarism detection, literary forensics, style coaching
- Behavior prediction: finance applications, election forecasting, marketing

- Creative writing: movie scripts, poetry, song lyrics, bot-powered financial and sports news stories
- Captioning: computer vision combined with natural language processing
- Translation: Google Translate and DeepL Translate

Even though the past decade has brought impressive feats, machines are still nowhere close to mastering natural language. The processes involved are tedious, requiring attentive statistical bookkeeping and substantive memory, the same way humans require memory to master languages. The point here is: there is plenty of room for new innovations and contributions to the field.

Language models have recently shifted from handcoded to data driven. They do not implement hardcoded logical and grammar rules. Instead, they rely on detecting the statistical relationships between words. Even though there is a school of thought in linguistics that asserts grammar is an innate property for humans, or in other words, is *hardcoded* into our brains, humans have a striking ability to master new languages without ever encountering any grammatical rules for these languages. From personal experience, attempting to learn the grammar of a new language seems to impede the learning process, but do not quote me on that.

One major challenge is that data for natural language is extremely high-dimensional. There are millions of words across thousands of languages. There are huge corpuses of documents, such as entire collections of authors' works, billions of tweets, Wikipedia articles, news articles, Facebook comments, movie reviews, etc. A first goal is then to reduce the number of dimensions for efficient storage, processing, and computation, while at the same time avoiding the loss of essential information. This has been a common theme in the AI field, and one cannot help but wonder how many mathematical innovations would have never seen the light of the day had we possessed unlimited storage and computational infrastructures.

Preparing Natural Language Data for Machine Processing

For a machine to process any natural language task, the first thing it must do is to break down text and organize it into building blocks that retain meaning, intent, context, topics, information, and sentiments. To this end, it must establish a correspondence between words and number tags, using processes called *tokenizing*, *stemming* (such as giving singular words and their plural variation the same token), *lemmatization* (associating several words of similar meaning together), *case normalization* (such as giving capitalized and lowercase words of the same spelling the same tokens), and others. This correspondence is not for individual characters that make up words, but for full words, pairs or more of words (2-grams or n-grams), punctuations, significant capitalizations, etc., that carry meaning. This creates a *vocabulary* or a *lexicon* of numerical tokens corresponding to a given corpus of natural language

documents. A vocabulary or a lexicon in this sense is similar to a Python dictionary: each individual natural language building block object has a unique token.

An *n*-gram is a sequence of *n* words that carry a meaning when kept ordered together that is different from the meaning of each word on its own. For example, a 2-gram is a couple of words together whose meaning would change if we unpair them, such as ice cream or was not, so the whole 2-gram gets one numerical token, retaining the meaning of the two words within their correct context. Similarly, a 3-gram is a triplet of ordered words, such as John F. Kennedy, and so on. A *parser* for natural language is the same as a compiler for computers. Do not worry if these new terms confuse you. For our mathematical purposes, all we need are the numerical tokens associated with unique words, n-grams, emojis, punctuation, etc., and the resulting vocabulary for a corpus of natural language documents. These are saved in a dictionary like objects, allowing us to flip back and forth easily between text and numerical tokens.

We leave the actual details of tokenizing, stemming, lemmatization, parsing, and other natural language data preparations for computer scientists and their collaborations with linguists. In fact, collaboration with linguists has become less important as the models mature in their ability to detect patterns directly from the data, thus, the need for coding handcrafted linguistic rules into natural language models has diminished. Note also that not all natural language pipelines include stemming and lemmatization. They all, however, involve tokenizing. The quality of tokenizing text data is crucial for the performance of our natural language pipeline. It is the first step containing fundamental building blocks representing the data that we feed into our models. The quality of both the data and the way it is tokenized affects the outputs of the entire natural language processing pipeline. For your production applications, use the *spaCy* parser, which does sentence segmentation, tokenization, and multiple other things in one pass.

After tokenizing and building a healthy vocabulary (the collection of numerical tokens and the entities they correspond to in the natural language text), we need to represent entire natural language documents using vectors of numbers. These documents can range from very long, such as a book series, to very short, such as a Twitter tweet or a simple search query for Google Search or DuckDuckGo. We can then express a corpus of one million documents as a collection of one million numerical vectors, or a matrix with one million columns. These columns will be as long as our chosen vocabulary, or shorter if we decide to *compress* these documents further. In linear algebra language, the length of these vectors is the dimension of our *vector space* that our documents are *embedded in*.

The whole point of this process is to obtain numerical vector representations of our documents so that we can do math on them: now comes linear algebra with its arsenal of linear combinations, projections, dot products, and singular value decompositions. There is, however, one caveat: for natural language applications, the

lengths of the vectors representing our documents, or the size of our vocabulary, are prohibitively enormous to do any useful computations with. The curse of dimensionality becomes a real thing.



The Curse of Dimensionality

Vectors become exponentially farther apart in terms of Euclidean distance as the number of dimensions increases. One natural language example is sorting documents based on their *distance* from another document, such as a search query. This simple operation becomes impractical when we go above 20 dimensions or so if we use the Euclidean distance to measure the *closeness* of documents (see Wikipedia’s “[curse of dimensionality](#)” for more details). Thus, for natural language applications, we must use another measure for distance between documents. We will discuss *cosine similarity* shortly, which measures the *angle* between two document vectors, as opposed to their Euclidean distance.

Therefore, a main driver for natural language processing models is to represent these documents using shorter vectors that convey the main topics and retain meaning. Think how many unique tokens or combinations of tokens we have to use to represent this book while at the same time preserving its most important information.

To summarize, our natural language processing pipeline proceeds as follows:

1. From text to numerical tokens, then to an acceptable vocabulary for an entire corpus of documents.
2. From documents of tokens to high-dimensional vectors of numbers.
3. From high-dimensional vectors of numbers to lower-dimensional vectors of topics using techniques like *direct projection onto a smaller subset of the vocabulary space* (just dropping part of the vocabulary, making the corresponding entries zero), *latent semantic analysis* (projecting onto special vectors determined by special linear combinations of the document vectors), *word2vec*, *Doc2Vec*, *thought vectors*, *Dirichlet allocation*, and others. We discuss these shortly.

As is usually the case in mathematical modeling, there is more than one way to represent a given document as a vector of numbers. We decide on the vector space that our documents inhabit, or get *embedded in*. Each vector representation has advantages and disadvantages, depending on the goal of our natural language task. Some are simpler than others too.

Statistical Models and the log Function

When representing a document as a vector of numbers starts with counting the number of times certain terms appear in the document, then our document vectorizing model is statistical, since it is frequency based.

When we deal with term frequencies, it is better to apply the *log* function to our counts as opposed to using raw counts. The log function is advantageous when we deal with quantities that could get extremely large, extremely small, or could have extreme variations in scale. Viewing these extreme counts or variations within a logarithmic scale brings them back to the normal realm.

For example, the number 10^{23} is huge, but $\log(10^{23}) = 23 \log(10)$ is not. Similarly, if the term “shark” appears in 2 documents of a corpus of 20 million documents ($20 \text{ million}/2 = 10 \text{ million}$), and the term “whale” appears in 20 documents of this corpus ($20 \text{ million}/20 = 1 \text{ million}$), then that is a 9 million difference, which seems excessive for terms that appeared in 2 and 20 documents, respectively. Computing the same quantities but on a *log* scale, we get $7\log(10)$ and $6\log(10)$, respectively (it doesn’t matter which log base we use), which doesn’t seem excessive anymore, and more in line with the terms’ occurrence in the corpus.

The need for using the log function when dealing with word counts in particular is reinforced by *Zipf’s law*. This law says that term counts in a corpus of natural language naturally follow a power law, so it is best to temper that with a log function, transforming differences in term frequencies into a linear scale. We discuss this next.

Zipf’s Law for Term Counts

Zipf’s law for natural language has to do with word counts. It is very interesting and so surprising that I am tempted to try and see if it applies to my own book. It is hard to imagine that as I write each word in this book, my unique word counts are actually following some law. Are we, along with the way we word our ideas and thoughts, *that predictable*? It turns out that Zipf’s law extends to counting many things around us, not only words in documents and corpuses.

Zipf’s law reads as follows: *for a corpus of natural language where the terms have been ordered according to their frequencies, the frequency of the first item is twice that of the second item, three times the third item, and so on.* That is, the frequency with which an item appears in a corpus is related to its ranking: $f_1 = 2f_2 = 3f_3 = \dots$

We can verify if Zipf’s law applies by plotting the frequency of the terms against their respective ranks and verifying the power law: $f_r = f(r) = f_1 r^{-1}$. To verify power laws, it is easier to make a *log-log* plot, plotting $\log(f_r)$ against $\log(r)$. If we obtain

a straight line in the log-log plot, then $f_r = f(r) = f_1 r^\alpha$, where α is the slope of the straight line.

Various Vector Representations for Natural Language Documents

Let's list the most common document vector representations for state-of-the-art natural language processing models. The first two, *term frequency* and *term frequency-inverse document frequency* (TF-IDF), are statistical representations since they are frequency based, relying on counting word appearances in documents. They are slightly more involved than a simple binary representation detecting the presence or nonpresence of certain words, nevertheless, they are still shallow, merely counting words. Even with this shallowness, they are very useful for applications such as spam filtering and sentiment analysis.

Term Frequency Vector Representation of a Document or Bag of Words

Here, we represent a document using a *bag of words*, discarding the order in which words appear in the document. Even though word order encodes important information about a document's content, ignoring it is usually an OK approximation for short sentences and phrases.

Suppose we want to embed our given document in a *vocabulary space* of 10,000 tokens. Then the vector representing this document will have 10,000 entries, with each entry counting how many times each particular token appears in the document. For obvious reasons, this is called the *term frequency* or *bag-of-words* vector representation of the document, where each entry is a nonnegative integer (a whole number).

For example, the Google Search query “What’s the weather tomorrow?” will be vectorized as zeros everywhere except for ones at the tokens representing the words “what,” “the,” “weather,” and “tomorrow,” if they exist in the vocabulary. We then *normalize* this vector, dividing each entry by the total number of terms in the document so that the length of the document doesn't skew our analysis. That is, if a document has 50,000 terms and the term “cat” gets mentioned a hundred times, and another document has a hundred terms only and the term “cat” gets mentioned 10 times, then obviously the word “cat” is more important for the second document than for the first, and a mere word count without normalizing would not be able to capture that.

Finally, some natural language processing classes take the log of each term in the document vector for the reasons mentioned in the previous two sections.

Term Frequency-Inverse Document Frequency Vector Representation of a Document

Here, for each entry of the vector representing the document, we still count the number of times the token appears in the document, *but then we divide by the number of documents in our corpus in which the token occurs*.

The idea is that if a term appears many times in one document and not as much in the others, then this term must be important for this one document, getting a higher score in the corresponding entry of the vector representing this document.

To avoid division by zero, if a term does not appear in any document, it is common practice to add one to the denominator. For example, the inverse document frequency of the token *cat* is:

$$\text{IDF for cat} = \frac{\text{number of documents in corpus}}{\text{number of documents containing cat} + 1}$$

Obviously, using TF-IDF representation, the entries of the document vectors will be nonnegative rational numbers, each providing a measure of the *importance* of that particular token to the document. Finally, we take the log of each entry in this vector, for the same reasons stated in the previous section.

There are many alternative TF-IDF approaches relevant to information retrieval systems, such as [Okapi BM25](#), and [Molino 2017](#).

Topic Vector Representation of a Document Determined by Latent Semantic Analysis

TF-IDF vectors are very high-dimensional (as many dimensions as tokens in the corpus, so it could be in the millions), sparse, and have no special meaning when added or subtracted from each other. We need more compact vectors, in the hundreds of dimensions or less, which is a big squeeze from millions of dimensions. In addition to the dimension reduction advantage, these vectors capture some meaning, not only word counts and statistics. We call them *topic vectors*. Instead of focusing on the *statistics of words in documents*, we focus on the *statistics of connections between words in documents and across corpuses*. The topics produced here will be linear combinations of word counts.

First, we process the whole TF-IDF matrix X of our corpus, producing our *topic space*. Processing in this case means that we compute the *singular value decomposition* of the TF-IDF matrix from linear algebra, namely, $X = U\Sigma V^t$. [Chapter 6](#) is dedicated to singular value decomposition, so for now we will only explain how it is used for producing our topic space for a corpus. Singular value decomposition from linear

algebra is called *latent semantic analysis* in natural language processing. We will use both terms synonymously.

We have to pay attention to whether the columns of the corpus's TF-IDF matrix X represent the word tokens or the documents. Different authors and software packages use one or the other, so we must be careful and process either the matrix or its transpose to produce our topic space. In this section we follow the representation that the rows are all the words (tokens for words, n-grams, etc.) of the entire corpus, and the columns are the TF-IDF vector representations for each document in the corpus. This is slightly divergent from the usual representation of a data matrix, where the features (the words within each document) are in the columns, and the instances (the documents) are in the rows. The reason for this switch will be apparent shortly. However, this is not divergent from our representation for documents as column vectors.

Next, given a new document with its TF-IDF vector representation, we convert it to a much more compact *topic vector* by *projecting it onto* the topic space produced by the singular value decomposition of the corpus's TF-IDF matrix. *Projecting* in linear algebra is merely computing the *dot product* between the appropriate vectors and saving the resulting scalar numbers into the entries of a new *projected vector*:

- We have a TF-IDF vector of a document that has as many entries as the number of tokens in the entire corpus.
- We have *topic weight vectors*, which are the columns of the matrix U produced by the singular value decomposition of the TF-IDF matrix $X = U\Sigma V^t$. Again, each topic weight vector has as many entries as tokens in our corpus. Initially, we also have as many topic weight vectors as tokens in our entire corpus (columns of U). The *weights* in the column of U tell us how much a certain token contributes to the topic, with a big contribution if it is a positive number close to 1, an ambivalent contribution if it is close to 0, and even a negative contribution if it is a negative number close to -1. Note that the entries of U are always numbers between -1 and 1, so we interpret them as weighing factors for our corpus's tokens.

Topic selection and dimension reduction

You might be wondering, if we have as many topic weight vectors as tokens in our corpus, each having as many entries as tokens as well, then where are the savings, and when will compression or dimension reduction happen? Keep reading.

Goal 1

Compute how much of a certain topic our document contains. This is simply the dot product between the document's TF-IDF vector and the column of U corresponding to the topic that we care for. Record this as the first scalar number.

Goal 2

Compute how much of *another* topic our document contains. This is the dot product between the document's TF-IDF vector and the column of U corresponding to this other topic that we care for. Record this as the second scalar number.

Goal 3

Repeat this for as many (as there are columns of U , which is the same as the total number of tokens in the corpus) or as few (as one) topics as we like, recording the scalar number from each dot product that we compute. It is clear now that “a topic” in this context means a column vector containing weights between -1 and 1 assigned to each token in the corpus.

Goal 4

Reduce the dimension by keeping only the topics that matter. That is, if we decide to keep only two topics, then the *compressed vector representation* of our document will be the *two-dimensional* vector containing the two scalar numbers produced using the two dot products between the document's TF-IDF vector and the two topics' weight vectors. This way, we would have reduced the dimension of our document from possibly *millions* to just two. Pretty cool stuff.

Goal 5

Choose the right topics to represent our documents. This is where the singular value decomposition works its magic. The columns of U are organized in order, from the most important topic across the corpus to the least important. In the language of statistics, the columns are organized from the topic with the *most variance* across the corpus and hence encodes more information, to the one with the *least variance* and hence encodes little information. We explain how variance and singular value decomposition are related in [Chapter 10](#). Thus, if we decide to project our high-dimensional document onto the first few column vectors of U only, we are guaranteed that we are not missing much in terms of capturing enough variation of possible topics across the corpus, and assessing how much of these our document contains.

Goal 6

Understand that *this is still a statistical method for capturing topics in a document*. We started with the TF-IDF matrix of a corpus, simply counting token occurrences in documents. In this sense, a topic is captured based only on the premise that documents that refer to similar things use similar words. This is different

from capturing topics based on the *meanings* of the words they use. That is, if we have two documents discussing the same topic but using entirely different vocabulary, they will be far apart in topic space. The remedy to this would be to store words together with other words of similar meaning, which is the word2vec approach, discussed later in this chapter.

Question 1

What happens if we add another document to our corpus? Luckily, we do not have to reprocess the whole corpus to produce the document's topic vector, we just project it onto the corpus's existing topic space. This of course breaks down if we add a new document that has nothing in common with our corpus, such as an article on pure mathematics added to a corpus on Shakespeare's love sonnets. Our math article in this case will be represented by a bunch of zeros or close-to-zero entries, which does not capture the ideas in the article adequately.

Now we have the following questions:

Question 2

What about the matrix V^t in the singular value decomposition $X = U\Sigma V^t$, what does it mean in the context of natural language processing of our corpus? The matrix V^t has the same number of rows and columns as the number of documents in our corpus. It is the document-document matrix and gives the shared meaning between documents.

Question 3

When we move to a lower-dimensional topic space using latent semantic analysis, are large distances between documents preserved? Yes, since the singular value decomposition focuses on *maximizing* the variance across the corpus's documents.

Question 4

Are small distances preserved, meaning does latent semantic analysis preserve the *fine structure* of a document that separates it from *not so different* other documents? No. Latent Dirichlet allocation, discussed soon, does a better job here.

Question 5

Can we improve latent semantic analysis to also keep close document vectors together in the lower-dimensional topic space? Yes, we can *steer* the vectors by taking advantage of extra information, or *metadata*, of the documents, such as messages having the same sender, or by penalizing using a cost function so that the method spills out topic vectors that preserve *closeness* as well.

To summarize, latent semantic analysis chooses the topics in an optimal way that maximizes the diversity in the topics across the corpus. The matrix U from the

singular value decomposition of the TF-IDF matrix is very important for us. It returns the directions along which the variance is maximal. We usually get rid of the topics that have the least amount of variance between the documents in the corpus, throwing away the last columns of U . This is similar to manually getting rid of stop words (and, a, the, etc.) during text preparation, but latent semantic analysis does that for us in an optimized way. The matrix U has the same number of rows and columns as our vocabulary. It is the cross-correlation between words and topics based on word co-occurrence in the same document. When we multiply a new document by U (project it onto the columns of U), we would get the amount of each topic in the document. We can truncate U as we wish and throw away less important topics, reducing the dimension to as few topics as we want.

Shortcomings of latent semantic analysis

The topic spaces it produces, or the columns of U , are mere *linear combinations* of tokens that are thrown together in a way that captures as much variance in the usage across the vocabulary's tokens as possible. This doesn't necessarily translate into word combinations that are in any way meaningful to humans. Bummer. Word2vec, discussed later, addresses these shortcomings.

Finally, the topic vectors produced via latent semantic analysis are just linear transformations performed on the TF-IDF vectors. They should be the first choice for semantic searches, clustering documents, and content-based recommendation engines. All of this can be accomplished by measuring distances between these topic vectors, which we explain later in this chapter.

Topic Vector Representation of a Document Determined by Latent Dirichlet Allocation

Unlike topic vectors using latent semantic analysis, with *latent Dirichlet allocation* (LDA) we do have to reprocess the entire corpus if we add a new document to the corpus to produce its topic vector. Moreover, we use a nonlinear statistical approach to bundle words into topics: we assume a Dirichlet distribution of word frequencies. This makes the method more precise than latent semantic analysis in terms of the statistics of allocating words to topics. Thus, the method is explainable: the way words are allocated to topics, based on how often they occurred together in a document, and the way topics are allocated to documents, tend to make sense to us as humans.

This nonlinear method takes longer to train than the linear latent semantic analysis. For this reason it is impractical for applications involving corpuses of documents, even though it is explainable. We can use it instead for summarizing single documents, where each sentence in the document becomes its own *document*, and the mother document becomes the corpus.

LDA was invented in 2000 by geneticists for the purpose of inferring population structure, and adopted in 2003 for natural language processing. The following are its assumptions:

- We start with raw word counts (rather than normalized TF-IDF vectors), but there is still no sequencing of words to make sense of them. Instead, we still rely on modeling the statistics of words for each document, except this time we incorporate the word distribution explicitly into the model.
- A document is a linear combination of an arbitrary number of topics (specify this number ahead of time so that the method allocates the document's tokens to this number of topics).
- We can represent each topic by a certain distribution of words based on their term frequencies.
- The probability of occurrence of a certain topic in a document follows a Dirichlet probability distribution.
- The probability of a certain word being assigned to a topic also follows a Dirichlet probability distribution.

As a result, topic vectors obtained using Dirichlet allocation are sparse, indicating clean separation between the topics in the sense of which words they contain, which makes them explainable.

With Dirichlet allocation, words that occur frequently together are assigned to the same topics. So this method keeps tokens that were close together, close together, when we move to the lower-dimensional topic space. Latent semantic analysis, on the other hand, keeps tokens that were spread apart, spread apart, when we move to the lower-dimensional topic space, so this is better for classification problems where the separation between the classes is maintained even as we move to the lower-dimensional space.

Topic Vector Representation of a Document Determined by Latent Discriminant Analysis

Unlike latent semantic analysis and latent Dirichlet allocation, which break down a document into as many topics as we choose, latent discriminant analysis breaks down a document into *only one topic*, such as spamness, sentiment, etc. This is good for binary classification, such as classifying messages as spam or nonspam, or classifying reviews as positive or negative. Rather than what latent semantic analysis does, maximizing the separation between all the vectors in the new topic space, latent discriminant analysis maximizes the separation only between the centroids of the vectors belonging to each class.

But how do we determine the vector representing this one topic? Given the TF-IDF vectors of labeled spam and nonspam documents, we compute the centroid of each class, then our vector is along the line connecting the two centroids (see [Figure 7-1](#)).

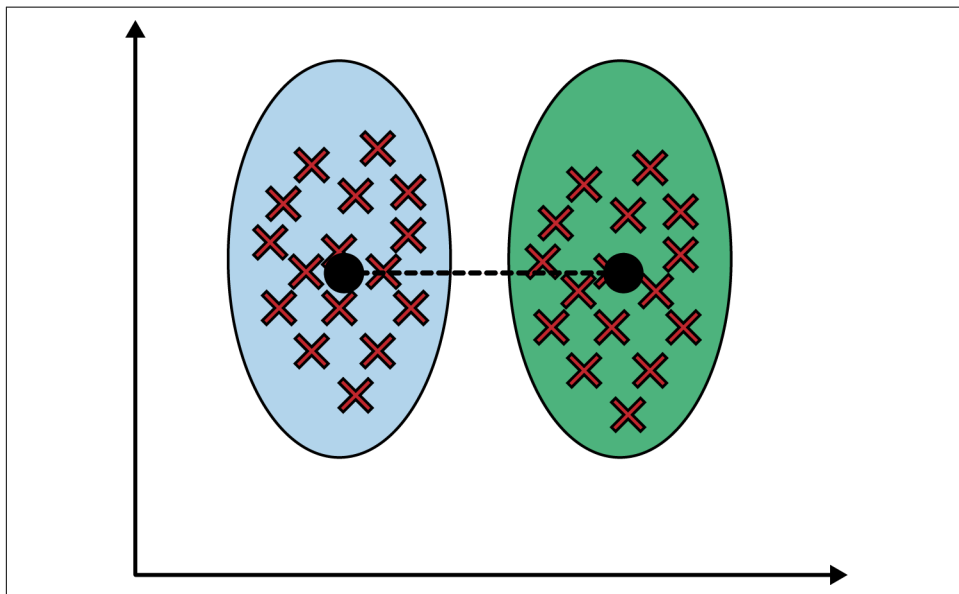


Figure 7-1. Latent discriminant analysis

Each new document can now be projected onto this one dimension. The coordinate of our document along that line is the dot product between its TF-IDF and the direction vector of the centroids line. The whole document (with millions of dimensions) is now squashed into one number along one dimension (one axis) that carries the two centroids along with their midpoint. We can then classify the document as belonging to one class or the other depending on its distance from each centroid along that one line. Note that the decision boundary for separating classes using this method is linear.

Meaning Vector Representations of Words and of Documents Determined by Neural Network Embeddings

The previous models for vectorizing documents of natural language text only considered linear relationships between words, or in latent Dirichlet allocation, we had to use human judgment to select the model's parameters and extract features. We now know that the power of neural networks lies in their ability to capture nonlinear relationships, extract features, and find appropriate model parameters automatically. We will now use neural networks to create vectors that represent individual words and terms, and we will employ similar methods to create vectors representing the

meanings of entire paragraphs. Since these vectors encode the meaning and the logical and contextual usage of each term, we can reason with them simply by doing the usual vector additions and subtractions.

Word2vec vector representation of individual terms by incorporating continuous-ness attributes

By using TF vectors or TF-IDF vectors as a starting point for our topic vector models, we have ignored the nearby context of words and the effect that has on their meanings. Word vectors solve this problem. A word vector is a numerical vector representation of a word's meaning, so every single term in the corpus becomes a vector of semantics. This vector representation with floating-point number entries of single words enables semantic queries and logical reasoning.

Word vector representations are learned using a neural network. They usually have 100 to 500 dimensions encoding how much of each meaning dimension a word carries within it. When training a word vector model, the text data is unlabeled. Once trained, two terms can be determined to be close in meaning or far apart by comparing their vectors via some closeness metrics. Cosine similarity, discussed next, is the go-to method.

In 2013, Google created a word-to-vector model, word2vec, that it trained on the Google News feed containing 100 billion words. The resulting pre-trained word2vec model contains 300 dimensional vectors for 3 million words and phrases. It is freely available to download at the [Google Code Archive page for the word2vec project](#).

The vector that word2vec builds up captures much more of a word's meaning than the topic vectors discussed earlier in this chapter. The abstract of the paper “[Efficient Estimation of Word Representations in Vector Space](#)” (Mikolov et al. 2013) is informative:

We propose two novel model architectures for computing continuous vector representations of words from very large data sets. The quality of these representations is measured in a word similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks. We observe large improvements in accuracy at much lower computational cost, i.e. it takes less than a day to learn high quality word vectors from a 1.6 billion words data set. Furthermore, we show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.

A month later, the paper “[Distributed Representations of Words and Phrases and their Compositionality](#)” (Mikolov et al. 2013) addressed the representation of word phrases that mean something different than their individual components, such as “Air Canada”:

The recently introduced continuous Skip-gram model is an efficient method for learning high quality distributed vector representations that capture a large number of precise syntactic and semantic word relationships. In this paper we present several extensions that improve both the quality of the vectors and the training speed. By subsampling of the frequent words we obtain significant speedup and also learn more regular word representations. We also describe a simple alternative to the hierarchical softmax called negative sampling. An inherent limitation of word representations is their indifference to word order and their inability to represent idiomatic phrases. For example, the meanings of “Canada” and “Air” cannot be easily combined to obtain “Air Canada”. Motivated by this example, we present a simple method for finding phrases in text, and show that learning good vector representations for millions of phrases is possible.

The publication that introduced word2vec representations, “[Linguistic Regularities in Continuous Space Word Representations](#)” (Mikolov et al. 2013), demonstrates how these meaning vectors for words encode logical regularities and how this enables us to answer regular analogy questions:

Continuous space language models have recently demonstrated outstanding results across a variety of tasks. In this paper, we examine the vector space word representations that are implicitly learned by the input layer weights. We find that these representations are surprisingly good at capturing syntactic and semantic regularities in language, and that each relationship is characterized by a relation specific vector offset. This allows vector oriented reasoning based on the offsets between words. For example, the male/female relationship is automatically learned, and with the induced vector representations, “King - Man + Woman” results in a vector very close to “Queen.” We demonstrate that the word vectors capture syntactic regularities by means of syntactic analogy questions (provided with this paper), and are able to correctly answer almost 40% of the questions. We demonstrate that the word vectors capture semantic regularities by using the vector offset method to answer SemEval-2012 Task 2 questions. Remarkably, this method outperforms the best previous systems.

The performance of word2vec has improved dramatically since 2013 by training it on much larger corpuses.

Word2vec takes one word and assigns to it a vector of attributes, such as place-ness, animal-ness, city-ness, positivity (sentiment), brightness, gender, etc. Each attribute is a dimension, capturing how much of the attribute the meaning of the word contains.

These word meaning vectors and the attributes are not encoded manually, but during training, where the model learns the meaning of a word from the company it keeps: the five or so nearby words in the same sentence. This is different from latent semantic analysis, where the topics are learned only from words occurring in the same document, not necessarily close to each other. For applications involving short documents and statements, word2vec embeddings have actually replaced topic vectors obtained through latent semantic analysis. We can also use word vectors to derive word clusters from huge data sets by performing k-means clustering on top of

the word vector representations. See the [Google Code Archive page for the word2vec project](#) for more information.

The advantage of representing words through vectors that mean something (rather than count something) is that we can reason with them. For example, as mentioned earlier, if we subtract the vector representing “man” from the vector representing “king” and add the vector representing “woman,” then we get a vector very close to the vector representing the word “queen.” Another example is capturing the relationship between singular and plural words. If we subtract vectors representing the singular form of words from vectors representing their plural forms, we obtain vectors that are roughly the same for all words.

The next questions are: how do we compute word2vec embeddings? That is, how do we train a word2vec model? What are training data, the neural network’s architecture, and its input and output? The neural networks that train word2vec models are shallow with only one hidden layer. The input is a large corpus of text, and the outputs are vectors of several hundred dimensions, one for each unique term in the corpus. Words that share common linguistic contexts end up with vectors that are *close* to each other.

There are two learning algorithms for word2vec, which we will not detail here. However, by now we have a very good idea of how neural networks work, especially shallow ones with only one hidden layer. The two learning algorithms are:

Continuous bag-of-words

This predicts the current word from a window of surrounding context words; the order of the context words does not influence the prediction.

Continuous skip-gram

This uses the current word to predict the surrounding window of context words; the algorithm weighs nearby context words more heavily than more distant context words.

Both algorithms learn the vector representation of a term that is useful for prediction of other terms in a sentence. Continuous bag-of-words is apparently faster than continuous skip-gram, while skip-gram is better for infrequent words.

For more details, refer to the tutorial “[The Amazing Power of Word Vectors](#)” (Colyer 2016), the [Wikipedia page on word2vec](#), and the three original papers on the subject: “Efficient Estimation of Word Representations in Vector Space” (Mikolov et al. 2013), “Distributed Representations of Words and Phrases and their Compositionality” (Mikolov et al. 2013), and “Linguistic Regularities in Continuous Space Word Representations” (Mikolov et al. 2013).

The trained Google News word2vec model has 3 million words, each represented with a vector of 300 dimensions. To download this, you would need 3 GB of available memory. There are ways around downloading the whole pre-trained model if we have limited memory or if we only care about a fraction of the words.

How to visualize vectors representing words

Word vectors are very high-dimensional (100–500 dimensions), but humans can only visualize two- and three-dimensional vectors, so we need to project our high-dimensional vectors onto these drastically lower-dimensional spaces and still retain their most essential characteristics. By now we know that the singular value decomposition (principal component analysis) accomplishes that for us, giving us the vectors along which to project in decreasing order of importance, or the directions along which a given collection of word vectors varies the most. That is, the singular value decomposition ensures that this projection gives the best possible view of the word vectors, keeping them as far apart as possible.

There are many nice examples on the web. In the publication “[Word Embedding-Topic Distribution Vectors for MOOC \(Massive Open Online Courses\) video lectures dataset](#)” (Kastrati et al. 2020), the authors use a data set from the education domain with the transcripts of 12,032 video lectures from 200 courses collected from [Coursera](#) to generate two things: word vectors using the word2vec model, and document topic vectors using latent Dirichlet allocation. The data set has 878,000 sentences and more than 79,000,000 tokens. The vocabulary size is over 68,000 unique words. The individual video transcripts are of different lengths, varying from 228 to 32,767 tokens, with an average of 6,622 tokens per video transcript. The authors use word2vec and latent Dirichlet allocation implementations in the Gensim package in Python. [Figure 7-2](#) shows the publication’s three-dimensional visualization of a subset of the word vectors using principal component analysis.

Note that word vectors and document topic vectors are not an end unto themselves; instead they are a means to an end, which is usually a natural language processing task, such as: classification within specific domains (such as the massive open online courses in the example), benchmarking and performance analysis of existing and new models, transfer learning, recommendation systems, contextual analysis, short text enrichment with topics, personalized learning, and organizing content that is easy to search and for maximum visibility. We visit such tasks shortly.

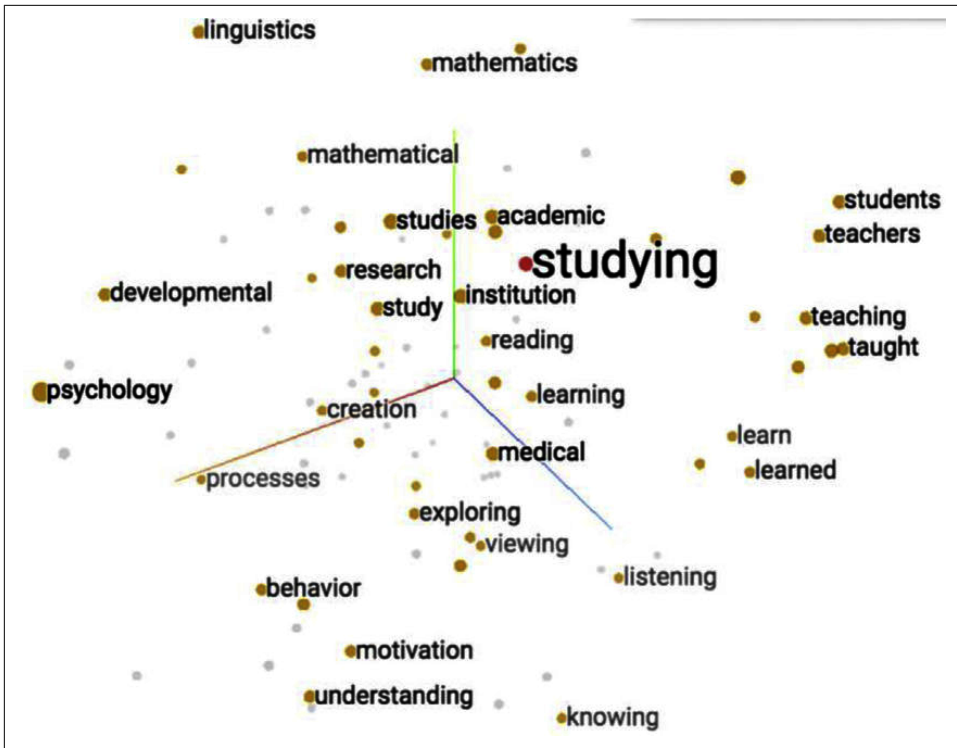


Figure 7-2. Three-dimensional visualization of word vectors using the first three principal components. This example highlights the vector representing the word “studying” and its neighbors: academic, studies, institution, reading, etc. (image source).

Facebook’s fastText vector representation of individual n-character grams

Facebook’s fastText is similar to word2vec, but instead of representing full words or n-grams as vectors, it is trained to output a vector representation for every *n-character* gram. This enables fastText to handle rare, misspelled, and even partial words, such as the ones frequently appearing in social media posts. During training, word2vec’s skip-gram algorithm learns to predict the surrounding context of a given word. Similarly, fastText’s n-character gram algorithm learns to predict a word’s surrounding n-character grams, providing more granularity and flexibility. For example, instead of only representing the full word “lovely” as a vector, it will represent the 2- and 3-grams as vectors as well: lo, lov, ov, ove, ve, vel, el, ely, and ly.

Facebook released its pre-trained fastText models for 294 languages, trained on available Wikipedia collections for these languages. These range from Abkhazian to Zulu, and include rare languages spoken by only a handful of people. Of course, the accuracy of the released models varies across languages and depends on the availability and quality of the training data.

Doc2vec or par2vec vector representation of a document

How about representing documents semantically? In previous sections we were able to represent entire documents as topic vectors, but word2vec only represents individual words or phrases as vectors. Can we then extend the word2vec model to represent entire documents as vectors carrying meaning? The paper “[Distributed Representations of Sentences and Documents](#)” (Le et al. 2014) does exactly that, with an unsupervised algorithm that learns fixed-length dense vectors from variable-length pieces of texts, such as sentences, paragraphs, and documents. The tutorial “[Doc2Vec tutorial using Gensim](#)” (Klintberg 2015) walks through the Python implementation process, producing a fixed-size vector for each full document in a given corpus.

Global vector or vector representation of words

There are other ways to produce vectors representing meanings of words. [Global vector or GloVe \(2014\)](#) is a model that obtains such vectors using the singular value decomposition. It is trained only on the nonzero entries of a global word-word co-occurrence matrix, which tabulates how frequently words co-occur with one another across an entire corpus.

GloVe is essentially a *log-bilinear model* with a weighted least-squares objective. The log-bilinear model is perhaps the simplest neural language model. Given the preceding $n-1$ words, the log-bilinear model computes an initial vector representation for the next word simply by linearly combining the vector representations of these preceding $n-1$ words. Then the probability of the occurrence of the next word given those $n-1$ preceding words is computed based on computing the similarity (dot product) between the linear combination vector representation and the representations of all words in the vocabulary:

$$\begin{aligned} & Prob(w_n = w | w_1, w_2, \dots, w_{n-1}) \\ &= \frac{\exp(w_{vocab_i}^t w)}{\exp(w_{vocab_1}^t w) + \exp(w_{vocab_2}^t w) + \dots + \exp(w_{vocab_{vocab_size}}^t w)} \end{aligned}$$

The main intuition underlying the Global Vector model is the simple observation that ratios of word-word co-occurrence probabilities potentially encode some form of meaning. The example on the GloVe project website considers the co-occurrence probabilities for the target words “ice” and “steam” with various probe words from the vocabulary. [Table 7-1](#) shows the actual probabilities from a six-billion-word corpus.

Table 7-1. Probability table showing the occurrence of the words “ice” and “steam” with the words “solid,” “gas,” “water,” and “fashion”

Probability and ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

Observing the table in Table 7-1, we notice that, as expected, the word *ice* co-occurs more frequently with the word *solid* than it does with the word *gas*, whereas the word *steam* co-occurs more frequently with the word *gas* than it does with the word *solid*. Both *ice* and *steam* co-occur with their shared property *water* frequently, and both co-occur with the unrelated word *fashion* infrequently. Calculating the ratio of probabilities cancels out the noise from nondiscriminative words like *water*, so that large values, much greater than 1, correlate well with properties specific to *ice*, and small values, much less than 1, correlate well with properties specific to *steam*. This way, the ratio of probabilities encodes some crude form of meaning associated with the abstract concept of the thermodynamic phase.

The training objective of GloVe is to learn word vectors such that their dot product equals the logarithm of the probability of co-occurrence of words. Since the logarithm of a ratio is equal to the difference of logarithms, this objective considers vector differences in the word vector space. Because these ratios can encode some form of meaning, this information gets encoded as vector differences as well. For this reason, the resulting word vectors perform very well on word analogy tasks, such as those discussed in the word2vec package.

Since singular value decomposition algorithms have been optimized for decades, GloVe has an advantage in training over word2vec, which is a neural network and relies on gradient descent and backpropagation to perform its error minimization. If training our own word vectors from a certain corpus that we care about, we are probably better off using a Global Vector model than word2vec, even though word2vec is the first to accomplish semantic and logical reasoning with words, since Global Vector trains faster, has better RAM and CPU efficiency, and gives more accurate results than word2vec, even on smaller corporuses.

Cosine Similarity

So far in this chapter we have worked toward one goal only: convert a document of natural language text into a vector of numbers. Our document can be one word, one sentence, a paragraph, multiple paragraphs, or longer. We discovered multiple ways to get our vectors—some are more semantically representative of our documents than others.

Once we have a document's vector representation, we can feed it into machine learning models, such as classification algorithms, clustering algorithms, or others. One example is to cluster the document vectors of a corpus with some clustering algorithm such as *k-means* to create a document classifier. We can also determine how semantically similar our document is to other documents, for search engines, information retrieval systems, and other applications.

We have established that due to the curse of dimensionality, measuring the Euclidean distance between two very high-dimensional document vectors is useless, since they would come out extremely far apart, only because of the vastness of the space they inhabit. So how do we determine whether vectors representing documents are *close* or *far*, or *similar* or *different*? One successful way is to use *cosine similarity*, measuring the cosine of the angle between the two document vectors. This is given by the dot product of the vectors, each normalized by its length (had we normalized the document vectors ahead of time, then their lengths would have already been one):

$$\cos \left(\text{angle between } \overrightarrow{doc_1} \text{ and } \overrightarrow{doc_2} \right) = \frac{\overrightarrow{doc_1}^t \overrightarrow{doc_2}}{\text{length}(\overrightarrow{doc_1}) \text{length}(\overrightarrow{doc_2})}$$

Figure 7-3 shows three documents represented in a two-dimensional vector space. We care about the angles between them.

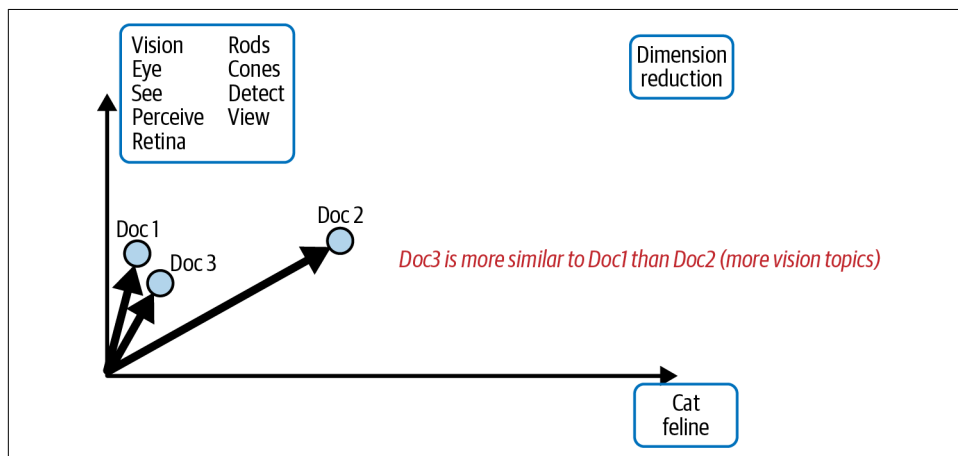


Figure 7-3. Three documents represented in a two-dimensional vector space

The cosine of an angle is always a number between -1 and 1. When two document vectors are perfectly aligned and pointing in the same direction along all the dimensions, their cosine similarity is 1; when they are perfect opposites of each other

regarding every single dimension, their cosine similarity is -1; and when they are orthogonal to each other, their cosine similarity is 0.

Natural Language Processing Applications

The bulk of this chapter has been about converting a given document of natural language text to a vector of numbers. We have established that there are multiple ways to get our document vectors, all leading to varying representations (and hence conclusions), or emphasizing certain aspects of the given natural language data over others. For people entering the natural language processing subfield of AI, this is one of the hardest barriers to overcome, especially if they are from a quantitative background, where the entities they work with are inherently numerical, ripe for mathematical modeling and analysis. Now that we have overcome this barrier, equipped with concrete vector representations for natural language data, we can think mathematically about popular applications. It is important to be aware that there are multiple ways to accomplish each of the following. Traditional approaches are hardcoded rules, assigning scores to words, punctuations, emojis, etc., then relying on the existence of these in a data sample to produce a result. Modern approaches rely on various machine learning models, which in turn rely on (mostly) labeled training data sets. To excel in this field, we must set time aside and try different models on the same task, compare performance, and gain an in-depth understanding of each model along with its strengths, weaknesses, and mathematical justifications of its successes and failures.

Sentiment Analysis

The following are common approaches for extracting sentiment from natural language text:

Hardcoded rules

A successful algorithm is VADER, or Valence Aware Dictionary for sEntiment Reasoning. A tokenizer here needs to handle punctuation and emojis properly, since these convey a lot of sentiment. We also have to manually compile thousands of words along with their sentiment score, as opposed to having the machine accomplish this automatically.

Naive Bayes classifier

This is a set of classifying algorithms based on the Bayes' Theorem from probability, the decision rule for classification on maximum likelihood. This will be discussed in [Chapter 11](#).

Latent discriminant analysis

In the previous section, we learned how to classify documents into two classes using latent discriminant analysis. To recap, we start with the data labeled into

two classes, then we compute the centroid of each class and find the direction connecting them. We project each new data instance along that direction, and classify it according to which centroid it falls closer to.

Using latent semantic analysis

Clusters of document vectors formed using latent semantic analysis can be used for classification. Ideally, positive reviews cluster away from negative reviews in latent semantic analysis topic spaces. Given a bunch of reviews labeled positive or negative, we first compute their topic vectors using latent semantic analysis. Now, to classify a new review, we can compute its topic vector, then that topic vector's cosine similarity with the positive and negative topic vectors. Finally, we classify the review as positive if it is more similar to positive topic vectors, and negative if more similar to negative topic vectors.

Transformers, convolutional neural network, recurrent long short-term memory neural network

All of these modern machine learning methods require passing our document in vector form into a neural network with a certain architecture. We will spend time on these state-of-the-art methods shortly.

Spam Filter

Mathematically, spam filtering is a similar classification problem to sentiment analysis discussed previously, when the sentiment of a document is either positive or negative. Thus, the same methods for sentiment classification apply for spam filtering. In all cases, it doesn't matter how we create our document vectors—we can use them to predict whether a social post is spam or not spam, predict how likely it is to get likes, etc.

Search and Information Retrieval

Again, no matter how we create the numerical vectors representing documents, we can use them for search and information retrieval tasks. The search can be index based or semantic based.

Full text search

When we search for a document based on a word or a partial word that it contains. Search engines break documents into words that can be indexed, similar to the indexes we find at the end of textbooks. Of course, spelling errors and typos require a lot of tracking and sometimes guessing. Indices, when available, work pretty well.

Semantic search

Here, our search for documents takes into account the meaning of the words in both our query and in the documents within which we are searching.

The following are common approaches for search and information retrieval:

Based on cosine similarity between the TF-IDF of documents

This is good for corpuses containing billions of documents. Any search engine with a millisecond response time employs an underlying TF-IDF matrix.

Based on semantics

Cosine similarity between topic vectors of documents obtained through latent semantic analysis (for corpuses containing millions of documents) or latent Dirichlet allocation (for much smaller corpuses). This is similar to how we classified whether a message is spam or not spam using latent semantic analysis, except that now we compute the cosine similarity between the new document's topic vector and *all* the topic vectors of our database, returning the ones that are most similar to our document.

Based on eigenvector iteration

This has to do with ranking algorithms in search results, such as the PageRank algorithm (which we go over in “[Example: PageRank Algorithm](#)” on page 302). The following is a useful excerpt from the paper “[Role of Ranking Algorithms for Information Retrieval](#)” (Choudhary and Burdak 2012):

There are three important components in a search engine. They are the crawler, the indexer, and the ranking mechanism. The crawler, also called a robot or spider, traverses the web and downloads the web pages. These downloaded pages are sent to an indexing module that parses the web pages and builds the index based on the keywords in those pages. An index is generally maintained using keywords. When a user types a query using keywords into a search engine's interface, the query processor component matches the query keywords with the index and returns URLs to the user. But before showing the pages to the user, a ranking mechanism is used by the search engine to show the most relevant pages at the top and less relevant ones at the bottom.

Semantic search and queries using word vectors (word2vec or GloVe)

Consider a search like this one, adapted from the book *Natural Language Processing in Action*: “She invented something to do with physics in Europe in the early 20th century.” When we enter our search sentence into Google or Bing, we may not get the direct answer *Marie Curie*. Google Search will most likely only give us links to lists of famous physicists, both men and women. After searching several pages we find *Marie Curie*, our answer. Google will take note of that, and refine our results next time we search. Now using word vectors, we can do simple arithmetic on the word vectors representing woman+Europe+physics+scientist+famous, then we would obtain a new vector, close in cosine similarity to the vector representing *Marie Curie*, and voilà! We have our answer. We can even subtract gender bias in the natural sciences from word vectors by simply subtracting the vector representing the token *man*, *male*, etc., so we can search for the word vector closest to: woman+Europe+physics+scientist-male-2*man.

Search based on analogy questions

To compute a search such as *They are to music what Marie Curie is to science*, all we have to do is simple vector arithmetic of the word vectors representing Marie Curie-science+music.

The following paragraph, regarding indexing and semantic searches is paraphrased from the book *Natural Language Processing in Action*:

Traditional indexing approaches work with binary word occurrence vectors, discrete vectors (bag-of-word vectors), sparse floating-point number vectors (TF-IDF vectors), and low-dimensional floating-point number vectors (such as three-dimensional geographic information system data). But high-dimensional floating-point number vectors, such as topic vectors from latent semantic analysis or latent Dirichlet allocation, are challenging. Inverted indexes work for discrete vectors or binary vectors, because the index only needs to maintain an entry for each nonzero discrete dimension. That value of that dimension is either present or not present in the referenced vector or document. Because TF-IDF vectors are sparse, mostly zero, we do not need an entry in our index for most dimensions for most documents. Latent semantic analysis and latent Dirichlet allocation produce topic vectors that are high dimensional, continuous, and dense, where zeros are rare. Moreover, the semantic analysis algorithm does not produce an efficient index for scalable search. This is exacerbated by the curse of dimensionality, which makes an exact index impossible. One solution to the challenge of high-dimensional vectors is to index them with a locality-sensitive hash, like a zip code, that designates a region of hyperspace. Such a hash is similar to a regular hash: it is discrete and only depends on the values in the vector. But even this doesn't work perfectly once we exceed about 12 dimensions. An exact semantic search wouldn't work for a large corpus, such as a Google search or even a Wikipedia semantic search. The key is to settle for *good enough* rather than striving for a perfect index or a latent hashing algorithm for our high-dimensional vectors. There are now several open source implementations of some efficient and accurate approximate nearest neighbors algorithms that use latent semantic hashing to efficiently implement semantic search. Technically, these indexing or hashing solutions cannot guarantee that we will find all the best matches for our semantic search query. But they can get a good list of close matches almost as fast as with a conventional reverse index on a TF-IDF vector or bag-of-words vector if we are willing to give up a little precision. Neural network models fine-tune the concepts of topic vectors so that the vectors associated with words are more precise and useful, hence enhancing searches.

Machine Translation

The goal here is to translate a sequence of tokens of any length (such as a sentence or a paragraph) to a sequence of any length in a different language. The encoder-decoder architecture, discussed in the context of transformers and recurrent neural networks, has proven successful for translation tasks. The encoder-decoder architecture is different than the auto-encoder architecture.

Image Captioning

This combines computer vision with natural language processing.

Chatbots

This is the ultimate application of natural language processing. A chatbot requires more than one kind of processing: parse language, search, analyze, generate responses, respond to requests, and execute them. Moreover, it requires a database to maintain a memory of past statements and responses.

Other Applications

Other applications include *named-entity recognition*, *conceptual focus*, relevant information extraction from text (such as dates), and language generation, which we visit in [Chapter 8](#).

Transformers and Attention Models

Transformers and attention models are the state-of-the-art for natural language processing applications such as machine translation, question answering, language generation, named-entity recognition, image captioning, and chatbots (as of 2022). Currently, they underlie large language models such as [Google’s BERT \(Bidirectional Encoder Representations from Transformers\)](#) and OpenAI’s [GPT-2 \(Generative Pre-trained Transformer\)](#) and [GPT-3](#).

Transformers bypass both recurrence and convolution architectures, which were the go-to architectures for natural language processing applications up until 2017, when the paper [“Attention Is All You Need” \(Vaswani et al. 2017\)](#) introduced the first transformer model.

The dethroned recurrent and convolutional neural network architectures are still in use (and work well) for certain natural language processing applications, as well as other applications such as finance. We elaborate on these models later in this chapter. However, the reasons that led to abandoning them for natural language are:

- For short input sequences of natural language tokens, the attention layers that are involved in transformer models are faster than recurrent layers. Even for long sequences, we can modify attention layers to focus on only certain neighborhoods within the input.
- The number of sequential operations required by a recurrent layer depends on the length of the input sequence. This number stays constant for an attention layer.

- In convolutional neural networks, the width of the kernel directly affects the long-term dependencies between pairs of inputs and corresponding outputs. Tracking long-term dependencies then requires large kernels, or stacks of convolutional layers, all increasing the computational cost of the natural language model employing them.

The Transformer Architecture

Transformers are an integral part of enormous language models, such as GPT-2, GPT-3, ChatGPT, Google's BERT (which trains the language model by looking at the sequential text data from both left to right and right to left) and **Wu Dao's transformer**. These models are massive: GPT-2 has around 1.5 billion parameters trained on millions of documents, drawn from 8 million websites from all around the internet. GPT-3 has 175 billion parameters trained on an even larger data set. Wu Dao's transformer has a whopping 1.75 trillion parameters, consuming a ton of computational resources for training and inference.

Transformers were originally designed for language translation tasks, so they have an *encoder-decoder* structure. **Figure 7-4** illustrates the architecture of the transformer model originally introduced by the paper “Attention Is All You Need” (Vaswani et al. 2017). However, each encoder and decoder is its own module, so they can be used separately to perform various tasks. For example, we can use the encoder alone to perform a classification task such as a part of speech tagging, meaning we input the sentence: *I love cooking in my kitchen*, and the output will be a class for each word: *I*: pronoun; *love*: verb; *cooking*: noun, etc.

The input to the full transformer model (with both the encoder and decoder included) is a sequence of natural language tokens of any length, such as a question to a chatbot, a paragraph in English that requires translation to French, or a summarization into a headline. The output is another sequence of natural language tokens, also of any length, such as the chatbot's answer, the translated paragraph in French, or the headline.

Do not confuse the training phase with the inference phase of a model:

During training

The model is fed both the data and the labels, such as an English sentence (input data sample) along with its French translation (label), and the model learns a mapping from the input to the target label that hopefully generalizes well to the entire vocabularies and grammars of both languages.

During inference

The model is fed only the English sentence, and outputs its French translation. Transformers output the French sentence one new token at a time.

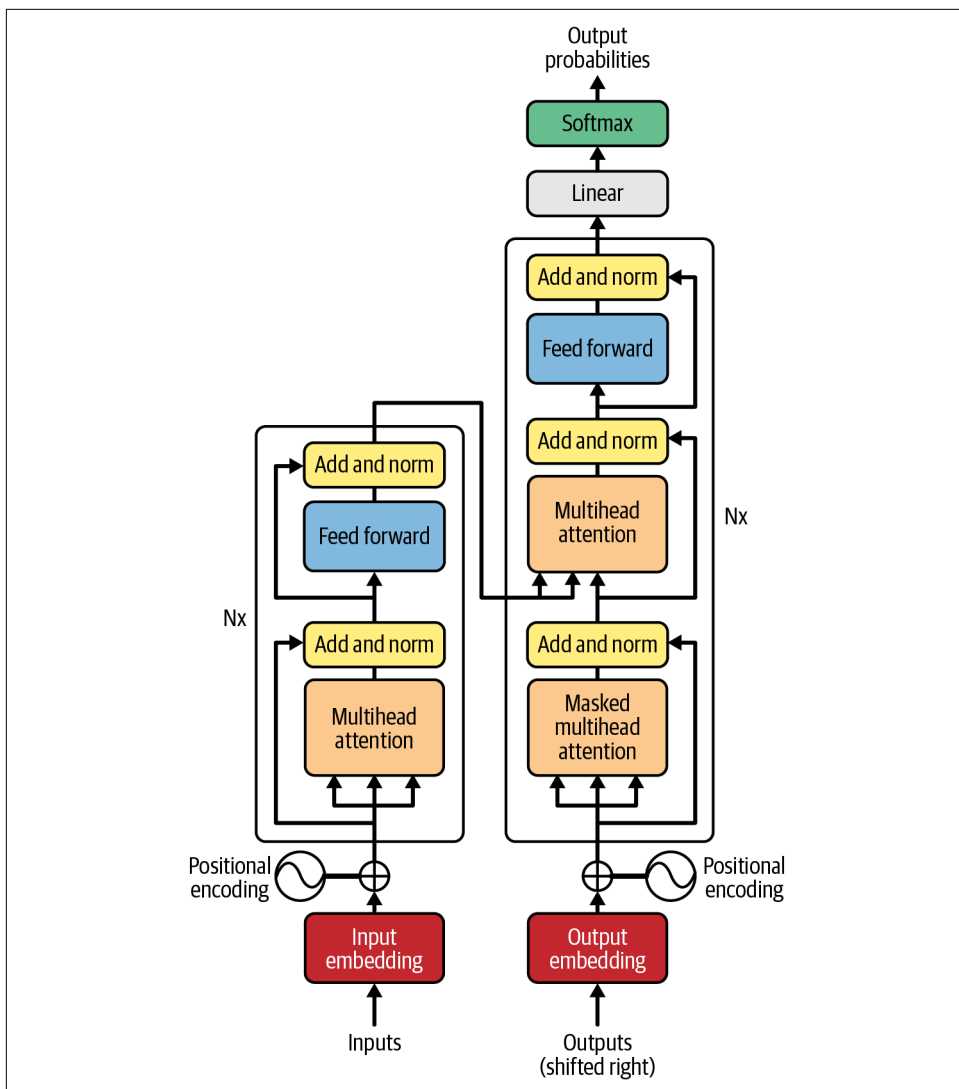


Figure 7-4. The simple encoder-decoder architecture of a transformer model ([image source](#))

The encoder, on the left half of the transformer architecture in [Figure 7-4](#), receives an input of tokens, such as an English sentence, *How was your day?*, and produces multiple numerical vector representations for each token of this sentence—along with their linear transformations into new directions that capture alignment with other word vectors—encoding the token’s contextual information from within the sentence. The linear transformations happen (multiplying word vectors by weight matrices)

because the encoder wants to learn the important directions in which certain word vectors of a sentence align with others (contextual alignment).

The decoder, on the right half of the architecture in [Figure 7-4](#), receives these vectors as part of its input, together with its own output from the previous time step. Ultimately, it generates an output of tokens, such as the French translation of the input sentence, *Comme se passe ta journée* (see [Figure 7-5](#)). What the decoder actually computes is a *probability* for each word in the French vocabulary (say, 50,000 tokens) using a softmax function, then produces the token with the highest probability. In fact, since computing a softmax for such a high-dimensional vocabulary is expensive, the decoder uses a *sampled softmax*, which computes the probability for each token in a random sample of the French vocabulary at each step. During training, it has to include the target token in this sample, but during inference, there is no target token.

Transformers use a process called *attention* to capture long-term dependencies in sequences of tokens. The word *sequence* is confusing here, especially for mathematicians, who have clear distinctions among the terms *sequence*, *series*, *vector*, and *list*. Sequences are usually processed one term at a time, meaning one term is processed, then the next, then the next, and so on, until the whole input is consumed. Transformers do not process input tokens sequentially. They process them all together, in parallel. This is different from the way recurrent neural networks process input tokens, which have to be fed sequentially, in effect prohibiting parallel computation. If it was up to us to correct this terminology, we should call a natural language sentence a *vector* if we are processing it using a transformer model, or a *matrix* since each word in the sentence is represented as its own vector, or a *tensor* if we process a batch of sentences at a time, which the architecture of the transformer allows. If we want to process the same exact sentence using a recurrent neural network model, then we should call it a *sequence*, since this model consumes its input data sequentially, one token at a time. If we process it using a convolutional neural network, then we would call it a vector (or matrix) again, since the network consumes it as a whole, not broken down into one token at a time.

It is an advantage when a model does not need to consume the input sequentially, because such architectures allow for parallel processing. That said, even though parallelization makes transformers computationally efficient, they cannot take full advantage of the inherent sequential nature of the natural language input and the information encoded within this sequentiality. Think of how humans process text. There are new transformer models that try to leverage this.

The transformer model runs as follows:

1. Represent each word from the input sequence as a d -dimensional vector.
2. Incorporate the order of words into the model by adding to the word vector information about its position (*positional encoding*). Introduce positional information into the input by accompanying each vector of each word with a positional encoding vector of the same length. The positional encoding vectors have the same dimension as the word vector embeddings (this allows the two vectors to be added together). There are many choices of positional encodings: some are learned during training, others are fixed. Discretized sine and cosine functions with varying frequencies are common.
3. Next, feed the positionally encoded word vectors to the encoder block. The encoder attends to all words in the input sequence, irrespective of if they precede or succeed the word under consideration, thus the transformer encoder is bidirectional.
4. The decoder receives as input its own predicted output word at time step $t-1$, along with the output vectors of the encoder.
5. The input to the decoder is also augmented by positional encoding.
6. The augmented decoder input is fed into the three sublayers. The decoder cannot attend to succeeding words, so we apply masking in its first sublayer. At the second sublayer, the decoder also receives the output of the encoder, which now allows the decoder to attend to all of the words in the input sequence.
7. The output of the decoder finally passes through a fully connected layer, followed by a softmax layer, to generate a prediction for the next word of the output sequence.

The Attention Mechanism

The transformer's magic is largely due to built-in *attention mechanisms*. An attention mechanism comes with bonuses:

Explainability

Pointing out which parts of the input sentence (or document) the model paid attention to when producing a particular output (see [Figure 7-5](#)).

Leveraging pre-trained attention models

We can adapt pre-trained models to domain-specific tasks. That is, we can further tweak their parameter values with extra training on domain-specific data.

More accurate modeling of longer sentences

Another value of attention mechanisms is that they allow the modeling of dependencies in sequences of natural language tokens without regard to how far apart related tokens occur in these sequences.

Figure 7-5 illustrates attention for a translation task from English to French.

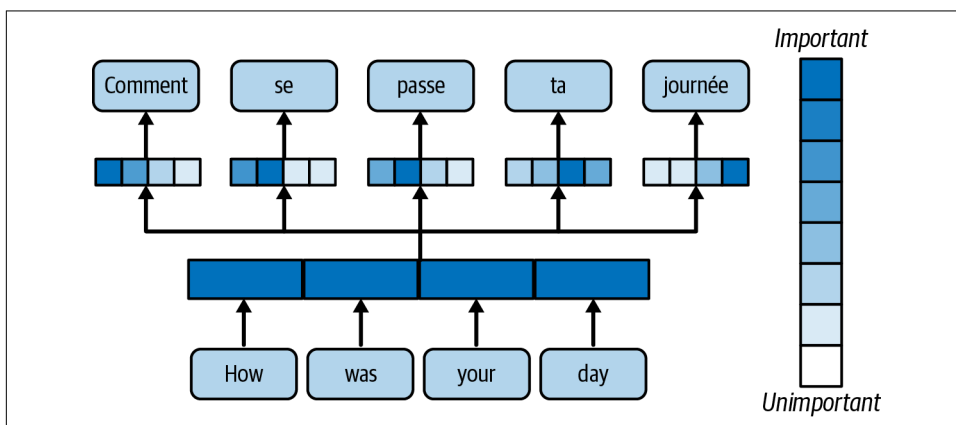


Figure 7-5. Illustrating attention via a translation task: how weights assigned to input tokens show which ones the model paid more attention to in order to produce each output token (*image source*)

There is no hardcore mathematics involved in an attention mechanism: we only have to compute a scaled dot product. The main goal of attention is to highlight the most relevant parts of the input sequence, how strongly they relate to each other within the input itself, and how strongly they contribute to certain parts of the output. We compute these word alignment scores in multiple directions because words can relate to each other in multiple ways.

Self attention is when a sequence of vectors computes alignment within its own members. We are now familiar with the fact that the dot product measures the compatibility between two vectors. We can compute the simplest possible self attention weights by finding the dot products between all the members of the sequence of vectors. For example, for the sentence *I love cooking in my kitchen*, we would compute all the dot products between the word vectors representing the words *I*, *love*, *cooking*, *in*, *my*, and *kitchen*. We would expect the dot product between *I* and *my* to be high, similarly between *cooking* and *kitchen*. However, the dot product will be highest between *I* and *I*, *love* and *love*, etc., because these vectors are perfectly aligned with themselves, but there is no valuable information gleaned there.

The transformer's solution to avoiding this waste is multifold:

1. Apply three different linear transformations to each vector of the input sequence (each word of the sentence), multiplying them by three different weight matrices. We then obtain three different sets of vectors corresponding to each input word vector $\vec{w}$:

- The *query* vector $\overrightarrow{query} = W_q \overrightarrow{w}$, the vector *attended from*.
 - The *key* vector $\overrightarrow{key} = W_k \overrightarrow{w}$, the vector *attended to*.
 - The *value* vector $\overrightarrow{value} = W_v \overrightarrow{w}$, to capture the context that is being generated.
2. Obtain alignment scores between the query and key vectors for all words in the sentence by computing their dot product scaled by the inverse of the square root of the length of these vectors $\sqrt{l}$. We apply this scaling for numerical stability to keep the dot products from becoming large. (These dot products will soon be passed into a softmax function. Since the softmax function has a very small gradient when its input has a large magnitude, we offset this effect by dividing each dot product by $\sqrt{l}$.) Moreover, alignment of two vectors is independent from the lengths of these vectors. Therefore, the alignment score between *cooking* and *kitchen* in our sentence will be:

$$alignment_{cooking,kitchen} = \frac{1}{\sqrt{l}} \overrightarrow{query}_{cooking}^t \overrightarrow{key}_{kitchen}$$

Note that this will be different than the alignment score between *kitchen* and *cooking*, since the query and key vectors for each are different. Thus, the resulting alignment matrix is not symmetric.

3. Transform each alignment score between each two words in the sentence into a probability by passing the score into the *softmax* function. For example:

$$\begin{aligned} \omega_{cooking,kitchen} = & \\ & softmax(alignment_{cooking,kitchen}) = \\ & \frac{exp(alignment_{cooking,kitchen})}{\{exp(alignment_{cooking,I}) + \\ & exp(alignment_{cooking,love}) + \\ & exp(alignment_{cooking,cooking}) + \\ & exp(alignment_{cooking,in}) + \\ & exp(alignment_{cooking,my}) + \\ & exp(alignment_{cooking,kitchen})\}} \end{aligned}$$

4. Finally, encode the context of each word by linearly combining the value vectors using the alignment probabilities as weights for the linear combination. For example:

$$\begin{aligned}
context_{cooking} &= \overrightarrow{\omega_{cooking,I} value_I} + \overrightarrow{\omega_{cooking,love} value_{love}} \\
&+ \overrightarrow{\omega_{cooking,cooking} value_{cooking}} + \overrightarrow{\omega_{cooking,in} value_{in}} + \overrightarrow{\omega_{cooking,my} value_{my}} \\
&+ \overrightarrow{\omega_{cooking,kitchen} value_{kitchen}}
\end{aligned}$$

We have thus managed to capture in one vector the context of each word in the given sentence, with high worth assigned to those words in the sentence it mostly aligns with.

The good news here is that we can compute the context vector for all the words of a sentence (data sample) simultaneously, since we can pack the vectors previously mentioned in matrices and use efficient and parallel matrix computations to get the contexts for all the terms at once.

We implement all of the above in one *attention head*. That is, one attention head produces one context vector for each token in the data sample. We would benefit from producing multiple context vectors for the same token, since some information gets lost with all the averaging happening on the way to a context vector. The idea here is to be able to extract information using different representations of the terms of a sentence (data sample), as opposed to a single representation corresponding to a single attention head. So we implement a *multihead attention*, choosing for each head new transformation matrices W_q , W_k , and W_v .

Note that during the training process, the entries of the transformation matrices are model parameters that have to be learned from the training data samples. Imagine then how fast the number of the model's parameters will balloon.

Figure 7-6 illustrates the *multihead attention mechanism*, implementing h heads that receive different linearly transformed versions of the queries, keys, and values to produce h context vectors for each token, which are then concatenated to produce the output of the *multihead attention* part of the model's structure.

The decoder uses a similar self attention mechanism, but here each word can only attend to the words before it, since text is generated from left to right. Moreover, the decoder has an extra attention mechanism attending to the outputs it receives from the encoder.

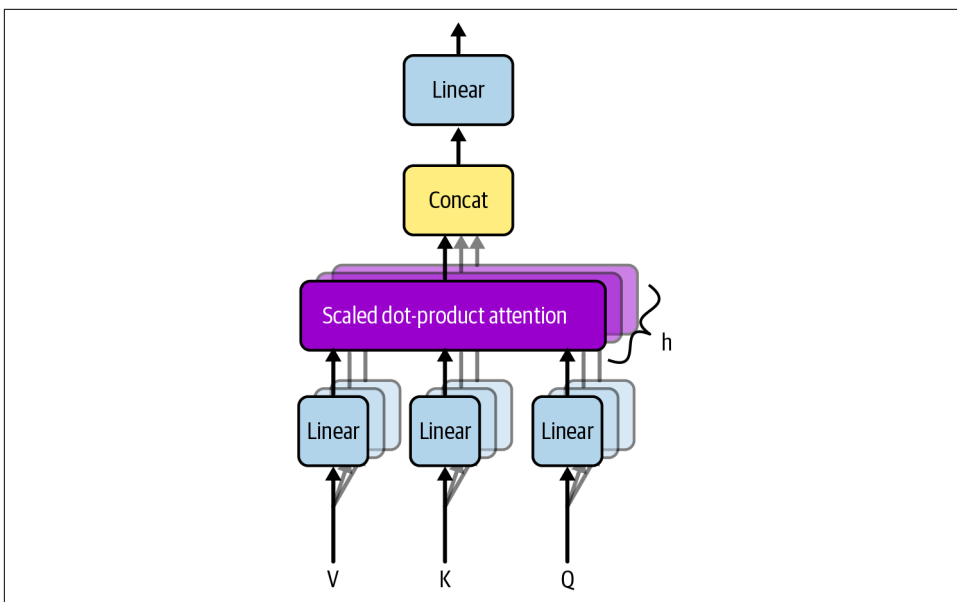


Figure 7-6. Multihead attention mechanism (*image source*)

Transformers Are Far from Perfect

Even though transformer models have revolutionized the natural language processing field, they are far from perfect. Language models, in general, are *mindless mimics*. They understand neither their inputs nor their outputs. Critical articles, such as [this](#) and [this](#) by the *MIT Technology Review*, among others, detail their shortcomings, such as lack of comprehension of language, repetition when used to generate long passages of text, and so on. That said, the transformer model brought about a tidal wave for natural language, and it is making its way to other AI fields, such as biomedicine, computer vision, and image generation.

Convolutional Neural Networks for Time Series Data

The term *time series* in the natural language processing and finance domains should be *time sequence* instead. *Series* in mathematics refers to adding up the terms of an infinite *sequence*. So when our data is *not summed*, which is the case for all natural language data and most finance data, we actually have *sequences*, not *series*, of numbers, vectors, etc. Oh well, vocabulary collisions are unavoidable, even across different fields that heavily rely on one another.

Other than the definitions of words in a dictionary, their meanings are mostly correlated to the way they occur relative to each other. This is conveyed through the

way words are *ordered* in sentences, as well as their context and proximity to other words in sentences.

We first emphasize the two ways in which we can explore the meanings behind words and terms in documents:

Spatially

Exploring a sentence all at once as one vector of tokens, whichever way these tokens are represented mathematically.

Temporarily

Exploring a sentence sequentially, one token at a time.

Convolutional neural networks, discussed in [Chapter 5](#), explore sentences spatially, by sliding a fixed-width window (kernel or filter) along the tokens of the sentence. When using convolutional neural networks to analyze text data, the network expects an input of fixed dimensions. On the other hand, when using recurrent neural networks (discussed next) to analyze text data, the network expects tokens sequentially, hence, the input does not need to be of fixed length.

In [Chapter 5](#), we were sliding two-dimensional windows (kernels or filters) over images, and in this chapter, we will slide one-dimensional kernels over text tokens. We now know that each token is represented as a vector of numbers. We can either use one-hot encoding or word vectors of the word2vec model. One-hot encoded tokens are represented with a very long vector that has a 0 for every possible vocabulary word that we want to include from the corpus, and a 1 in the position of the token we are encoding. Alternatively, we can use trained word vectors produced via word2vec. Thus, an input data sample to the convolutional neural network is a matrix made up of column vectors, one column for each token in the data sample. If we use word2vec to represent tokens, then each column vector would have 100 to 500 entries, depending on the particular word2vec model used. Recall that for a convolutional neural network, each data sample has to have the exact same number of tokens.

Therefore, one data sample (a sentence or a paragraph) is represented with a two-dimensional matrix, where the number of rows is the full length of the word vector. In this context, saying that we are sliding a *one-dimensional* kernel over our data sample is slightly misleading, but here is the explanation. The vector representation of the sample's tokens extends *downward*; however, the filter covers the whole length of that dimension all at once. That is, if the filter is three tokens wide, it would be a matrix of weights with three columns and as many rows as the vector representation of our tokens. Thus, one-dimensional convolution here refers to convolving only *horizontally*. This is different than two-dimensional convolution for images, where the two-dimensional filter travels across the image both horizontally and vertically.

As in [Chapter 5](#), during a forward pass, the weight values in the filters are the same for one data sample. This means that we can parallelize the process, which is why convolutional neural networks are efficient to train.

Recall that convolutional neural networks can also process more than one channel of input at the same time, that is, three-dimensional tensors of input, not only two-dimensional matrices of numbers. For images this was processing the red, green, and blue channels of an input image all at once. For natural language, one input sample is a bunch of words represented as column vectors lined up next to each other. We now know that there are multiple ways to represent the same word as a vector of numbers, each perhaps capturing different semantics of the same word. These different vector representations of the same word are not necessarily of the same length. If we restrict them to be of the same length, then each representation can be a word's *channel*, and the convolutional neural network can process all the channels of the same data sample at once.

As in [Chapter 5](#), convolutional neural networks are efficient due to weight sharing, pooling layers, dropout, and small filter sizes. We can run the model with multiple size filters, then concatenate the output of each size filter into a longer thought vector before passing it into the fully connected last layer. Of course, the last layer of the network accomplishes the desired task, such as sentiment classification, spam filtering, text generation, and others. We went over this in [Chapters 5 and 6](#).

Recurrent Neural Networks for Time Series Data

Consider the following three sentences:

- She bought tickets to watch the movie.
- She, having free time, bought tickets to watch the movie.
- She, having heard about it nonstop for two weeks in a row, finally bought tickets to watch the movie.

In all three sentences, the predicate *bought tickets to watch the movie* corresponds to the sentence's subject *She*. A natural language model will be able to learn this if it is designed to handle long-term dependencies. Let's explore how different models handle such long-term dependencies:

Convolutional neural networks and long-term dependencies

A convolutional neural network, with its narrow filtering window ranging three to five tokens scanning the sentence, will be able to learn from the first sentence easily, and maybe the second sentence, given that the predicate's position changed only a little bit (pooling layers help with the network's resistance to small variations). The third sentence will be tough, unless we use larger filters (which increases the computation cost and makes the network more like a fully

connected network than a convolutional network), or if we deepen the network, stacking convolutional layers on top of each other so that the coverage widens as the sentence makes its way deeper into the network.

Recurrent neural networks with memory units

A completely different approach is to feed the sentence into the network sequentially, one token at a time, and maintain a *state* and a *memory* that hold on to important information for a certain amount of time. The network produces an outcome when all the tokens in the sentence have passed through it. If this is during training, only the outcome produced after the last token has been processed gets compared to the sentence's label, then the error *backpropagates through time*, to adjust the weights. Compare this to the way we hold on to information when reading a long sentence or paragraph. Recurrent neural networks with long short-term memory units are designed this way.

Transformer models and long-term dependencies

Transformer models, which we discussed earlier, abolish both convolution and recurrence, relying only on attention to capture the relationship between the subject of the sentence *She*, and the predicate *bought tickets to watch the movie*.

One more thing differentiates recurrence models from convolutional and transformer models: does the model expect its input to be of the same length for all data samples? Can we only input sentences of the same length? The answer for transformers and convolutional networks is that they expect only fixed-length data samples, so we have to preprocess our samples and make them all the same length. On the other hand, recurrent neural networks handle variable length inputs really well, since after all they only take them one token at a time.

The main idea for a recurrent neural network is that it holds on to past information as it processes new information. How does this holding happen? In a feed forward network, the output of a neuron leaves it and never gets back to it. In a recurrent network, the output loops back into the neuron, along with new input, in essence creating a *memory*. Such algorithms are great for autocompletion and grammar check. They have been integrated into Gmail's Smart Compose since 2018.

How Do Recurrent Neural Networks Work?

Here are the steps for how a recurrent neural network gets trained on a set of labeled data samples. Each data sample is made up of a bunch of tokens and a label. As always, the goal of the network is to learn the general features and patterns within the data that end up producing a certain label (or output) versus others. When tokens of each sample are input sequentially, our goal is then to detect the features, across all the data samples, that emerge when certain tokens appear in patterns relative to each other:

1. Grab one tokenized and labeled data sample from your data set (such as a movie review labeled positive, or a tweet labeled fake news).
2. Pass the first token $token_0$ of your sample into the network. Remember that tokens are vectorized, so you are really passing a vector of numbers into the network. In mathematical terms, we are evaluating a function at that token's vector and producing another vector. So far, our network has calculated $f(token_0)$.
3. Now pass the second token $token_1$ of your sample into the network, *along with the output of the first token, $f(token_0)$* . The network now will evaluate $f(token_1 + f(token_0))$. This is the recurrence step, and this is how the network does not forget $token_0$ as it processes $token_1$.
4. Now pass the third token $token_2$ of your sample into the network, *along with the output of the previous step, $f(token_1 + f(token_0))$* . The network now will evaluate $f(token_2 + f(token_1 + f(token_0)))$.
5. Keep going until you finish all the tokens of your one sample. Suppose this sample only had five tokens, then our recurrent network will output $f(token_4 + f(token_3 + f(token_2 + f(token_1 + f(token_0)))))$. Note that this output looks very similar to the output of the feed forward fully connected networks that we discussed in [Chapter 4](#), except that this output *unfolds through time* as we input a sample's tokens one at a time *into one recurrent neuron*, while in [Chapter 4](#), the network's output *unfolds through space*, as one data sample moves from one layer of the neural network to the next. Mathematically, when we write the formulas of each, they are the same, so we don't need more math beyond what we learned in the past three chapters. That's why we love math.
6. When training the network to produce the right thing, it is the final output of the sample, $f(token_4 + f(token_3 + f(token_2 + f(token_1 + f(token_0)))))$, that gets compared against the sample's true label via evaluating a loss function, exactly as we did in [Chapters 3, 4, and 5](#).
7. Now pass the next data sample one token at a time into the network and do the same thing again.
8. We update the network's weights in exactly the same way we updated them in [Chapter 4](#), by minimizing the loss function via a gradient descent-based algorithm, where we calculate the required gradient (the derivatives with respect to all the network's weights) via backpropagation. As we just said, this is exactly the same backpropagation mathematics we learned in [Chapter 4](#), except now, of course, we get to say *we're backpropagating through time*.

In finance, dynamics, and feedback control, this process is called an **autoregressive moving average (ARMA) model**.

Training a recurrent neural net can be expensive, especially for data samples of any significant length, say 10 tokens or more, since the number of weights to learn is directly related to the number of tokens in data samples: the more tokens, the more *depth in time* the recurrent network has. Other than the computational cost, this depth comes with all the troubles encountered by regular feed forward networks with many layers: vanishing or exploding gradients, especially with samples of data with hundreds of tokens, which will be the mathematical equivalent of a fully connected feed forward neural network with hundreds of layers! The same remedies for exploding and vanishing gradients for feed forward networks work here.

Gated Recurrent Units and Long Short-Term Memory Units

Recurrent neurons in recurrent networks are not enough to capture long-term dependencies in a sentence. A token's effect gets diluted and stepped on by the new information as more tokens pass through the recurrent neuron. In fact, a token's information is almost completely gone only after two tokens have passed. This problem can be addressed if we add memory units, called *long short-term memory units*, to the architecture of the network. These help learning dependencies stretching across a whole data sample.

Long short-term memory units contain neural networks, and they can be trained to find only the new information that needs to be retained for the upcoming input, and to forget, or reset to zero, information that is no longer relevant to learning. Therefore, long short-term memory units learn which information to hold on to, while the rest of the network learns to predict the target label.

There is no new mathematics beyond what we learned in [Chapter 4](#) here, so we will not go into the weeds digging into the specific architecture of a long short-term memory unit, or a *gated unit*. In summary, the input token for each time step passes through the forget and update gates (functions), gets multiplied by weights and masks, then gets stored in a memory cell. The network's next output depends on a combination of the input token and the memory unit's current state. Moreover, long short-term memory units share the weights they learned across samples, so they do not have to relearn basic information about language as they go through each sample's tokens.

Humans are able to process language on a subconscious level, and long short-term memory units are a step into modeling that. They are able to detect patterns in language that allow us to address more complex tasks than mere classification, such as language generation. We can generate novel text from learned probability distributions. This is the topic of [Chapter 8](#).

An Example of Natural Language Data

When faced with narratives about different models, it always makes things easier when we have specific examples in mind with real data, along with the models' hyperparameters. We can find the [IMDb](#) movie review data set at [the Stanford AI website](#). Each data sample is labeled with a 0 (negative review) or a 1 (positive review). We can start with the raw text data if we want to practice preprocessing natural language text. Then we can tokenize it and vectorize it using one-hot encoding over a chosen vocabulary, the Google word2vec model, or some other model. Do not forget to split the data into the training and test sets. Then choose the hyperparameters, for example, length of word vectors around 300, number of tokens per data sample around 400, mini-batches of 32, and number of epochs 2. We can play around with these to get a feel for the models' performance.

Finance AI

AI models have a wide use for the finance field. By now, we know the underlying structure of most AI models (except for graphs, which have a different mathematical structure; we discuss graph networks in [Chapter 9](#)). At this point, only mentioning an application area from finance is enough for us to have a very good idea about how to go about modeling it using AI. Moreover, many finance applications are naturally intertwined with natural language processing applications, such as marketing decisions based on customer reviews, or a natural language processing system used to predict economic trends and trigger large financial transactions based only on the models' outputs.

The following are only two AI applications in finance, among many. Think of ways we can put what we have learned so far to good use modeling these problems:

- Stock market time series prediction. A recurrent neural network can take a sequence of inputs and produce a sequence of outputs. This is useful for the time series prediction required for stock prices. We input the prices over the past n days, and the network outputs the prices from the past $n-1$ days *along with tomorrow's price*.
- Autoregressive moving average (ARMA) model in finance, dynamics, and feedback control.

The stock market appears multiple times in this book. Keep an eye out for it when we are discussing stochastic processes in [Chapter 11](#) on probability.

Summary and Looking Ahead

There was almost no new math in this chapter; however, it was one of the hardest to write. The goal was to summarize the most important ideas in the whole natural language processing field. Moving from words to relatively low-dimensional vectors of numbers that carry meaning was the main barrier to overcome. Once we learned multiple ways to do this, whether vectorizing one word at a time or the main topics in a long document or an entire corpus, feeding those vectors to different machine learning models with different architectures and purposes was just business as usual.

Calculus

- The $\log$ scale for term frequencies and inverse document frequencies

Statistics

- Zipf's law for word counts
- The Dirichlet probability distribution for assigning words to topics and topics to documents

Linear algebra

- Vectorizing documents of natural language
- The dot product of two vectors and how it provides a measure of similarity or compatibility between the entities that the vectors represent
- Cosine similarity
- Singular value decomposition, i.e., latent semantic analysis

Probability

- Conditional probabilities
- Bilinear log model

Time series data

- What it means
- How it is fed into machine learning models (as one bulk, or one item at a time)

AI model of the day

- The transformer

Probabilistic Generative Models

AI ties up all the math that I know together, and I have been getting to know math for years.

—H.

If machines are ever to be endowed with an understanding of the world around them, and an ability to recreate it, like we do when we imagine, dream, draw, create songs, watch movies, or write books, then generative models are one significant step in that direction. We need to get these models right if we are ever going to achieve general artificial intelligence.

Generative models are built on the assumption that we can only interpret input data correctly if our model has learned the underlying statistical structure of this data. This is loosely analogous to our dreaming process, which points to the possibility that our brain has learned a model that is able to virtually recreate our environment.

In this chapter, we still have the mathematical structure of training function, loss function, and optimization presented throughout the book. However, unlike in the first few chapters, we aim to learn probability distributions, instead of deterministic functions. The overarching theme is that there is training data, and we want to come up with a mathematical model that generates new data similar to it.

There are two quantities of interest:

- The true (and unknown) joint probability distribution of the features of the input data $p_{data}(\vec{x})$.
- The model joint probability distribution of the features of the data along with the parameters of the model: $p_{model}(\vec{x}; \vec{\theta})$.

Ideally, we want these two as close as possible. In practice, we settle for parameter values $\vec{\theta}$ that allow $p_{model}(\vec{x}; \vec{\theta})$ to work well for our particular use cases.

Throughout the chapter, we make use of three rules for probability distributions:

1. The product rule that decomposes the multivariable joint probability distribution into a product of single variable conditional probability distributions.
2. Bayes' Rule, which allows us to flip between variables seamlessly.
3. Independence or conditional independence assumptions on the features or on latent (hidden) variables, which allow us to simplify the product of single variable conditional probabilities even further.

In previous chapters we were minimizing the *loss function*. In this chapter the analogous function is the *log likelihood function*, and the optimization process always attempts to *maximize* this log likelihood (careful, we are not minimizing a loss function, we are maximizing an objective function instead). More on this soon.

Before we dive in, let's make a note that puts our previous deterministic machine learning models into probability language. Our previous models learned a training function that mapped the features of the input data $\vec{x}$ to an output y (target or label), or $f(\vec{x}; \vec{\theta}) = y$. When our goal was classification, f returned the label y that had the highest probability. That is, a classifier learns a direct map from input data $\vec{x}$ to class labels y ; in other words, they model the posterior probability $p(y|\vec{x})$ directly. We will elaborate on this later in the chapter.

What Are Generative Models Useful For?

Generative models have made it possible to blur the lines between true and computer-generated data. They have been improving and are achieving impressive successes: machine-generated images, including those of humans, are increasingly more realistic. It is hard to tell whether an image of a model in the fashion industry is that of a real person or the output of a generative machine learning model.

The goal of a generative model is to use a machine to generate novel data, such as audio waveforms containing speech, images, videos, or natural language text. Generative models sample data from a learned probability distribution, where the samples mimic reality as much as possible. The assumption here is that there is some unknown probability distribution underlying the real-life data that we want to mimic (otherwise our whole reality will be some random chaotic noise, lacking any coherence or structure), and the model's goal is to learn an approximation of this probability distribution using the training data.

After collecting a large amount of data from a specific domain, we train a generative model to generate data similar to the collected data. The collected data can be millions of images or videos, thousands of audio recordings, or entire corpuses of natural language.

Generative models are useful for many applications, including augmenting data when data is scarce and more of it is needed, inputting missing values for higher-resolution images, and simulating new data for reinforcement learning or for semi-supervised learning when only few labels are available. Another application is image-to-image translation, such as converting aerial images into maps or converting hand-drawn sketches to images. More applications include image denoising, inpainting, super-resolution, and image editing, such as making smiles wider, cheekbones higher, and faces slimmer.

Moreover, generative models are built to generate more than one acceptable output by drawing multiple samples from the desired probability distribution. This is different than our deterministic models that average over the output with different features during training using a mean squared error loss function or some other averaging loss function. The downside here is that a generative model can draw some bad samples as well.

One type of generative model, namely *generative adversarial networks* (invented in 2014 by Ian Goodfellow et al.), are incredibly promising and have a wide range of applications, from augmenting data sets to completing masked human faces to astrophysics and high energy physics, such as simulating data sets similar to those produced at the CERN Large Hadron Collider, or simulating distribution of dark matter and predicting gravitational lensing. Generative adversarial models set up two neural networks that compete against each other in a zero-sum game (think game theory in mathematics) until the machine itself cannot tell the difference between a real image and a computer-generated one. This is why their outputs seem very close to reality.

Chapter 7, which was heavily geared toward natural language processing, flirted with generative models without explicitly pointing them out. Most applications of natural language processing, which are not simple classification models (spam or not spam, positive sentiment or negative sentiment, and part of speech tagging), include language generation. Such examples are autocomplete on our smartphones or email, machine translation, text summarization, chatbots, and image captioning.

The Typical Mathematics of Generative Models

Generative models perceive and represent the world through probability distributions. That is, a color image is one sample from the joint probability distribution of pixels that together form a meaningful image (try to count the dimensions of such a

joint probability distribution with all the red, green, and blue channels included), an audio wave is one sample from the joint probability distribution of audio signals that together make up meaningful sounds (these are also extremely high dimensional), and a sentence is one sample from the joint probability distribution of words or characters that together represent coherent sentences.

The glaring question is then: how do we compute these amazingly representative joint probability distributions that are able to capture the complexity of the world around us, but sadly happen to be extremely high dimensional?

The machine learning answer is predictable at this point. Start with an easy probability distribution that we know of, such as the Gaussian distribution, then find a way to mold it into another distribution that well approximates the empirical distribution of the data at hand. But how do we mold one distribution into another? We can apply a deterministic function to its probability density. So we must understand the following:

How do we apply a deterministic function to a probability distribution, and what is the probability distribution of the resulting random variable?

We use the following transformation formula:

$$p_x(\vec{x}) = p_z\left(g^{-1}(\vec{x})\right) \left| \det \left(\frac{\partial g^{-1}\vec{x}}{\partial \vec{x}} \right) \right|$$

This is very well documented in many probability books and we will extract what we need from there shortly.

What is the correct function that we must apply?

One way is to train our model to *learn* it. We now know that neural networks have the capacity to represent a wide range of functions, so we can pass the simple probability distribution that we start with through a neural network (the neural network would be the formula of the deterministic function that we are looking for), then we learn the network's parameters by minimizing the error between the empirical distribution of the given data and the distribution output by the network.

How do we measure errors between probability distributions?

Probability theory provides us with some measures of how two probability distributions diverge from each other, such as the Kullback-Leibler (KL) divergence. This is also related to cross-entropy from information theory.

Do all generative models work this way?

Yes and no. *Yes* in the sense that they are all trying to learn the joint probability distribution that presumably generated the training data. In other words, generative models attempt to learn the formula and the parameters of a joint probability